

claude-windows / b2baac41-c08a-478d-a717-a673b57603e0

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\b2baac41-c08a-478d-a717-a673b57603e0.jsonl`  
- SHA-256: `e35d27ed848f2c551be43a54792dd92ebdd1a43372dac35c832ed17b95f09d7e`  
- Source modified: `2026-06-29T14:11:16+00:00`  
- Imported at: `2026-07-05T16:48:20+00:00`  
- project: `C--Users-avidu-Projects-muneem`  
- session_id: `b2baac41-c08a-478d-a717-a673b57603e0`
```

Transcript

1. user (2026-05-31T08:14:46.921Z)

can you resync from remote and refresh your state and tell me the next steps please?

2. assistant (2026-05-31T08:14:58.821Z)

I'll resync from remote and refresh my understanding of the project state. Let me start with the git operations and exploration in parallel.

3. user (2026-05-31T08:15:02.829Z)

From <https://github.com/avidullu/muneem>

```
- [deleted]      (none)      -> origin/docs/axis-known-issue  
- [deleted]      (none)      -> origin/docs/parser-architecture  
- [deleted]      (none)      -> origin/docs/parsers-reconciliation  
- [deleted]      (none)      -> origin/feat/axis-validation  
- [deleted]      (none)      -> origin/feat/hdfc-parser  
- [deleted]      (none)      -> origin/feat/icici-validation  
- [deleted]      (none)      -> origin/fix/axis-refuse-unreconciled  
- [deleted]      (none)      -> origin/fix/parser-registry-autoload  
- [deleted]      (none)      -> origin/refactor/shared-reconciler  
- [deleted]      (none)      -> origin/security/precommit-sensitive-file-guard
```

---STATUS---

On branch main

Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

---LOG---

```
6f6141f docs: parser architecture proposal (concept-core, verification-first) [DRAFT] (#21)  
128fc36 docs(roadmap): flag Axis savings parser as a must-fix (needs thorough rewrite) (#20)  
8e1e2a8 fix(parsers): Axis savings refuses to store unreconciled rows (stop-gap) (#19)  
bdee68d docs: CURRENT_STATE reflects Axis/ICICI now reconcile (shared validation) (#18)  
90579f8 feat(parsers): retrofit ICICI with parse/persist split + reconciliation (#17)  
1e12e21 feat(parsers): retrofit Axis with parse/persist split + reconciliation (#16)  
988a3ca refactor(parsers): extract shared running-balance reconciler; HDFC adopts it (#15)  
a40eebf feat(parsers): HDFC savings-statement parser with reconciliation + SUMMARY bookend (#14)  
6879e15 fix(parsers): auto-load submodules so the registry is populated in production (#13)  
7659548 security: pre-commit guard blocking statements/secrets; tighten hygiene test (#12)  
d72a49b chore: comprehensive tech debt audit & targeted cleanups (no behavior change) (#11)  
e036123 docs: add CURRENT_STATE.md summarizing latest milestone  
d2397d0 Add Axis Credit Card Parser and rigorous local test suite (#10)  
8bef285 Add Axis and ICICI parser architecture (#9)  
5ed5238 feat: statement unlocking (pypdf) + Gmail ingestion plumbing (#8)
```

4. user (2026-05-31T08:15:03.000Z)

CLAUDE.md

```
testdata\promo-emails\README.md
docs\TESTING.md
.venv\Lib\site-packages\pip\_vendor\idna\LICENSE.md
.venv\Lib\site-packages\pip-26.1.1.dist-info\licenses\src\pip\_vendor\idna\LICENSE.md
.venv\Lib\site-packages\typer\.agents\skills\typer\SKILL.md
.pytest_cache\README.md
DESIGN.md
docs\prior-art-email-mining.md
.venv\Lib\site-packages\pypdfium2-5.8.0.dist-
info\licenses\data\windows_x64\BUILD_LICENSES\libjpeg_turbo.md
README.md
.venv\Lib\site-packages\pyparsing\ai\best_practices.md
.venv\Lib\site-packages\idna-3.17.dist-info\licenses\LICENSE.md
docs\DEPENDENCIES.md
docs\CURRENT_STATE.md
docs\parser-architecture.md
docs\ROADMAP.md
```

5. assistant (2026-05-31T08:15:07.508Z)

Already in sync with `origin/main` (working tree clean, remote feature branches pruned post-merge). Now let me refresh my understanding of the current state by reading the key state docs.

6. user (2026-05-31T08:15:08.467Z)

```
1      # Current State
2
3      > **Last Updated:** 2026-06 (post tech-debt audit + ruff cleanup)
4      > This document summarizes the current capabilities, completed features, and near-term
next steps for `muneem`.
5      > See [ROADMAP.md](./ROADMAP.md) for the authoritative milestone tracking.
6
7      ## 1. Core Ingestion Pipeline
8      The ingestion pipeline supports a generic `SearchCriteria` interface and multiple
providers (Track B plumbing):
9
10     * **Gmail** (`muneem.ingestion.gmail`): Full OAuth2 read-only + search + attachment
download implemented.
11     * **IMAP** (`muneem.ingestion.imap`): Basic auth + search implemented; `get_headers` /
attachment methods are NotImplemented (experimental).
12     * **Outlook** (`muneem.ingestion.outlook`): Device-flow auth + basic search implemented;
attachment/header/download NotImplemented (experimental).
13
14     Companion one-time auth tools live in `tools/`. No high-level `muneem fetch` CLI command
yet (see Roadmap B.4).
15
16     ## 2. Statement Parsers (Track A + C progress)
17     Deterministic parsers live under `muneem.parsers`, registered via decorator and selected
by `get_parser` on text heuristics + sender. Submodules are **auto-loaded on package import**
(`_autoload_parsers`), so the registry is populated in the production path (not just under
tests).
18
19     * **HDFC Bank** (`hdfc.py`): Savings / transaction-account statements via **coordinate
clustering** (deliberately *not* `extract_tables()` ? HDFC's table is semi-ruled). Extracts
date, narration, value date, ref, withdrawal, deposit, closing balance. **Self-validating:**
balance reconciliation + a cross-check against the bank's own end-of-statement SUMMARY (Dr/Cr
counts, opening/closing balance); records `documents.status` = `reconciled`/`unreconciled`.
Verified complete on real statements (37 & 31 txns) without inspecting any value.
20     * **Axis Bank** (`axis.py`): Savings (`axis_transactions`) + Credit Card
(`axis_cc_transactions`). ?? The savings `extract_tables()` mapping is **unreliable on real Axis
layouts** ? column counts and date placement vary between statements (e.g. Jun vs Oct 2025), so
it can mislabel rows. Reconciliation catches this, so as a **stop-gap the savings parser refuses
to store rows that don't reconcile** (marks the document `unreconciled`, inserts 0) instead of
persisting garbage. **Pending real fix: an HDFC-style coordinate-clustering rewrite** validated
by total reconciliation (`opening` + `?Deposits` ? `?Withdrawals` == `closing`). Credit-card has no
```

```

running balance (parse/persist split + count only).
21     * **ICICI Bank** (`icici.py`): Transaction / savings statements (`icici_transactions`);
**reconciles** + records status.
22
23     All balance-bearing parsers share one reconciler (`muneem.parsers.validation`) and split
parsing from DB-insert, so the row logic is unit-testable without a PDF/DB and a parse is
validated before it's trusted. Parsers use `pdfplumber` (with optional password); HDFC parses by
word-position clustering, Axis/ICICI by `extract_tables()` (reconciliation will flag it if that
drops rows on a real statement). **Password resolution** (password_rules + hybrid unlock) is
**not yet wired** into the main `muneem ingest --doc-type bank_statement` path; it is exercised
only via the `@pytest.mark.local_statements` test in `tests/parsers/test_local_axis.py`. Unlock
hybrid logic exists in `unlock.py` (pypdf + candidates/prompt) and is tested.
24
25     See `password_rules.py` (now prefers user config dir) and `cli.py:ingest`.
26
27     ## 3. Database & CLI
28     * Schema v4 with `documents`, `offers`, and per-bank txn tables (`axis_`,
`icici_transactions`, `hdfc_transactions`). Dedup on sha256. No versioned migration scripts yet
(IF NOT EXISTS + user_version pragma).
29     * CLI: `ingest <pdf>` (promo or bank_statement via parser), `offers list`, `txns list
<bank>`.
30     * Config uses platformdirs for local dirs; secrets via keyring.
31
32     ## 4. Testing & Hygiene
33     * Strong offline regression on `testdata/promo-emails/` (T1/T3 in default CI).
34     * Security guards in `test_repo_hygiene.py` + license allowlist test.
35     * Local-only tests for real statements (gitignored data).
36
37     ## 5. Next Steps (Roadmap Alignment)
38     Focus remains on completing Milestone C (HDFC vertical slice + unlock wiring) then D
(harden, logging, evaluation) before generalize. Open items (DESIGN §8.3 LLM for sensitive, §8.5
merchant canonicalization) stay parked.
39
40     **Tech debt reduced in recent cleanup PR** (see PR for details): deduped helpers, DRY DB
inserts, improved password rule loading, ruff format enforcement, doc accuracy.
41
42     The project stays **local-first**, secrets in OS keychain, promo corpus safe for CI/LLM
experiments.
43

```

7. user (2026-05-31T08:15:08.949Z)

```

1     # muneem ? Project Plan & Roadmap
2
3     > Status: **active plan, pre-implementation.** This is the "how and in what order"
companion to [`DESIGN.md`](../DESIGN.md) (the "what and why"). DESIGN.md owns the architecture
and the threat model; this doc owns sequencing, milestones, and concrete tasks. Where they
disagree, DESIGN.md wins and this doc should be corrected.
4     >
5     > Created 2026-05-30.
6
7     ---
8
9     ## ?? Known issues ? must-fix
10
11     - **Axis savings parser needs a thorough rewrite.** The current `extract_tables()`-based
mapping is **wrong and unreliable on real Axis layouts**: the real columns are
12     `Txn Date | Transaction | Withdrawals | Deposits | Balance | Other Information` (not
the
13     assumed `Debit/Credit`), the column count and date placement **vary between
statements**
14     (Jun 2025 = 8 cols, per-row balance, date merged into the narration cell; Oct 2025 = 9
15     cols, clean date in col 0, no per-row balance), `can_handle` matches "Statement of
Account"
16     but the text says "Account Statement", and the account-number regex misses. **Stop-

```

```

gapped
18     in PR #19** ? `AxisParser` now refuses to persist a parse that doesn't reconcile
(stores 0
19     rows, marks the document `unreconciled`), so no mislabeled data is saved, but it
parses
20     nothing useful yet. **Real fix:** an HDFC-style **coordinate-clustering** rewrite,
validated
21     by **total reconciliation** (`opening + ?Deposits ? ?Withdrawals == closing`,
confirmed TRUE
22     on the Oct 2025 statement) plus the opening/closing bookend. Validate against the June
+
23     October 2025 statements and any future Axis files. Credit-card
(`AxisCreditCardParser`) is
24     separate and unverified on real data. **The rewrite should land on the new concept-
core
25     architecture ? see [parser-architecture.md](./parser-architecture.md) (it's step 3
of that
26     plan and the first real test of "generic-first").**
27
28     ---
29
30     ## 1. Decisions locked (2026-05-30)
31
32     These resolve open decisions from DESIGN.md § 8. They were made by the user explicitly;
this section is the start of the decision log.
33
34     | DESIGN.md ref | Decision | Choice |
35     |---|---|---|
36     | § 8.1 Language | Implementation language for v1 | **Python-only.** Accept the heavier
deployment story in exchange for the best PDF/OCR/LLM ecosystem and fastest path on the hard
parsing work. Revisit only if the ingestion daemon outgrows Python. |
37     | § 8.4 First slice | What to build first | **Both tracks in parallel** ? the promo-
corpus parse?store?query spine *and* the Gmail ingestion plumbing, developed as two decoupled
tracks that converge at the first bank slice. |
38     | § 8.2 Passwords | Password sourcing strategy | **Hybrid: candidate dictionary ? per-
sender rule template ? interactive prompt.** Cache the winning method per sender. Secrets never
touch the repo or logs. |
39     | (practical) | First real bank parser target | **HDFC** ? born-digital statements,
name+DDMM-style PDF password (per CLAUDE.md), and the issuer with the longest consistent
statement history to test against. |
40
41     **Still parked** (do not pick silently ? see § 9 below): § 8.3 LLM model for *sensitive*
categories, § 8.5 merchant canonicalization, and the timing of encryption-at-rest (SQLCipher).
42
43     ---
44
45     ## 2. Guiding principles (inherited from DESIGN.md § 5 + CLAUDE.md)
46
47     1. **Local by default.** All raw statement data and the DB stay on the machine. OCR/LLM
work runs locally for any sensitive category. No cloud sync of staging or DB, ever.
48     2. **No cloud LLM on sensitive contents** (bank, CC, broker, MF, insurance) without
explicit per-category approval in-session. The **promo corpus is the one non-sensitive
playground** where cloud LLMs are policy-permitted ? so it's where we validate any LLM-
extraction pattern *before* it could ever see a statement.
49     3. **Test on safe, committed data wherever possible.** `testdata/promo-emails/` is the
regression corpus and runs in CI. Statement parsing gets CI coverage via **synthetic/redacted
fixtures** in the issuer's layout ? never real statements.
50     4. **Secrets discipline.** OAuth tokens ? OS keychain (Windows Credential Manager).
Nothing sensitive in git; don't weaken `.gitignore`. Logs are redacted (no account numbers,
balances, line items, or statement text).
51     5. **Decisions parked for humans stay parked.** Don't write code that silently locks in
§ 9's open items.
52
53     ---
54

```

```
55     ## 3. Proposed Python stack
56
57     User-locked is Python-only. The libraries below are *recommendations* (well-known OSS,
per CLAUDE.md's "batteries-included stdlib + well-known OSS over novel dependencies") ? open to
override, not parked decisions. Anything that is a cloud **service** is flagged and needs a nod
before adoption.
58
59     | Concern | Proposed | Notes |
60     |---|---|---|
61     | Packaging / deps / venv | **uv** + `pyproject.toml` | Fast, modern. Poetry is an
equally fine fallback. |
62     | Lint + format | **ruff** | One tool, fast. |
63     | Tests | **pytest** | Regression suite runs offline in CI on the promo corpus +
synthetic fixtures. |
64     | PDF text + image extraction | **PyMuPDF (`fitz`)** | Carried from Email-Mining;
confirmed the right library. |
65     | Table recovery (born-digital statements) | **pdfplumber** | Coordinate/word-level
extraction for HDFC's transaction tables. |
66     | PDF decryption | **pikepdf** (libqpdf) | Robust standard-security-handler support.
`pypdf` is the fallback. |
67     | Image dedup | **Pillow + scikit-image SSIM** | Carried from Email-Mining (MD5 hash +
SSIM ? 0.95). |
68     | OCR (image PDFs, later) | **Tesseract** via `pytesseract` | Local. Windows needs the
Tesseract binary installed (setup task). |
69     | Gmail | **google-api-python-client + google-auth-oauthlib** | Read-only scope
`gmail.readonly`. |
70     | Secret storage | **keyring** | Uses Windows Credential Manager on win32. |
71     | DB | **sqlite3 (stdlib)** + lightweight versioned migrations | Encryption-at-rest
(SQLCipher via `sqlcipher3`) deferred to Milestone D per DESIGN.md. |
72     | CLI | **Typer** | Clean ergonomics; `argparse` (stdlib) is the zero-dependency
fallback. |
73     | LLM (promo only, non-sensitive) | deterministic-first; cloud LLM **only if needed and
approved** | See § 5 Track A. Sensitive-doc LLM is parked (§ 9 / DESIGN § 8.3). |
74
75     ---
76
77     ## 4. Build order at a glance
78
79     ...
80     Milestone 0 ? Decisions & scaffolding (no sensitive data)
81         ?
82         ?????????????????????????????????????????????????????????????????????
83         ?                                     ?
84         TRACK A: Promo spine                TRACK B: Gmail ingestion                (run in parallel,
85         parse ? store ? query                fetch ? stage ? catalog                fully decoupled)
86         (committed corpus, CI)                (real email, local only)
87         ?                                     ?
88         ?????????????????????????????????????????????
89         ?
90         Milestone C ? HDFC vertical slice (CONVERGENCE)
91         unlock (hybrid passwords) ? HDFC parser ? transactions ? CLI
92         ? completes DESIGN.md § 7
93         ?
94         ?
95         Milestone D ? Harden & evaluate (the pre-generalize checkpoint)
96         ?
97         ?
98         Generalize (more issuers, local-LLM fallback, holdings, items, analytics, UI)
99     ...
100
101     Tracks A and B are genuinely independent: A needs no OAuth and no secrets; B needs no
parsing. As a single developer you can interleave them in any order ? neither blocks the other.
**Both must land before Milestone C**, where a fetched-and-unlocked HDFC PDF flows through the
parser for the first time.
102
```

```

103     Effort tags below are rough (S / M / L), for relative sequencing only ? not calendar
estimates.
104
105     ---
106
107     ## 5. Milestones & tasks
108
109     ### Milestone 0 ? Decisions & scaffolding · no sensitive data
110
111     - [ ] **0.1 (S)** Project skeleton: `pyproject.toml` (uv), package dir (name TBD, e.g.
`muneem/`), `ruff` + `pytest` configured. *Do not over-materialize the source tree (DESIGN § 9)
? create only what 0.x needs.*
112     - [ ] **0.2 (S)** Offline CI: GitHub Actions (or local pre-commit) that installs deps
and runs `pytest` against committed data only. A trivial `test_smoke` proves the loop is green.
113     - [ ] **0.3 (S)** Config + secrets foundation: a config loader (env + a gitignored
`config.toml`), and a `keyring`-backed secret accessor stub. Confirm `.gitignore` covers
`config.toml` and any new secret paths.
114     - [ ] **0.4 (S)** `gitignore` audit: verify the proposed local-only dirs
(`testdata/statements/`, staging dir, `*.db`) are blocked. Add a
`testdata/statements/.gitkeep`-style note or README so the expected-but-absent dir is
documented.
115     - **Exit:** `uv run pytest` is green in CI on a clean checkout; repo layout agreed.
116
117     ### Track A ? Promo spine: parse ? store ? query · committed corpus, fully CI-testable
118
119     > Resurrects the Email-Mining deals feature as muneem's first working output. Zero
secrets, zero OAuth.
120
121     - [ ] **A.1 (M)** Port Email-Mining's extraction into a real module: PyMuPDF per-page
`get_text` + `get_images`, size filter (?300×300), MD5 + SSIM dedup. Replace the edit-line-55
hardcoded path with proper arguments/config. Replace one-shot stdout with a return value.
122     - [ ] **A.2 (S)** Minimal schema slice + migration v1: `documents` (id, source,
external_id, sender, subject, received_date, doc_type, sha256, path, status, created_at) and
`offers` (id, document_id, merchant, promo_code, description, expiry_date, redeemed,
source_text). Full schema is future work (`docs/schema.md`).
123     - [ ] **A.3 (M)** Promo extraction, **deterministic-first**: regex/heuristics for promo
codes + expiry dates over the unified text. Where deterministic extraction misses, an **LLM
fallback** is permitted *because promo is non-sensitive* ? but that pulls in a cloud service, so
it's gated on a quick okay (reuse Email-Mining's Gemini path, go local via Ollama, or use
Anthropic ? your call when we get there).
124     - [ ] **A.4 (M)** `muneem ingest <pdf>` writes a `documents` + `offers` row; `muneem
offers list` reads them back. **Ground-truth + regression test:** establish the known promo
codes for each of the 10 corpus PDFs, then a `pytest` that runs the pipeline over
`testdata/promo-emails/` and asserts recovery. (Building the ground-truth answer key is itself
part of this task.)
125     - **Exit:** `ingest ? SQLite ? offers list` works end-to-end; green regression test on
the committed corpus. Stages 3?4?5 proven with zero risk.
126
127     ### Track B ? Gmail ingestion: fetch ? stage ? catalog · real email, local only
128
129     > First contact with real (sensitive) email. Everything it writes is gitignored.
130
131     - [ ] **B.1 (M)** Gmail OAuth2: register a Google Cloud project, `gmail.readonly` scope,
store the token in `keyring`. Document the one-time consent flow.
132     - [ ] **B.2 (M)** Financial-document detection v1: a **sender allowlist** + server-side
query (`has:attachment filename:pdf from:? newer_than:?`). Start narrow and precise; broaden
later. Pull matching messages + PDF attachments into a gitignored staging dir.
133     - [ ] **B.3 (S)** Incremental sync: persist Gmail `historyId`; on re-run, fetch only
what's new. Idempotent (re-running never duplicates `documents`, keyed by message id +
attachment hash).
134     - [ ] **B.4 (S)** Catalog fetched attachments as `documents` rows (status
`locked`/`unparsed`), with sha256 + staging path. `muneem fetch --since 90d` is the entry point.
135     - **Exit:** `muneem fetch` populates the staging dir + `documents` rows from a narrow
real-Gmail query. (Manual/local verification ? not CI; it touches a real account.)
136

```

```

137     ### Milestone C ? HDFC vertical slice · CONVERGENCE · completes DESIGN.md § 7
138
139     > A fetched HDFC statement gets unlocked and parsed into real transactions. This is the
moment every stage runs once on real data the user cares about.
140
141     - [ ] **C.1 (M)** Unlocking (Stage 2) with the **hybrid** strategy: candidate dictionary
derived from user attributes (e.g. `first4(name).upper() + DDMM(dob)`) ? per-sender rule
template ? interactive prompt. Cache the winning method per sender in the keychain. **No
password material in repo or logs.** Decrypt with pikepdf; advance `documents.status` to
`unlocked`.
142     - [ ] **C.2 (L)** HDFC parser (Stage 3, the hard 80%): born-digital text-layer +
pdfplumber table recovery ? normalized transactions (txn_date, value_date, amount, direction,
narration, running balance). **Synthetic HDFC-layout fixtures** committed for CI; real
statements stay gitignored locally.
143     - [ ] **C.3 (M)** Schema migration v2: `accounts` (issuer, account_type, last4, holder)
+ `transactions` (account_id, document_id, txn fields, category, merchant_id-nullable). Populate
from the parser. `muneem txns list --last-month` prints real parsed transactions.
144     - **Exit:** **DESIGN.md § 7 vertical slice complete** ? Gmail ? HDFC statement ?
unlocked ? parsed ? SQLite ? CLI lists last month's transactions, on real data, locally.
145
146     ### Milestone D ? Harden & evaluate · the pre-generalize checkpoint
147
148     - [ ] **D.1 (M)** Redacted structured logging (assert no statement text / account
numbers leak); idempotent, resumable re-runs across all stages.
149     - [ ] **D.2 (S)** Encryption-at-rest decision (DESIGN § 5.4): adopt SQLCipher now or
schedule it. (Parked item ? bring evidence, then decide.)
150     - [ ] **D.3 (M)** **Evaluation writeup:** how well did the deterministic HDFC parser
hold across statement vintages? Where does it break? Which categories will need the local-LLM
fallback? This is the evidence DESIGN § 7 asks for *before* generalizing. Capture in
`docs/parser-notes-hdfc.md`.
151     - **Exit:** the slice is robust and re-runnable; a written go/no-go on generalization
with real evidence.
152
153     ---
154
155     ## 6. Generalize (post-checkpoint backlog ? scope after Milestone D)
156
157     Ordered roughly by leverage, not committed:
158
159     - Second/third bank parsers + the first **credit-card bill** parser (reuses unlock +
parser scaffolding).
160     - **Local-LLM long-tail fallback** (Ollama) for issuers that don't justify a bespoke
parser ? **local, because these are sensitive** (resolves DESIGN § 8.3).
161     - Broker / MF statements ? `holdings` (point-in-time positions; the investment-timeline
use-case).
162     - Itemized receipts (Swiggy/Amazon/BigBasket/etc.) ? `items` (the price-intelligence +
catalog use-cases).
163     - **Merchant canonicalization** ? `merchants` (DESIGN § 8.5 ? defer until real data
exposes the actual mess, then decide rules-file vs. library vs. learned).
164     - Analytics surfaces for the DESIGN § 2 use-cases (CLI views first; optional small
**local** web UI later).
165
166     ---
167
168     ## 7. Cross-cutting workstreams (run throughout, not a phase)
169
170     - **Security & secrets hygiene** ? keychain for tokens + cached passwords; redacted
logs; gitignore audits each milestone; encryption-at-rest tracked to D.2.
171     - **Schema migrations from day one** ? versioned migration scripts; the schema *will*
churn as new issuers land. Keep `docs/schema.md` current.
172     - **Test-corpus curation** ? ground-truth answer key for the promo corpus (A.4);
synthetic/redacted statement fixtures for CI (C.2). Never commit real statements.
173     - **Decision log** ? append resolutions to § 1 here and mark the corresponding DESIGN.md
§ 8 item resolved.
174

```

```

175     ---
176
177     ## 8. Risks & mitigations
178
179     | Risk | Mitigation |
180     |---|---|
181     | Sensitive data leaks into git, logs, or a cloud LLM | Local-by-default; promo-only LLM
playground; redacted logging (D.1); gitignore audits; the § 2 principles are non-negotiable. |
182     | HDFC parser is brittle across statement vintages | Synthetic fixtures + the D.3
evaluation gate *before* generalizing; deterministic-first with a planned local-LLM fallback for
the long tail. |
183     | Password sourcing fails noisily on backfill | Hybrid strategy degrades to interactive
prompt; cache per sender so it's a one-time cost. |
184     | Gmail OAuth / quota friction | Read-only scope, server-side queries (cheap),
incremental `historyId` sync; narrow allowlist first. |
185     | Scope creep into the full § 2 vision before the slice works | Hard checkpoint at
Milestone D; generalization is explicitly post-checkpoint. |
186     | Two parallel tracks diverge / integration surprises at C | Both write through the
*same* `documents` table and schema-migration discipline from A.2/B.4, so convergence is a join,
not a rewrite. |
187
188     ---
189
190     ## 9. Still-open decisions (do NOT pick silently)
191
192     Carried from DESIGN.md § 8, not yet resolved ? surface and ask when a milestone forces
them:
193
194     - **§ 8.3 ? LLM model for sensitive categories.** Coupled to your hardware (GPU? RAM?)
and latency tolerance. Forced at the start of the generalize phase (local-LLM fallback). Ollama
+ Llama/Qwen/Mistral is the likely shape, but unconfirmed.
195     - **§ 8.5 ? Merchant canonicalization.** Deferred until real receipt data exposes the
mess. Forced when `items`/`merchants` land.
196     - **Encryption-at-rest timing** (D.2) ? SQLCipher now vs. later.
197     - **Promo-track LLM** (A.3) ? only reached if deterministic extraction underperforms;
cloud is policy-permitted for promo but still needs a quick okay on which vendor.
198
199     ---
200
201     ## 10. Definition of done for v1
202
203     The DESIGN.md § 7 slice, hardened: **`muneem fetch` pulls HDFC statements from Gmail ?
unlocks them with the hybrid password strategy ? the HDFC parser writes normalized
`transactions` to SQLite ? `muneem txns list` shows last month's activity ? all locally, with
redacted logs, idempotent re-runs, green CI on synthetic fixtures, and a written evaluation of
where to go next.** The promo "forgotten deals" feature ships alongside as the Track A
deliverable.
204

```

8. user (2026-05-31T08:15:09.247Z)

```

1     # muneem ? Parser Architecture (concept-core, verification-first)
2
3     > **Status: DRAFT for discussion.** Direction chosen 2026-06: "concept core + profiles-
as-data."
4     > The deterministic core below (L0?L2, schema, profiles, verifier, traps) is settled
enough to
5     > build against. The **L3 local-vision-model fallback** and the **numeric-hallucination
traps** are
6     > marked _pending_ the local-VLM deep-research run, which will populate them.
7     > Companion to [`DESIGN.md`](./DESIGN.md) (what/why) and [`ROADMAP.md`](./ROADMAP.md)
(sequencing).
8
9     ---
10

```



```

11     ## 1. Why we're rethinking this
12
13     The current parsers are brittle for two reasons:
14
15     1. They encode *syntax*, not *semantics*. HDFC hard-codes x-positions; Axis hard-
codes column
16         indices. They know where a value sits, not what it is ? so any format nudge maps
the wrong
17         column and produces wrong data.
18     2. They parse open-loop. They emit whatever they extracted with no check, so a wrong
parse looks
19         like a right one. (The Axis parser confidently produced 15 mislabeled rows ? see
ROADMAP Known
20         Issues.)
21
22     Goals for the new architecture: format-resilient (small layout/label changes don't
break us),
23     extensible (a new issuer or a changed format is mostly data, not code), semantic
(we know
24     the critical placeholders ? balance, spend, deposit, ?), and failsafe (drift is
loud, never
25     silent).
26
27     ## 2. The key idea ? reconciliation makes parsing a closed-loop search
28
29     We do not need to perfectly parse every format. We need to propose candidate
interpretations
30     and let arithmetic pick the correct one. The identity
31
32     ```
33     opening_balance + ?(deposits) ? ?(withdrawals) == closing_balance      (+ the per-row
balance walk)
34     ```
35
36     is a truth oracle that is independent of layout. That reframes everything:
37
38     - We can identify concepts aggressively / fuzzily, because a wrong guess is
rejected, not stored.
39     - It is what makes a probabilistic extractor (a vision model) safe on money ? we
never trust the
40     model, we trust the math that checks it.
41     - A changed format is not "rewrite the parser" ? it's "the search finds a mapping that
reconciles."
42
43     Principle: extraction proposes, reconciliation disposes.
44
45     ## 3. The canonical concept schema (the contract)
46
47     One bank-agnostic vocabulary that every extractor targets. These are the "critical
placeholders."
48
49     - Row concepts: `txn_date`, `value_date`, `narration`, `reference`, `debit`
(spend/withdrawal),
50     `credit` (deposit), `balance`; or `{amount, direction}` for credit cards (no running
balance).
51     - Statement concepts: `opening_balance`, `closing_balance`, `debit_count`,
`credit_count`,
52     `account_number`, `period_start/end`, `currency`.
53
54     Parsing = map this bank's columns/fields onto these concepts, then verify.
Provenance + a
55     confidence verdict travel with every parse (see §7).
56
57     ## 4. The extraction ladder (cheapest first; every layer gated by the verifier)
58

```

```

59 | Layer | How it identifies a concept (e.g. `balance`) | Resilience it buys |
60 |---|---|---|
61 | L0 Header lexicon | alias + fuzzy match: `balance` ? {Balance, Closing Bal, Running
Bal}`, `debit` ? {Withdrawal, Debit, Dr, Spend, Paid Out}`, ? | renamed/abbreviated labels |
62 | L1 Structure & content | dates by pattern; amounts are right-aligned numerics;
`balance` = the numeric column whose running-balance recurrence holds | missing/merged
headers (the Axis-June case) |
63 | L2 Mapping search | enumerate plausible concept?column assignments; keep the
one(s) that reconcile | column reorder / count drift between statements |
64 | L3 Local vision fallback _(pending research)_ | page image ? JSON in the concept
schema; accepted only if it reconciles | scans, no text layer, novel layouts |
65
66 The self-identifying balance column (L1) is the heart of it: we find "balance" not
by its label
67 but by which column makes the math work ? fully format-independent. L0/L1 usually
agree; L2 is the
68 fallback search when they don't; L3 is the last resort, and the research decides which
local VLM
69 (GPU, Python, commercial-safe weights) and what number-hallucination guards it needs.
70
71 ## 5. The verifier (reconciliation-as-oracle)
72
73 Already exists in seed form as `muneem.parsers.validation`. It grows into the component
that both
74 selects and validates:
75
76 - Per-row walk: `balance[i] == balance[i-1] ? debit[i] + credit[i]` (catches a
missed/misread row
77 between any two balances).
78 - Total reconcile: `opening + ?credits ? ?debits == closing` (works even when
there's no per-row
79 balance, e.g. Axis savings).
80 - Bookend: parsed Dr/Cr counts + opening/closing vs the bank's printed SUMMARY
(catches a missing
81 first/last row ? HDFC has this; others where available).
82 - Selection: among L0/L1/L2 candidate mappings, pick the reconciling one; if several
reconcile,
83 prefer the highest-confidence (header-backed) one.
84 - Confidence verdict: `reconciled` (per-row or total) + `bookended` ? trust; else
quarantine.
85
86 ## 6. Bank knowledge as *data* (profiles), not code
87
88 A bank profile is a small declarative object (likely YAML/dict): concept?label
aliases, accepted
89 date formats, anchor strings to locate the table, account-number patterns, known quirks
(multi-account
90 sections, "Other Information" column), and the password rule. Generic detectors run
first; the
91 profile supplies priors/hints to disambiguate ? it is not bespoke parsing logic.
Absorbing a small
92 format change = tweak a profile (often nothing ? the generic path copes). A brand-new
issuer = add a
93 profile, and only drop to custom code if the generic ladder genuinely can't cope.
94
95 ## 7. Failsafes ? the "traps" (loud drift, never silent)
96
97 The Axis lesson: a broken parse must announce itself. Built-in tripwires, each ? flag
+ provenance,
98 never store-and-pretend:
99
100 - header label matches no lexicon entry ? "unknown field, mapped by structure ? review"
101 - reconciliation breaks, or parsed counts ? printed summary ? quarantine (store
nothing or store
102 flagged; today's stop-gap already refuses to persist unreconciled rows)

```

103 - a profile's expected anchor is missing ? "format may have changed for <bank>"

104 - _(L3)_ VLM output won't reconcile, or disagrees with a deterministic parse ? reject the VLM

105 - **provenance + confidence on every parsed statement:** which layer/mapping produced it, did it

106 reconcile / bookend. Trust is *recorded*, not assumed.

107

108 _Pending research:_ the deep-research run will enumerate VLM failure modes on financial numbers

109 (digit transposition, dropped/duplicated rows) ? these become concrete L3 traps.

110

111 ## 8. Two tiers of "semantic"

112

113 - **Tier 1 ? field semantics** (this doc): what each column *means*. The resilience win. Build first.

114 - **Tier 2 ? transaction semantics** (later): classify each row (UPI / NEFT / ATM / charge / interest

115 / sweep), extract merchant/counterparty, assign category. Same schema, layered on top; ties to the

116 parked merchant-canonicalization decision (DESIGN § 8.5).

117

118 ## 9. Data-model evolution

119

120 Today: per-bank raw tables (`hdfc\_transactions`, `axis\_transactions`, ?) ? fine for fidelity, but the

121 concept schema points at a **unified `transactions` table** keyed on the canonical concepts, plus

122 `account`/`document` context and **`provenance` + `confidence`** columns. Migration is additive (the

123 per-bank tables can become views or be backfilled); sequence it after the core lands so we don't churn

124 storage twice.

125

126 ## 10. Implementation plan (incremental, merge-as-we-go)

127

128 1. **Core extraction.** Promote `validation.py` into a `verifier` (per-row + total + bookend +

129 candidate selection + confidence). Define the concept schema as dataclasses.

130 **Refactor HDFC onto

131 it as the reference implementation** (no behavior change ? it already reconciles + bookends).

132 2. **Concept-mapping engine (L0/L1/L2)** as generic, bank-agnostic code; express HDFC + ICICI as

133 profiles (data) on top of it.

134 3. **Axis clustering rewrite onto the core** ? this is the must-fix *and* the first real test of

135 "generic-first": coordinate clustering ? concept mapping ? total reconcile. Validates on Jun + Oct

136 2025 + future Axis files.

137 4. **L3 local-vision fallback** (after the research): a verified extractor that only fires when the

138 deterministic ladder fails to reconcile, with the number-traps from §7.

139 5. **Unify storage** (§9) + begin **Tier-2 transaction semantics**.

140 Each step is a green, reversible PR; the verifier guarantees we never regress into storing garbage.

141

142 ## 11. Open questions (for discussion)

143

144 - **Storage unification timing** ? keep per-bank tables until the concept core is proven, then migrate?

145 - **Generic-vs-profile balance** ? how much to lean on profiles as priors vs. pure content inference.

146 - **L3 scope** ? once the research lands: which local VLM, and is it a rare fallback or a routine

147 cross-check on every statement?
148 - ****Tier-2**** ? when do transaction classification + merchant canonicalization become in-
scope?
149 - ****Confidence policy**** ? what does the CLI/ledger do with a `quarantined` statement
(skip, show with
150 a warning, queue for the VLM)?
151

9. assistant (2026-05-31T08:15:28.134Z)

Let me ground this against the actual code and GitHub state rather than just the docs.

10. user (2026-05-31T08:15:31.482Z)

---OPEN PRs---
---OPEN ISSUES---
---BRANCHES (local)---
* main 6f6141f [origin/main] docs: parser architecture proposal (concept-core, verification-
first) [DRAFT] (#21)

11. user (2026-05-31T08:15:31.643Z)

No files found

12. user (2026-05-31T08:15:31.659Z)

tests\test_cli.py
tests\test_smoke.py
tests\test_db.py
tests\test_extract.py
tests\test_promo_regression.py
tests\test_ocr.py
tests\test_unlock.py
tests\conftest.py
tests\ingestion\test_gmail.py
tests\ingestion\test_providers.py
tests\test_dependency_licenses.py
tests\test_offers.py
tests\test_precommit_hook.py
tests\test_repo_hygiene.py
tests\parsers\test_registry.py
tests\parsers\test_hdfc.py
tests\parsers\test_validation.py
tests\parsers\test_icici.py
tests\parsers\test_axis.py
tests\parsers\test_local_axis.py

13. assistant (2026-05-31T08:15:35.869Z)

The `muneem/` glob came up empty ? package is likely under a `src/` layout. Let me locate it.

14. user (2026-05-31T08:15:36.420Z)

src\muneem\parsers\base.py
src\muneem\parsers__init__.py
tests\parsers\test_registry.py
tests\parsers\test_hdfc.py
src\muneem\parsers\hdfc.py
tests\parsers\test_validation.py
src\muneem\parsers\validation.py
src\muneem\parsers\icici.py
tests\parsers\test_icici.py
src\muneem\parsers\axis.py
tests\parsers\test_axis.py

tests\parsers\test_local_axis.py

15. user (2026-05-31T08:15:37.404Z)

```
src/muneem/parsers/__init__.py
src/muneem/parsers/axis.py
src/muneem/parsers/base.py
src/muneem/parsers/hdfc.py
src/muneem/parsers/icici.py
src/muneem/parsers/validation.py
tests/parsers/test_axis.py
tests/parsers/test_hdfc.py
tests/parsers/test_icici.py
tests/parsers/test_local_axis.py
tests/parsers/test_registry.py
tests/parsers/test_validation.py
```

16. assistant (2026-05-31T08:15:43.770Z)

Let me confirm the baseline test suite is green so my recommendation stands on solid ground.

17. user (2026-05-31T08:15:47.341Z)

```
2026-05-31
--- running offline test suite (excluding local_statements) ---
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_unlock.py:4: in <module>
    import pypdf
E   ModuleNotFoundError: No module named 'pypdf'
===== short test summary info =====
ERROR tests/ingestion/test_gmail.py
ERROR tests/ingestion/test_providers.py
ERROR tests/parsers/test_axis.py
ERROR tests/parsers/test_hdfc.py
ERROR tests/parsers/test_icici.py
ERROR tests/parsers/test_local_axis.py
ERROR tests/parsers/test_validation.py
ERROR tests/test_cli.py
ERROR tests/test_db.py
ERROR tests/test_extract.py
ERROR tests/test_ocr.py
ERROR tests/test_offers.py
ERROR tests/test_promo_regression.py
ERROR tests/test_smoke.py
ERROR tests/test_unlock.py
!!!!!!!!!!!!!!!!!!!! Interrupted: 15 errors during collection !!!!!!!!!!!!!!!!!!!!!!!
15 errors in 0.62s
```

18. assistant (2026-05-31T08:15:51.700Z)

I invoked the wrong interpreter ? the project has a `.venv`. Let me use it.

19. user (2026-05-31T08:15:59.939Z)

```
.....X..... [ 75%]
..... [100%]
===== XPASSES =====
===== short test summary info =====
XPASS tests/test_ocr.py::test_swiggy_image_code_recovered_fuzzily - recovery of the stylized
boxed code is OCR-engine/version dependent ? succeeds on Tesseract 5.4, returns nothing on the
apt build; the planned fix is a purpose-trained model (docs/DEPENDENCIES.md). strict=False so it
```

never fails the build.
94 passed, 9 deselected, 1 xpassed in 6.77s

20. assistant (2026-05-31T08:16:54.074Z)

Resynced and refreshed. Here's where **muneem** actually stands, grounded in the code and remote ? not just the docs.

Repo state

- **On `main`**, clean, synced with `origin/main`. All feature branches merged and pruned; **no** open PRs or issues.
- **Offline test suite is green**: 94 passed, 9 deselected (the `local\_statements` real-PDF tests), 1 expected xpass. Solid baseline.
- The last three commits are all **docs**, not code ? the work has reached a *decision point*, which is why you're being asked to weigh in.

What works vs. what's stubbed

Area	State
HDFC parser	? Works on real statements. Coordinate-clustering, reconciles + bookends against the bank's SUMMARY, records `documents.status`. The reference-quality parser.
ICICI parser	? Reconciles + records status (via `extract_tables()`).
Axis savings	?? Broken , stop-gapped. `extract_tables()` mislabels columns across layouts; since #19 it <i>refuses to store unreconciled rows</i> (saves 0, marks `unreconciled`) ? so no garbage, but it parses nothing useful. This is the flagged must-fix .
Axis credit card	?? Parse/persist split + count only; unverified on real data.
Shared verifier	? `validation.py` ? the running-balance reconciler all balance-bearing parsers share (the "seed" the new architecture grows from).
Gmail ingestion	? Full OAuth2 read-only + search + attachment download. IMAP/Outlook partial/experimental.
Passwords	?? Hybrid unlock logic exists + tested, but not wired into the main `ingest --doc-type bank_statement` path ? only exercised via a local test.
`muneem fetch` CLI	? Doesn't exist yet (Roadmap B.4). Gmail plumbing is there; no high-level command.

The fork in the road

The genuine blocker is a **decision**, not code: [`docs/parser-architecture.md`](docs/parser-architecture.md) (#21) is an explicit **DRAFT** for discussion ? the "concept-core, verification-first" rework (extraction proposes, reconciliation disposes). The ROADMAP says the **Axis must-fix** should land *on* that new architecture (it's step 3 of its plan and "the first real test of generic-first"). So I shouldn't start the must-fix until the architecture is signed off ? and per your standing rules, I won't silently pick it for you.

Its §11 has four open questions (storage-unification timing, generic-vs-profile balance, L3 scope, confidence/quarantine policy), and the **L3 local-vision** fallback is parked behind a "local-VLM deep-research run" that hasn't happened yet.

My recommendation: **lock the architecture** (L0?L2 + verifier + profiles are settled enough to build) and start with its zero-risk Step 1 ? promote **`validation.py`** into a real verifier, define the concept schema as dataclasses, and refactor HDFC onto it with *no behavior change*. That's a green, reversible PR that lays the spine before touching the broken Axis parser. The local-VLM research can run in parallel since it's non-sensitive and only gates L3 (step 4).

Let me get your direction on what to pick up:

21. user (2026-05-31T08:17:20.652Z)

Your questions have been answered: "The parser track is gated on the architecture DRAFT (#21). What should I pick up next?"="Build architecture Step 1". You can now continue with these answers in mind.

22. assistant (2026-05-31T08:17:51.043Z)

Locked in ? ****architecture Step 1****. Per your PR cadence I'll do this on a branch as a green, reversible PR with no behavior change. Before writing anything, I need to read the current code thoroughly: the seed verifier, the HDFC reference parser, the shared base/registry, and the tests that define the no-regression contract.

23. user (2026-05-31T08:17:52.009Z)

```
1      """Shared transaction-validation helpers for bank-statement parsers.
2
3      Reconciliation proves a parse is **complete and correct without inspecting any
4      sensitive value**: if a transaction is missed or misread, the running balance
5      stops adding up. This is the generic core used by every balance-bearing parser
6      (HDFC, Axis savings, ICICI). Reports carry counts/booleans only ? never an amount.
7      """
8
9      from __future__ import annotations
10
11     import re
12     from collections.abc import Sequence
13     from dataclasses import dataclass
14
15     _CR_DR_SUFFIX = re.compile(r"(?i)\s*(cr|dr)\s*$")
16
17
18     def parse_amount(value: object) -> float | None:
19         """Parse a statement money cell to a float.
20
21         Returns ``None`` only for a genuinely absent cell (None / empty / ``"-`` /
22         non-numeric); ``"0.00"`` parses to ``0.0``. This deliberately distinguishes
23         *absent* from *zero* ? unlike ``base.safe_amount``, which maps ``"0.00"`` to
24         ``None`` and would corrupt a real zero balance and the reconciliation math.
25         """
26         if value is None:
27             return None
28         text = _CR_DR_SUFFIX.sub("", str(value).strip()).replace(",", "").strip()
29         if text in ("", "-"):
30             return None
31         try:
32             return float(text)
33         except ValueError:
34             return None
35
36
37     @dataclass
38     class ReconcileReport:
39         """Result of a running-balance walk. Counts only, never an amount."""
40
41         n: int # rows considered
42         compared: int # consecutive (balance-bearing) pairs actually checked
43         breaks: int
44         first_break: int | None = None
45
46         @property
47         def ok(self) -> bool:
48             return self.compared > 0 and self.breaks == 0
49
50
51     def reconcile_running_balance(
52         entries: Sequence[tuple[float | None, float | None, float | None]],
53         tol: float = 0.01,
54     ) -> ReconcileReport:
55         """Verify ``balance[i] == balance[i-1] - debit[i] + credit[i]`` across rows.
56
57         ``entries`` is an ordered list of ``(debit, credit, balance)``; ``debit`` and
58         ``credit`` may be ``None`` (treated as 0.0). A row whose ``balance`` is ``None``
```

```

59         is skipped (can't anchor on it). Because the walk spans the whole statement, a
60         clean result also rules out any row missed *between* two balances ? which is the
61         failure mode silent ``extract_tables()`` row-drops would otherwise hide.
62         """
63         breaks = 0
64         first_break: int | None = None
65         compared = 0
66         prev: float | None = None
67         for i, (debit, credit, balance) in enumerate(entries):
68             if balance is None:
69                 continue
70             if prev is not None:
71                 expected = prev - (debit or 0.0) + (credit or 0.0)
72                 compared += 1
73                 if abs(expected - balance) > tol:
74                     breaks += 1
75                     if first_break is None:
76                         first_break = i
77             prev = balance
78         return ReconcileReport(
79             n=len(entries), compared=compared, breaks=breaks, first_break=first_break
80         )
81
82
83     __all__ = ["ReconcileReport", "reconcile_running_balance", "parse_amount"]
84

```

24. user (2026-05-31T08:17:52.470Z)

```

1     """HDFC savings / transaction-account statement parser.
2
3     **Approach:** coordinate / column clustering on **pdfplumber (MIT)** ? not the
4     default ``extract_tables()`` and not a ready-made library. HDFC's transaction
5     table is *semi-ruled*, where ``extract_tables()`` is unreliable; the deep-research
6     decision (see the ``hdfc-parser-tech-decision`` memory) is to cluster words by
7     position. pdfplumber's whole chain is permissive and it opens password-protected
8     statements directly.
9
10    **Layout.** Each transaction is one *primary* line ? the transaction date at the
11    far left (``x0?52``) plus three right-aligned amount columns (Withdrawal, Deposit,
12    Closing Balance) ? followed by zero or more *continuation* lines that wrap the
13    narration and carry the value date (``dd/mm/yyyy`` at ``x0>110``) and the
14    cheque/reference number. We group words into visual lines by ``top``, detect
15    primary lines by a leading date + a closing-balance amount, attach continuations,
16    and read columns by x-position.
17
18    **Correctness is checked, not assumed.** ``reconcile`` verifies the running
19    balance (closing[i] == closing[i-1] - withdrawal + deposit) and
20    ``validate_against_summary`` cross-checks the parse against HDFC's own end-of-
21    statement SUMMARY (Dr/Cr counts + opening/closing balance). Together they prove a
22    parse complete without inspecting any individual value ? which is how we validate
23    against real (sensitive) statements. No statement content is logged here.
24    """
25
26    from __future__ import annotations
27
28    import csv
29    import re
30    from dataclasses import dataclass, field
31    from pathlib import Path
32
33    import pdfplumber
34
35    from muneem import db as dbmod
36

```



```

37 from . import register_parser
38 from .base import StatementParser
39 from .validation import ReconcileReport, reconcile_running_balance
40
41 # --- Column geometry (PDF points; A4-portrait HDFC detailed statement) -----
42 # Left-aligned columns key on x0; right-aligned amount columns key on their right
43 # edge (x1), which is stable regardless of the number's magnitude.
44 _DATE_MAX_X0 = 70.0 # transaction date sits at x0?52; the value date is at x0>110
45 _NARR_MIN_X0 = 100.0 # narration band starts at x0?111
46 _AMOUNT_MIN_X0 = 295.0 # amount columns start at x0?306; narration never crosses ?290
47 _WITHDRAWAL_MAX_X1 = 400.0 # withdrawal right edge ?358
48 _DEPOSIT_MAX_X1 = 490.0 # deposit right edge ?443; closing balance right edge ?561
49 _LINE_TOL = 2.5 # words within this many points of each other share a visual line
50 _CONT_MAX_SPAN = 70.0 # a continuation line is at most this far below its primary line
51
52 _DATE_RE = re.compile(r"^d{2}/d{2}/d{4}$")
53 # Indian-format money: optional minus, comma groups, exactly two decimals, optional
Cr/Dr.
54 _AMOUNT_RE = re.compile(r"^-[\\d,]+\\.\\d{2}(?:cr|dr)?$", re.IGNORECASE)
55 _FOOTER_MARKERS = {"STATEMENT", "SUMMARY"}
56 _ACCOUNT_RE = re.compile(r"Account\\s*Number\\s*[:.]?\\s*(\\d{6,})", re.IGNORECASE)
57
58
59 @dataclass
60 class Transaction:
61     date: str # dd/mm/yyyy, as printed
62     narration: str
63     withdrawal: float
64     deposit: float
65     closing_balance: float
66     value_date: str | None = None
67     ref: str | None = None
68     page: int = 0
69
70
71 @dataclass
72 class Statement:
73     path: Path
74     transactions: list[Transaction] = field(default_factory=list)
75     account_number: str | None = None
76
77
78 @dataclass
79 class StatementSummary:
80     """The figures HDFC prints in its own end-of-statement SUMMARY block."""
81
82     opening_balance: float | None = None
83     closing_balance: float | None = None
84     debit_count: int | None = None
85     credit_count: int | None = None
86     debit_amount: float | None = None
87     credit_amount: float | None = None
88
89
90 @dataclass
91 class ValidationReport:
92     """Parse cross-checked against the bank's own SUMMARY. Booleans + counts only.
93
94     ``None`` means a figure wasn't available to check (not a failure). ``ok``
95     requires the balance walk plus every available check to pass.
96     """
97
98     n: int
99     reconciles: bool
100     debit_count_ok: bool | None = None

```

```

101         credit_count_ok: bool | None = None
102         opening_ok: bool | None = None
103         closing_ok: bool | None = None
104
105         @property
106         def ok(self) -> bool:
107             checks = [self.debit_count_ok, self.credit_count_ok, self.opening_ok,
self.closing_ok]
108             return self.n > 0 and self.reconciles and all(c is True for c in checks)
109
110
111     def _to_amount(token: str) -> float | None:
112         """Parse an HDFC money token to a float, or None if it is not money.
113
114         Unlike base.safe_amount we keep ``0.00`` as ``0.0`` (not None): a zero amount
115         is real information and the reconciliation math needs it.
116         """
117         if not _AMOUNT_RE.match(token):
118             return None
119         cleaned = re.sub(r"(?i)(cr|dr)$", "", token).replace(",", "")
120         try:
121             return float(cleaned)
122         except ValueError:
123             return None
124
125
126     def _group_lines(words: list[dict]) -> list[list[dict]]:
127         """Group words into visual lines by ``top`` (tolerant of sub-pixel jitter)."""
128         lines: list[list[dict]] = []
129         anchor_top: float | None = None
130         for w in sorted(words, key=lambda w: (w["top"], w["x0"])):
131             if lines and anchor_top is not None and abs(w["top"] - anchor_top) <= _LINE_TOL:
132                 lines[-1].append(w)
133             else:
134                 lines.append([w])
135                 anchor_top = w["top"]
136         return [sorted(line, key=lambda w: w["x0"]) for line in lines]
137
138
139     def _line_top(line: list[dict]) -> float:
140         return min(w["top"] for w in line)
141
142
143     def _has_narration(line: list[dict]) -> bool:
144         return any(_NARR_MIN_X0 <= w["x0"] < _AMOUNT_MIN_X0 for w in line)
145
146
147     def _has_amount(line: list[dict]) -> bool:
148         return any(w["x0"] >= _AMOUNT_MIN_X0 and _AMOUNT_RE.match(w["text"]) for w in line)
149
150
151     def _has_leading_date(line: list[dict]) -> bool:
152         first = line[0]
153         return bool(_DATE_RE.match(first["text"]) and first["x0"] < _DATE_MAX_X0)
154
155
156     def _has_footer_marker(line: list[dict]) -> bool:
157         return any(w["text"].upper() in _FOOTER_MARKERS for w in line)
158
159
160     def _is_primary(line: list[dict]) -> bool:
161         """A transaction's first line: a date at the far left + a closing-balance amount.
162
163         Requiring the closing-balance amount guards against stray dates in the page
164         header / account-summary box (e.g. the statement period).

```

```

165     """
166     if not _has_leading_date(line):
167         return False
168     return any(
169         w["x0"] >= _AMOUNT_MIN_X0 and w["x1"] >= _DEPOSIT_MAX_X1 and
170         _AMOUNT_RE.match(w["text"])
171         for w in line
172     )
173
174     def _is_continuation(line: list[dict]) -> bool:
175         """A narration-wrap line: narration-band text, no leading date, no amounts."""
176         return _has_narration(line) and not _has_leading_date(line) and not
177         _has_amount(line)
178
179     def _parse_transaction(
180         primary: list[dict], continuations: list[list[dict]], page_no: int
181     ) -> Transaction:
182         withdrawal = deposit = 0.0
183         closing: float | None = None
184         narr: list[tuple[float, float, str]] = [] # (top, x0, text) for ordered reassembly
185
186         for line in (primary, *continuations):
187             for w in line:
188                 x0, x1, text = w["x0"], w["x1"], w["text"]
189                 if x0 >= _AMOUNT_MIN_X0:
190                     val = _to_amount(text)
191                     if val is None:
192                         continue
193                     if x1 < _WITHDRAWAL_MAX_X1:
194                         withdrawal = val
195                     elif x1 < _DEPOSIT_MAX_X1:
196                         deposit = val
197                     else:
198                         closing = val
199                 elif x0 >= _NARR_MIN_X0:
200                     narr.append((w["top"], x0, text))
201
202         value_date: str | None = None
203         parts: list[str] = []
204         for _top, _x0, text in sorted(narr):
205             if value_date is None and _DATE_RE.match(text):
206                 value_date = text # the value date, lifted out of the narration band
207                 continue
208             parts.append(text)
209
210         ref_candidates = [p for p in parts if p.isdigit() and len(p) >= 9]
211         ref = max(ref_candidates, key=len) if ref_candidates else None
212
213         return Transaction(
214             date=primary[0]["text"],
215             narration=" ".join(parts),
216             withdrawal=withdrawal,
217             deposit=deposit,
218             closing_balance=closing if closing is not None else 0.0,
219             value_date=value_date,
220             ref=ref,
221             page=page_no,
222         )
223
224
225     def extract_transactions_from_lines(lines: list[list[dict]], page_no: int = 0) ->
226     list[Transaction]:
227         """Pure, PDF-free core: turn grouped table-body lines into transactions.

```

```

227
228     Kept separate from PDF I/O so the column logic is unit-testable on synthetic
229     word lists, with no real data.
230     """
231     primaries = [i for i, line in enumerate(lines) if _is_primary(line)]
232     transactions: list[Transaction] = []
233     for k, idx in enumerate(primaries):
234         stop = primaries[k + 1] if k + 1 < len(primaries) else len(lines)
235         primary = lines[idx]
236         primary_top = _line_top(primary)
237         continuations: list[list[dict]] = []
238         for m in range(idx + 1, stop):
239             line = lines[m]
240             if _has_footer_marker(line) or _line_top(line) - primary_top >
_CONT_MAX_SPAN:
241                 break
242             if _is_continuation(line):
243                 continuations.append(line)
244             transactions.append(_parse_transaction(primary, continuations, page_no))
245     return transactions
246
247
248 def _table_body(lines: list[list[dict]]) -> list[list[dict]]:
249     """Lines below the column header (drops the per-page account-summary box)."""
250     header_top: float | None = None
251     for line in lines:
252         texts = {w["text"] for w in line}
253         if "Narration" in texts and any(t.startswith("Closing") for t in texts):
254             header_top = _line_top(line)
255             break
256     if header_top is None:
257         return []
258     return [line for line in lines if _line_top(line) > header_top + 1]
259
260
261 def _extract_account_number(text: str) -> str | None:
262     match = _ACCOUNT_RE.search(text)
263     return match.group(1) if match else None
264
265
266 def parse_statement(pdf_path: Path | str, password: str = "") -> Statement:
267     """Parse an HDFC statement PDF into a :class:`Statement` of transactions.
268
269     ``password`` is forwarded to pdfplumber for encrypted statements and used
270     in-memory only ? never stored or logged.
271     """
272     path = Path(pdf_path)
273     transactions: list[Transaction] = []
274     account_number: str | None = None
275     with pdfplumber.open(str(path), password=password or "") as pdf:
276         for page_no, page in enumerate(pdf.pages):
277             if account_number is None and page_no < 2:
278                 account_number = _extract_account_number(page.extract_text() or "")
279             words = page.extract_words(use_text_flow=False, keep_blank_chars=False)
280             body = _table_body(_group_lines(words))
281             transactions.extend(extract_transactions_from_lines(body, page_no))
282     return Statement(path=path, transactions=transactions,
account_number=account_number)
283
284
285 def reconcile(statement: Statement, tol: float = 0.01) -> ReconcileReport:
286     """Check the running balance via the shared reconciler (closing[i] ==
287     closing[i-1] - withdrawal + deposit).
288
289     Spans page breaks (transactions are in document order), so a clean report also

```

```

290     validates page stitching. Reports counts only ? no amounts.
291     """
292     entries = [(t.withdrawal, t.deposit, t.closing_balance) for t in
statement.transactions]
293     return reconcile_running_balance(entries, tol)
294
295
296 def _parse_summary_lines(lines: list[str]) -> StatementSummary | None:
297     """Pure core: read HDFC's SUMMARY block from extracted text lines.
298
299     Layout (stable across statements)::
300
301         SUMMARY
302         Opening Balance  Debit Amount  Credit Amount  Closing Balance
303         <opening>        <debitAmt>    <creditAmt>    <closing>
304         Debit Count  Credit Count
305         <debitCount> <creditCount>
306
307     The values land on their own lines, so we take the first post-SUMMARY line
308     carrying ?4 money tokens (the balances) and the first carrying ?2 bare
309     integers and no money (the counts).
310     """
311     start = next((i for i, ln in enumerate(lines) if ln.strip() == "SUMMARY"), None)
312     if start is None:
313         return None
314     balances: list[float] | None = None
315     counts: list[int] | None = None
316     for ln in lines[start + 1 : start + 8]:
317         tokens = ln.split()
318         monies = [v for v in (_to_amount(t) for t in tokens) if v is not None]
319         ints = [int(t) for t in tokens if t.isdigit()]
320         if balances is None and len(monies) >= 4:
321             balances = monies[:4]
322         elif counts is None and not monies and len(ints) >= 2:
323             counts = ints[:2]
324     if balances is None and counts is None:
325         return None
326     opening, debit_amt, credit_amt, closing = (balances or [None, None, None, None])[:4]
327     debit_count, credit_count = (counts or [None, None])[:2]
328     return StatementSummary(
329         opening_balance=opening,
330         closing_balance=closing,
331         debit_count=debit_count,
332         credit_count=credit_count,
333         debit_amount=debit_amt,
334         credit_amount=credit_amt,
335     )
336
337
338 def parse_summary(pdf_path: Path | str, password: str = "") -> StatementSummary | None:
339     """Read the bank's own SUMMARY figures from the statement, or None if absent."""
340     with pdfplumber.open(str(Path(pdf_path)), password=password or "") as pdf:
341         for page in pdf.pages:
342             text = page.extract_text() or ""
343             if "SUMMARY" in text:
344                 summary = _parse_summary_lines(text.splitlines())
345                 if summary is not None:
346                     return summary
347     return None
348
349
350 def validate_against_summary(
351     statement: Statement, summary: StatementSummary, tol: float = 0.01
352 ) -> ValidationReport:
353     """Cross-check a parse against the bank's SUMMARY: reconciliation + bookends.

```

```

354
355 Reconciliation catches any mid-statement error/omission, the Dr/Cr counts catch
356 any extra/missing row, and the opening & closing balances catch a missing
357 first/last row.
358 """
359 txns = statement.transactions
360 reconciles = reconcile(statement, tol).ok
361 debit_count_ok = credit_count_ok = opening_ok = closing_ok = None
362 if summary.debit_count is not None:
363     debit_count_ok = sum(1 for t in txns if t.withdrawal > 0) == summary.debit_count
364 if summary.credit_count is not None:
365     credit_count_ok = sum(1 for t in txns if t.deposit > 0) == summary.credit_count
366 if txns and summary.closing_balance is not None:
367     closing_ok = abs(txns[-1].closing_balance - summary.closing_balance) <= tol
368 if txns and summary.opening_balance is not None:
369     implied_opening = txns[0].closing_balance - txns[0].deposit + txns[0].withdrawal
370     opening_ok = abs(implied_opening - summary.opening_balance) <= tol
371 return ValidationReport(
372     n=len(txns),
373     reconciles=reconciles,
374     debit_count_ok=debit_count_ok,
375     credit_count_ok=credit_count_ok,
376     opening_ok=opening_ok,
377     closing_ok=closing_ok,
378 )
379
380
381 def write_csv(statement: Statement, out_path: Path | str) -> Path:
382     """Write parsed transactions to CSV for local QA (the 'export to a sheet' idea).
383
384     Writes real data ? call only for a local, gitignored destination.
385     """
386     out = Path(out_path)
387     out.parent.mkdir(parents=True, exist_ok=True)
388     with open(out, "w", newline="", encoding="utf-8") as fh:
389         writer = csv.writer(fh)
390         writer.writerow(
391             [
392                 "date",
393                 "value_date",
394                 "narration",
395                 "ref",
396                 "withdrawal",
397                 "deposit",
398                 "closing_balance",
399                 "page",
400             ]
401         )
402         for t in statement.transactions:
403             writer.writerow(
404                 [
405                     t.date,
406                     t.value_date or "",
407                     t.narration,
408                     t.ref or "",
409                     f"{t.withdrawal:.2f}",
410                     f"{t.deposit:.2f}",
411                     f"{t.closing_balance:.2f}",
412                     t.page,
413                 ]
414             )
415         return out
416
417 @register_parser

```

```

419     class HDFCParser(StatementParser):
420         """Registered parser for HDFC savings / transaction-account statements."""
421
422         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
423             low = (text or "").lower()
424             is_hdfc = "hdfc bank" in low or "hdfcbank" in low or "hdfcbank" in (sender or
 "").lower()
425             is_statement = "savings account details" in low or (
426                 "narration" in low and "closing balance" in low
427             )
428             return is_hdfc and is_statement
429
430         def parse_and_insert(
431             self,
432             pdf_path: Path,
433             conn,
434             doc_id: int,
435             password: str | None = None,
436         ) -> int:
437             """Parse, insert rows into ``hdfc_transactions``, and record parse quality.
438
439             Sets ``documents.status`` to 'reconciled' / 'unreconciled' based on the
440             balance walk (and the bank's SUMMARY when present) so a downstream consumer
441             can tell a trustworthy parse from a suspect one. Returns the row count.
442             """
443             stmt = parse_statement(pdf_path, password=password or "")
444             for t in stmt.transactions:
445                 dbmod.insert_hdfc_transaction(
446                     conn,
447                     document_id=doc_id,
448                     account_number=stmt.account_number,
449                     txn_date=t.date,
450                     value_date=t.value_date,
451                     particulars=t.narration,
452                     ref=t.ref,
453                     withdrawal=t.withdrawal,
454                     deposit=t.deposit,
455                     closing_balance=t.closing_balance,
456                 )
457             summary = parse_summary(pdf_path, password=password or "")
458             ok = (
459                 validate_against_summary(stmt, summary).ok
460                 if summary is not None
461                 else reconcile(stmt).ok
462             )
463             conn.execute(
464                 "UPDATE documents SET status = ? WHERE id = ?",
465                 ("reconciled" if ok else "unreconciled", doc_id),
466             )
467             return len(stmt.transactions)
468

```

25. user (2026-05-31T08:17:52.659Z)

```

1     from __future__ import annotations
2
3     import abc
4     import sqlite3
5     from pathlib import Path
6
7
8     def safe_amount(s: str) -> float | None:
9         """Parse a monetary string to float or None. Shared across bank parsers."""
10         if not s or s.strip() in ("-", "", "0.00"):

```

```

11         return None
12     try:
13         return float(s.replace(",", "").strip())
14     except ValueError:
15         return None
16
17
18     class StatementParser(abc.ABC):
19         @abc.abstractmethod
20         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
21             """Return True if this parser can handle the given document."""
22             pass
23
24         @abc.abstractmethod
25         def parse_and_insert(
26             self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
27         ) -> int:
28             """Extract txns and insert them into the bank-specific table. Returns count of
rows."""
29             pass
30
31
32     __all__ = ["StatementParser", "safe_amount"]
33

```

26. assistant (2026-05-31T08:17:56.745Z)

Now the ICICI and Axis parsers (to avoid breaking the shared interface), the registry `\_\_init\_\_`, and the tests that pin behavior.

27. user (2026-05-31T08:17:57.706Z)

```

1     """ICICI Bank statement parser (transaction / savings account).
2
3     Same trustworthy pattern as the Axis savings parser: row-building is split from the
4     DB insert (pure, unit-testable) and the parse is **reconciled** before we trust it ?
5     ``documents.status`` records ``reconciled`` / ``unreconciled`` from a running-balance
6     walk, so a silent ``extract_tables()`` row-drop surfaces instead of passing quietly.
7     No ICICI statement is in the corpus yet, so this is synthetic-tested; the
8     ``local_statements`` path will validate it against a real one when available.
9     """
10
11     from __future__ import annotations
12
13     import re
14     import sqlite3
15     from dataclasses import dataclass
16     from pathlib import Path
17
18     import pdfplumber
19
20     from muneem import db as dbmod
21
22     from . import register_parser
23     from .base import StatementParser
24     from .validation import parse_amount, reconcile_running_balance
25
26     _DATE_RE = re.compile(r"\d{2}/\d{2}/\d{4}") # ICICI: dd/mm/yyyy
27     _ACCOUNT_RES = [
28         re.compile(r"Account Number\s*:\s*([\d-]+)", re.IGNORECASE),
29         re.compile(r"A/c\s*No\s*[:\s]*([\d-]+)", re.IGNORECASE),
30     ]
31

```



```

32
33 @dataclass
34 class ICICITxn:
35     txn_date: str
36     mode: str
37     particulars: str
38     withdrawals: float | None
39     deposits: float | None
40     balance: float | None
41
42
43 def _first_match(text: str, patterns: list[re.Pattern]) -> str | None:
44     for rx in patterns:
45         m = rx.search(text)
46         if m:
47             return m.group(1).strip()
48     return None
49
50
51 def _rows_from_icici_tables(tables: list) -> list[ICICITxn]:
52     """Build transactions from extracted tables. Pure: no PDF, no DB.
53
54     ICICI row: [Date, Mode, Particulars, Withdrawals, Deposits, Balance].
55     Opening/closing-balance and non-transaction rows are skipped.
56     """
57     rows: list[ICICITxn] = []
58     for table in tables or []:
59         for row in table or []:
60             if not row or len(row) < 5:
61                 continue
62             date_str = str(row[0]).strip() if row[0] else ""
63             low = date_str.lower()
64             if "opening balance" in low or "closing balance" in low:
65                 continue
66             if not _DATE_RE.match(date_str):
67                 continue
68             rows.append(
69                 ICICITxn(
70                     txn_date=date_str,
71                     mode=str(row[1]).strip() if len(row) > 1 and row[1] else "",
72                     particulars=str(row[2]).strip() if len(row) > 2 and row[2] else "",
73                     withdrawals=parse_amount(row[3]) if len(row) > 3 else None,
74                     deposits=parse_amount(row[4]) if len(row) > 4 else None,
75                     balance=parse_amount(row[5]) if len(row) > 5 else None,
76                 )
77             )
78     return rows
79
80
81 def parse_icici(pdf_path: Path, password: str | None = None) -> tuple[str,
list[ICICITxn]]:
82     """Open an ICICI statement and return (account_number, transactions)."""
83     account_number = "UNKNOWN"
84     rows: list[ICICITxn] = []
85     with pdfplumber.open(pdf_path, password=password) as pdf:
86         head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))
87         account_number = _first_match(head, _ACCOUNT_RES) or "UNKNOWN"
88         for page in pdf.pages:
89             rows.extend(_rows_from_icici_tables(page.extract_tables()))
90     return account_number, rows
91
92
93 @register_parser
94 class ICICIParser(StatementParser):

```

```

95         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
96             if sender and "icicibank.com" in sender.lower():
97                 return True
98             return "ICICI Bank" in text and "Statement of Transactions" in text
99
100         def parse_and_insert(
101             self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
102         ) -> int:
103             try:
104                 account_number, rows = parse_icici(pdf_path, password)
105             except Exception as exc:
106                 raise RuntimeError(f"Failed to parse ICICI statement {pdf_path}: {exc}")
107             from exc
108             for t in rows:
109                 dbmod.insert_icici_transaction(
110                     conn,
111                     document_id=doc_id,
112                     account_number=account_number,
113                     txn_date=t.txn_date,
114                     mode=t.mode,
115                     particulars=t.particulars,
116                     withdrawals=t.withdrawals,
117                     deposits=t.deposits,
118                     balance=t.balance,
119                 )
120             report = reconcile_running_balance([(t.withdrawals, t.deposits, t.balance) for t
in rows])
121             conn.execute(
122                 "UPDATE documents SET status = ? WHERE id = ?",
123                 ("reconciled" if report.ok else "unreconciled", doc_id),
124             )
125             return len(rows)

```

28. user (2026-05-31T08:17:58.175Z)

```

1     """Axis Bank statement parsers: savings/transaction account + credit card.
2
3     The savings parse is split from the DB insert (unit-testable) and validated by a
4     running-balance reconciliation.
5
6     **STOP-GAP:** ``extract_tables()`` has proven unreliable on *real* Axis savings
7     layouts ? column counts and date placement vary between statements (Jun vs Oct
8     2025), so the current mapping can mislabel rows. Until the real fix lands,
9     ``AxisParser`` refuses to persist a parse that doesn't reconcile (marks the
10    document ``unreconciled`` and stores 0 rows) rather than save mislabeled data.
11    The real fix is a coordinate-clustering rewrite (HDFC-style), validated by total
12    reconciliation ``opening + ?Deposits ? ?Withdrawals == closing``; see
13    docs/CURRENT_STATE.md. Credit-card statements have no running balance, so
14    ``AxisCreditCardParser`` can't reconcile (parse/persist split + count only).
15    """
16
17    from __future__ import annotations
18
19    import re
20    import sqlite3
21    from dataclasses import dataclass
22    from pathlib import Path
23
24    import pdfplumber
25
26    from muneem import db as dbmod
27

```

```

28 from . import register_parser
29 from .base import StatementParser
30 from .validation import parse_amount, reconcile_running_balance
31
32 _SAVINGS_DATE_RE = re.compile(r"\d{2}-\d{2}-\d{4}") # Axis savings: dd-mm-yyyy
33 _CC_DATE_RE = re.compile(r"\d{2}/\d{2}/\d{4}") # Axis credit card: dd/mm/yyyy
34 _ACCOUNT_RES = [
35     re.compile(r"Account No\s*:\s*([\d-]+)", re.IGNORECASE),
36     re.compile(r"A/c\s*No\s*:\s*([\d-]+)", re.IGNORECASE),
37 ]
38 _CARD_RES = [re.compile(r"(\d{4}XXX\d{4})", re.compile(r"(\d{6}\*\d{4})")]
39
40
41 @dataclass
42 class AxisTxn:
43     txn_date: str
44     value_date: str
45     particulars: str
46     debit: float | None
47     credit: float | None
48     balance: float | None
49
50
51 @dataclass
52 class AxisCCTxn:
53     txn_date: str
54     particulars: str
55     merchant_category: str
56     amount: float
57     type: str # "Cr" or "Dr"
58
59
60 def _first_match(text: str, patterns: list[re.Pattern]) -> str | None:
61     for rx in patterns:
62         m = rx.search(text)
63         if m:
64             return m.group(1).strip()
65     return None
66
67
68 def _rows_from_savings_tables(tables: list) -> list[AxisTxn]:
69     """Build savings transactions from extracted tables. Pure: no PDF, no DB.
70
71     Axis savings row: [Tran Date, Value Date, Particulars, Debit, Credit, Balance].
72     Opening/closing-balance rows and non-transaction rows are skipped.
73     """
74     rows: list[AxisTxn] = []
75     for table in tables or []:
76         for row in table or []:
77             if not row or len(row) < 5:
78                 continue
79             date_str = str(row[0]).strip() if row[0] else ""
80             low = date_str.lower()
81             if "opening balance" in low or "closing balance" in low:
82                 continue
83             if not _SAVINGS_DATE_RE.match(date_str):
84                 continue
85             rows.append(
86                 AxisTxn(
87                     txn_date=date_str,
88                     value_date=str(row[1]).strip() if len(row) > 1 and row[1] else "",
89                     particulars=str(row[2]).strip() if len(row) > 2 and row[2] else "",
90                     debit=parse_amount(row[3]) if len(row) > 3 else None,
91                     credit=parse_amount(row[4]) if len(row) > 4 else None,
92                     balance=parse_amount(row[5]) if len(row) > 5 else None,

```

```

93         )
94     )
95     return rows
96
97
98     def _rows_from_cc_tables(tables: list) -> list[AxisCCTxn]:
99         """Build credit-card transactions from extracted tables. Pure: no PDF, no DB."""
100         rows: list[AxisCCTxn] = []
101         for table in tables or []:
102             for row in table or []:
103                 if not row or len(row) < 9:
104                     continue
105                 date_str = str(row[0]).strip() if row[0] else ""
106                 if not _CC_DATE_RE.match(date_str):
107                     continue
108                 amount_str = str(row[8]).strip() if row[8] else ""
109                 is_cr = "Cr" in amount_str
110                 amount = parse_amount(re.sub(r"^\d.", "", amount_str)) or 0.0
111                 rows.append(
112                     AxisCCTxn(
113                         txn_date=date_str,
114                         particulars=str(row[2]).strip() if row[2] else "",
115                         merchant_category=str(row[7]).strip() if row[7] else "",
116                         amount=amount,
117                         type="Cr" if is_cr else "Dr",
118                     )
119                 )
120         return rows
121
122
123     def parse_axis_savings(pdf_path: Path, password: str | None = None) -> tuple[str,
list[AxisTxn]]:
124         """Open an Axis savings statement and return (account_number, transactions)."""
125         account_number = "UNKNOWN"
126         rows: list[AxisTxn] = []
127         with pdfplumber.open(pdf_path, password=password) as pdf:
128             head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))
129             account_number = _first_match(head, _ACCOUNT_RES) or "UNKNOWN"
130             for page in pdf.pages:
131                 rows.extend(_rows_from_savings_tables(page.extract_tables()))
132         return account_number, rows
133
134
135     def parse_axis_cc(pdf_path: Path, password: str | None = None) -> tuple[str,
list[AxisCCTxn]]:
136         """Open an Axis credit-card statement and return (card_number, transactions)."""
137         card_number = "UNKNOWN"
138         rows: list[AxisCCTxn] = []
139         with pdfplumber.open(pdf_path, password=password) as pdf:
140             head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))
141             card_number = _first_match(head, _CARD_RES) or "UNKNOWN"
142             for page in pdf.pages:
143                 rows.extend(_rows_from_cc_tables(page.extract_tables()))
144         return card_number, rows
145
146
147     @register_parser
148     class AxisParser(StatementParser):
149         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
150             if sender and "axisbank.com" in sender.lower():
151                 return True
152             return "Axis Bank" in text and "Statement of Account" in text

```

```

153
154     def parse_and_insert(
155         self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
156     ) -> int:
157         try:
158             account_number, rows = parse_axis_savings(pdf_path, password)
159         except Exception as exc:
160             raise RuntimeError(f"Failed to parse Axis statement {pdf_path}: {exc}") from
exc
161
162         # STOP-GAP (pending the coordinate-clustering rewrite): extract_tables() is
163         # unreliable across real Axis layouts ? column counts and date placement vary
164         # between statements, so the current mapping can mislabel rows. Rather than
165         # persist data we can't trust, we REFUSE to insert anything that doesn't
166         # reconcile: mark the document 'unreconciled' and store 0 rows. A correct
167         # parse (which reconciles) inserts normally. See docs/CURRENT_STATE.md.
168         report = reconcile_running_balance([(t.debit, t.credit, t.balance) for t in
rows])
169         if not report.ok:
170             conn.execute("UPDATE documents SET status = ? WHERE id = ?",
("unreconciled", doc_id))
171             return 0
172
173         for t in rows:
174             dbmod.insert_axis_transaction(
175                 conn,
176                 document_id=doc_id,
177                 account_number=account_number,
178                 txn_date=t.txn_date,
179                 value_date=t.value_date,
180                 particulars=t.particulars,
181                 debit=t.debit,
182                 credit=t.credit,
183                 balance=t.balance,
184             )
185             conn.execute("UPDATE documents SET status = ? WHERE id = ?", ("reconciled",
doc_id))
186             return len(rows)
187
188
189     @register_parser
190     class AxisCreditCardParser(StatementParser):
191         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
192             return "Axis Bank" in text and "Credit Card Statement" in text
193
194         def parse_and_insert(
195             self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
196         ) -> int:
197             try:
198                 card_number, rows = parse_axis_cc(pdf_path, password)
199             except Exception as exc:
200                 raise RuntimeError(f"Failed to parse Axis CC statement {pdf_path}: {exc}")
from exc
201
202             for t in rows:
203                 dbmod.insert_axis_cc_transaction(
204                     conn,
205                     document_id=doc_id,
206                     account_number=card_number,
207                     txn_date=t.txn_date,
208                     particulars=t.particulars,
209                     merchant_category=t.merchant_category,
210                     amount=t.amount,

```

```

210         type=t.type,
211     )
212     # Credit-card statements carry no running balance, so reconciliation does
213     # not apply; documents.status is left as set by the caller.
214     return len(rows)
215

```

29. user (2026-05-31T08:17:58.543Z)

```

1     from __future__ import annotations
2
3     from .base import StatementParser, safe_amount
4
5     _REGISTRY: list[type[StatementParser]] = []
6
7
8     def register_parser(parser_cls: type[StatementParser]) -> type[StatementParser]:
9         """Decorator to register a parser."""
10        _REGISTRY.append(parser_cls)
11        return parser_cls
12
13
14    def get_parser(
15        text: str, sender: str | None = None, subject: str | None = None
16    ) -> StatementParser | None:
17        """Return an instantiated parser that can handle this document, or None."""
18        for parser_cls in _REGISTRY:
19            parser = parser_cls()
20            if parser.can_handle(text, sender, subject):
21                return parser
22        return None
23
24
25    def _autoload_parsers() -> None:
26        """Import every parser submodule so its ``@register_parser`` runs.
27
28        Registration is an import *side-effect*, so the registry is only populated for
29        modules that have actually been imported. Without this, the production path
30        (``from muneem.parsers import get_parser``, as cli.py uses) would leave
31        ``_REGISTRY`` EMPTY ? only the tests imported the submodules. Auto-discovery
32        fixes that and means new parser modules register themselves with no edit here.
33        """
34        import importlib
35        import pkgutil
36
37        for info in pkgutil.iter_modules(__path__):
38            if info.name != "base" and not info.name.startswith("_"):
39                importlib.import_module(f"{__name__}.{info.name}")
40
41
42    _autoload_parsers()
43
44    __all__ = ["StatementParser", "register_parser", "get_parser", "safe_amount"]
45

```

30. assistant (2026-05-31T08:18:02.007Z)

Now the tests that pin the contract I must not break ? especially validation and HDFC.

31. user (2026-05-31T08:18:02.961Z)

```

1     """Tests for the shared running-balance reconciler (muneem.parsers.validation)."""
2
3     from muneem.parsers.validation import reconcile_running_balance

```

```

4
5
6 def test_reconciles_clean():
7     entries = [(500.0, 0.0, 9500.0), (0.0, 2000.0, 11500.0), (1500.5, 0.0, 9999.5)]
8     r = reconcile_running_balance(entries)
9     assert r.n == 3 and r.compared == 2 and r.breaks == 0 and r.ok
10
11
12 def test_detects_break():
13     entries = [(500.0, 0.0, 9500.0), (0.0, 2000.0, 11500.0), (1500.5, 0.0, 9999.49)]
14     r = reconcile_running_balance(entries)
15     assert r.breaks == 1 and r.first_break == 2 and not r.ok
16
17
18 def test_none_debit_or_credit_treated_as_zero():
19     # Axis/ICICI rows: exactly one of (debit, credit) is filled; the other is None.
20     entries = [(None, None, 1000.0), (250.0, None, 750.0), (None, 1250.0, 2000.0)]
21     r = reconcile_running_balance(entries)
22     assert r.compared == 2 and r.breaks == 0 and r.ok
23
24
25 def test_a_dropped_balance_surfaces_as_a_gap():
26     # If a row's balance goes missing (e.g. a silent extract_tables drop), the walk
27     # across the gap no longer adds up ? exactly the failure we want surfaced.
28     entries = [(None, None, 1000.0), (250.0, None, None), (None, None, 750.0)]
29     r = reconcile_running_balance(entries)
30     assert r.breaks == 1 and r.first_break == 2 and not r.ok
31
32
33 def test_empty_and_single_are_not_ok():
34     assert not reconcile_running_balance([]).ok
35     assert not reconcile_running_balance([(None, None, 100.0)]).ok # nothing to compare
36

```

32. user (2026-05-31T08:18:03.029Z)

```

1     """Tests for the HDFC statement parser (muneem.parsers.hdfc).
2
3     Column logic is exercised on *synthetic* word lists that mimic the real statement
4     geometry ? no real data, fast, offline. ``parse_and_insert`` is tested by
5     monkeypatching the parse functions (not by mocking pdfplumber), and a
6     ``local_statements`` test runs the real parser on the user's PDFs (skipped in CI).
7     """
8
9     import os
10    from pathlib import Path
11
12    import pytest
13
14    from muneem import db as dbmod
15    from muneem.parsers import hdfc
16
17
18    def W(text, x0, top, x1=None):
19        """A minimal pdfplumber-style word dict."""
20        return {
21            "text": text,
22            "x0": float(x0),
23            "x1": float(x1 if x1 is not None else x0 + 5 * len(text)),
24            "top": float(top),
25            "bottom": float(top) + 8,
26        }
27
28
29    def _sample_words():

```

```

30     """A 3-transaction table whose running balance reconciles, with header, a
31     per-page summary box above it, wrapped narration, and a footer marker."""
32     words = []
33     words += [W("Opening", 37, 50), W("Balance", 72, 50), W("01/02/2026", 52, 60)]
34     words += [
35         W("Date", 72, 80),
36         W("Narration", 175, 80),
37         W("Withdrawals", 296, 80),
38         W("Deposits", 386, 80),
39         W("Closing", 473, 80),
40         W("Balance", 504, 80),
41     ]
42     words += [
43         W("05/02/2026", 52, 100, 92),
44         W("UPI-SWIGGY-PAY", 111, 100),
45         W("500.00", 330, 100, 358),
46         W("0.00", 428, 100, 443),
47         W("9,500.00", 519, 100, 561),
48     ]
49     words += [W("okhdfcbank-CONTINUED", 111, 109)]
50     words += [W("04/02/2026", 122, 118, 162), W("123456789012", 180, 118, 255)]
51     words += [
52         W("08/02/2026", 52, 140, 92),
53         W("SALARY-CREDIT", 111, 140),
54         W("0.00", 340, 140, 358),
55         W("2,000.00", 415, 140, 443),
56         W("11,500.00", 515, 140, 561),
57     ]
58     words += [
59         W("10/02/2026", 52, 160, 92),
60         W("ATM-WDL", 111, 160),
61         W("1,500.50", 320, 160, 358),
62         W("0.00", 428, 160, 443),
63         W("9,999.50", 519, 160, 561),
64     ]
65     words += [W("STATEMENT", 200, 180), W("SUMMARY", 260, 180)]
66     return words
67
68
69 def _parse_sample():
70     body = hdfc._table_body(hdfc._group_lines(_sample_words()))
71     return hdfc.extract_transactions_from_lines(body, page_no=3)
72
73
74 def _sample_statement():
75     return hdfc.Statement(
76         path=Path("x"), transactions=_parse_sample(), account_number="50100000000000"
77     )
78
79
80 def test_columns_and_continuation_assembly():
81     t1, t2, t3 = _parse_sample()
82     assert t1.date == "05/02/2026"
83     assert t1.withdrawal == 500.0 and t1.deposit == 0.0 and t1.closing_balance == 9500.0
84     assert t1.value_date == "04/02/2026"
85     assert t1.ref == "123456789012"
86     assert t1.narration == "UPI-SWIGGY-PAY okhdfcbank-CONTINUED 123456789012"
87     assert t1.page == 3
88     assert t2.deposit == 2000.0 and t2.closing_balance == 11500.0
89     assert t3.withdrawal == 1500.5 and t3.closing_balance == 9999.5
90
91
92 def test_summary_box_and_footer_excluded():
93     txns = _parse_sample()
94     assert [t.date for t in txns] == ["05/02/2026", "08/02/2026", "10/02/2026"]

```



```

95         assert "STATEMENT" not in txns[-1].narration and "SUMMARY" not in txns[-1].narration
96
97
98     def test_reconciliation():
99         assert hdfc.reconcile(_sample_statement()).ok
100         txns = _parse_sample()
101         txns[2].closing_balance = 9999.49 # off by a paisa
102         report = hdfc.reconcile(hdfc.Statement(path=Path("x"), transactions=txns))
103         assert report.breaks == 1 and report.first_break == 2 and not report.ok
104
105
106     def test_parse_summary_lines_and_validation():
107         lines = [
108             "SUMMARY",
109             "Opening Balance Debit Amount Credit Amount Closing Balance",
110             "10,000.00 2,000.50 2,000.00 9,999.50",
111             "Debit Count Credit Count",
112             "2 1",
113         ]
114         s = hdfc._parse_summary_lines(lines)
115         assert s.opening_balance == 10000.0 and s.closing_balance == 9999.5
116         assert s.debit_count == 2 and s.credit_count == 1
117         report = hdfc.validate_against_summary(_sample_statement(), s)
118         assert report.ok and report.debit_count_ok and report.credit_count_ok
119         assert report.opening_ok and report.closing_ok
120
121
122     def test_validation_catches_missing_last_row():
123         stmt = hdfc.Statement(path=Path("x"), transactions=_parse_sample()[:-1])
124         summary = hdfc.StatementSummary(
125             opening_balance=10000.0, closing_balance=9999.50, debit_count=2, credit_count=1
126         )
127         report = hdfc.validate_against_summary(stmt, summary)
128         assert report.reconciles and report.closing_ok is False and not report.ok
129
130
131     @pytest.mark.parametrize(
132         "token,expected",
133         [
134             ("1,23,456.78", 123456.78),
135             ("0.00", 0.0),
136             ("1,000.00Cr", 1000.0),
137             ("9,999.50Dr", 9999.50),
138             ("abc", None),
139             ("100", None),
140             ("10/02/2026", None),
141         ],
142     )
143     def test_to_amount(token, expected):
144         assert hdfc._to_amount(token) == expected
145
146
147     def test_can_handle():
148         parser = hdfc.HDFCParser()
149         statement = "HDFC BANK Savings Account Details Narration Closing Balance"
150         assert parser.can_handle(statement)
151         assert parser.can_handle(statement, sender="alerts@hdfcbank.net")
152         # sender alone, with no statement markers, must NOT match (e.g. a promo from HDFC)
153         assert not parser.can_handle("anything", sender="alerts@hdfcbank.net")
154         assert not parser.can_handle("HDFC Bank promo: use code SAVE50")
155         assert not parser.can_handle("Axis Bank Statement of Account Narration Closing
Balance")
156         assert not parser.can_handle("ICICI Bank Statement of Transactions")
157
158

```

```

159     def test_parse_and_insert_records_reconciled(tmp_path, monkeypatch):
160         monkeypatch.setattr(hdfc, "parse_statement", lambda *a, **k: _sample_statement())
161         monkeypatch.setattr(
162             hdfc,
163             "parse_summary",
164             lambda *a, **k: hdfc.StatementSummary(
165                 opening_balance=10000.0, closing_balance=9999.50, debit_count=2,
166                 credit_count=1
167             ),
168         )
169         with dbmod.connect(tmp_path / "t.db") as conn:
170             dbmod.init_db(conn)
171             doc_id = dbmod.insert_document(conn, source="file", sha256="h1",
172             status="parsed")
173             n = hdfc.HDFCParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
174             assert n == 3
175             rows = dbmod.list_hdfc_transactions(conn, limit=100)
176             assert len(rows) == 3
177             assert rows[0]["account_number"] == "50100000000000"
178             assert {r["particulars"] for r in rows} # narration mapped to particulars
179             assert dbmod.get_document_by_sha(conn, "h1")["status"] == "reconciled"
180
181     def test_parse_and_insert_records_unreconciled(tmp_path, monkeypatch):
182         broken = _parse_sample()
183         broken[2].closing_balance = 1.0 # break the balance walk
184         monkeypatch.setattr(
185             hdfc,
186             "parse_statement",
187             lambda *a, **k: hdfc.Statement(path=Path("x"), transactions=broken),
188         )
189         monkeypatch.setattr(hdfc, "parse_summary", lambda *a, **k: None) # fall back to
190         reconcile()
191         with dbmod.connect(tmp_path / "t.db") as conn:
192             dbmod.init_db(conn)
193             doc_id = dbmod.insert_document(conn, source="file", sha256="h2",
194             status="parsed")
195             hdfc.HDFCParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
196             assert dbmod.get_document_by_sha(conn, "h2")["status"] == "unreconciled"
197
198     @pytest.mark.local_statements
199     @pytest.mark.parametrize(
200         "pdf_path",
201         sorted(Path("testdata/hdfc-statements").glob("*.pdf")),
202         ids=lambda p: p.name,
203     )
204     def test_real_hdfc_statement_validates(pdf_path, tmp_path):
205         """Local-only: parse the real statements and require the parse to agree with
206         HDFC's own SUMMARY (Dr/Cr counts + opening/closing) and reconcile fully.
207
208         Set MUNEEM_TEST_HDFC_PW; never runs in CI. Surfaces counts/booleans only.
209         """
210         pw = os.environ.get("MUNEEM_TEST_HDFC_PW")
211         if not pw:
212             pytest.skip("needs MUNEEM_TEST_HDFC_PW + real statements (never in CI)")
213
214         stmt = hdfc.parse_statement(pdf_path, password=pw)
215         assert stmt.transactions, f"{pdf_path.name}: no transactions parsed"
216         summary = hdfc.parse_summary(pdf_path, password=pw)
217         assert summary is not None, f"{pdf_path.name}: SUMMARY block not found"
218         report = hdfc.validate_against_summary(stmt, summary)
219         assert report.ok, (
220             f"{pdf_path.name}: parse disagrees with bank SUMMARY ?
221             reconciles={report.reconciles} "

```

```

219         f"dr={report.debit_count_ok} cr={report.credit_count_ok} "
220         f"open={report.opening_ok} close={report.closing_ok}"
221     )
222
223     # End-to-end through the registered parser + DB, incl. the status it records.
224     with dbmod.connect(tmp_path / "t.db") as conn:
225         dbmod.init_db(conn)
226         doc_id = dbmod.insert_document(conn, source="file", sha256=pdf_path.name,
status="parsed")
227         n = hdfc.HDFCParser().parse_and_insert(pdf_path, conn, doc_id, password=pw)
228         assert n == len(stmt.transactions)
229         assert dbmod.get_document_by_sha(conn, pdf_path.name)["status"] == "reconciled"
230

```

33. assistant (2026-05-31T08:21:30.661Z)

This gives me the full contract. Let me check the DB layer and project config (ruff/line-length, since the repo enforces `ruff format`), and confirm who imports the verifier symbols so I don't break a consumer.

34. user (2026-05-31T08:21:31.815Z)

```

docs\CURRENT_STATE.md:23:[Omitted long matching line]
docs\parser-architecture.md:73:Already exists in seed form as `muneem.parsers.validation`. It
grows into the component that both
tests\parsers\test_axis.py:15:from muneem.parsers.validation import reconcile_running_balance
tests\parsers\test_icici.py:14:from muneem.parsers.validation import reconcile_running_balance
tests\parsers\test_validation.py:1: """Tests for the shared running-balance reconciler
(muneem.parsers.validation)."""
tests\parsers\test_validation.py:3:from muneem.parsers.validation import
reconcile_running_balance
src\muneem\parsers\icici.py:24:from .validation import parse_amount, reconcile_running_balance
src\muneem\parsers\hdfc.py:39:from .validation import ReconcileReport, reconcile_running_balance
src\muneem\parsers\axis.py:30:from .validation import parse_amount, reconcile_running_balance

```

35. user (2026-05-31T08:21:32.546Z)

```

1     [build-system]
2     requires = ["setuptools>=61"]
3     build-backend = "setuptools.build_meta"
4
5     [project]
6     name = "muneem"
7     description = "A local-first personal-finance bookkeeper that ingests financial
documents from email into a consolidated ledger."
8     readme = "README.md"
9     requires-python = ">=3.11"
10    dynamic = ["version"]
11    # Private, single-user tool ? this classifier makes packaging tools refuse to upload it
to PyPI.
12    classifiers = ["Private :: Do Not Upload"]
13    dependencies = [
14        "typer>=0.12",
15        "platformdirs>=4.0",
16        "keyring>=25.0",
17        "pdfminer.six>=20240706",
18        "pypdfium2>=4.30",
19        "pytesseract>=0.3.10",
20        "pillow>=10.0",
21        "pypdf>=4.0",
22        "google-api-python-client>=2.100",
23        "google-auth>=2.30",
24        "google-auth-oauthlib>=1.2",
25        "pdfplumber>=0.11",

```

```

26     ]
27
28     [project.scripts]
29     muneem = "muneem.cli:app"
30
31     [project.optional-dependencies]
32     outlook = ["msal>=1.0", "requests>=2.0"]
33     dev = [
34         "pytest>=8.0",
35         "ruff>=0.6",
36         "pyyaml>=6.0",
37     ]
38
39     [tool.setuptools.dynamic]
40     version = { attr = "muneem.__version__" }
41
42     [tool.setuptools.packages.find]
43     where = ["src"]
44
45     [tool.ruff]
46     line-length = 100
47     src = ["src", "tests"]
48
49     [tool.ruff.lint]
50     select = ["E", "F", "I", "UP", "B"]
51
52     [tool.ruff.format]
53     quote-style = "double"
54     indent-style = "space"
55     skip-magic-trailing-comma = false
56     line-ending = "auto"
57
58     [tool.ruff.lint.flake8-bugbear]
59     # Typer uses Option()/Argument() as parameter defaults by design.
60     extend-immutable-calls = ["typer.Option", "typer.Argument"]
61
62     [tool.pytest.ini_options]
63     # Default runs are offline and deterministic: the ocr/llm/local_statements tiers
64     # (see docs/TESTING.md) are opt-in and excluded here.
65     addopts = "-ra -m 'not ocr and not llm and not local_statements'"
66     testpaths = ["tests"]
67     markers = [
68         "ocr: requires the Tesseract binary (image-OCR tier)",
69         "llm: makes live LLM calls (never in default CI)",
70         "local_statements: reads gitignored real statements (local only)",
71     ]
72

```

36. user (2026-05-31T08:21:33.915Z)

```

29:     status          TEXT NOT NULL DEFAULT 'parsed',
139: def insert_document(conn: sqlite3.Connection, **fields: object) -> int:
190: def insert_hdfc_transaction(conn: sqlite3.Connection, **fields: object) -> int:

```

37. assistant (2026-05-31T08:23:02.917Z)

I have everything I need. Here's the concrete plan for this PR before I write it.

Plan ? architecture Step 1 (no behavior change)

```

**New file `concepts.py`** ? the canonical concept schema as pure dataclasses (the contract
every extractor targets):
- `ConceptRow` ? `txn_date, narration, debit, credit, balance, value_date?, reference?` (CC rows
map `amount`?debit/credit by direction, `balance=None` ? documented).

```

- `StatementConcepts` ? `opening\_balance, closing\_balance, debit\_count, credit\_count, debit\_amount, credit\_amount, account\_number, period\_start/end, currency` (all optional).

****Grow `validation.py` into the verifier**** (keeping `parse\_amount`, `ReconcileReport`, `reconcile\_running\_balance` intact so ICICI/Axis/tests don't break) by adding:

- `reconcile\_total(...)` ? the layout-independent identity `opening + ?credit ? ?debit == closing` (Axis's future foundation).
- `verify(rows, concepts)` ? the ****generic bookend****, relocated verbatim from HDFC's `validate\_against\_summary` but expressed over `ConceptRow`/`StatementConcepts`.
- `Confidence` enum (`reconciled`/`unreconciled` ? same strings the DB already stores).
- `select\_reconciling(candidates)` ? the selection primitive the L2 mapping search (Step 2) will plug into.

****Refactor `hdfc.py` onto the core**** ? map its `Transaction`/`ConceptRow` and `StatementSummary`/`StatementConcepts`, route `validate\_against\_summary`/`status` through the shared verifier. Keep the public names (`StatementSummary`, `ValidationReport`, `validate\_against\_summary`, `reconcile`, `Transaction` fields) as back-compat aliases so ****every existing test passes unchanged**** ? that's the proof of no behavior change.

****Explicitly NOT touched**** (parked per `DESIGN`/§11): storage stays per-bank tables (unification is Step 5); ICICI/Axis stay as-is (they become profiles in Step 2); CLI/quarantine policy unchanged; no L3/VLM. New `reconcile\_total`/`select\_reconciling` get unit tests so they aren't dead code.

Let me set up the branch and start.

38. user (2026-05-31T08:23:03.651Z)

Switched to a new branch 'feat/parser-concept-core'
--- on branch ---
feat/parser-concept-core

39. user (2026-05-31T08:23:07.395Z)

Chapter marked: "Concept-core Step 1". Continue your current work.

40. assistant (2026-05-31T08:23:09.378Z)

Now the new concept schema module:

41. assistant (2026-05-31T08:23:22.062Z)

""The canonical *concept schema* ? one bank-agnostic vocabulary every extractor targets.

This is the ****contract**** at the heart of the concept-core parser architecture (see [docs/parser-architecture.md](../docs/parser-architecture.md) §3): a parser's job is to **map this bank's columns/fields onto these concepts, then let the verifier check them.** Because the concepts are layout-independent, a format change becomes "re-map onto the same concepts," not "rewrite the parser."

These are pure data ? no PDF, no DB, no logic ? so they import nothing and are safe to use anywhere.

The verifier (`muneem.parsers.validation`) consumes them; bank extractors produce them.

""

```
from __future__ import annotations
```

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class ConceptRow:
```

```
    """One normalized transaction, in the bank-agnostic vocabulary.
```

```

    ``debit`` is money out (withdrawal / spend / Dr); ``credit`` is money in (deposit / Cr).
    Exactly one is typically non-``None`` on a real row, but both may be ``None`` for a
    non-amount marker row. ``balance`` is the running account balance after this row, or
    ``None`` when the statement carries no per-row balance.

    **Credit cards** (no running balance) map onto the same row: put the charge in ``debit`` or
    the payment/refund in ``credit`` according to the printed Dr/Cr direction, and leave
    ``balance=None``. That keeps one row type for the whole ladder; total reconciliation (not
the
per-row walk) is what validates a balance-less statement.
"""

    txn_date: str
    narration: str
    debit: float | None = None
    credit: float | None = None
    balance: float | None = None
    value_date: str | None = None
    reference: str | None = None

@dataclass
class StatementConcepts:
    """Statement-level concepts ? the figures a bank prints in its own SUMMARY / header.

    Every field is optional: ``None`` means "this statement didn't expose it," which the
verifier
    treats as not checkable (never as a failure). ``debit_amount`` / ``credit_amount`` are the
    bank's own printed totals; they are recorded here for completeness but are not yet asserted
by
    :func:`muneem.parsers.validation.verify` (the current bookend checks counts +
opening/closing,
    matching the established HDFC behavior).
    """

    opening_balance: float | None = None
    closing_balance: float | None = None
    debit_count: int | None = None
    credit_count: int | None = None
    debit_amount: float | None = None
    credit_amount: float | None = None
    account_number: str | None = None
    period_start: str | None = None
    period_end: str | None = None
    currency: str | None = None

```

```
__all__ = ["ConceptRow", "StatementConcepts"]
```

42. user (2026-05-31T08:23:22.078Z)

File created successfully at: C:\Users\avidu\Projects\muneem\src\muneem\parsers\concepts.py
(file state is current in your context ? no need to Read it back)

43. assistant (2026-05-31T08:23:47.524Z)

Now grow `validation.py` into the verifier ? keeping the existing three public symbols intact,
adding the new capabilities:

44. assistant (2026-05-31T08:24:20.451Z)

```
"""The verifier ? reconciliation as a layout-independent truth oracle.
```

Reconciliation proves a parse is **complete and correct** without inspecting any sensitive

```

value**:
if a transaction is missed or misread, the arithmetic stops adding up. That is what lets us
trust a
parse of a statement we never eyeball, and (later) what makes a probabilistic extractor safe on
money ? *extraction proposes, reconciliation disposes* (see
[docs/parser-architecture.md](../docs/parser-architecture.md) $2, $5).

```

This module grew from a single running-balance walk into the verifier the architecture calls for:

```

* **per-row walk** ? ``reconcile_running_balance``: ``balance[i] == balance[i-1] ? debit +
credit``.
* **total reconcile** ? ``reconcile_total``: ``opening + ?credit ? ?debit == closing`` (works
even
with no per-row balance, e.g. Axis savings).
* **bookend** ? ``verify``: parsed Dr/Cr counts + opening/closing vs the bank's printed SUMMARY.
* **selection** ? ``select_reconciling``: among candidate concept mappings, pick the one
arithmetic
endorses (the seam the L2 mapping search plugs into).
* **confidence** ? ``Confidence`` + ``BookendReport.confidence``: a recorded verdict, not an
assumption.

```

```

Reports carry counts / booleans / a verdict only ? **never an amount**.
"""

```

```

from __future__ import annotations

```

```

import re
from collections.abc import Sequence
from dataclasses import dataclass
from enum import Enum

```

```

from .concepts import ConceptRow, StatementConcepts

```

```

_CR_DR_SUFFIX = re.compile(r"(?i)\s*(cr|dr)\s*$")

```

```

def parse_amount(value: object) -> float | None:
    """Parse a statement money cell to a float.

```

```

Returns ``None`` only for a genuinely absent cell (None / empty / ``"-"" /
non-numeric); ``"0.00"`` parses to ``0.0``. This deliberately distinguishes
*absent* from *zero* ? unlike ``base.safe_amount``, which maps ``"0.00"`` to
``None`` and would corrupt a real zero balance and the reconciliation math.
"""

```

```

if value is None:
    return None
text = _CR_DR_SUFFIX.sub("", str(value).strip()).replace(",", "").strip()
if text in ("", "-"):
    return None
try:
    return float(text)
except ValueError:
    return None

```

```

@dataclass

```

```

class ReconcileReport:
    """Result of a running-balance walk. Counts only, never an amount."""

    n: int # rows considered
    compared: int # consecutive (balance-bearing) pairs actually checked
    breaks: int
    first_break: int | None = None

```

```

@property
def ok(self) -> bool:
    return self.compared > 0 and self.breaks == 0

def reconcile_running_balance(
    entries: Sequence[tuple[float | None, float | None, float | None]],
    tol: float = 0.01,
) -> ReconcileReport:
    """Verify ``balance[i] == balance[i-1] - debit[i] + credit[i]`` across rows.

    ``entries`` is an ordered list of ``(debit, credit, balance)``; ``debit`` and
    ``credit`` may be ``None`` (treated as 0.0). A row whose ``balance`` is ``None``
    is skipped (can't anchor on it). Because the walk spans the whole statement, a
    clean result also rules out any row missed *between* two balances ? which is the
    failure mode silent ``extract_tables()`` row-drops would otherwise hide.
    """
    breaks = 0
    first_break: int | None = None
    compared = 0
    prev: float | None = None
    for i, (debit, credit, balance) in enumerate(entries):
        if balance is None:
            continue
        if prev is not None:
            expected = prev - (debit or 0.0) + (credit or 0.0)
            compared += 1
            if abs(expected - balance) > tol:
                breaks += 1
                if first_break is None:
                    first_break = i
        prev = balance
    return ReconcileReport(
        n=len(entries), compared=compared, breaks=breaks, first_break=first_break
    )

def _row_entries(
    rows: Sequence[ConceptRow],
) -> list[tuple[float | None, float | None, float | None]]:
    """Project concept rows onto the ``(debit, credit, balance)`` tuples the walk consumes."""
    return [(r.debit, r.credit, r.balance) for r in rows]

def reconcile_total(
    rows: Sequence[ConceptRow], opening: float, closing: float, tol: float = 0.01
) -> bool:
    """Verify the layout-independent identity ``opening + ?credit ? ?debit == closing``.

    This is the oracle for statements with **no reliable per-row balance** (e.g. Axis savings):
    it checks the whole statement against its own opening/closing bookends without needing each
    row's running balance. ``opening`` and ``closing`` come from the bank's SUMMARY/header
    (``StatementConcepts``); the caller must have them.
    """
    delta = sum(r.credit or 0.0 for r in rows) - sum(r.debit or 0.0 for r in rows)
    return abs((opening + delta) - closing) <= tol

class Confidence(str, Enum):
    """Recorded trust verdict for a parsed statement (mirrors ``documents.status`` values).

    ``RECONCILED`` ? the parse passed every available check, so it is trusted. ``UNRECONCILED``
    ?
    a check failed (or none could run); the parse is suspect and is quarantined rather than
    trusted.
    """

```



```

"""

RECONCILED = "reconciled"
UNRECONCILED = "unreconciled"

@dataclass
class BookendReport:
    """A parse cross-checked against the bank's own SUMMARY. Booleans + counts only.

    ``None`` for a check means the figure wasn't available to verify (not a failure). ``ok``
    requires the per-row balance walk *plus* every available check to pass.
    """

    n: int
    reconciles: bool
    debit_count_ok: bool | None = None
    credit_count_ok: bool | None = None
    opening_ok: bool | None = None
    closing_ok: bool | None = None

    @property
    def ok(self) -> bool:
        checks = [self.debit_count_ok, self.credit_count_ok, self.opening_ok, self.closing_ok]
        return self.n > 0 and self.reconciles and all(c is True for c in checks)

    @property
    def confidence(self) -> Confidence:
        return Confidence.RECONCILED if self.ok else Confidence.UNRECONCILED

def verify(
    rows: Sequence[ConceptRow], concepts: StatementConcepts, tol: float = 0.01
) -> BookendReport:
    """Bookend a parse against the bank's printed SUMMARY: reconciliation + the printed figures.

    The per-row walk catches any mid-statement error/omission; the Dr/Cr counts catch any
    extra/missing row; the opening & closing balances catch a missing first/last row. Each
    printed
    figure is only checked when present (``None`` ? not checked). The opening balance is checked
    by
    implication (``balance[0] ? credit[0] + debit[0]`` should equal the printed opening).
    """
    reconciles = reconcile_running_balance(_row_entries(rows), tol).ok
    debit_count_ok = credit_count_ok = opening_ok = closing_ok = None
    if concepts.debit_count is not None:
        debit_count_ok = sum(1 for r in rows if (r.debit or 0.0) > 0) == concepts.debit_count
    if concepts.credit_count is not None:
        credit_count_ok = sum(1 for r in rows if (r.credit or 0.0) > 0) == concepts.credit_count
    if rows and rows[-1].balance is not None and concepts.closing_balance is not None:
        closing_ok = abs(rows[-1].balance - concepts.closing_balance) <= tol
    if rows and rows[0].balance is not None and concepts.opening_balance is not None:
        implied_opening = rows[0].balance - (rows[0].credit or 0.0) + (rows[0].debit or 0.0)
        opening_ok = abs(implied_opening - concepts.opening_balance) <= tol
    return BookendReport(
        n=len(rows),
        reconciles=reconciles,
        debit_count_ok=debit_count_ok,
        credit_count_ok=credit_count_ok,
        opening_ok=opening_ok,
        closing_ok=closing_ok,
    )

def select_reconciling(candidates: Sequence[Sequence[ConceptRow]], tol: float = 0.01) -> int |

```

None:

```
"""Among candidate concept-row mappings, return the index of the first that reconciles.
```

```
This is the *selection* half of "extraction proposes, reconciliation disposes": when the
extraction ladder can't be sure which column?concept assignment is right, it proposes
several
```

```
and lets the per-row walk pick the correct one. Returns ``None`` if none reconcile.
```

```
The candidate *generation* (the L2 mapping search) is Step 2 of the architecture plan; this
is
```

```
the verifier-side seam it plugs into. Selection here uses the per-row walk, so candidates
need a
```

```
running balance ? balance-less statements select via :func:`reconcile_total` at the call
site.
```

```
"""
```

```
for i, rows in enumerate(candidates):
    if reconcile_running_balance(_row_entries(rows), tol).ok:
        return i
return None
```

```
__all__ = [
    "parse_amount",
    "ReconcileReport",
    "reconcile_running_balance",
    "reconcile_total",
    "Confidence",
    "BookendReport",
    "verify",
    "select_reconciling",
]
```

45. user (2026-05-31T08:24:20.482Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\validation.py has been updated successfully. (file state is current in your context ? no need to Read it back)

46. assistant (2026-05-31T08:25:19.687Z)

Now refactor `hdfc.py` onto the core. First the imports:

47. user (2026-05-31T08:25:21.417Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\hdfc.py has been updated successfully. (file state is current in your context ? no need to Read it back)

48. assistant (2026-05-31T08:25:24.077Z)

Add the `Transaction ? ConceptRow` mapping (the boundary onto the canonical vocabulary):

49. user (2026-05-31T08:25:28.780Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\hdfc.py has been updated successfully. (file state is current in your context ? no need to Read it back)

50. assistant (2026-05-31T08:25:38.387Z)

Replace HDFC's local `StatementSummary`/`ValidationReport` dataclasses with back-compat aliases onto the shared types:

51. user (2026-05-31T08:25:43.666Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\hdfc.py has been updated successfully. (file state is current in your context ? no need to Read it back)

52. assistant (2026-05-31T08:25:57.172Z)

Rewrite `validate\_against\_summary` to map onto concepts and delegate to the shared verifier ?
this is the core of "refactor HDFC onto it":

53. user (2026-05-31T08:26:01.239Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\hdfc.py has been updated successfully. (file state is current in your context ? no need to Read it back)

54. user (2026-05-31T08:26:11.685Z)

```
320         elif counts is None and not monies and len(ints) >= 2:
321             counts = ints[:2]
322         if balances is None and counts is None:
323             return None
324         opening, debit_amt, credit_amt, closing = (balances or [None, None, None, None])[:4]
325         debit_count, credit_count = (counts or [None, None])[:2]
326         return StatementSummary(
327             opening_balance=opening,
328             closing_balance=closing,
329             debit_count=debit_count,
330             credit_count=credit_count,
331             debit_amount=debit_amt,
332             credit_amount=credit_amt,
333         )
334
335
336     def parse_summary(pdf_path: Path | str, password: str = "") -> StatementSummary | None:
337         """Read the bank's own SUMMARY figures from the statement, or None if absent."""
338         with pdfplumber.open(str(Path(pdf_path)), password=password or "") as pdf:
339             for page in pdf.pages:
340                 text = page.extract_text() or ""
341                 if "SUMMARY" in text:
342                     summary = _parse_summary_lines(text.splitlines())
343                     if summary is not None:
344                         return summary
345             return None
346
347
348     def validate_against_summary(
349         statement: Statement, summary: StatementSummary, tol: float = 0.01
350     ) -> ValidationReport:
351         """Cross-check a parse against the bank's SUMMARY: reconciliation + bookends.
352
353         Reconciliation catches any mid-statement error/omission, the Dr/Cr counts catch
354         any extra/missing row, and the opening & closing balances catch a missing
355         first/last row.
356         """
357         rows = [t.to_concept() for t in statement.transactions]
358         return verify(rows, summary, tol)
359
360
361     def write_csv(statement: Statement, out_path: Path | str) -> Path:
362         """Write parsed transactions to CSV for local QA (the 'export to a sheet' idea).
363
364         Writes real data ? call only for a local, gitignored destination.
```

55. user (2026-05-31T08:26:17.242Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\hdfc.py has been updated successfully. (file state is current in your context ? no need to Read it back)

56. assistant (2026-05-31T08:26:19.833Z)

Now the status-recording logic in `parse\_and\_insert` ? use the `Confidence` enum (same stored strings):

57. user (2026-05-31T08:26:23.035Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\hdfc.py has been updated successfully. (file state is current in your context ? no need to Read it back)

58. assistant (2026-05-31T08:26:30.993Z)

Refactor is in place. Before adding new tests, let me verify the existing suite still passes (proving no behavior change) and that formatting/lint is clean:

59. user (2026-05-31T08:26:33.665Z)

```
Exit code 1
3 files left unchanged
--- ruff lint ---
UP042 Class Confidence inherits from both `str` and `enum.Enum`
--> src\muneem\parsers\validation.py:121:7
|
|
121 | class Confidence(str, Enum):
|     ^^^^^^^^^^^^^
122 |     """Recorded trust verdict for a parsed statement (mirrors ``documents.status``
values).
|
help: Inherit from `enum.StrEnum`

Found 1 error.
No fixes available (1 hidden fix can be enabled with the `--unsafe-fixes` option).
```

60. assistant (2026-05-31T08:26:37.944Z)

`ruff` wants `StrEnum` (the project targets 3.11+). Let me switch:

61. user (2026-05-31T08:26:39.026Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\validation.py has been updated successfully. (file state is current in your context ? no need to Read it back)

62. user (2026-05-31T08:26:42.096Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\validation.py has been updated successfully. (file state is current in your context ? no need to Read it back)

63. user (2026-05-31T08:26:55.702Z)

```
All checks passed!
--- pytest (offline) ---
.....X..... [ 75%]
..... [100%]
===== XPASSES =====
===== short test summary info =====
XPASS tests/test_ocr.py::test_swiggy_image_code_recovered_fuzzily - recovery of the stylized
boxed code is OCR-engine/version dependent ? succeeds on Tesseract 5.4, returns nothing on the
apt build; the planned fix is a purpose-trained model (docs/DEPENDENCIES.md). strict=False so it
never fails the build.
94 passed, 9 deselected, 1 xpassed in 6.50s
```

64. assistant (2026-05-31T08:27:42.459Z)

All 94 existing tests pass unchanged ? that's the no-behavior-change proof. Now let me add tests for the new verifier capabilities so they're exercised, not dead code. First the verifier tests:

65. user (2026-05-31T08:27:46.832Z)

The file C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py has been updated successfully. (file state is current in your context ? no need to Read it back)

66. user (2026-05-31T08:28:04.559Z)

The file C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py has been updated successfully. (file state is current in your context ? no need to Read it back)

67. assistant (2026-05-31T08:28:07.091Z)

Now the `to\_concept` mapping test in the HDFC reference parser's test file:

68. user (2026-05-31T08:28:10.472Z)

The file C:\Users\avidu\Projects\muneem\tests\parsers\test_hdfc.py has been updated successfully. (file state is current in your context ? no need to Read it back)

69. user (2026-05-31T08:28:15.350Z)

```
Exit code 1
2 files left unchanged
E501 Line too long (106 > 100)
--> tests\parsers\test_validation.py:15:101
|
14 | def _concept_rows():
15 |     """3 rows whose running balance walks cleanly: 10000 ?(?500) 9500 ?(+2000) 11500
? (?1500.5) 9999.5."""
|
^^^^^^
16 |     return [
17 |         ConceptRow(txn_date="05/02/2026", narration="a", debit=500.0, credit=0.0,
balance=9500.0),
|
```

Found 1 error.

70. assistant (2026-05-31T08:28:18.555Z)

Docstring line too long ? shorten it:

71. user (2026-05-31T08:28:20.704Z)

The file C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py has been updated successfully. (file state is current in your context ? no need to Read it back)

72. user (2026-05-31T08:28:31.200Z)

```
All checks passed!
--- pytest (offline) ---
.....X..... [ 70%]
..... [100%]
===== XPASSES =====
===== short test summary info =====
XPASS tests/test_ocr.py::test_swiggy_image_code_recovered_fuzzily - recovery of the stylized
boxed code is OCR-engine/version dependent ? succeeds on Tesseract 5.4, returns nothing on the
apt build; the planned fix is a purpose-trained model (docs/DEPENDENCIES.md). strict=False so it
never fails the build.
101 passed, 9 deselected, 1 xpassed in 6.29s
```

73. assistant (2026-05-31T08:29:25.319Z)

101 passing (94 existing + 7 new), lint clean. Now I'll update the docs to reflect that Step 1 landed ? the architecture doc is the authority for the parser-rework sequence, so I'll mark Step 1 there and add a note to CURRENT_STATE.

74. user (2026-05-31T08:29:29.579Z)

The file C:\Users\avidu\Projects\muneem\docs\parser-architecture.md has been updated successfully. (file state is current in your context ? no need to Read it back)

75. user (2026-05-31T08:29:36.730Z)

The file C:\Users\avidu\Projects\muneem\docs\parser-architecture.md has been updated successfully. (file state is current in your context ? no need to Read it back)

76. user (2026-05-31T08:29:42.238Z)

The file C:\Users\avidu\Projects\muneem\docs\CURRENT_STATE.md has been updated successfully. (file state is current in your context ? no need to Read it back)

77. assistant (2026-05-31T08:29:51.878Z)

Let me do a final review of the full diff and run the complete offline suite once more before committing:

78. user (2026-05-31T08:30:00.462Z)

```
--- diff stat ---
docs/CURRENT_STATE.md          |   2 +-
docs/parser-architecture.md    |  12 +++-
src/muneem/parsers/hdfc.py     |  94 ++++++-----
src/muneem/parsers/validation.py | 145 ++++++-----
tests/parsers/test_hdfc.py     |   8 +++++
tests/parsers/test_validation.py |  86 ++++++-----
6 files changed, 288 insertions(+), 69 deletions(-)

--- new untracked ---
M docs/CURRENT_STATE.md
M docs/parser-architecture.md
M src/muneem/parsers/hdfc.py
M src/muneem/parsers/validation.py
M tests/parsers/test_hdfc.py
M tests/parsers/test_validation.py
?? src/muneem/parsers/concepts.py

--- final offline run ---
===== XPASSES =====
===== short test summary info =====
XPASS tests/test_ocr.py::test_swiggy_image_code_recovered_fuzzily - recovery of the stylized
boxed code is OCR-engine/version dependent ? succeeds on Tesseract 5.4, returns nothing on the
apt build; the planned fix is a purpose-trained model (docs/DEPENDENCIES.md). strict=False so it
never fails the build.
101 passed, 9 deselected, 1 xpassed in 6.19s
```

79. assistant (2026-05-31T08:30:06.338Z)

All green: 101 passed, clean lint, `concepts.py` added. Let me eyeball the HDFC refactor diff ? that's the substantive change ? to confirm it reads cleanly:

80. user (2026-05-31T08:30:07.093Z)

```
diff --git a/src/muneem/parsers/hdfc.py b/src/muneem/parsers/hdfc.py
index 92a8533..4eff5e3 100644
--- a/src/muneem/parsers/hdfc.py
+++ b/src/muneem/parsers/hdfc.py
```

```

@@ -36,7 +36,14 @@ from muneem import db as dbmod

    from . import register_parser
    from .base import StatementParser
    -from .validation import ReconcileReport, reconcile_running_balance
    +from .concepts import ConceptRow, StatementConcepts
    +from .validation import (
    +    BookendReport,
    +    Confidence,
    +    ReconcileReport,
    +    reconcile_running_balance,
    +    verify,
    +)

    # --- Column geometry (PDF points; A4-portrait HDFC detailed statement) -----
    # Left-aligned columns key on x0; right-aligned amount columns key on their right
    @@ -67,6 +74,22 @@ class Transaction:
        ref: str | None = None
        page: int = 0

    +    def to_concept(self) -> ConceptRow:
    +        """Map HDFC's native row onto the bank-agnostic concept vocabulary (the verifier's
    input).
    +
    +        Withdrawal ? ``debit``, deposit ? ``credit``, closing balance ? ``balance``. The
    ``page``
    +        field is HDFC-internal provenance and has no concept equivalent.
    +        """
    +        return ConceptRow(
    +            txn_date=self.date,
    +            narration=self.narration,
    +            debit=self.withdrawal,
    +            credit=self.deposit,
    +            balance=self.closing_balance,
    +            value_date=self.value_date,
    +            reference=self.ref,
    +        )
    +

    @dataclass
    class Statement:
    @@ -75,37 +98,12 @@ class Statement:
        account_number: str | None = None

    -@dataclass
    -class StatementSummary:
    -    """The figures HDFC prints in its own end-of-statement SUMMARY block."""
    -
    -    opening_balance: float | None = None
    -    closing_balance: float | None = None
    -    debit_count: int | None = None
    -    credit_count: int | None = None
    -    debit_amount: float | None = None
    -    credit_amount: float | None = None
    -
    -
    -@dataclass
    -class ValidationReport:
    -    """Parse cross-checked against the bank's own SUMMARY. Booleans + counts only.
    -
    -    ``None`` means a figure wasn't available to check (not a failure). ``ok``
    -    requires the balance walk plus every available check to pass.
    -    """
    -

```

```

-     n: int
-     reconciles: bool
-     debit_count_ok: bool | None = None
-     credit_count_ok: bool | None = None
-     opening_ok: bool | None = None
-     closing_ok: bool | None = None
-
-     @property
-     def ok(self) -> bool:
-         checks = [self.debit_count_ok, self.credit_count_ok, self.opening_ok, self.closing_ok]
-         return self.n > 0 and self.reconciles and all(c is True for c in checks)
+ # HDFC's SUMMARY block and its cross-check are now expressed in the shared concept vocabulary +
+ # verifier (the concept-core refactor). These back-compat names keep HDFC's public surface
+ # stable:
+ # StatementSummary == the statement-level concepts a SUMMARY block supplies
+ # ValidationReport == the verifier's bookend report (per-row walk + printed-figure checks)
+ StatementSummary = StatementConcepts
+ ValidationReport = BookendReport

def _to_amount(token: str) -> float | None:
@@ -352,30 +350,13 @@ def validate_against_summary(
) -> ValidationReport:
    """Cross-check a parse against the bank's SUMMARY: reconciliation + bookends.

-     Reconciliation catches any mid-statement error/omission, the Dr/Cr counts catch
-     any extra/missing row, and the opening & closing balances catch a missing
-     first/last row.
+     Maps HDFC's rows onto the canonical concepts and delegates to the shared verifier
+     (:func:`muneem.parsers.validation.verify`): reconciliation catches any mid-statement
+     error/omission, the Dr/Cr counts catch any extra/missing row, and the opening & closing
+     balances catch a missing first/last row.
    """
-     txns = statement.transactions
-     reconciles = reconcile(statement, tol).ok
-     debit_count_ok = credit_count_ok = opening_ok = closing_ok = None
-     if summary.debit_count is not None:
-         debit_count_ok = sum(1 for t in txns if t.withdrawal > 0) == summary.debit_count
-     if summary.credit_count is not None:
-         credit_count_ok = sum(1 for t in txns if t.deposit > 0) == summary.credit_count
-     if txns and summary.closing_balance is not None:
-         closing_ok = abs(txns[-1].closing_balance - summary.closing_balance) <= tol
-     if txns and summary.opening_balance is not None:
-         implied_opening = txns[0].closing_balance - txns[0].deposit + txns[0].withdrawal
-         opening_ok = abs(implied_opening - summary.opening_balance) <= tol
-     return ValidationReport(
-         n=len(txns),
-         reconciles=reconciles,
-         debit_count_ok=debit_count_ok,
-         credit_count_ok=credit_count_ok,
-         opening_ok=opening_ok,
-         closing_ok=closing_ok,
-     )
+     rows = [t.to_concept() for t in statement.transactions]
+     return verify(rows, summary, tol)

def write_csv(statement: Statement, out_path: Path | str) -> Path:
@@ -460,8 +441,9 @@ class HDFCParser(StatementParser):
    if summary is not None
    else reconcile(stmt).ok
)
+     verdict = Confidence.RECONCILED if ok else Confidence.UNRECONCILED
+     conn.execute(
+         "UPDATE documents SET status = ? WHERE id = ?",

```



```
-         ("reconciled" if ok else "unreconciled", doc_id),
+         (verdict.value, doc_id),
    )
    return len(stmt.transactions)
```

81. assistant (2026-05-31T08:31:07.136Z)

The refactor reads cleanly ? the bookend logic moved wholesale into the shared verifier, HDFC now maps-and-delegates, and the old dataclasses became back-compat aliases. That's the no-behavior-change story in the diff itself.

Per your PR cadence, let me commit, push, and open the PR.

82. user (2026-05-31T08:31:12.568Z)

```
[feat/parser-concept-core 705d5d2] feat(parsers): concept-core verifier + schema; HDFC
refactored onto it (no behavior change)
7 files changed, 353 insertions(+), 69 deletions(-)
create mode 100644 src/muneem/parsers/concepts.py
```

83. user (2026-05-31T08:31:18.748Z)

```
remote:      https://github.com/avidullu/muneem/pull/new/feat/parser-concept-core
remote:
branch 'feat/parser-concept-core' set up to track 'origin/feat/parser-concept-core'.
To https://github.com/avidullu/muneem.git
 * [new branch]      feat/parser-concept-core -> feat/parser-concept-core
```

84. user (2026-05-31T08:31:35.869Z)

<https://github.com/avidullu/muneem/pull/22>

85. assistant (2026-05-31T08:31:45.523Z)

PR opened: [muneem#22](<https://github.com/avidullu/muneem/pull/22>). Verified locally (101 green, lint clean, no behavior change), so per your squash-merge cadence I'll merge it and sync `main`.

86. user (2026-05-31T08:31:52.016Z)

```
docs/parser-architecture.md      | 12 +++-
src/muneem/parsers/concepts.py   | 65 ++++++
src/muneem/parsers/hdfc.py       | 94 ++++++-----
src/muneem/parsers/validation.py | 145 ++++++-----
tests/parsers/test_hdfc.py       | 18 +++++
tests/parsers/test_validation.py | 86 ++++++
7 files changed, 353 insertions(+), 69 deletions(-)
create mode 100644 src/muneem/parsers/concepts.py
```

87. user (2026-05-31T08:31:56.247Z)

```
Your branch is up to date with 'origin/main'.
Already on 'main'
From https://github.com/avidullu/muneem
- [deleted]      (none)      -> origin/feat/parser-concept-core
Already up to date.
--- log ---
7555fb2 feat(parsers): concept-core verifier + schema; HDFC refactored onto it (no behavior
change) (#22)
6f6141f docs: parser architecture proposal (concept-core, verification-first) [DRAFT] (#21)
128fc36 docs(roadmap): flag Axis savings parser as a must-fix (needs thorough rewrite) (#20)
```

88. assistant (2026-05-31T08:32:03.023Z)

Merged as [muneem#22](https://github.com/avidullu/muneem/pull/22), branch deleted, `main` synced. Quick green check on the merged commit:

89. user (2026-05-31T08:32:10.624Z)

```
===== short test summary info =====
XPASS tests/test_ocr.py::test_swiggy_image_code_recovered_fuzzily - recovery of the stylized
boxed code is OCR-engine/version dependent ? succeeds on Tesseract 5.4, returns nothing on the
apt build; the planned fix is a purpose-trained model (docs/DEPENDENCIES.md). strict=False so it
never fails the build.
101 passed, 9 deselected, 1 xpassed in 6.23s
```

90. assistant (2026-05-31T08:32:39.148Z)

Done ? **Step 1 is shipped.** [muneem#22](https://github.com/avidullu/muneem/pull/22) is squash-merged to `main`, branch cleaned up, offline suite green on the merged commit (**101 passed**, ruff clean).

What's now in place

The concept-core contract that the rest of the parser rework hangs off:

- **concepts.py** ? the bank-agnostic vocabulary (`ConceptRow`, `StatementConcepts`) every extractor will target.
- **validation.py** is now the verifier ? per-row walk + `reconcile_total` + the concept-level `verify` bookend + `select_reconciling` (the L2 seam) + a recorded `Confidence` verdict.
- **HDFC** is the reference implementation ? maps its rows to concepts and verifies through the shared verifier. Proven no-behavior-change: every prior test passes verbatim, +7 new ones cover the new capabilities.

Parked exactly as agreed: per-bank storage, ICICI/Axis, CLI/quarantine policy, L3/VLM ? none touched.

Next steps (the plan's remaining sequence)

Following [`parser-architecture.md`](docs/parser-architecture.md) §10:

1. **Step 2 ? concept-mapping engine (L0/L1/L2)** as generic, bank-agnostic code, then express **HDFC + ICICI** as profiles (data) on top of it. This is where "bank knowledge becomes data, not code" gets proven.
2. **Step 3 ? the Axis must-fix** ? coordinate-clustering rewrite onto the core, validated by total reconcile on the Jun + Oct 2025 statements. This is the headline payoff ? it kills the stop-gap and is the first real test of generic-first.
3. **Parallel, anytime** ? the **local-VLM deep-research run** (still pending) that populates L3 + the numeric-hallucination traps for Step 4. Non-sensitive, doesn't block Steps 2/3.

My recommendation is to **continue to Step 2** next (it's the prerequisite that makes the Step 3 Axis rewrite a profile + generic-path exercise rather than another bespoke parser). Want me to pick up Step 2, jump straight to the Axis must-fix (Step 3), or kick off the local-VLM research in parallel first?

91. user (2026-05-31T08:32:39.433Z)

yes keep the per bank tables as is for now and revisit later. Transaction classification will be soon in scope after this phase when I would like to include financial statements eg. stock sales, mutual funds, ETF buy/sell/dividend/fees etc. classifications
This may be a memory/feedback as well that Banks are a starting point conceptually a "muneem" does all the finances so different (equity, mutual funds, ETFs, government and private bonds, etc. statement parsing is also well within scope)

92. assistant (2026-05-31T08:33:58.702Z)

Two things there: a decision to confirm, and a scope steer you've explicitly asked me to remember. Let me record the scope first, then confirm the rest.

93. assistant (2026-05-31T08:34:06.282Z)

```

---
name: scope-all-personal-finance
description: muneem's scope is the user's ENTIRE financial life, not just banks; investment-
statement parsing + instrument-event classification are core near-term goals, not stretch
backlog.
metadata:
  type: project
---

```

A *muneem* is a bookkeeper/accountant ? conceptually it handles *all* of the user's finances. *Bank statements* are only the starting point / first vertical slice, not the boundary of scope. *Squarely in scope* (the user's words: "well within scope," not hypothetical): parsing statements for *equities, mutual funds, ETFs, government & private bonds*, and similar instruments. *Tier-2 transaction classification* is a *near-term* goal ? planned for *after* the current concept-core parser phase (the step 1?3/5 rework begun 2026-05) ? and must classify *instrument events*: stock / MF / ETF *buy / sell*, *dividend*, *fees*, etc.

Why: the repo (DESIGN/ROADMAP) currently frames brokers/MF/holdings as a *post-checkpoint* "generalize" backlog item; the user is elevating it to *conceptually central and near-term*. This sets the altitude for design decisions made *now*.

How to apply: don't over-fit the concept core to bank-statement rows (``txn_date/debit/credit/balance``). Keep a clear path open for a *second concept family* ? instrument, ISIN, quantity, unit price, event-type, holdings snapshots ? with *its own* reconciliation oracle (e.g. units/holdings continuity, cost-basis math) distinct from the bank running-balance walk. The "extraction proposes, reconciliation disposes" principle (`[[parser-validation-pattern]]`, `parser-architecture.md`) generalizes, but the *oracle* differs per family. Treat broker/MF/ETF/bond parsing as planned milestones, not "maybe later." Per-bank raw tables stay for now; storage unification is deferred (`parser-architecture.md` step 5).

94. user (2026-05-31T08:34:06.307Z)

File created successfully at: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-muneem\memory\scope-all-personal-finance.md (file state is current in your context ? no need to Read it back)

95. user (2026-05-31T08:34:17.245Z)

```

1      # muneem ? Memory Index
2
3      - [PR workflow cadence](pr-workflow-cadence.md) ? one PR per work item; verify locally,
then squash-merge as you go.
4      - [Prefer local over paid APIs](prefer-local-over-paid-apis.md) ? self-hosted/local/own-
trained first; paid APIs only as a fallback.
5      - [Commercial license safety](commercial-license-safety.md) ? only MIT/BSD/Apache deps;
avoid (A)GPL. PyMuPDF (AGPL) is banned.
6      - [HDFC parser tech decision](hdfc-parser-tech-decision.md) ? hand-roll coordinate
clustering on pdfplumber; camelot-py is the license-safe fallback.
7      - [Parser validation pattern](parser-validation-pattern.md) ? every parser: split
parse/persist, reconcile via shared validation, bookend vs the bank's SUMMARY, record
documents.status.
8

```

96. user (2026-05-31T08:34:22.493Z)

The file C:\Users\avidu\claude\projects\C--Users-avidu-Projects-muneem\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

97. assistant (2026-05-31T08:34:44.335Z)

Both noted.

Storage decision ? confirmed and already aligned. Per-bank tables stay; storage unification is parked to Step 5 of the architecture plan. No action needed ? this PR didn't touch storage,

and I won't until you revisit it.

****Scope steer ? saved to memory**** ([scope-all-personal-finance.md](../.claude/projects/C--Users-avidu-Projects-muneem/memory/scope-all-personal-finance.md), project-type). The sharpening I captured: muneem is the **whole** financial picture, banks are just the first slice, and ****equity / MF / ETF / govt & private bonds parsing + instrument-event classification (buy/sell/dividend/fees)** are core near-term goals ? not the "generalize backlog" the repo docs currently frame them as.

The design implication I'm carrying forward (not building yet): the concept schema I just shipped is bank-shaped (`debit/credit/balance` + running-balance walk). Investments will need a ****second concept family**** ? instrument/ISIN, quantity, unit price, event type, holdings snapshots ? with ****its own reconciliation oracle**** (units/holdings continuity, cost basis) rather than the running-balance walk. The verifier's "propose ? reconcile ? record confidence" spine generalizes cleanly; only the oracle gets swapped per family. So I'll make sure ****Step 2's concept-mapping engine + profile mechanism** don't hard-code bank-only assumptions at their seams, even though the first two profiles (HDFC, ICICI) happen to be banks. That keeps the door open for instruments without over-building now.

This actually **reinforces** Step 2 as the right next move: the generic engine is no longer just about bank format-resilience ? it's the load-bearing foundation for a second instrument family soon. Your "yes" answered the storage question but didn't pick a build target, so to confirm ? want me to:

- ****Step 2**** (recommended) ? build the generic L0/L1/L2 concept-mapping engine, express HDFC + ICICI as data profiles, with seams kept family-agnostic; or
- ****Step 3**** ? jump straight to the Axis must-fix on the new core; or
- ****local-VLM research**** ? kick that off in parallel first.

98. user (2026-05-31T08:35:54.080Z)

Please go ahead with your improved Step 2 and then Step 3. Would be great to see the precision of Axis vs HDFC at the end of your Step 3

99. assistant (2026-05-31T08:38:12.970Z)

On it ? ****Step 2 then Step 3****, two PRs per your cadence, and I'll close with an Axis-vs-HDFC precision comparison. This is a meaty arc, so let me set up task tracking and gather the few things I still need: the ICICI/Axis test contracts, the DB insert signatures, and whether real statements + passwords are present locally (for the final comparison).

100. user (2026-05-31T08:38:14.336Z)

```
1      """Tests for the ICICI parser.
2
3      Row-building is tested on synthetic tables (no PDF/DB). ``parse_and_insert`` is
4      tested via a pdfplumber mock and via monkeypatching for the reconcile->status
5      behaviour. No real ICICI statement is in the corpus yet.
6      """
7
8      from pathlib import Path
9      from unittest.mock import MagicMock, patch
10
11     import muneem.db as dbmod
12     from muneem.parsers import icici
13     from muneem.parsers.icici import ICICIParser
14     from muneem.parsers.validation import reconcile_running_balance
15
16
17     def test_can_handle():
18         parser = ICICIParser()
19         assert parser.can_handle("Some text", sender="statements@icicibank.com")
20         assert parser.can_handle("ICICI Bank Statement of Transactions")
21         assert not parser.can_handle("HDFC Bank")
```

```

22
23
24 def test_rows_from_icici_tables_skips_non_txn_rows():
25     tables = [
26         [
27             ["Date", "Mode", "Particulars", "Withdrawals", "Deposits", "Balance"], #
header
28             ["OPENING BALANCE", "", "", "", "", "1,000.00"], # explicit skip
29             ["12/05/2023", "UPI", "Amazon", "250.00", "", "750.00"],
30             ["13/05/2023", "NEFT", "Salary", "", "1,250.00", "2,000.00"],
31             ["bad", "x", "y", "", "", ""], # not a date -> skipped
32         ]
33     ]
34     rows = icici._rows_from_icici_tables(tables)
35     assert len(rows) == 2
36     assert rows[0].withdrawals == 250.0 and rows[0].deposits is None and rows[0].balance
== 750.0
37     assert rows[1].deposits == 1250.0 and rows[1].balance == 2000.0
38     report = reconcile_running_balance([(r.withdrawals, r.deposits, r.balance) for r in
rows])
39     assert report.ok and report.compared == 1 # 750 - 0 + 1250 == 2000
40
41
42 def test_parse_and_insert_via_pdfplumber_mock(tmp_path):
43     mock_pdf = MagicMock()
44     mock_page = MagicMock()
45     mock_page.extract_text.return_value = "Account Number: 987654321"
46     mock_page.extract_tables.return_value = [
47         [
48             ["12/05/2023", "UPI", "Amazon", "500.00", "", "2000.00"],
49             ["Opening Balance", "", "", "", "", "2500.00"],
50         ]
51     ]
52     mock_pdf.pages = [mock_page]
53     with dbmod.connect(tmp_path / "test.db") as conn:
54         dbmod.init_db(conn)
55         doc_id = dbmod.insert_document(conn, source="file", sha256="fakehash2",
status="parsed")
56         with patch("pdfplumber.open") as mock_open:
57             mock_open.return_value.__enter__.return_value = mock_pdf
58             count = ICICIParser().parse_and_insert(Path("fake.pdf"), conn, doc_id)
59             assert count == 1
60             txns = dbmod.list_icici_transactions(conn)
61             assert len(txns) == 1
62             assert txns[0]["account_number"] == "987654321"
63             assert txns[0]["withdrawals"] == 500.0
64
65
66 def _icici_rows(last_balance):
67     return [
68         icici.ICICITxn("12/05/2023", "UPI", "Amazon", 250.0, None, 750.0),
69         icici.ICICITxn("13/05/2023", "NEFT", "Salary", None, 1250.0, last_balance),
70     ]
71
72
73 def test_parse_and_insert_records_reconciled(tmp_path, monkeypatch):
74     monkeypatch.setattr(icici, "parse_icici", lambda *a, **k: ("ACC999",
_icici_rows(2000.0)))
75     with dbmod.connect(tmp_path / "t.db") as conn:
76         dbmod.init_db(conn)
77         doc_id = dbmod.insert_document(conn, source="file", sha256="ic1",
status="parsed")
78         n = ICICIParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
79         assert n == 2
80         assert dbmod.get_document_by_sha(conn, "ic1")["status"] == "reconciled"

```

```

81
82
83     def test_parse_and_insert_records_unreconciled(tmp_path, monkeypatch):
84         monkeypatch.setattr(icici, "parse_icici", lambda *a, **k: ("ACC999",
85 _icici_rows(9999.0)))
86         with dbmod.connect(tmp_path / "t.db") as conn:
87             dbmod.init_db(conn)
88             doc_id = dbmod.insert_document(conn, source="file", sha256="ic2",
status="parsed")
89             ICICIParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
90             assert dbmod.get_document_by_sha(conn, "ic2")["status"] == "unreconciled"

```

101. user (2026-05-31T08:38:14.866Z)

```

1     """Tests for the Axis parsers (savings + credit card).
2
3     Row-building is tested on synthetic tables (no PDF/DB). ``parse_and_insert`` is
4     tested both via a pdfplumber mock (the open->extract->insert flow) and via
5     monkeypatching for the reconcile->status behaviour. Real encrypted statements run
6     under the local_statements marker (see test_local_axis.py).
7     """
8
9     from pathlib import Path
10    from unittest.mock import MagicMock, patch
11
12    import muneem.db as dbmod
13    from muneem.parsers import axis
14    from muneem.parsers.axis import AxisCreditCardParser, AxisParser
15    from muneem.parsers.validation import reconcile_running_balance
16
17
18    def test_can_handle():
19        assert AxisParser().can_handle("Some text", sender="alerts@axisbank.com")
20        assert AxisParser().can_handle("Axis Bank Statement of Account")
21        assert not AxisParser().can_handle("HDFC Bank")
22        assert AxisCreditCardParser().can_handle("Axis Bank Credit Card Statement")
23        assert not AxisCreditCardParser().can_handle("Axis Bank Statement of Account")
24
25
26    def test_rows_from_savings_tables_skips_non_txn_rows():
27        tables = [
28            [
29                ["Tran Date", "Value Date", "Particulars", "Debit", "Credit", "Balance"], #
header
30                ["OPENING BALANCE", "", "", "", "", "1,000.00"], # explicit skip
31                ["12-05-2023", "12-05-2023", "UPI/Zomato", "250.00", "", "750.00"],
32                ["13-05-2023", "13-05-2023", "Salary", "", "1,250.00", "2,000.00"],
33                ["bad", "x", "junk", "", "", ""], # not a date -> skipped
34            ]
35        ]
36        rows = axis._rows_from_savings_tables(tables)
37        assert len(rows) == 2
38        assert rows[0].debit == 250.0 and rows[0].credit is None and rows[0].balance ==
750.0
39        assert rows[1].credit == 1250.0 and rows[1].debit is None and rows[1].balance ==
2000.0
40        report = reconcile_running_balance([(r.debit, r.credit, r.balance) for r in rows])
41        assert report.ok and report.compared == 1 # 750 - 0 + 1250 == 2000
42
43
44    def test_rows_from_cc_tables():
45        tables = [
46            [
47                ["Date", "c1", "Particulars", "c3", "c4", "c5", "c6", "Category", "Amount"],

```

```

48         ["05/02/2026", "x", "Swiggy", "", "", "", "", "Food", "250.00 Dr"],
49         ["06/02/2026", "x", "Refund", "", "", "", "", "Food", "100.00 Cr"],
50         ["bad", "too", "short"], # < 9 cols -> skipped
51     ]
52 ]
53 rows = axis._rows_from_cc_tables(tables)
54 assert len(rows) == 2
55 assert rows[0].amount == 250.0 and rows[0].type == "Dr" and
rows[0].merchant_category == "Food"
56     assert rows[1].amount == 100.0 and rows[1].type == "Cr"
57
58
59 def test_parse_and_insert_via_pdfplumber_mock(tmp_path):
60     # A reconciling 2-row table (in the assumed layout) exercises the open->extract->
61     # insert path. The third row has no date and is skipped.
62     mock_pdf = MagicMock()
63     mock_page = MagicMock()
64     mock_page.extract_text.return_value = "Account No: 123456789"
65     mock_page.extract_tables.return_value = [
66         [
67             ["12-05-2023", "12-05-2023", "UPI/Zomato", "250.00", "", "1000.00"],
68             ["13-05-2023", "13-05-2023", "Salary", "", "500.00", "1500.00"],
69             ["Invalid Date", "12-05-2023", "Skipped", "", "", ""],
70         ]
71     ]
72     mock_pdf.pages = [mock_page]
73     with dbmod.connect(tmp_path / "test.db") as conn:
74         dbmod.init_db(conn)
75         doc_id = dbmod.insert_document(conn, source="file", sha256="fakehash",
status="parsed")
76         with patch("pdfplumber.open") as mock_open:
77             mock_open.return_value.__enter__.return_value = mock_pdf
78             count = AxisParser().parse_and_insert(Path("fake.pdf"), conn, doc_id)
79             assert count == 2
80             txns = dbmod.list_axis_transactions(conn)
81             assert {t["account_number"] for t in txns} == {"123456789"}
82             assert any(t["debit"] == 250.0 for t in txns)
83             assert dbmod.get_document_by_sha(conn, "fakehash")["status"] == "reconciled"
84
85
86 def _savings_rows(last_balance):
87     return [
88         axis.AxisTxn("12-05-2023", "12-05-2023", "UPI/Zomato", 250.0, None, 750.0),
89         axis.AxisTxn("13-05-2023", "13-05-2023", "Salary", None, 1250.0, last_balance),
90     ]
91
92
93 def test_parse_and_insert_records_reconciled(tmp_path, monkeypatch):
94     monkeypatch.setattr(
95         axis, "parse_axis_savings", lambda *a, **k: ("ACC123", _savings_rows(2000.0))
96     )
97     with dbmod.connect(tmp_path / "t.db") as conn:
98         dbmod.init_db(conn)
99         doc_id = dbmod.insert_document(conn, source="file", sha256="ax1",
status="parsed")
100         n = AxisParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
101         assert n == 2
102         assert dbmod.list_axis_transactions(conn)[0]["account_number"] == "ACC123"
103         assert dbmod.get_document_by_sha(conn, "ax1")["status"] == "reconciled"
104
105
106 def test_parse_and_insert_refuses_to_store_unreconciled(tmp_path, monkeypatch):
107     # Stop-gap: a parse that doesn't reconcile must store NOTHING (no mislabeled
108     # rows), mark the document unreconciled, and return 0.
109     monkeypatch.setattr(

```

```

110         axis, "parse_axis_savings", lambda *a, **k: ("ACC123", _savings_rows(9999.0))
111     )
112     with dbmod.connect(tmp_path / "t.db") as conn:
113         dbmod.init_db(conn)
114         doc_id = dbmod.insert_document(conn, source="file", sha256="ax2",
status="parsed")
115         n = AxisParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
116         assert n == 0
117         assert dbmod.list_axis_transactions(conn) == []
118         assert dbmod.get_document_by_sha(conn, "ax2")["status"] == "unreconciled"
119

```

102. user (2026-05-31T08:38:15.104Z)

```

1     """Local-only checks for the Axis parsers against real encrypted statements.
2
3     Skipped unless an Axis password is available (`password_rules.yaml` or
4     `MUNEEM_TEST_AXIS_PW`); never runs in CI. The Axis savings parser is mid-rewrite
5     (`extract_tables()` is unreliable on real layouts ? coordinate-clustering rewrite
6     pending), so this asserts the **stop-gap safety invariant** rather than full
7     success: the savings parser must never persist rows it couldn't reconcile. Surfaces
8     counts / booleans / parser names only ? no statement content.
9     """
10
11     import os
12     from pathlib import Path
13
14     import pdfplumber
15     import pytest
16
17     from muneem import db as dbmod
18     from muneem.parsers import get_parser
19     from muneem.password_rules import get_password_for_bank
20
21     AXIS_PDFS = sorted(Path("testdata/axis-statements").glob("*.pdf"))
22
23
24     @pytest.mark.local_statements
25     @pytest.mark.parametrize("pdf_path", AXIS_PDFS, ids=lambda p: p.name)
26     def test_local_axis_never_stores_unreconciled(pdf_path, tmp_path):
27         pw = get_password_for_bank("axis") or os.environ.get("MUNEEM_TEST_AXIS_PW")
28         if not pw:
29             pytest.skip("needs the Axis password (password_rules.yaml or
MUNEEM_TEST_AXIS_PW)")
30
31         with pdfplumber.open(pdf_path, password=pw) as pdf:
32             text = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(2,
len(pdf.pages))))
33             parser = get_parser(text)
34             if parser is None:
35                 pytest.skip(f"{pdf_path.name}: no parser recognizes it yet (Axis rewrite
pending)")
36
37             with dbmod.connect(tmp_path / "t.db") as conn:
38                 dbmod.init_db(conn)
39                 doc_id = dbmod.insert_document(conn, source="file", sha256=pdf_path.name,
status="parsed")
40                 n = parser.parse_and_insert(pdf_path, conn, doc_id, password=pw)
41                 status = dbmod.get_document_by_sha(conn, pdf_path.name)["status"]
42                 stored = dbmod.list_axis_transactions(conn)
43
44             # Stop-gap invariant: the savings parser either stores nothing (and flags the
45             # document unreconciled) or, if it stored rows, they reconciled. Never garbage.
46             if type(parser).__name__ == "AxisParser":
47                 assert (n == 0 and stored == []) or status == "reconciled", (

```



```
48         f"{pdf_path.name}: stored {n} row(s) with status={status} ? "  
49         "stop-gap must not persist unreconciled rows"  
50     )  
51
```

103. assistant (2026-05-31T08:38:25.216Z)

Now let me check what real-statement data and passwords are available locally (existence only ? never values), and the DB insert signatures I must preserve:

104. user (2026-05-31T08:38:29.656Z)

```
--- testdata dirs ---  
testdata/hdfc-statements: 4 pdf(s)  
testdata/axis-statements: 4 pdf(s)  
testdata/icici-statements: (absent)  
--- passwords available? (existence only, never values) ---  
MUNEEM_TEST_HDFC_PW=unset  
MUNEEM_TEST_AXIS_PW=unset  
password_rules.yaml: absent  
/c/Users/avidu/.config/muneem/password_rules.yaml: absent
```

105. user (2026-05-31T08:38:30.991Z)

```
160: def insert_axis_transaction(conn: sqlite3.Connection, **fields: object) -> int:  
161-     return _insert_row(conn, "axis_transactions", fields)  
162-  
163-  
164: def insert_icici_transaction(conn: sqlite3.Connection, **fields: object) -> int:  
165-     return _insert_row(conn, "icici_transactions", fields)  
166-  
167-  
168: def insert_axis_cc_transaction(conn: sqlite3.Connection, **fields: object) -> int:  
169-     return _insert_row(conn, "axis_cc_transactions", fields)  
170-  
171-  
--  
178: def list_axis_transactions(conn: sqlite3.Connection, limit: int = 50) -> list[sqlite3.Row]:  
179-     return conn.execute(  
180-         "SELECT * FROM axis_transactions ORDER BY id DESC LIMIT ?", (limit,) )  
181-     ).fetchall()  
--  
184: def list_icici_transactions(conn: sqlite3.Connection, limit: int = 50) -> list[sqlite3.Row]:  
185-     return conn.execute(  
186-         "SELECT * FROM icici_transactions ORDER BY id DESC LIMIT ?", (limit,) )  
187-     ).fetchall()
```

106. assistant (2026-05-31T08:40:07.674Z)

Good ? that tells me what I need. Key facts:

- **Real statements are present** (4 HDFC + 4 Axis PDFs, gitignored) but **no passwords are available** to me (env vars unset, no `password_rules.yaml`). So I **cannot decrypt them myself** ? and per your security rules I won't ask you to paste a password. The real-data precision comparison will be a harness **you** run (or enable via `password_rules.yaml`/keyring, which surfaces only counts/booleans to me). I'll build CI-safe synthetic Jun/Oct Axis fixtures for the in-loop precision check, and the real-statement comparison on top.
- DB inserts are `**fields`-flexible, so migrating ICICI/Axis onto the engine won't disturb storage.

Let me set up tasks and lock the Step 2 design.

107. user (2026-05-31T08:40:11.434Z)

Task #1 created successfully: Step 2: generic concept-mapping engine (L0/L1/L2) + Profile

108. user (2026-05-31T08:40:13.925Z)

Task #2 created successfully: Step 2: migrate ICICI onto the engine; give HDFC a profile

109. user (2026-05-31T08:40:15.782Z)

Task #3 created successfully: Step 2: tests green + PR + squash-merge

110. user (2026-05-31T08:40:19.055Z)

Task #4 created successfully: Step 3: Axis coordinate-clustering rewrite onto the core

111. user (2026-05-31T08:40:23.396Z)

Task #6 created successfully: Axis-vs-HDFC precision comparison (counts/booleans only)

112. user (2026-05-31T08:40:20.918Z)

Task #5 created successfully: Step 3: synthetic Axis fixtures + tests + PR + squash-merge

113. user (2026-05-31T08:42:12.092Z)

Updated task #1 status

114. assistant (2026-05-31T08:42:09.367Z)

Real statements present but locked ? so I'll verify in-loop on synthetic Jun/Oct fixtures and hand you the real-data comparison to run. Design locked. Let me start Step 2.

****Step 2 design**** (flagging one architecture call): the engine runs ****generic detectors first**, with the profile supplying aliases/anchors/quirks as priors** ? that's a pragmatic answer to \$11's open "generic-vs-profile balance" question, not a hard lock; easy to revisit. The ****family-agnostic seam**** is a `FamilySpec`` (concept names + which are date/amount/balance + a typed-row builder) plus an ****injected reconciliation oracle**** ? so the same L0/L1/L2 control flow will later serve an instrument family with a different oracle. HDFC stays bespoke coordinate-clustering (the documented exception where `extract_tables`` fails) but gets a Profile as `*data*`; ICICI (and Axis next) run through the generic engine.

115. user (2026-05-31T08:42:11.012Z)

Switched to a new branch 'feat/concept-mapping-engine'
feat/concept-mapping-engine

116. user (2026-05-31T08:42:11.782Z)

Chapter marked: "Step 2 ? concept-mapping engine". Continue your current work.

117. assistant (2026-05-31T08:44:58.031Z)

Now the engine ? the substantive module. Writing `engine.py`` (the generic L0/L1/L2 mapping ladder + the family-agnostic `FamilySpec``):

118. assistant (2026-05-31T08:45:55.578Z)

""The generic concept-mapping engine ? columns in, concepts out, arithmetic decides.

This is the ***generic-first*** half of the concept-core architecture (see [`docs/parser-architecture.md``](`../..../docs/parser-architecture.md`) \$4, \$6): bank-agnostic code that takes a **\*\*cell-matrix\*\*** (the shape `pdfplumber.extract_tables()`` returns, or what a coordinate-clustering extractor emits) and maps its columns onto the canonical concepts via a cheap-first

ladder, with a profile supplying priors:

```
* **L0 ? header lexicon.** Match header-row labels against the profile's concept?label aliases.
* **profile-order.** If there is no header, fall back to the profile's known column order (a
prior).
* **L1/L2 ? content + search.** Classify columns by content (dates, money), then enumerate the
plausible amount-column ? {debit, credit, balance} assignments and **keep the one the
reconciliation
oracle endorses** ? the self-identifying balance column, found by *which mapping makes the
math work*
rather than by its label.
```

The engine is **family-agnostic**: it is parameterized by a :class:`FamilySpec` (the target concept names, which are date/amount/balance, and a typed-row builder) and an injected `oracle` (`rows -> bool`). Only the **bank** family ships today; an instrument family (equity/MF/ETF/bond ? see the project scope) would reuse this control flow with its own concepts + oracle. Nothing here logs a value; provenance is the winning layer + column map.

```
from __future__ import annotations
```

```
import itertools
import re
from collections.abc import Callable, Sequence
from dataclasses import dataclass, field
```

```
from .concepts import ConceptRow
from .validation import parse_amount
```

```
Cell = object # a table cell: str | None (we coerce defensively)
Row = Sequence[Cell]
```

Generic Indian-statement date shapes; a profile can pin its own to avoid false positive

```
_GENERIC_DATE_PATTERNS: tuple[re.Pattern, ...] = (
    re.compile(r"^\d{2}[/-]\d{2}[/-]\d{4}$"), # dd/mm/yyyy or dd-mm-yyyy
    re.compile(r"^\d{2}[- ][A-Za-z]{3}[- ]\d{2,4}$"), # dd-Mon-yyyy / dd Mon yy
)
```

"Money" requires a decimal point so bare integers (e.g. a long reference number) are I for an amount column during content classification.

```
_MONEY_RE = re.compile(r"(?i)^\d{1,2}(?:\s*(?:cr|dr))?$")
_BALANCE_MARKERS = ("opening balance", "closing balance", "balance b/f", "balance c/f")
```

The generic bank lexicon. Profiles extend/override per issuer; keys are concept names; lower-cased header labels matched exactly (not by substring) during L0.

```
_BANK_ALIASES: dict[str, tuple[str, ...]] = {
    "txn_date": ("date", "txn date", "tran date", "transaction date", "posting date"),
    "value_date": ("value date", "val date", "value dt"),
    "narration": (
        "narration", "particulars", "description", "transaction details", "details",
        "remarks", "transaction", "transaction remarks",
    ),
    "reference": (
        "ref", "reference", "ref no", "reference no", "cheque no", "chq no", "instrument no",
    ),
    "debit": ("debit", "withdrawal", "withdrawals", "dr", "paid out", "debit amount"),
    "credit": ("credit", "deposit", "deposits", "cr", "paid in", "credit amount"),
    "balance": ("balance", "closing balance", "running balance", "closing bal", "available
balance"),
```

```

}

def _text(cell: Cell) -> str:
    return "" if cell is None else str(cell).strip()

def _norm(label: str) -> str:
    return re.sub(r"\s+", " ", label.strip().lower())

def _matches_date(text: str, patterns: Sequence[re.Pattern]) -> bool:
    return any(p.match(text) for p in patterns)

def _looks_like_money(text: str) -> bool:
    return bool(_MONEY_RE.match(text))

@dataclass
class FamilySpec:
    """A concept *family*: the target vocabulary + how to build typed rows from a column
    mapping.

    The engine's L0/L1/L2 logic is written against this spec rather than against bank concepts,
    so a
    future instrument family (with its own concepts + reconciliation oracle) reuses the same
    engine.
    ``build`` turns per-row ``{concept: raw_cell}`` dicts into typed rows the oracle
    understands.
    """

    name: str
    concepts: tuple[str, ...]
    date_concepts: tuple[str, ...]
    amount_concepts: tuple[str, ...]
    balance_concept: str | None
    default_aliases: dict[str, tuple[str, ...]]
    build: Callable[[list[dict[str, str]], list]

def _build_bank_rows(mapped: list[dict[str, str]]) -> list[ConceptRow]:
    """Turn mapped cells into :class:`ConceptRow`s (amounts parsed; absent stays ``None``)."""
    rows: list[ConceptRow] = []
    for m in mapped:
        rows.append(
            ConceptRow(
                txn_date=_text(m.get("txn_date")),
                narration=_text(m.get("narration")),
                debit=parse_amount(m.get("debit")),
                credit=parse_amount(m.get("credit")),
                balance=parse_amount(m.get("balance")),
                value_date=_text(m.get("value_date")) or None,
                reference=_text(m.get("reference")) or None,
            )
        )
    return rows

BANK_FAMILY = FamilySpec(
    name="bank",
    concepts=("txn_date", "value_date", "narration", "reference", "debit", "credit", "balance"),
    date_concepts=("txn_date", "value_date"),
    amount_concepts=("debit", "credit", "balance"),
    balance_concept="balance",

```

```

    default_aliases=_BANK_ALIASES,
    build=_build_bank_rows,
)

@dataclass
class MappingResult:
    """The outcome of mapping one table: typed rows + how we got there (provenance)."""

    rows: list # typed rows from family.build (e.g. list[ConceptRow])
    mapped: list[dict[str, str]] # per-row {concept: raw cell}, incl. profile extras (e.g.
    ICICI mode)
    column_map: dict[str, int] # the winning concept -> column-index assignment
    layer: str # "L0" / "profile-order" / "L2" ? which rung produced it
    verified: bool # did the oracle endorse it? (False if no oracle was supplied)

@dataclass
class Profile:
    """Declarative per-issuer knowledge ? *data, not code* (architecture §6).

    Generic detectors run first; the profile supplies priors: extra concept?label aliases, the
    accepted date shape, a positional fallback when there's no header, table anchors, account-
    number
    patterns, quirks, and (placeholder, wiring parked) the password rule.
    """

    name: str
    aliases: dict[str, tuple[str, ...]] = field(default_factory=dict)
    date_patterns: tuple[re.Pattern, ...] = ()
    column_order: tuple[str, ...] | None = None
    table_anchors: tuple[str, ...] = ()
    account_patterns: tuple[re.Pattern, ...] = ()
    quirks: frozenset[str] = frozenset()
    password_rule: str | None = None # data only; sourcing is parked (DESIGN §8.2)

    def first_account(self, text: str) -> str | None:
        for rx in self.account_patterns:
            m = rx.search(text)
            if m:
                return m.group(1).strip()
        return None

    def _alias_index(profile: Profile, family: FamilySpec) -> dict[str, str]:
        """Build ``normalized-label -> concept``. Profile entries win over family defaults on
        conflict."""
        index: dict[str, str] = {}
        for concept, labels in family.default_aliases.items():
            for label in labels:
                index[_norm(label)] = concept
        for concept, labels in profile.aliases.items():
            for label in labels:
                index[_norm(label)] = concept
        return index

    def _date_patterns(profile: Profile) -> tuple[re.Pattern, ...]:
        return profile.date_patterns or _GENERIC_DATE_PATTERNS

    def _detect_header(rows: list[list[Cell]], alias_index: dict[str, str]) -> tuple[int | None,
    dict[str, int]]:
        """Find the header row (most exact alias hits, >=2) and its concept->column map."""
        best_idx: int | None = None

```

```

best_map: dict[str, int] = {}
for i, row in enumerate(rows):
    cmap: dict[str, int] = {}
    for col, cell in enumerate(row):
        concept = alias_index.get(_norm(_text(cell)))
        if concept and concept not in cmap:
            cmap[concept] = col
    if len(cmap) > len(best_map):
        best_map, best_idx = cmap, i
if len(best_map) < 2:
    return None, {}
return best_idx, best_map

def _is_balance_marker(row: Row) -> bool:
    return any(any(mk in _norm(_text(c)) for mk in _BALANCE_MARKERS) for c in row)

def _is_transaction_row(row: Row, date_col: int | None, patterns: Sequence[re.Pattern]) -> bool:
    if date_col is not None:
        cell = row[date_col] if date_col < len(row) else None
        return _matches_date(_text(cell), patterns)
    return any(_matches_date(_text(c), patterns) for c in row)

def _classify_columns(
    data: list[list[Cell]], ncols: int, patterns: Sequence[re.Pattern]
) -> dict[int, str]:
    """Tag each column date / amount / text / empty by majority content."""
    kinds: dict[int, str] = {}
    for c in range(ncols):
        cells = [_text(r[c]) for r in data if c < len(r) and _text(r[c])]
        if not cells:
            kinds[c] = "empty"
            continue
        thresh = max(1, int(len(cells) * 0.6))
        if sum(1 for x in cells if _matches_date(x, patterns)) >= thresh:
            kinds[c] = "date"
        elif sum(1 for x in cells if _looks_like_money(x)) >= thresh:
            kinds[c] = "amount"
        else:
            kinds[c] = "text"
    return kinds

def _base_map(col_kinds: dict[int, str], data: list[list[Cell]]) -> dict[str, int]:
    """Date + narration columns from content (the non-ambiguous part of an L1 mapping)."""
    base: dict[str, int] = {}
    date_cols = [c for c, k in sorted(col_kinds.items()) if k == "date"]
    text_cols = [c for c, k in sorted(col_kinds.items()) if k == "text"]
    if date_cols:
        base["txn_date"] = date_cols[0]
    if len(date_cols) > 1:
        base["value_date"] = date_cols[1]
    if text_cols:
        base["narration"] = max(
            text_cols, key=lambda c: sum(len(_text(r[c])) for r in data if c < len(r))
        )
    return base

def _search_candidates(
    col_kinds: dict[int, str], data: list[list[Cell]], family: FamilySpec
) -> list[tuple[str, dict[str, int]]]:
    """Enumerate plausible amount-column assignments (the L2 search space)."""

```

```

base = _base_map(col_kinds, data)
amount_cols = [c for c, k in sorted(col_kinds.items()) if k == "amount"]
cands: list[dict[str, int]] = []
if family.balance_concept and len(amount_cols) >= 3:
    for bal in amount_cols:
        rest = [c for c in amount_cols if c != bal]
        for d, cr in itertools.permutations(rest, 2):
            cands.append(**base, "debit": d, "credit": cr, family.balance_concept: bal)
if len(amount_cols) >= 2: # no per-row balance (total-reconcile families, e.g. Axis Oct)
    for d, cr in itertools.permutations(amount_cols, 2):
        cands.append(**base, "debit": d, "credit": cr)
return [{"L2", c} for c in cands[:24]]

def _apply_map(data: list[list[Cell]], col_map: dict[str, int]) -> list[dict[str, str]]:
    out: list[dict[str, str]] = []
    for row in data:
        out.append({c: (_text(row[i]) if i < len(row) else "") for c, i in col_map.items()})
    return out

def map_table(
    table: list[Row] | None,
    profile: Profile,
    family: FamilySpec = BANK_FAMILY,
    *,
    oracle: Callable[[list], bool] | None = None,
) -> MappingResult | None:
    """Map one extracted table onto ``family``'s concepts; ``oracle`` selects among candidates.

    Resolution order: header lexicon (L0) ? profile column order ? content search (L2). With an
    ``oracle`` the first candidate it endorses wins (``verified=True``); without one, the first
    structurally-complete candidate is returned unverified (the caller reconciles separately).
    Returns ``None`` if no transaction rows are found, or the best unverified candidate if an
    oracle
    was supplied but nothing reconciled.
    """
    if not table:
        return None
    rows_in = [list(r) for r in table if r is not None]
    if not rows_in:
        return None
    alias_index = _alias_index(profile, family)
    patterns = _date_patterns(profile)

    header_idx, col_map_hdr = _detect_header(rows_in, alias_index)
    start = header_idx + 1 if header_idx is not None else 0
    date_col_hint = col_map_hdr.get("txn_date")
    if date_col_hint is None and profile.column_order and "txn_date" in profile.column_order:
        date_col_hint = profile.column_order.index("txn_date")

    data = [
        r
        for r in rows_in[start:]
        if not _is_balance_marker(r) and _is_transaction_row(r, date_col_hint, patterns)
    ]
    if not data:
        return None
    ncols = max(len(r) for r in data)

    candidates: list[tuple[str, dict[str, int]]] = []
    if "debit" in col_map_hdr and "credit" in col_map_hdr: # L0: header pinned the amounts
        col_kinds = _classify_columns(data, ncols, patterns)
        candidates.append(("L0", {*_base_map(col_kinds, data), **col_map_hdr}))
    elif col_map_hdr: # header present but didn't pin amounts: keep its hints, search the rest

```

```

        candidates.append(("L0", col_map_hdr))
    if profile.column_order: # positional prior (no/partial header)
        candidates.append(("profile-order", {c: i for i, c in enumerate(profile.column_order)}))
    candidates += _search_candidates(_classify_columns(data, ncols, patterns), data, family)

    best: MappingResult | None = None
    for layer, cmap in candidates:
        if "txn_date" not in cmap or not ({ "debit", "credit", "balance" } & cmap.keys()):
            continue
        mapped = _apply_map(data, cmap)
        typed = family.build(mapped)
        if oracle is None:
            return MappingResult(typed, mapped, cmap, layer, verified=False)
        if oracle(typed):
            return MappingResult(typed, mapped, cmap, layer, verified=True)
        if best is None:
            best = MappingResult(typed, mapped, cmap, layer, verified=False)
    return best

```

```
__all__ = ["Profile", "FamilySpec", "BANK_FAMILY", "MappingResult", "map_table"]
```

119. user (2026-05-31T08:45:55.598Z)

File created successfully at: C:\Users\avidu\Projects\muneem\src\muneem\parsers\engine.py (file state is current in your context ? no need to Read it back)

120. assistant (2026-05-31T08:46:24.633Z)

Now the concrete profiles (`profiles.py`) ? ICICI and HDFC as data:

121. assistant (2026-05-31T08:46:34.818Z)

```
"""Per-issuer profiles ? bank knowledge expressed as **data, not code** (architecture §6).
```

A profile carries the priors the generic engine needs to disambiguate one issuer: extra concept?label aliases, the accepted date shape, a positional fallback when there's no header, table anchors, account-number patterns, quirks, and (parked) the password rule. Adding an issuer is ideally **adding a profile**, not writing a parser.

```
`ICICI_PROFILE` drives ICICI fully through `engine.map_table`. `HDFC_PROFILE` is data consumed by
HDFC's parser, whose *extraction* stays bespoke coordinate-clustering ? the documented exception
where extract_tables() fails on a semi-ruled table (see the hdfc-parser-tech-decision`
memory`).
```

The Axis profile lands with the Step 3 coordinate-clustering rewrite.

```
"""
```

```
from __future__ import annotations
```

```
import re
```

```
from .engine import Profile
```

```
ICICI_PROFILE = Profile(
    name="icici",
    aliases={
        "narration": ("particulars",),
        "debit": ("withdrawals",),
        "credit": ("deposits",),
        "mode": ("mode",), # ICICI-specific extra (UPI/NEFT/...); rides along in
MappingResult.mapped
    },
```



```

date_patterns=(re.compile(r"^d{2}/d{2}/d{4}$"),), # dd/mm/yyyy
# Known ICICI layout, used when a table has no detectable header row.
column_order=("txn_date", "mode", "narration", "debit", "credit", "balance"),
account_patterns=(
    re.compile(r"Account Number\s*:\s*([\d-]+)", re.IGNORECASE),
    re.compile(r"A/c\s*No\s*[:\s]*([\d-]+)", re.IGNORECASE),
),
)

HDFC_PROFILE = Profile(
    name="hdfc",
    aliases={
        "debit": ("withdrawal", "withdrawals"),
        "credit": ("deposit", "deposits"),
        "balance": ("closing balance",),
    },
    date_patterns=(re.compile(r"^d{2}/d{2}/d{4}$"),), # dd/mm/yyyy
    table_anchors=("Narration", "Closing Balance"),
    account_patterns=(re.compile(r"Account\s*Number\s*[:.]\s*(\d{6,})", re.IGNORECASE),),
    # HDFC's transaction table is semi-ruled; extract_tables() is unreliable on it, so HDFC
extracts
    # by word-position clustering rather than through the generic cell-matrix engine.
    quirks=frozenset({"semi-ruled-table", "coordinate-clustering"}),
)

__all__ = ["ICICI_PROFILE", "HDFC_PROFILE"]

```

122. user (2026-05-31T08:46:34.827Z)

File created successfully at: C:\Users\avidu\Projects\muneem\src\muneem\parsers\profiles.py
(file state is current in your context ? no need to Read it back)

123. assistant (2026-05-31T08:47:11.079Z)

Now migrate ICICI onto the engine and wire HDFC's account extraction to its profile (profile-as-data, behavior-identical). First, rewrite `icici.py`:

124. assistant (2026-05-31T08:47:24.795Z)

```

"""ICICI Bank statement parser (transaction / savings account).

```

```

Row-building runs through the **generic concept-mapping engine** (`muneem.parsers.engine`)
driven
by `ICICI_PROFILE` ? the engine maps each extracted table's columns onto the canonical
concepts
(header lexicon, else ICICI's known column order), so the parser is now mostly *data*. Parsing
is
still split from the DB insert and the parse is **reconciled** before we trust it
(`documents.status` records `reconciled` / `unreconciled` from a running-balance walk), so
a
silent `extract_tables()` row-drop surfaces instead of passing quietly. No ICICI statement is
in
the corpus yet, so this is synthetic-tested; the `local_statements` path will validate it
against a
real one when available.
"""

```

```

from __future__ import annotations

```

```

import sqlite3
from dataclasses import dataclass
from pathlib import Path

```

```

import pdfplumber

```

```

from muneem import db as dbmod

from . import register_parser
from .base import StatementParser
from .engine import map_table
from .profiles import ICICI_PROFILE
from .validation import parse_amount, reconcile_running_balance

@dataclass
class ICICITxn:
    txn_date: str
    mode: str
    particulars: str
    withdrawals: float | None
    deposits: float | None
    balance: float | None

def _rows_from_icici_tables(tables: list) -> list[ICICITxn]:
    """Build transactions from extracted tables via the generic engine. Pure: no PDF, no DB.

    The engine maps columns onto the canonical concepts; ICICI's ``mode`` column rides along in
    the
    mapping. Reconciliation happens in :meth:`ICICIParser.parse_and_insert`, so this stays
    oracle-free (pure row-building). Opening/closing-balance and non-transaction rows are
    dropped by
    the engine (no date / balance marker).
    """
    rows: list[ICICITxn] = []
    for table in tables or []:
        result = map_table(table, ICICI_PROFILE)
        if result is None:
            continue
        for m in result.mapped:
            rows.append(
                ICICITxn(
                    txn_date=m.get("txn_date", ""),
                    mode=m.get("mode", ""),
                    particulars=m.get("narration", ""),
                    withdrawals=parse_amount(m.get("debit")),
                    deposits=parse_amount(m.get("credit")),
                    balance=parse_amount(m.get("balance")),
                )
            )
    return rows

def parse_icici(pdf_path: Path, password: str | None = None) -> tuple[str, list[ICICITxn]]:
    """Open an ICICI statement and return (account_number, transactions)."""
    account_number = "UNKNOWN"
    rows: list[ICICITxn] = []
    with pdfplumber.open(pdf_path, password=password) as pdf:
        head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))
        account_number = ICICI_PROFILE.first_account(head) or "UNKNOWN"
        for page in pdf.pages:
            rows.extend(_rows_from_icici_tables(page.extract_tables()))
    return account_number, rows

@register_parser
class ICICIParser(StatementParser):
    def can_handle(self, text: str, sender: str | None = None, subject: str | None = None) ->
bool:

```

```

        if sender and "icicibank.com" in sender.lower():
            return True
        return "ICICI Bank" in text and "Statement of Transactions" in text

def parse_and_insert(
    self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str | None = None
) -> int:
    try:
        account_number, rows = parse_icici(pdf_path, password)
    except Exception as exc:
        raise RuntimeError(f"Failed to parse ICICI statement {pdf_path}: {exc}") from exc
    for t in rows:
        dbmod.insert_icici_transaction(
            conn,
            document_id=doc_id,
            account_number=account_number,
            txn_date=t.txn_date,
            mode=t.mode,
            particulars=t.particulars,
            withdrawals=t.withdrawals,
            deposits=t.deposits,
            balance=t.balance,
        )
    report = reconcile_running_balance([(t.withdrawals, t.deposits, t.balance) for t in
rows])
    conn.execute(
        "UPDATE documents SET status = ? WHERE id = ?",
        ("reconciled" if report.ok else "unreconciled", doc_id),
    )
    return len(rows)

```

125. user (2026-05-31T08:47:24.811Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\icici.py has been updated successfully. (file state is current in your context ? no need to Read it back)

126. assistant (2026-05-31T08:47:27.251Z)

Now wire HDFC's account extraction to its profile (behavior-identical):

127. user (2026-05-31T08:47:30.049Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\hdfc.py has been updated successfully. (file state is current in your context ? no need to Read it back)

128. user (2026-05-31T08:47:34.496Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\hdfc.py has been updated successfully. (file state is current in your context ? no need to Read it back)

129. user (2026-05-31T08:47:38.575Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\hdfc.py has been updated successfully. (file state is current in your context ? no need to Read it back)

130. assistant (2026-05-31T08:47:45.718Z)

Let me lint and run the suite to verify the engine + ICICI migration before writing new engine tests:

131. user (2026-05-31T08:47:47.904Z)

Exit code 1
2 files reformatted, 2 files left unchanged

```

E501 Line too long (102 > 100)
--> src\muneem\parsers\engine.py:4:101
|
3 | This is the *generic-first* half of the concept-core architecture (see
4 | [`docs/parser-architecture.md`](../../docs/parser-architecture.md) §4, §6): bank-agnostic
code that
|
^^
5 | takes a **cell-matrix** (the shape `pdfplumber.extract_tables()` returns, or what a
coordinate-
6 | clustering extractor emits) and maps its columns onto the canonical concepts via a cheap-
first
|

```

```

E501 Line too long (102 > 100)
--> src\muneem\parsers\engine.py:12:101
|
10 | * **profile-order.** If there is no header, fall back to the profile's known column order
(a prior).
11 | * **L1/L2 ? content + search.** Classify columns by content (dates, money), then enumerate
the
12 | plausible amount-column ? {debit, credit, balance} assignments and **keep the one the
reconciliation
|
^^
13 | oracle endorses** ? the self-identifying balance column, found by *which mapping makes
the math work*
14 | rather than by its label.
|

```

```

E501 Line too long (103 > 100)
--> src\muneem\parsers\engine.py:13:101
|
11 | * **L1/L2 ? content + search.** Classify columns by content (dates, money), then enumerate
the
12 | plausible amount-column ? {debit, credit, balance} assignments and **keep the one the
reconciliation
13 | oracle endorses** ? the self-identifying balance column, found by *which mapping makes
the math work*
|
^^^
14 | rather than by its label.
|

```

```

E501 Line too long (102 > 100)
--> src\muneem\parsers\engine.py:185:101
|
184 | def _alias_index(profile: Profile, family: FamilySpec) -> dict[str, str]:
185 |     """Build ``normalized-label -> concept``. Profile entries win over family defaults on
conflict."""
|
^^
186 |     index: dict[str, str] = {}
187 |     for concept, labels in family.default_aliases.items():
|

```

```

E501 Line too long (101 > 100)
--> src\muneem\parsers\icici.py:9:101
|
7 | (`documents.status` records `reconciled` / `unreconciled` from a running-balance
walk), so a
8 | silent `extract_tables()` row-drop surfaces instead of passing quietly. No ICICI
statement is in
9 | the corpus yet, so this is synthetic-tested; the `local_statements` path will validate it
against a

```

```
|
^
10 | real one when available.
11 | ""
|
```

Found 5 errors.

132. assistant (2026-05-31T08:48:24.908Z)

Just E501 in docstrings/comments. Let me trim the long lines:

133. user (2026-05-31T08:48:30.628Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\engine.py has been updated successfully. (file state is current in your context ? no need to Read it back)

134. user (2026-05-31T08:48:34.721Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\engine.py has been updated successfully. (file state is current in your context ? no need to Read it back)

135. user (2026-05-31T08:48:38.828Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\icici.py has been updated successfully. (file state is current in your context ? no need to Read it back)

136. user (2026-05-31T08:48:49.310Z)

```
All checks passed!
--- pytest (offline) ---
.....X..... [ 70%]
..... [100%]
===== XPASSES =====
===== short test summary info =====
XPASS tests/test_ocr.py::test_swiggy_image_code_recovered_fuzzily - recovery of the stylized
boxed code is OCR-engine/version dependent ? succeeds on Tesseract 5.4, returns nothing on the
apt build; the planned fix is a purpose-trained model (docs/DEPENDENCIES.md). strict=False so it
never fails the build.
101 passed, 9 deselected, 1 xpassed in 6.35s
```

137. assistant (2026-05-31T08:50:27.166Z)

101 green ? ICICI migration and the HDFC profile preserved all behavior. But before writing tests I need to fix a real flaw: my L2 search and `map\_table` gate hardcode `debit`/`credit`/`balance`, which ****breaks the family-agnostic claim****. Let me read the current state and generalize those over `FamilySpec`.

138. user (2026-05-31T08:50:28.188Z)

```
98     @dataclass
99     class FamilySpec:
100         """A concept *family*: the target vocabulary + how to build typed rows from a column
mapping.
101
102         The engine's L0/L1/L2 logic is written against this spec rather than against bank
concepts, so a
103         future instrument family (with its own concepts + reconciliation oracle) reuses the
same engine.
104         ``build`` turns per-row ``{concept: raw_cell}`` dicts into typed rows the oracle
understands.
105         """
106
107         name: str
```

```

108     concepts: tuple[str, ...]
109     date_concepts: tuple[str, ...]
110     amount_concepts: tuple[str, ...]
111     balance_concept: str | None
112     default_aliases: dict[str, tuple[str, ...]]
113     build: Callable[[list[dict[str, str]]], list]
114
115
116     def _build_bank_rows(mapped: list[dict[str, str]]) -> list[ConceptRow]:
117         """Turn mapped cells into :class:`ConceptRow`s (amounts parsed; absent stays
118         ``None``)."""
119         rows: list[ConceptRow] = []
120         for m in mapped:
121             rows.append(
122                 ConceptRow(
123                     txn_date=_text(m.get("txn_date")),
124                     narration=_text(m.get("narration")),
125                     debit=parse_amount(m.get("debit")),
126                     credit=parse_amount(m.get("credit")),
127                     balance=parse_amount(m.get("balance")),
128                     value_date=_text(m.get("value_date")) or None,
129                     reference=_text(m.get("reference")) or None,
130                 )
131             )
132         return rows
133
134     BANK_FAMILY = FamilySpec(
135         name="bank",
136         concepts=("txn_date", "value_date", "narration", "reference", "debit", "credit",
137 "balance"),
138         date_concepts=("txn_date", "value_date"),
139         amount_concepts=("debit", "credit", "balance"),
140         balance_concept="balance",
141         default_aliases=_BANK_ALIASES,
142         build=_build_bank_rows,
143     )
144
145     @dataclass
146     class MappingResult:
147         """The outcome of mapping one table: typed rows + how we got there (provenance)."""
148
149         rows: list # typed rows from family.build (e.g. list[ConceptRow])
150         mapped: list[
151             dict[str, str]
152         ] # per-row {concept: raw cell}, incl. profile extras (e.g. ICICI mode)
153         column_map: dict[str, int] # the winning concept -> column-index assignment
154         layer: str # "L0" / "profile-order" / "L2" ? which rung produced it
155         verified: bool # did the oracle endorse it? (False if no oracle was supplied)
156
157

```

139. user (2026-05-31T08:50:31.708Z)

```

236     cells = [_text(r[c]) for r in data if c < len(r) and _text(r[c])]
237     if not cells:
238         kinds[c] = "empty"
239         continue
240     thresh = max(1, int(len(cells) * 0.6))
241     if sum(1 for x in cells if _matches_date(x, patterns)) >= thresh:
242         kinds[c] = "date"
243     elif sum(1 for x in cells if _looks_like_money(x)) >= thresh:
244         kinds[c] = "amount"
245     else:

```

```

246         kinds[c] = "text"
247     return kinds
248
249
250     def _base_map(col_kinds: dict[int, str], data: list[list[Cell]]) -> dict[str, int]:
251         """Date + narration columns from content (the non-ambiguous part of an L1
mapping)."""
252         base: dict[str, int] = {}
253         date_cols = [c for c, k in sorted(col_kinds.items()) if k == "date"]
254         text_cols = [c for c, k in sorted(col_kinds.items()) if k == "text"]
255         if date_cols:
256             base["txn_date"] = date_cols[0]
257         if len(date_cols) > 1:
258             base["value_date"] = date_cols[1]
259         if text_cols:
260             base["narration"] = max(
261                 text_cols, key=lambda c: sum(len(_text(r[c])) for r in data if c < len(r))
262             )
263         return base
264
265
266     def _search_candidates(
267         col_kinds: dict[int, str], data: list[list[Cell]], family: FamilySpec
268     ) -> list[tuple[str, dict[str, int]]]:
269         """Enumerate plausible amount-column assignments (the L2 search space)."""
270         base = _base_map(col_kinds, data)
271         amount_cols = [c for c, k in sorted(col_kinds.items()) if k == "amount"]
272         cands: list[dict[str, int]] = []
273         if family.balance_concept and len(amount_cols) >= 3:
274             for bal in amount_cols:
275                 rest = [c for c in amount_cols if c != bal]
276                 for d, cr in itertools.permutations(rest, 2):
277                     cands.append(**base, "debit": d, "credit": cr, family.balance_concept:
bal})
278         if len(amount_cols) >= 2: # no per-row balance (total-reconcile families, e.g. Axis
Oct)
279             for d, cr in itertools.permutations(amount_cols, 2):
280                 cands.append(**base, "debit": d, "credit": cr)
281         return [("L2", c) for c in cands[:24]]
282
283
284     def _apply_map(data: list[list[Cell]], col_map: dict[str, int]) -> list[dict[str, str]]:
285         out: list[dict[str, str]] = []
286         for row in data:
287             out.append({c: (_text(row[i]) if i < len(row) else "") for c, i in
col_map.items()})
288         return out
289
290
291     def map_table(
292         table: list[Row] | None,
293         profile: Profile,
294         family: FamilySpec = BANK_FAMILY,
295         *,
296         oracle: Callable[[list], bool] | None = None,
297     ) -> MappingResult | None:
298         """Map one extracted table onto ``family``'s concepts; ``oracle`` selects among
candidates.
299
300         Resolution order: header lexicon (L0) ? profile column order ? content search (L2).
With an
301         ``oracle`` the first candidate it endorses wins (``verified=True``); without one,
the first
302         structurally-complete candidate is returned unverified (the caller reconciles
separately).

```

```

303         Returns ``None`` if no transaction rows are found, or the best unverified candidate
if an oracle
304         was supplied but nothing reconciled.
305         """
306         if not table:
307             return None
308         rows_in = [list(r) for r in table if r is not None]
309         if not rows_in:
310             return None
311         alias_index = _alias_index(profile, family)
312         patterns = _date_patterns(profile)
313
314         header_idx, col_map_hdr = _detect_header(rows_in, alias_index)
315         start = header_idx + 1 if header_idx is not None else 0
316         date_col_hint = col_map_hdr.get("txn_date")
317         if date_col_hint is None and profile.column_order and "txn_date" in
profile.column_order:
318             date_col_hint = profile.column_order.index("txn_date")
319
320         data = [
321             r
322             for r in rows_in[start:]
323             if not _is_balance_marker(r) and _is_transaction_row(r, date_col_hint, patterns)
324         ]
325         if not data:
326             return None
327         ncols = max(len(r) for r in data)
328
329         candidates: list[tuple[str, dict[str, int]]] = []
330         if "debit" in col_map_hdr and "credit" in col_map_hdr: # L0: header pinned the
amounts
331             col_kinds = _classify_columns(data, ncols, patterns)
332             candidates.append(("L0", {**_base_map(col_kinds, data), **col_map_hdr}))
333         elif col_map_hdr: # header present but didn't pin amounts: keep its hints, search
the rest
334             candidates.append(("L0", col_map_hdr))
335         if profile.column_order: # positional prior (no/partial header)
336             candidates.append(("profile-order", {c: i for i, c in
enumerate(profile.column_order)}))
337         candidates += _search_candidates(_classify_columns(data, ncols, patterns), data,
family)
338
339         best: MappingResult | None = None
340         for layer, cmap in candidates:
341             if "txn_date" not in cmap or not ({ "debit", "credit", "balance" } & cmap.keys()):
342                 continue
343             mapped = _apply_map(data, cmap)
344             typed = family.build(mapped)
345             if oracle is None:
346                 return MappingResult(typed, mapped, cmap, layer, verified=False)
347             if oracle(typed):
348                 return MappingResult(typed, mapped, cmap, layer, verified=True)
349             if best is None:
350                 best = MappingResult(typed, mapped, cmap, layer, verified=False)
351         return best
352
353
354     __all__ = ["Profile", "FamilySpec", "BANK_FAMILY", "MappingResult", "map_table"]
355

```

140. assistant (2026-05-31T08:51:08.480Z)

Now I'll generalize the engine over `FamilySpec` so L2 and the gate aren't bank-bound. Five edits:

141. user (2026-05-31T08:51:12.303Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\engine.py has been updated successfully. (file state is current in your context ? no need to Read it back)

142. user (2026-05-31T08:51:19.536Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\engine.py has been updated successfully. (file state is current in your context ? no need to Read it back)

143. user (2026-05-31T08:51:35.404Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\engine.py has been updated successfully. (file state is current in your context ? no need to Read it back)

144. user (2026-05-31T08:51:47.674Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\engine.py has been updated successfully. (file state is current in your context ? no need to Read it back)

145. assistant (2026-05-31T08:52:57.357Z)

Engine is now genuinely family-parameterized. Let me update task status and write the engine tests (including a non-bank family that proves the seam):

146. user (2026-05-31T08:52:57.836Z)

Updated task #1 status

147. user (2026-05-31T08:52:58.308Z)

Updated task #2 status

148. user (2026-05-31T08:52:58.780Z)

Updated task #3 status

149. assistant (2026-05-31T08:53:21.272Z)

```
"""Tests for the generic concept-mapping engine (muneem.parsers.engine).
```

```
Exercises the ladder on synthetic cell-matrices (no PDF/DB): L0 header lexicon, the profile-
order
fallback, L1/L2 content search with the oracle picking the self-identifying balance column, and
a
deliberately non-bank ``FamilySpec`` to prove the engine is family-agnostic.
"""
```

```
import re
from dataclasses import dataclass
```

```
from muneem.parsers.engine import BANK_FAMILY, FamilySpec, Profile, map_table
from muneem.parsers.validation import parse_amount, reconcile_running_balance
```

```
DMY = (re.compile(r"^\d{2}/\d{2}/\d{4}$"),)
PROFILE = Profile(name="t", date_patterns=DMY)
```

```
def _walk(rows):
    return reconcile_running_balance([(r.debit, r.credit, r.balance) for r in rows]).ok
```

--- L0: header lexicon (generic labels, no custom aliases needed) -----

```

def test_l0_header_lexicon_maps_generic_labels():
    table = [
        ["Date", "Particulars", "Withdrawals", "Deposits", "Balance"],
        ["01/01/2026", "Opening Balance", "", "", "1000.00"], # marker -> skipped
        ["02/01/2026", "UPI Amazon", "250.00", "", "750.00"],
        ["03/01/2026", "Salary", "", "1250.00", "2000.00"],
    ]
    r = map_table(table, PROFILE, BANK_FAMILY)
    assert r is not None and r.layer == "L0"
    assert [row.txn_date for row in r.rows] == ["02/01/2026", "03/01/2026"]
    assert r.rows[0].debit == 250.0 and r.rows[0].balance == 750.0
    assert r.rows[1].credit == 1250.0 and r.rows[1].balance == 2000.0
    assert r.column_map["balance"] == 4

def test_l1_detects_value_date_as_second_date_column():
    table = [
        ["Tran Date", "Value Date", "Particulars", "Debit", "Credit", "Balance"],
        ["02/01/2026", "01/01/2026", "UPI", "250.00", "", "750.00"],
        ["03/01/2026", "02/01/2026", "Salary", "", "1250.00", "2000.00"],
    ]
    r = map_table(table, PROFILE, BANK_FAMILY)
    assert r is not None
    assert r.column_map["txn_date"] == 0 and r.column_map["value_date"] == 1
    assert r.rows[0].value_date == "01/01/2026"

```

--- profile-order: positional prior when there is no header -----

```

def test_profile_order_fallback_when_no_header():
    p = Profile(
        name="t",
        date_patterns=DMY,
        column_order=("txn_date", "narration", "debit", "credit", "balance"),
    )
    table = [
        ["02/01/2026", "UPI", "250.00", "", "750.00"],
        ["03/01/2026", "Salary", "", "1250.00", "2000.00"],
    ]
    r = map_table(table, p, BANK_FAMILY)
    assert r is not None and r.layer == "profile-order"
    assert r.rows[0].debit == 250.0 and r.rows[1].credit == 1250.0

```

--- L2: the self-identifying balance column, found by the oracle, not by a label -----

```

def test_l2_search_finds_self_identifying_balance_via_oracle():
    # Unusual order, no header, no column_order: balance sits BEFORE the flows. Only the
    assignment
    # whose running balance walks cleanly should survive ? the oracle picks it.
    table = [
        ["02/01/2026", "UPI", "750.00", "250.00", ""], # date, narr, BALANCE, DEBIT, CREDIT
        ["03/01/2026", "Salary", "2000.00", "", "1250.00"],
    ]
    r = map_table(table, PROFILE, BANK_FAMILY, oracle=walk)
    assert r is not None and r.verified and r.layer == "L2"
    assert r.column_map["balance"] == 2
    assert r.rows[0].balance == 750.0 and r.rows[0].debit == 250.0
    assert r.rows[1].credit == 1250.0 and r.rows[1].balance == 2000.0

```

```

def test_returns_unverified_when_nothing_reconciles():
    # The Axis-style safety path: with an oracle, a non-reconciling parse comes back
    verified=False
    # so the caller can refuse to persist it.
    table = [
        ["02/01/2026", "UPI", "250.00", "", "750.00"],
        ["03/01/2026", "Salary", "", "1250.00", "9999.00"], # balance broken
    ]
    r = map_table(table, PROFILE, BANK_FAMILY, oracle=_walk)
    assert r is not None and not r.verified

def test_returns_none_when_no_transaction_rows():
    table = [["Date", "Particulars", "Balance"], ["x", "y", "z"]]
    assert map_table(table, PROFILE, BANK_FAMILY) is None

```

--- family-agnostic seam: the same engine, a different concept family + oracle -----

```

@dataclass
class PointsRow:
    txn_date: str
    earned: float | None
    redeemed: float | None
    balance: float | None

def _build_points(mapped):
    return [
        PointsRow(
            txn_date=m.get("txn_date", ""),
            earned=parse_amount(m.get("earned")),
            redeemed=parse_amount(m.get("redeemed")),
            balance=parse_amount(m.get("balance")),
        )
        for m in mapped
    ]

POINTS_FAMILY = FamilySpec(
    name="points",
    concepts=("txn_date", "earned", "redeemed", "balance"),
    date_concepts=("txn_date",),
    amount_concepts=("earned", "redeemed", "balance"),
    balance_concept="balance",
    text_concepts=(),
    default_aliases={
        "txn_date": ("date",),
        "earned": ("earned", "points in"),
        "redeemed": ("redeemed", "points out"),
        "balance": ("balance", "points balance"),
    },
    build=_build_points,
)

def _points_walk(rows):
    prev, compared = None, 0
    for r in rows:
        if r.balance is None:
            continue
        if prev is not None:
            compared += 1

```


152. assistant (2026-05-31T08:53:32.268Z)

`zip` needs explicit `strict=`. Fixing the three:

153. user (2026-05-31T08:53:33.643Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\engine.py has been updated successfully. (file state is current in your context ? no need to Read it back)

154. user (2026-05-31T08:53:40.919Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\engine.py has been updated successfully. (file state is current in your context ? no need to Read it back)

155. user (2026-05-31T08:53:51.841Z)

All checks passed!

--- full offline suite ---

```
.....X..... [ 66%]
..... [100%]
===== XPASSES =====
===== short test summary info =====
XPASS tests/test_ocr.py::test_swiggy_image_code_recovered_fuzzily - recovery of the stylized
boxed code is OCR-engine/version dependent ? succeeds on Tesseract 5.4, returns nothing on the
apt build; the planned fix is a purpose-trained model (docs/DEPENDENCIES.md). strict=False so it
never fails the build.
108 passed, 9 deselected, 1 xpassed in 6.29s
```

156. assistant (2026-05-31T08:54:09.541Z)

108 passed (101 + 7 new), lint clean ? **Step 2 is functionally complete.** Let me update the docs before shipping. Reading the current state of the relevant sections:

157. user (2026-05-31T08:54:10.643Z)

```
126     storage twice.
127
128     ## 10. Implementation plan (incremental, merge-as-we-go)
129
130     1. Core extraction ? landed. validation.py is now the verifier: the per-row
walk, total
131     reconcile (opening + ?credit ? ?debit == closing), the concept-level bookend
(verify),
132     candidate selection (select_reconciling), and a recorded confidence verdict
133     (Confidence). The concept schema lives in concepts.py (ConceptRow,
StatementConcepts).
134     HDFC is refactored onto it as the reference implementation ? it maps its rows
onto the
135     concepts and verifies through the shared verifier, with no behavior change (every
existing
136     test passes; the relocated bookend logic is identical).
137     2. Concept-mapping engine (L0/L1/L2) as generic, bank-agnostic code; express HDFC +
ICICI as
138     profiles (data) on top of it.
139     3. Axis clustering rewrite onto the core ? this is the must-fix and the first real
test of
140     "generic-first": coordinate clustering ? concept mapping ? total reconcile. Validates
on Jun + Oct
141     2025 + future Axis files.
142     4. L3 local-vision fallback (after the research): a verified extractor that only
fires when the
143     deterministic ladder fails to reconcile, with the number-traps from §7.
144     5. Unify storage (§9) + begin Tier-2 transaction semantics.
145
```

146 Each step is a green, reversible PR; the verifier guarantees we never regress into
storing garbage.
147
148 ## 11. Open questions (for discussion)
149
150 - **Storage unification timing** ? keep per-bank tables until the concept core is
proven, then migrate?
151 - **Generic-vs-profile balance** ? how much to lean on profiles as priors vs. pure
content inference.
152 - **L3 scope** ? once the research lands: which local VLM, and is it a rare fallback or
a routine
153 cross-check on every statement?
154 - **Tier-2** ? when do transaction classification + merchant canonicalization become in-
scope?
155 - **Confidence policy** ? what does the CLI/ledger do with a `quarantined` statement
(skip, show with

158. user (2026-05-31T08:54:17.305Z)

The file C:\Users\avidu\Projects\muneem\docs\parser-architecture.md has been updated
successfully. (file state is current in your context ? no need to Read it back)

159. user (2026-05-31T08:54:21.535Z)

The file C:\Users\avidu\Projects\muneem\docs\parser-architecture.md has been updated
successfully. (file state is current in your context ? no need to Read it back)

160. user (2026-05-31T08:54:28.391Z)

The file C:\Users\avidu\Projects\muneem\docs\parser-architecture.md has been updated
successfully. (file state is current in your context ? no need to Read it back)

161. user (2026-05-31T08:54:32.963Z)

16 ## 2. Statement Parsers (Track A + C progress)
17 Deterministic parsers live under `muneem.parsers`, registered via decorator and selected
by `get\_parser` on text heuristics + sender. Submodules are **auto-loaded** on package import
(`\_autoload\_parsers`), so the registry is populated in the production path (not just under
tests).
18
19 * **HDFC Bank** (`hdfc.py`): Savings / transaction-account statements via **coordinate**
clustering (deliberately **not** `extract\_tables` ? HDFC's table is semi-ruled). Extracts
date, narration, value date, ref, withdrawal, deposit, closing balance. **Self-validating**:
balance reconciliation + a cross-check against the bank's own end-of-statement SUMMARY (Dr/Cr
counts, opening/closing balance); records `documents.status` = `reconciled`/`unreconciled`.
Verified complete on real statements (37 & 31 txns) without inspecting any value.
20 * **Axis Bank** (`axis.py`): Savings (`axis\_transactions`) + Credit Card
(`axis\_cc\_transactions`). ?? The savings `extract\_tables` mapping is **unreliable** on real Axis
layouts ? column counts and date placement vary between statements (e.g. Jun vs Oct 2025), so
it can mislabel rows. Reconciliation catches this, so as a **stop-gap** the savings parser refuses
to store rows that don't reconcile (marks the document `unreconciled`, inserts 0) instead of
persisting garbage. **Pending real fix**: an HDFC-style coordinate-clustering rewrite validated
by total reconciliation (`opening` + ?Deposits ? ?Withdrawals == closing). Credit-card has no
running balance (parse/persist split + count only).
21 * **ICICI Bank** (`icici.py`): Transaction / savings statements (`icici\_transactions`);
reconciles + records status.
22
23 All balance-bearing parsers share one **verifier** (`muneem.parsers.validation`) and
split parsing from DB-insert, so the row logic is unit-testable without a PDF/DB and a parse is
validated before it's trusted. **The concept-core rework** (parser-architecture.md step 1) has
landed: the verifier now spans the per-row walk + total reconcile (`opening` + ?Cr ? ?Dr ==
closing) + a concept-level bookend + a recorded confidence verdict, all targeting a bank-
agnostic **concept schema** (`muneem.parsers.concepts`); **HDFC** is refactored onto it as the
reference implementation with no behavior change. Axis/ICICI migrate onto the core next

(express as profiles ? step 2). Parsers use `pdfplumber` (with optional password); HDFC parses by word-position clustering, Axis/ICICI by `extract\_tables()` (reconciliation will flag it if that drops rows on a real statement). ****Password resolution**** (password_rules + hybrid unlock) is ****not yet wired**** into the main `muneem ingest --doc-type bank\_statement` path; it is exercised only via the `@pytest.mark.local\_statements` test in `tests/parsers/test\_local\_axis.py`. Unlock hybrid logic exists in `unlock.py` (pypdf + candidates/prompt) and is tested.

24

25 See `password\_rules.py` (now prefers user config dir) and `cli.py:ingest`.

26

27 `## 3. Database & CLI`

162. user (2026-05-31T08:54:42.397Z)

The file C:\Users\avidu\Projects\muneem\docs\CURRENT_STATE.md has been updated successfully.
(file state is current in your context ? no need to Read it back)

163. user (2026-05-31T08:54:47.886Z)

The file C:\Users\avidu\Projects\muneem\docs\CURRENT_STATE.md has been updated successfully.
(file state is current in your context ? no need to Read it back)

164. assistant (2026-05-31T08:54:55.570Z)

Docs updated. Final verification, then ship Step 2:

165. user (2026-05-31T08:55:03.578Z)

All checks passed!

```
===== XPASSES =====
===== short test summary info =====
XPASS tests/test_ocr.py::test_swiggy_image_code_recovered_fuzzily - recovery of the stylized
boxed code is OCR-engine/version dependent ? succeeds on Tesseract 5.4, returns nothing on the
apt build; the planned fix is a purpose-trained model (docs/DEPENDENCIES.md). strict=False so it
never fails the build.
108 passed, 9 deselected, 1 xpassed in 6.03s
--- status ---
M docs/CURRENT_STATE.md
M docs/parser-architecture.md
M src/muneem/parsers/hdfc.py
M src/muneem/parsers/icici.py
?? src/muneem/parsers/engine.py
?? src/muneem/parsers/profiles.py
?? tests/parsers/test_engine.py
```

166. user (2026-05-31T08:55:15.057Z)

```
[feat/concept-mapping-engine c981861] feat(parsers): generic concept-mapping engine (L0/L1/L2) +
profiles; ICICI runs through it
7 files changed, 635 insertions(+), 49 deletions(-)
create mode 100644 src/muneem/parsers/engine.py
create mode 100644 src/muneem/parsers/profiles.py
create mode 100644 tests/parsers/test_engine.py
```

167. user (2026-05-31T08:55:21.878Z)

branch 'feat/concept-mapping-engine' set up to track 'origin/feat/concept-mapping-engine'.
To <https://github.com/avidullu/muneem.git>

```
* [new branch] feat/concept-mapping-engine -> feat/concept-mapping-engine
```

168. user (2026-05-31T08:55:36.894Z)

<https://github.com/avidullu/muneem/pull/23>

169. assistant (2026-05-31T08:55:41.473Z)

PR [muneem#23](https://github.com/avidullu/muneem/pull/23) up. Verified locally (108 green, lint clean, no behavior change), so squash-merging per cadence and syncing `main`:

170. user (2026-05-31T08:55:49.120Z)

```
create mode 100644 src/muneem/parsers/engine.py
create mode 100644 src/muneem/parsers/profiles.py
create mode 100644 tests/parsers/test_engine.py
Already on 'main'
From https://github.com/avidullu/muneem
- [deleted]      (none)    -> origin/feat/concept-mapping-engine
Already up to date.
7ea7f7a feat(parsers): generic concept-mapping engine (L0/L1/L2) + profiles; ICICI runs through it (#23)
7555fb2 feat(parsers): concept-core verifier + schema; HDFC refactored onto it (no behavior change) (#22)
```

171. assistant (2026-05-31T08:59:58.867Z)

Step 2 merged as [muneem#23](https://github.com/avidullu/muneem/pull/23). Now ****Step 3 ? the Axis must-fix.**** Setting up:

172. user (2026-05-31T08:59:59.820Z)

Updated task #3 status

173. user (2026-05-31T09:00:00.279Z)

Updated task #4 status

174. user (2026-05-31T09:00:01.451Z)

Switched to a new branch 'feat/axis-coordinate-clustering'
feat/axis-coordinate-clustering

175. user (2026-05-31T09:00:02.104Z)

Chapter marked: "Step 3 ? Axis rewrite". Continue your current work.

176. user (2026-05-31T09:00:26.299Z)

File created successfully at: C:\Users\avidu\Projects\muneem\src\muneem\parsers\clustering.py
(file state is current in your context ? no need to Read it back)

177. assistant (2026-05-31T09:00:09.455Z)

The Axis design (flagging it since I'm building partly blind ? I can't decrypt the real PDFs): ****multi-candidate extraction****. I produce ***both*** a coordinate-clustered matrix and the ``extract_tables()`` matrix, feed each through the engine, and keep whichever the ****total-reconcile oracle**** endorses. This is safe by construction ? if neither reconciles, it stores nothing (same as today's stop-gap), so it can never persist garbage; the real-data run reveals actual coverage.

Writing the reusable coordinate-clustering module first:

178. assistant (2026-05-31T09:00:26.288Z)

```
"""Coordinate clustering ? turn positioned words into a clean cell-matrix.
```

```
When ``pdfplumber.extract_tables()`` mis-groups a semi-ruled or whitespace-aligned statement (the
```


Axis case: a date merged into the narration cell, column counts drifting between vintages), we fall back to reading **word positions** directly. Words are grouped into visual rows by their `top`, column boundaries are inferred from the vertical whitespace shared across all rows, and each word is dropped into the column its x-centre falls in. The result is a rectangular cell-matrix the generic `engine` can map ? and because columns come from geometry, a leading date sits in its *own* column instead of being glued to the narration.

This is bank-agnostic extraction (no issuer constants, no x-positions hard-coded); the engine's oracle still decides whether the result is trustworthy. Pure geometry in, strings out ? no value is logged.

```

from __future__ import annotations

from collections.abc import Sequence

Word = dict # a pdfplumber word: {"text", "x0", "x1", "top", "bottom", ...}

def group_lines(words: Sequence[Word], tol: float = 2.5) -> list[list[Word]]:
    """Group words into visual lines by `top` (tolerant of sub-pixel jitter), left-to-right."""
    lines: list[list[Word]] = []
    anchor_top: float | None = None
    for w in sorted(words, key=lambda w: (w["top"], w["x0"])):
        if lines and anchor_top is not None and abs(w["top"] - anchor_top) <= tol:
            lines[-1].append(w)
        else:
            lines.append([w])
            anchor_top = w["top"]
    return [sorted(line, key=lambda w: w["x0"]) for line in lines]

def infer_columns(words: Sequence[Word], gap: float = 10.0) -> list[tuple[float, float]]:
    """Infer column x-extents by merging word x-intervals separated by less than `gap` points.

    The gaps wider than `gap` are the inter-column whitespace; everything else coalesces into
    a
    column. Returns `(x_start, x_end)` per column, left to right.
    """
    intervals = sorted((float(w["x0"]), float(w["x1"])) for w in words)
    merged: list[list[float]] = []
    for x0, x1 in intervals:
        if merged and x0 <= merged[-1][1] + gap:
            merged[-1][1] = max(merged[-1][1], x1)
        else:
            merged.append([x0, x1])
    return [(a, b) for a, b in merged]

def _column_of(center: float, columns: list[tuple[float, float]]) -> int:
    for i, (a, b) in enumerate(columns):
        if a <= center <= b:
            return i
    return min(
        range(len(columns)),
        key=lambda i: min(abs(center - columns[i][0]), abs(center - columns[i][1])),
    )

```

```

def line_to_cells(line: Sequence[Word], columns: list[tuple[float, float]]) -> list[str]:
    """Place each word into its column; join words that share a column (in x order)."""
    cells = [""] for _ in columns
    for w in line:
        idx = _column_of((float(w["x0"]) + float(w["x1"])) / 2, columns)
        cells[idx] = (cells[idx] + " " + str(w["text"])).strip()
    return cells

def cluster_to_matrix(
    words_by_page: Sequence[Sequence[Word]], *, line_tol: float = 2.5, col_gap: float = 10.0
) -> list[list[str]]:
    """Cluster words from one or more pages into a single cell-matrix with consistent columns.

    Column boundaries are inferred **once across all pages** (a statement keeps its layout), so
    rows from every page land in the same columns; rows are emitted in page-then-top order.
    """
    all_words = [w for page in words_by_page for w in page]
    if not all_words:
        return []
    columns = infer_columns(all_words, col_gap)
    matrix: list[list[str]] = []
    for page in words_by_page:
        for line in group_lines(page, line_tol):
            matrix.append(line_to_cells(line, columns))
    return matrix

__all__ = ["group_lines", "infer_columns", "line_to_cells", "cluster_to_matrix"]

```

179. assistant (2026-05-31T09:00:29.108Z)

Now add `AXIS\_PROFILE` to profiles.py:

180. user (2026-05-31T09:00:38.929Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\profiles.py has been updated successfully. (file state is current in your context ? no need to Read it back)

181. user (2026-05-31T09:00:43.546Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\profiles.py has been updated successfully. (file state is current in your context ? no need to Read it back)

182. user (2026-05-31T09:00:50.387Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\profiles.py has been updated successfully. (file state is current in your context ? no need to Read it back)

183. user (2026-05-31T09:01:09.802Z)

[Request interrupted by user]

184. user (2026-05-31T09:01:09.810Z)

Can you make sure to update the README and relevant docs (CURRENT STATE, architecture etc.) with this new "total finance" view and not just banks?

185. assistant (2026-05-31T09:02:06.499Z)

Good call ? the "total finance" scope should be reflected in the docs, and right now they're bank-centric. That's a clean standalone work item, so I'll stash the in-progress Axis work, do the docs update as its own PR, then resume Axis. First, let me stash and read the two docs I

haven't seen yet (README, DESIGN):

186. user (2026-05-31T09:02:08.323Z)

```
M src/muneem/parsers/profiles.py
?? src/muneem/parsers/clustering.py
--- stashing Axis WIP ---
Saved working directory and index state On feat/axis-coordinate-clustering: axis-step3-wip
--- clean? ---
(clean if nothing above)
```

187. user (2026-05-31T09:02:08.347Z)

```
1      # muneem
2
3      **Your automated muneem ? a personal-finance bookkeeper that pulls financial documents
from your inbox, unlocks them, and builds a local consolidated ledger.**
4
5      In traditional Indian households a *muneem* (?????) was the trusted bookkeeper who
tracked every statement, bill, and transaction on the family's behalf. This project is the same
idea, automated and personal.
6
7      ## What it does (intended)
8
9      Reads Gmail and Outlook for financial documents (bank statements, credit-card bills,
brokerage statements, mutual-fund reports, utility bills, grocery and shopping receipts),
decrypts the password-protected ones, parses them into a normalized local database, and surfaces
insights:
10
11      - Grocery/shopping price-tracking across merchants over time
12      - Investment portfolio timeline and cross-broker view
13      - Bank/card performance comparison
14      - Catalog of purchased items with their bills (warranty, returns, age)
15      - Forgotten deals and coupons sitting unread in your inbox
16
17      ## Status
18
19      **Early implementation.** The design and plan are settled ? see
[`DESIGN.md`](./DESIGN.md) (architecture + threat model), [`docs/ROADMAP.md`](./docs/ROADMAP.md)
(milestones), and [`docs/TESTING.md`](./docs/TESTING.md) (test strategy). The Python project
skeleton, CLI, and offline CI are in place; build proceeds by the roadmap's milestones. Prior
art: [`docs/prior-art-email-mining.md`](./docs/prior-art-email-mining.md).
20
21      ## Getting started
22
23      Runs on Linux, macOS, and Windows. Requires **Python ? 3.11** and git.
24
25      ### Linux / macOS
26
27      ```bash
28      python3 -m venv .venv
29      source .venv/bin/activate
30      pip install -e ".[dev]"
31
32      muneem --version
33      muneem info
34      muneem ingest testdata/promo-emails/dataquest.pdf    # -> JS231sz
35      muneem offers list
36      ```
37
38      Or use the `Makefile`: `make install`, `make test`, `make lint`, `make run`.
39
40      ### Windows (PowerShell)
41
42      ```powershell
```

```

43     python -m venv .venv
44     .venv\Scripts\Activate.ps1
45     pip install -e ".[dev]"
46     muneem info
47     ```
48
49     ### Optional: OCR for codes inside banner images
50
51     The `--ocr` flag uses Tesseract (Apache-2.0, fully local ? no paid API):
52
53     | OS | Install |
54     |---|---|
55     | Debian / Ubuntu | `sudo apt-get install tesseract-ocr` |
56     | Fedora | `sudo dnf install tesseract` |
57     | macOS | `brew install tesseract` |
58     | Windows | `winget install UB-Mannheim.TesseractOCR` |
59
60     muneem finds it on `PATH`, at the usual install locations, or via the
61     `MUNEEM_TESSERACT_CMD` env var. Then `muneem ingest <pdf> --ocr` (run OCR tests with `make test-ocr`).
62
63     ### Notes
64
65     - Headless Linux: secret storage (used once Gmail/unlock land) needs a keyring
66     backend ? a desktop keyring (GNOME Keyring / KWallet), or `pip install keyrings.alt` for a file
67     backend. Not needed yet.
68     - CI runs the offline suite on Linux across Python 3.11?3.13 on every push.
69
70     ## Privacy
71
72     Single-user, fully local by design. All raw statement data stays on the machine. See
73     `DESIGN.md` § Security for the threat model and standing rules.
74
75

```

188. assistant (2026-05-31T09:05:31.244Z)

Several new directives ? let me reconcile them, with one important note first.

On the passwords: thank you ? but note they're now in this chat's transcript, which is a minor exposure. The durable path is the gitignored `password\_rules.yaml` or your OS keyring (the design already supports both). I'll use what you gave me **only** as transient env vars for local test runs ? never written to a file, never committed, never echoed back. The precision tests surface counts/booleans only, never statement content.

Here's how I'll reconcile everything (ICICI cleanup, Axis fix, AU Bank, total-finance docs, precision testing). First, let me gather facts: what statement data exists, scope the ICICI removal, and run an immediate **HDFC precision baseline** to validate the mechanism.

189. user (2026-05-31T09:05:35.771Z)

```

Switched to branch 'main'
--- statement data present (gitignored) ---
testdata/hdfc-statements: 4 pdf(s)
testdata/axis-statements: 4 pdf(s)
testdata/au-statements: absent
testdata/aubank-statements: 4 pdf(s)
testdata/icici-statements: absent
--- HDFC precision baseline on real statements (booleans/counts only) ---
    self.laparams = None if laparams is None else LAParams(**laparams)
    self.password = password
    self.unicode_norm = unicode_norm
    self.raise_unicode_errors = raise_unicode_errors

    try:
        self.doc = PDFDocument(PDFParser(stream), password=password or "")

```

```

except Exception as e:
>         raise PdfminerException(e)
E         pdfplumber.utils.exceptions.PdfminerException

.venv\Lib\site-packages\pdfplumber\pdf.py:52: PdfminerException
===== short test summary info =====
FAILED
tests/parsers/test_hdfc.py::test_real_hdfc_statement_validates[Avi_Dullu_28022026_183854522.pdf]
FAILED
tests/parsers/test_hdfc.py::test_real_hdfc_statement_validates[Avi_Dullu_30042026_001858675.pdf]
FAILED
tests/parsers/test_hdfc.py::test_real_hdfc_statement_validates[Avi_Dullu_30112025_110905081.pdf]
FAILED
tests/parsers/test_hdfc.py::test_real_hdfc_statement_validates[Avi_Dullu_31082025_205409615.pdf]
===== 4 failed, 16 deselected in 0.71s =====

```

190. user (2026-05-31T09:05:35.876Z)

```

Found 14 files
src\muneem\parsers\profiles.py
docs\CURRENT_STATE.md
docs\parser-architecture.md
src\muneem\parsers\engine.py
src\muneem\parsers\icici.py
tests\parsers\test_hdfc.py
tests\parsers\test_validation.py
tests\parsers\test_icici.py
src\muneem\db.py
src\muneem\cli.py
tests\parsers\test_registry.py
docs\DEPENDENCIES.md
DESIGN.md
CLAUDE.md

```

191. assistant (2026-05-31T09:06:21.818Z)

Two key findings: ****AU Bank data is at `testdata/aubank-statements` (4 PDFs)**, and the ****HDFC baseline failed at decryption**** (`PdfminerException` when opening with the password) ? not at parsing. That's the unlock layer: `pdfplumber`/`pdfminer`` may not handle this PDF's encryption even with the right password. The project has `unlock.py`` (pypdf) for exactly this, but it's not wired into the parse path.**

Let me diagnose whether the passwords are correct (via pypdf, which gives a clear verdict) and check the existing unlock code ? this determines the whole precision-testing path:

192. user (2026-05-31T09:06:26.068Z)

HDFC Avi_Dullu_28022026_183854522.pdf	ERROR FileNotDecryptedError: File has not been decrypted
HDFC Avi_Dullu_30042026_001858675.pdf	ERROR FileNotDecryptedError: File has not been decrypted
HDFC Avi_Dullu_30112025_110905081.pdf	ERROR FileNotDecryptedError: File has not been decrypted
HDFC Avi_Dullu_31082025_205409615.pdf	ERROR FileNotDecryptedError: File has not been decrypted
AXIS AXIS BANK Statement for August 2024.pdf	encrypted=True decrypt=2 pages=4
AXIS AXIS BANK Statement for June 2025.pdf	encrypted=True decrypt=2 pages=4
AXIS AXIS BANK Statement for May 2025.pdf	encrypted=True decrypt=2 pages=4
AXIS AXIS BANK Statement for October 2025.pdf	encrypted=True decrypt=2 pages=4
AU Statement_2024MTH07_XXXXX859_146.pdf	encrypted=True decrypt=1 pages=6
AU Statement_2026MTH01_XXXXX859_721.pdf	encrypted=True decrypt=1 pages=6
AU Statement_2026MTH03_XXXXX859_108.pdf	encrypted=True decrypt=1 pages=6
AU Statement_2026MTH04_XXXXX859_849.pdf	encrypted=True decrypt=1 pages=6

193. user (2026-05-31T09:06:26.085Z)

```
1      """Decrypt password-protected statement PDFs (Stage 2).
2
3      Uses pypdf (BSD-3-Clause) ? permissive, pure-Python. Implements the hybrid
4      password strategy from DESIGN.md § 8.2: try a candidate dictionary first, then
5      fall back to an interactive prompt. The working password is returned so the
6      caller can cache it in the OS keychain (muneem.secrets); **it is never logged**.
7      """
8
9      from __future__ import annotations
10
11      import shutil
12      from collections.abc import Callable, Iterable
13      from dataclasses import dataclass
14      from pathlib import Path
15
16      import pypdf
17
18
19      @dataclass
20      class UnlockResult:
21          ok: bool
22          method: str | None # "plaintext" | "candidate" | "prompt"
23          password: str | None # working password ? do NOT log; None if plaintext/failed
24          output_path: Path | None # decrypted copy, if a destination was given
25
26
27      def is_encrypted(path: Path | str) -> bool:
28          return pypdf.PdfReader(str(path)).is_encrypted
29
30
31      def password_works(path: Path | str, password: str) -> bool:
32          reader = pypdf.PdfReader(str(path))
33          if not reader.is_encrypted:
34              return True
35          try:
36              return reader.decrypt(password) != pypdf.PasswordType.NOT_DECRYPTED
37          except Exception:
38              return False
39
40
41      def write_decrypted(path: Path | str, password: str | None, dest: Path | str) -> Path:
42          """Write a decrypted copy of the PDF to dest (password may be '' for plaintext)."""
43          src = Path(path)
44          dest = Path(dest)
45          dest.parent.mkdir(parents=True, exist_ok=True)
46          reader = pypdf.PdfReader(str(src))
47          if reader.is_encrypted:
48              reader.decrypt(password or "")
49          writer = pypdf.PdfWriter()
50          writer.append(reader)
51          with open(dest, "wb") as fh:
52              writer.write(fh)
53          return dest
54
55
56      def unlock(
57          path: Path | str,
58          candidates: Iterable[str] = (),
59          prompt_fn: Callable[[], str | None] | None = None,
60          max_prompts: int = 3,
61          dest: Path | str | None = None,
62      ) -> UnlockResult:
63          """Hybrid unlock: plaintext? -> candidate dictionary -> interactive prompt.
```

```

64
65     If dest is given and unlock succeeds, a decrypted copy is written there. The
66     working password is returned in the result but must never be logged.
67     """
68     src = Path(path)
69     if not is_encrypted(src):
70         out = Path(shutil.copyfile(src, dest)) if dest else None
71         return UnlockResult(True, "plaintext", None, out)
72
73     tried: set[str] = set()
74     for pw in candidates:
75         if pw in tried:
76             continue
77         tried.add(pw)
78         if password_works(src, pw):
79             out = write_decrypted(src, pw, dest) if dest else None
80             return UnlockResult(True, "candidate", pw, out)
81
82     if prompt_fn is not None:
83         for _ in range(max_prompts):
84             pw = prompt_fn()
85             if not pw:
86                 break
87             if password_works(src, pw):
88                 out = write_decrypted(src, pw, dest) if dest else None
89                 return UnlockResult(True, "prompt", pw, out)
90
91     return UnlockResult(False, None, None, None)
92

```

194. user (2026-05-31T09:07:25.338Z)

[Request interrupted by user]

195. user (2026-05-31T09:07:41.492Z)

My bad, the HDFC password is 135826722. Can you rerun

196. assistant (2026-05-31T09:08:03.775Z)

Got it ? rerunning HDFC with ****135826722****. (Good news: the earlier run already confirmed ****Axis (SUNE0106) and AU Bank (SUNE0106) decrypt cleanly**** via pypdf ? codes 2 and 1 ? so those passwords are correct.)

197. user (2026-05-31T09:08:14.764Z)

```

--- pypdf decrypt check: HDFC with corrected password ---
HDFC Avi_Dullu_28022026_183854522.pdf          encrypted=True decrypt=1 pages=7
HDFC Avi_Dullu_30042026_001858675.pdf          encrypted=True decrypt=1 pages=6
HDFC Avi_Dullu_30112025_110905081.pdf          encrypted=True decrypt=1 pages=8
HDFC Avi_Dullu_31082025_205409615.pdf          encrypted=True decrypt=1 pages=9
--- HDFC precision baseline via the real parser (booleans/counts only) ---
===== test session starts =====
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\Projects\muneem\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\muneem
configfile: pyproject.toml
collecting ... collected 20 items / 16 deselected / 4 selected

tests/parsers/test_hdfc.py::test_real_hdfc_statement_validates[Avi_Dullu_28022026_183854522.pdf]
PASSED [ 25%]
tests/parsers/test_hdfc.py::test_real_hdfc_statement_validates[Avi_Dullu_30042026_001858675.pdf]
PASSED [ 50%]

```

```
tests/parsers/test_hdfc.py::test_real_hdfc_statement_validates[Avi_Dullu_30112025_110905081.pdf]
PASSED [ 75%]
tests/parsers/test_hdfc.py::test_real_hdfc_statement_validates[Avi_Dullu_31082025_205409615.pdf]
PASSED [100%]
```

===== 4 passed, 16 deselected in 6.35s =====

198. assistant (2026-05-31T09:09:59.914Z)

****HDFC precision baseline: 4/4 real statements pass**** ? they parse, the running balance reconciles, ***and*** they bookend against HDFC's own SUMMARY (Dr/Cr counts + opening/closing). So HDFC is solid on real data; the earlier failure was purely the wrong password, and ``pdfplumber`` handles the encryption directly once the password is right. Axis and AU Bank both decrypt cleanly with ``SUNE0106``.

So the landscape:

Issuer	Data	Decrypt	Parser status
HDFC	4 PDFs	? <code>`135826722`</code>	? **4/4 reconcile + bookend**
Axis	4 PDFs (Aug'24, May/Jun/Oct'25)	? <code>`SUNE0106`</code>	?? stop-gapped ? Step 3 rewrite
AU Bank	4 PDFs	? <code>`SUNE0106`</code>	? no parser yet ? build it
ICICI	none	?	?? remove (your call)

****Plan**** (each a squash-merged PR per your cadence):

1. ****Refocus**** ? remove ICICI + reframe docs (README/DESIGN/CURRENT_STATE/architecture/ROADMAP) to the ****total-finance**** view + HDFC/Axis/AU focus.
2. ****Axis rewrite**** (Step 3) ? precision-test Axis.
3. ****AU Bank parser**** ? precision-test AU.
4. ****Final precision comparison**** ? HDFC vs Axis vs AU, side by side.

Starting with #1. Let me scope the ICICI removal precisely:

199. user (2026-05-31T09:10:02.716Z)

Task #7 created successfully: Refocus: remove ICICI + total-finance docs

200. user (2026-05-31T09:10:04.627Z)

Task #8 created successfully: AU Bank parser (engine-driven) + precision test

201. user (2026-05-31T09:10:05.540Z)

Updated task #4 status

202. user (2026-05-31T09:10:07.621Z)

Updated task #6 description

203. user (2026-05-31T09:10:08.624Z)

Switched to a new branch 'chore/refocus-remove-icici'
chore/refocus-remove-icici

204. user (2026-05-31T09:10:08.807Z)

```
1      """Local SQLite store: the ``documents`` and ``offers`` slice of the schema.
2
3      This is the minimal slice the promo spine needs (ROADMAP A.2). The full schema
4      (accounts, transactions, holdings, merchants, items) grows with later milestones.
5      ``documents.sha256`` is UNIQUE so re-ingesting the same file is a no-op.
6      ``offers.source`` records provenance ('text' = exact text layer, 'ocr' = approximate).
7      """
8
```



```

9      from __future__ import annotations
10
11     import sqlite3
12     from collections.abc import Iterator
13     from contextlib import contextmanager
14     from pathlib import Path
15
16     SCHEMA_VERSION = 4
17
18     _SCHEMA = """
19     CREATE TABLE IF NOT EXISTS documents (
20         id            INTEGER PRIMARY KEY,
21         source        TEXT NOT NULL,
22         external_id   TEXT,
23         sender        TEXT,
24         subject       TEXT,
25         received_date TEXT,
26         doc_type      TEXT,
27         sha256        TEXT NOT NULL UNIQUE,
28         path          TEXT,
29         status        TEXT NOT NULL DEFAULT 'parsed',
30         created_at    TEXT NOT NULL DEFAULT (datetime('now'))
31     );
32
33     CREATE TABLE IF NOT EXISTS offers (
34         id            INTEGER PRIMARY KEY,
35         document_id   INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
36         merchant      TEXT,
37         promo_code    TEXT,
38         description   TEXT,
39         expiry_raw    TEXT,
40         expiry_type   TEXT,
41         source        TEXT NOT NULL DEFAULT 'text',
42         redeemed      INTEGER NOT NULL DEFAULT 0,
43         source_text   TEXT
44     );
45
46     CREATE INDEX IF NOT EXISTS idx_offers_document ON offers(document_id);
47
48     -- We use separate tables for each bank (e.g. axis_transactions, icici_transactions)
49     -- instead of a unified table to preserve raw schema fidelity of the source PDFs.
50     -- Normalization into a unified view layer is planned for a later phase
51     -- once all bank schemas are stabilized.
52     CREATE TABLE IF NOT EXISTS axis_transactions (
53         id INTEGER PRIMARY KEY,
54         document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
55         account_number TEXT,
56         txn_date TEXT,
57         value_date TEXT,
58         particulars TEXT,
59         debit REAL,
60         credit REAL,
61         balance REAL
62     );
63
64     CREATE TABLE IF NOT EXISTS axis_cc_transactions (
65         id INTEGER PRIMARY KEY,
66         document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
67         account_number TEXT,
68         txn_date TEXT,
69         particulars TEXT,
70         merchant_category TEXT,
71         amount REAL,
72         type TEXT
73     );

```

```

74
75 CREATE TABLE IF NOT EXISTS icici_transactions (
76     id INTEGER PRIMARY KEY,
77     document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
78     account_number TEXT,
79     txn_date TEXT,
80     mode TEXT,
81     particulars TEXT,
82     withdrawals REAL,
83     deposits REAL,
84     balance REAL
85 );
86
87 CREATE TABLE IF NOT EXISTS hdfc_transactions (
88     id INTEGER PRIMARY KEY,
89     document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
90     account_number TEXT,
91     txn_date TEXT,
92     value_date TEXT,
93     particulars TEXT,
94     ref TEXT,
95     withdrawal REAL,
96     deposit REAL,
97     closing_balance REAL
98 );
99 """
100
101
102 @contextmanager
103 def connect(db_path: Path | str) -> Iterator[sqlite3.Connection]:
104     """Open a connection (commit on success, always close). Parent dirs created."""
105     path = str(db_path)
106     if path != ":memory:":
107         Path(path).parent.mkdir(parents=True, exist_ok=True)
108     conn = sqlite3.connect(path)
109     conn.row_factory = sqlite3.Row
110     conn.execute("PRAGMA foreign_keys = ON")
111     try:
112         yield conn
113         conn.commit()
114     finally:
115         conn.close()
116
117
118 def init_db(conn: sqlite3.Connection) -> None:
119     """Create tables if absent and stamp the schema version."""
120     conn.executescript(_SCHEMA)
121     conn.execute(f"PRAGMA user_version = {SCHEMA_VERSION}")
122
123
124 def get_document_by_sha(conn: sqlite3.Connection, sha256: str) -> sqlite3.Row | None:
125     return conn.execute("SELECT * FROM documents WHERE sha256 = ?",
126 (sha256,)).fetchone()
127
128
129 def _insert_row(conn: sqlite3.Connection, table: str, fields: dict) -> int:
130     """Internal helper for the simple INSERT pattern used by documents, offers, and bank
131 txns."""
132     cols = ", ".join(fields)
133     placeholders = ", ".join("?" for _ in fields)
134     cur = conn.execute(
135         f"INSERT INTO {table} ({cols}) VALUES ({placeholders})",
136         tuple(fields.values()),
137     )
138     return int(cur.lastrowid)

```

```

137
138
139 def insert_document(conn: sqlite3.Connection, **fields: object) -> int:
140     """Insert into documents table. Delegates to the shared helper for DRY."""
141     return _insert_row(conn, "documents", dict(fields))
142
143
144 def insert_offer(conn: sqlite3.Connection, **fields: object) -> int:
145     return _insert_row(conn, "offers", fields)
146
147
148 def list_offers(conn: sqlite3.Connection) -> list[sqlite3.Row]:
149     return conn.execute(
150         """
151         SELECT o.id, o.merchant, o.promo_code, o.expiry_raw, o.description, o.source,
152                d.external_id, d.path
153         FROM offers o
154         JOIN documents d ON d.id = o.document_id
155         ORDER BY o.id
156         """
157     ).fetchall()
158
159
160 def insert_axis_transaction(conn: sqlite3.Connection, **fields: object) -> int:
161     return _insert_row(conn, "axis_transactions", fields)
162
163
164 def insert_icici_transaction(conn: sqlite3.Connection, **fields: object) -> int:
165     return _insert_row(conn, "icici_transactions", fields)
166
167
168 def insert_axis_cc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
169     return _insert_row(conn, "axis_cc_transactions", fields)
170
171
172 def list_axis_cc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
173     return conn.execute(
174         "SELECT * FROM axis_cc_transactions ORDER BY id DESC LIMIT ?", (limit,)
175     ).fetchall()
176
177
178 def list_axis_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
179     return conn.execute(
180         "SELECT * FROM axis_transactions ORDER BY id DESC LIMIT ?", (limit,)
181     ).fetchall()
182
183
184 def list_icici_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
185     return conn.execute(
186         "SELECT * FROM icici_transactions ORDER BY id DESC LIMIT ?", (limit,)
187     ).fetchall()
188
189
190 def insert_hdfc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
191     return _insert_row(conn, "hdfc_transactions", fields)
192
193
194 def list_hdfc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
195     return conn.execute(
196         "SELECT * FROM hdfc_transactions ORDER BY id DESC LIMIT ?", (limit,)
197     ).fetchall()

```

205. user (2026-05-31T09:10:15.239Z)

```

1      """Regression test for the parser registry (the PR #9-11 production-path bug).
2
3      `from muneem.parsers import get_parser` must return a working selector ? i.e. the
4      registry must be populated by package import alone, without anything importing the
5      axis/icici/... submodules first. We check in a FRESH interpreter (subprocess) so a
6      prior test importing a submodule can't mask a regression.
7      """
8
9      from __future__ import annotations
10
11     import subprocess
12     import sys
13
14
15     def test_registry_is_populated_by_package_import_alone():
16         code = (
17             "from muneem.parsers import get_parser, _REGISTRY\n"
18             "assert len(_REGISTRY) >= 2, f'registry empty: {_REGISTRY}'\n"
19             "assert get_parser('Axis Bank Statement of Account') is not None\n"
20             "assert get_parser('ICICI Bank Statement of Transactions') is not None\n"
21             "assert get_parser('a wholly unrelated document') is None\n"
22             "print('REGISTRY_OK', len(_REGISTRY))\n"
23         )
24         result = subprocess.run([sys.executable, "-c", code], capture_output=True,
text=True)
25         assert result.returncode == 0, f"fresh-interpreter import failed:\n{result.stderr}"
26         assert "REGISTRY_OK" in result.stdout
27
28
29     def test_get_parser_selects_by_sender():
30         from muneem.parsers import get_parser
31
32         assert type(get_parser("", sender="alerts@axisbank.com")).__name__ == "AxisParser"
33         assert type(get_parser("", sender="noreply@icicibank.com")).__name__ ==
"ICICIParser"
34

```

206. assistant (2026-05-31T09:10:13.805Z)

Now the CLI and registry test ICICI references:

207. user (2026-05-31T09:10:14.802Z)

```

1      """The ``muneem`` command-line interface.
2
3      Track A ships the promo spine: ``ingest`` a PDF into the local store and
4      ``offers list`` to read back what was found. ``--ocr`` adds local image OCR for
5      codes rasterized into banners (approximate; flagged for verification). Pipeline
6      subcommands (``fetch``, ``txns``) land with their milestones.
7      """
8
9      from __future__ import annotations
10
11     from pathlib import Path
12
13     import typer
14
15     from muneem import __version__
16     from muneem import db as dbmod
17     from muneem.config import load_config

```

```

18
19 app = typer.Typer(
20     add_completion=False,
21     help="muneem ? a local-first personal-finance bookkeeper.",
22     no_args_is_help=True,
23 )
24 offers_app = typer.Typer(no_args_is_help=True, help="Work with extracted offers.")
25 app.add_typer(offers_app, name="offers")
26
27 txns_app = typer.Typer(no_args_is_help=True, help="Work with parsed transactions.")
28 app.add_typer(txns_app, name="txns")
29
30
31 # Dispatch table for supported banks (extensible as new parsers land).
32 BANK_DISPATCH = {
33     "axis": (dbmod.list_axis_transactions, "debit", "credit"),
34     "axis_cc": (dbmod.list_axis_cc_transactions, "amount", "type"),
35     "icici": (dbmod.list_icici_transactions, "withdrawals", "deposits"),
36     "hdfc": (dbmod.list_hdfc_transactions, "withdrawal", "deposit"),
37 }
38
39
40 def _version_callback(value: bool) -> None:
41     if value:
42         typer.echo(f"muneem {__version__}")
43         raise typer.Exit()
44
45
46 @app.callback()
47 def main(
48     version: bool = typer.Option(
49         False,
50         "--version",
51         callback=_version_callback,
52         is_eager=True,
53         help="Show the muneem version and exit.",
54     ),
55 ) -> None:
56     """muneem command-line interface."""
57
58
59 @app.command()
60 def info() -> None:
61     """Show resolved local paths (no secrets, no sensitive data)."""
62     cfg = load_config()
63     present = "present" if cfg.config_file.is_file() else "absent"
64     typer.echo(f"muneem {__version__}")
65     typer.echo(f"config dir : {cfg.config_dir}")
66     typer.echo(f"data dir   : {cfg.data_dir}")
67     typer.echo(f"db path    : {cfg.db_path}")
68     typer.echo(f"staging   : {cfg.staging_dir}")
69     typer.echo(f"config file: {cfg.config_file} ({present})")
70
71
72 @app.command()
73 def ingest(
74     pdf: Path = typer.Argument(
75         ..., exists=True, dir_okay=False, readable=True, help="Path to a PDF to ingest."
76     ),
77     doc_type: str = typer.Option("promo", help="Document category."),
78     db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
79 dir)."),
80     ocr: bool = typer.Option(
81         False, "--ocr", help="Also OCR images locally (Tesseract) for codes in banners."
82     ),

```

```

82 ) -> None:
83     """Extract a PDF and store its document + any offers in the local DB."""
84     from muneem import db as dbmod
85     from muneem.extract import extract
86     from muneem.offers import extract_offer
87
88     db_path = db or load_config().db_path
89     ex = extract(pdf)
90
91     if doc_type == "bank_statement":
92         from muneem.parsers import get_parser
93
94         parser = get_parser(ex.text)
95         if not parser:
96             typer.echo(f"No parser found for bank statement: {pdf.name}", err=True)
97             return
98
99         with dbmod.connect(db_path) as conn:
100             dbmod.init_db(conn)
101             existing = dbmod.get_document_by_sha(conn, ex.sha256)
102             if existing is not None:
103                 typer.echo(f"already ingested (doc {existing['id']}): {pdf.name}")
104                 return
105
106             doc_id = dbmod.insert_document(
107                 conn,
108                 source="file",
109                 external_id=pdf.name,
110                 doc_type=doc_type,
111                 sha256=ex.sha256,
112                 path=str(pdf.resolve()),
113                 status="parsed",
114             )
115
116             count = parser.parse_and_insert(pdf, conn, doc_id)
117
118             typer.echo(
119                 f"ingested {pdf.name} (doc {doc_id}); parsed {count} "
120                 f"transactions using {parser.__class__.__name__}"
121             )
122             return
123     text_fields = extract_offer(ex.text)
124     # (code, source) pairs; text-layer codes are exact, OCR codes are approximate.
125     found: list[tuple[str, str]] = [(c, "text") for c in text_fields.codes]
126     expiry_raw, expiry_type = text_fields.expiry_raw, text_fields.expiry_type
127
128     if ocr:
129         from muneem.ocr import ocr_pdf, tesseract_available
130
131         if not tesseract_available():
132             typer.echo("warning: --ocr requested but Tesseract not found; skipping
OCR.", err=True)
133         else:
134             ocr_fields = extract_offer(ocr_pdf(pdf))
135             known = {c for c, _ in found}
136             for code in ocr_fields.codes:
137                 if code not in known:
138                     found.append((code, "ocr"))
139                     known.add(code)
140             if expiry_raw is None:
141                 expiry_raw, expiry_type = ocr_fields.expiry_raw, ocr_fields.expiry_type
142
143     with dbmod.connect(db_path) as conn:
144         dbmod.init_db(conn)
145         existing = dbmod.get_document_by_sha(conn, ex.sha256)

```

```

146         if existing is not None:
147             typer.echo(f"already ingested (doc {existing['id']}): {pdf.name}")
148             return
149         doc_id = dbmod.insert_document(
150             conn,
151             source="file",
152             external_id=pdf.name,
153             doc_type=doc_type,
154             sha256=ex.sha256,
155             path=str(pdf.resolve()),
156             status="parsed",
157         )
158         for code, src in found:
159             dbmod.insert_offer(
160                 conn,
161                 document_id=doc_id,
162                 promo_code=code,
163                 expiry_raw=expiry_raw,
164                 expiry_type=expiry_type,
165                 source=src,
166                 source_text=ex.text[:2000],
167             )
168
169         summary = ", ".join(f"{c} [{s}]" for c, s in found) if found else "none"
170         typer.echo(f"ingested {pdf.name} (doc {doc_id}); {len(found)} offer(s): {summary}")
171
172
173     @offers_app.command("list")
174     def offers_list(
175         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
176         dir)."),
177     ) -> None:
178         """List extracted offers."""
179         from muneem import db as dbmod
180
181         db_path = db or load_config().db_path
182         if not Path(db_path).exists():
183             typer.echo("no offers yet (database not created).")
184             return
185
186         with dbmod.connect(db_path) as conn:
187             dbmod.init_db(conn)
188             rows = dbmod.list_offers(conn)
189
190         if not rows:
191             typer.echo("no offers found.")
192             return
193
194         for row in rows:
195             who = row["merchant"] or row["external_id"] or "?"
196             when = f" ? expires {row['expiry_raw']}" if row["expiry_raw"] else ""
197             tag = " [ocr ? verify]" if row["source"] == "ocr" else ""
198             typer.echo(f"[{row['id']}] {who}: {row['promo_code']}{when}{tag}")
199
200
201     @txns_app.command("list")
202     def txns_list(
203         bank: str = typer.Argument(
204             ..., help="Which bank's transactions to list (e.g. hdfc, axis, icici)."
205         ),
206         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
207         dir)."),
208         limit: int = typer.Option(50, help="Max number of transactions to show."),
209     ) -> None:
210         """List parsed transactions for a specific bank."""
211         from muneem import db as dbmod

```

```

209
210     db_path = db or load_config().db_path
211     if not Path(db_path).exists():
212         typer.echo("database not created yet.")
213         return
214
215     bank = bank.lower()
216     if bank not in BANK_DISPATCH:
217         typer.echo(f"Unknown bank: {bank}", err=True)
218         return
219
220     list_fn, neg_key, pos_key = BANK_DISPATCH[bank]
221     with dbmod.connect(db_path) as conn:
222         dbmod.init_db(conn)
223         rows = list_fn(conn, limit=limit)
224
225     if not rows:
226         typer.echo(f"no {bank} transactions found.")
227         return
228     for row in rows:
229         if bank == "axis_cc":
230             amount = f"{row[pos_key]}" if row["type"] == "Cr" else f"-{row[neg_key]}"
231         else:
232             amount = f"{row[pos_key]}" if row[pos_key] else f"-{row[neg_key]}"
233         part = (row["particulars"] or "")[:50]
234         typer.echo(f"[{row['account_number']}] {row['txn_date']} : {part:<50} |
{amount:>10}")
235
236
237     if __name__ == "__main__":
238         app()
239

```

208. assistant (2026-05-31T09:11:30.386Z)

Now I have the full ICICI footprint. Let me confirm the current `profiles.py` on this branch before editing:

209. user (2026-05-31T09:11:31.112Z)

```

1     """Per-issuer profiles ? bank knowledge expressed as **data, not code** (architecture
2     $6).
3
4     A profile carries the priors the generic engine needs to disambiguate one issuer: extra
5     concept?label aliases, the accepted date shape, a positional fallback when there's no
6     header,
7     table anchors, account-number patterns, quirks, and (parked) the password rule. Adding
8     an issuer is
9     ideally *adding a profile*, not writing a parser.
10
11     `ICICI_PROFILE` drives ICICI fully through `engine.map_table`. `HDFC_PROFILE` is data
12     consumed by
13     HDFC's parser, whose *extraction* stays bespoke coordinate-clustering ? the documented
14     exception
15     where `extract_tables()` fails on a semi-ruled table (see the `hdfc-parser-tech-
16     decision` memory).
17     The Axis profile lands with the Step 3 coordinate-clustering rewrite.
18     """
19
20     from __future__ import annotations
21
22     import re
23
24     from .engine import Profile
25

```



```

20     ICICI_PROFILE = Profile(
21         name="icici",
22         aliases={
23             "narration": ("particulars",),
24             "debit": ("withdrawals",),
25             "credit": ("deposits",),
26             "mode": (
27                 "mode",
28             ), # ICICI-specific extra (UPI/NEFT/...); rides along in MappingResult.mapped
29         },
30         date_patterns=(re.compile(r"^\d{2}/\d{2}/\d{4}$"),), # dd/mm/yyyy
31         # Known ICICI layout, used when a table has no detectable header row.
32         column_order=("txn_date", "mode", "narration", "debit", "credit", "balance"),
33         account_patterns=(
34             re.compile(r"Account Number\s*:\s*([\d-]+)", re.IGNORECASE),
35             re.compile(r"A/c\s*No\s*[:\s]*([\d-]+)", re.IGNORECASE),
36         ),
37     )
38
39     HDFC_PROFILE = Profile(
40         name="hdfc",
41         aliases={
42             "debit": ("withdrawal", "withdrawals"),
43             "credit": ("deposit", "deposits"),
44             "balance": ("closing balance",),
45         },
46         date_patterns=(re.compile(r"^\d{2}/\d{2}/\d{4}$"),), # dd/mm/yyyy
47         table_anchors=("Narration", "Closing Balance"),
48         account_patterns=(re.compile(r"Account\s*Number\s*[:.]\s*(\d{6,})",
re.IGNORECASE),),
49         # HDFC's transaction table is semi-ruled; extract_tables() is unreliable on it, so
HDFC extracts
50         # by word-position clustering rather than through the generic cell-matrix engine.
51         quirks=frozenset({"semi-ruled-table", "coordinate-clustering"}),
52     )
53
54     __all__ = ["ICICI_PROFILE", "HDFC_PROFILE"]
55

```

210. assistant (2026-05-31T09:13:27.710Z)

Confirmed. Now I'll remove ICICI across the code. Using full-file writes for the multi-change files (`profiles.py`, `db.py`, `test\_registry.py`) and edits for the rest:

211. user (2026-05-31T09:13:28.857Z)

```

rm 'src/muneem/parsers/icici.py'
rm 'tests/parsers/test_icici.py'

```

212. assistant (2026-05-31T09:13:34.870Z)

"""Per-issuer profiles ? bank knowledge expressed as **data**, not **code** (architecture §6).

A profile carries the priors the generic engine needs to disambiguate one issuer: extra concept?label aliases, the accepted date shape, a positional fallback when there's no header, table anchors, account-number patterns, quirks, and (parked) the password rule. Adding an issuer is ideally *adding a profile*, not writing a parser.

`HDFC\_PROFILE` is data consumed by HDFC's parser, whose *extraction* stays bespoke coordinate-clustering ? the documented exception where `extract_tables()` fails on a semi-ruled table (see the `hdfc-parser-tech-decision` memory). The Axis and AU Bank profiles land with their engine-driven parsers (coordinate clustering ? generic mapping ? total reconcile).

```

"""

from __future__ import annotations

import re

from .engine import Profile

HDFC_PROFILE = Profile(
    name="hdfc",
    aliases={
        "debit": ("withdrawal", "withdrawals"),
        "credit": ("deposit", "deposits"),
        "balance": ("closing balance",),
    },
    date_patterns=(re.compile(r"^\d{2}/\d{2}/\d{4}$"),), # dd/mm/yyyy
    table_anchors=("Narration", "Closing Balance"),
    account_patterns=(re.compile(r"Account\s*Number\s*[:.]\s*(\d{6,})", re.IGNORECASE),),
    # HDFC's transaction table is semi-ruled; extract_tables() is unreliable on it, so HDFC
    extracts
    # by word-position clustering rather than through the generic cell-matrix engine.
    quirks=frozenset({"semi-ruled-table", "coordinate-clustering"}),
)

__all__ = ["HDFC_PROFILE"]

```

213. user (2026-05-31T09:13:34.884Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\profiles.py has been updated successfully. (file state is current in your context ? no need to Read it back)

214. assistant (2026-05-31T09:13:55.516Z)

"""Local SQLite store: the ``documents`` and ``offers`` slice of the schema.

This is the minimal slice the promo spine needs (ROADMAP A.2). The full schema (accounts, transactions, holdings, merchants, items) grows with later milestones. ``documents.sha256`` is UNIQUE so re-ingesting the same file is a no-op. ``offers.source`` records provenance ('text' = exact text layer, 'ocr' = approximate).

"""

```

from __future__ import annotations

import sqlite3
from collections.abc import Iterator
from contextlib import contextmanager
from pathlib import Path

SCHEMA_VERSION = 4

_SCHEMA = """
CREATE TABLE IF NOT EXISTS documents (
    id            INTEGER PRIMARY KEY,
    source        TEXT NOT NULL,
    external_id   TEXT,
    sender        TEXT,
    subject       TEXT,
    received_date TEXT,
    doc_type      TEXT,
    sha256        TEXT NOT NULL UNIQUE,
    path          TEXT,
    status        TEXT NOT NULL DEFAULT 'parsed',
    created_at    TEXT NOT NULL DEFAULT (datetime('now'))
);

```

```

CREATE TABLE IF NOT EXISTS offers (
    id            INTEGER PRIMARY KEY,
    document_id  INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
    merchant     TEXT,
    promo_code   TEXT,
    description  TEXT,
    expiry_raw   TEXT,
    expiry_type  TEXT,
    source       TEXT NOT NULL DEFAULT 'text',
    redeemed     INTEGER NOT NULL DEFAULT 0,
    source_text  TEXT
);

CREATE INDEX IF NOT EXISTS idx_offers_document ON offers(document_id);

-- We use separate tables for each bank (e.g. axis_transactions, hdfc_transactions)
-- instead of a unified table to preserve raw schema fidelity of the source PDFs.
-- Normalization into a unified view layer is planned for a later phase
-- once all bank schemas are stabilized.
CREATE TABLE IF NOT EXISTS axis_transactions (
    id INTEGER PRIMARY KEY,
    document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
    account_number TEXT,
    txn_date TEXT,
    value_date TEXT,
    particulars TEXT,
    debit REAL,
    credit REAL,
    balance REAL
);

CREATE TABLE IF NOT EXISTS axis_cc_transactions (
    id INTEGER PRIMARY KEY,
    document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
    account_number TEXT,
    txn_date TEXT,
    particulars TEXT,
    merchant_category TEXT,
    amount REAL,
    type TEXT
);

CREATE TABLE IF NOT EXISTS hdfc_transactions (
    id INTEGER PRIMARY KEY,
    document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
    account_number TEXT,
    txn_date TEXT,
    value_date TEXT,
    particulars TEXT,
    ref TEXT,
    withdrawal REAL,
    deposit REAL,
    closing_balance REAL
);
"""

@contextmanager
def connect(db_path: Path | str) -> Iterator[sqlite3.Connection]:
    """Open a connection (commit on success, always close). Parent dirs created."""
    path = str(db_path)
    if path != ":memory:":
        Path(path).parent.mkdir(parents=True, exist_ok=True)
    conn = sqlite3.connect(path)
    conn.row_factory = sqlite3.Row

```

```

conn.execute("PRAGMA foreign_keys = ON")
try:
    yield conn
    conn.commit()
finally:
    conn.close()

def init_db(conn: sqlite3.Connection) -> None:
    """Create tables if absent and stamp the schema version."""
    conn.executescript(_SCHEMA)
    conn.execute(f"PRAGMA user_version = {SCHEMA_VERSION}")

def get_document_by_sha(conn: sqlite3.Connection, sha256: str) -> sqlite3.Row | None:
    return conn.execute("SELECT * FROM documents WHERE sha256 = ?", (sha256,)).fetchone()

def _insert_row(conn: sqlite3.Connection, table: str, fields: dict) -> int:
    """Internal helper for the simple INSERT pattern used by documents, offers, and bank
    txns."""
    cols = ", ".join(fields)
    placeholders = ", ".join("?" for _ in fields)
    cur = conn.execute(
        f"INSERT INTO {table} ({cols}) VALUES ({placeholders})",
        tuple(fields.values()),
    )
    return int(cur.lastrowid)

def insert_document(conn: sqlite3.Connection, **fields: object) -> int:
    """Insert into documents table. Delegates to the shared helper for DRY."""
    return _insert_row(conn, "documents", dict(fields))

def insert_offer(conn: sqlite3.Connection, **fields: object) -> int:
    return _insert_row(conn, "offers", fields)

def list_offers(conn: sqlite3.Connection) -> list[sqlite3.Row]:
    return conn.execute(
        """
        SELECT o.id, o.merchant, o.promo_code, o.expiry_raw, o.description, o.source,
               d.external_id, d.path
        FROM offers o
        JOIN documents d ON d.id = o.document_id
        ORDER BY o.id
        """
    ).fetchall()

def insert_axis_transaction(conn: sqlite3.Connection, **fields: object) -> int:
    return _insert_row(conn, "axis_transactions", fields)

def insert_axis_cc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
    return _insert_row(conn, "axis_cc_transactions", fields)

def list_axis_cc_transactions(conn: sqlite3.Connection, limit: int = 50) -> list[sqlite3.Row]:
    return conn.execute(
        "SELECT * FROM axis_cc_transactions ORDER BY id DESC LIMIT ?", (limit,)
    ).fetchall()

```

```

def list_axis_transactions(conn: sqlite3.Connection, limit: int = 50) -> list[sqlite3.Row]:
    return conn.execute(
        "SELECT * FROM axis_transactions ORDER BY id DESC LIMIT ?", (limit,)
    ).fetchall()

def insert_hdfc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
    return _insert_row(conn, "hdfc_transactions", fields)

def list_hdfc_transactions(conn: sqlite3.Connection, limit: int = 50) -> list[sqlite3.Row]:
    return conn.execute(
        "SELECT * FROM hdfc_transactions ORDER BY id DESC LIMIT ?", (limit,)
    ).fetchall()

```

215. user (2026-05-31T09:13:55.531Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\db.py has been updated successfully. (file state is current in your context ? no need to Read it back)

216. assistant (2026-05-31T09:13:59.751Z)

```

"""Regression test for the parser registry (the PR #9-11 production-path bug).

`from muneem.parsers import get_parser` must return a working selector ? i.e. the
registry must be populated by package import alone, without anything importing the
axis/hdfc/... submodules first. We check in a FRESH interpreter (subprocess) so a
prior test importing a submodule can't mask a regression.
"""

from __future__ import annotations

import subprocess
import sys

def test_registry_is_populated_by_package_import_alone():
    code = (
        "from muneem.parsers import get_parser, _REGISTRY\n"
        "assert len(_REGISTRY) >= 2, f'registry empty: {_REGISTRY}\n'"
        "assert get_parser('Axis Bank Statement of Account') is not None\n"
        "assert get_parser('HDFC Bank savings account details') is not None\n"
        "assert get_parser('a wholly unrelated document') is None\n"
        "print('REGISTRY_OK', len(_REGISTRY))\n"
    )
    result = subprocess.run([sys.executable, "-c", code], capture_output=True, text=True)
    assert result.returncode == 0, f"fresh-interpreter import failed:\n{result.stderr}"
    assert "REGISTRY_OK" in result.stdout

def test_get_parser_selects_by_sender():
    from muneem.parsers import get_parser

    assert type(get_parser("", sender="alerts@axisbank.com")).__name__ == "AxisParser"

```

217. user (2026-05-31T09:13:59.763Z)

The file C:\Users\avidu\Projects\muneem\tests\parsers\test_registry.py has been updated successfully. (file state is current in your context ? no need to Read it back)

218. user (2026-05-31T09:14:02.116Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\cli.py has been updated successfully. (file state is current in your context ? no need to Read it back)

219. user (2026-05-31T09:14:04.024Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\engine.py has been updated successfully. (file state is current in your context ? no need to Read it back)

220. user (2026-05-31T09:14:05.411Z)

The file C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py has been updated successfully. (file state is current in your context ? no need to Read it back)

221. user (2026-05-31T09:14:06.951Z)

The file C:\Users\avidu\Projects\muneem\tests\parsers\test_hdfc.py has been updated successfully. (file state is current in your context ? no need to Read it back)

222. assistant (2026-05-31T09:14:11.670Z)

Now the CLI help text, then verify the code removal is green:

223. user (2026-05-31T09:14:13.581Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\cli.py has been updated successfully. (file state is current in your context ? no need to Read it back)

224. user (2026-05-31T09:14:13.642Z)

Updated task #7 status

225. user (2026-05-31T09:14:25.824Z)

All checks passed!

--- grep leftover icici (should be docs only) ---

DESIGN.md

--- offline suite ---

.....X..... [69%]

..... [100%]

===== XPASSES =====

===== short test summary info =====

XPASS tests/test_ocr.py::test_swiggy_image_code_recovered_fuzzily - recovery of the stylized boxed code is OCR-engine/version dependent ? succeeds on Tesseract 5.4, returns nothing on the apt build; the planned fix is a purpose-trained model (docs/DEPENDENCIES.md). strict=False so it never fails the build.

103 passed, 9 deselected, 1 xpassed in 6.12s

226. assistant (2026-05-31T09:14:39.954Z)

ICICI is gone from the code (103 pass, lint clean; only `DESIGN.md` has a leftover mention, handled next). Now the docs reframe. Let me read DESIGN.md and the remaining ICICI doc contexts:

227. user (2026-05-31T09:14:40.198Z)

1 # muneem ? Design Document

2

3 > Status: ****draft, pre-implementation****. This document captures the vision, the pipeline shape, the security posture, and the open decisions to resolve before code lands. It is intentionally opinionated where the author has a strong view, and explicitly undecided where the trade-offs need a human call.

4

5 ---

6

7 ## 1. What muneem is

8

9 ****muneem**** is a personal-finance bookkeeper that ingests financial documents from the

user's email accounts, unlocks the password-protected ones, parses them into a normalized schema, stores them in a local database, and surfaces analytics across the user's full financial life.

10
11 The name is the Hindi word for the traditional household/business bookkeeper ? a trusted figure who maintained the family ledger by hand. This project is the same role, automated.

12
13 ****Single user. Local-first. Privacy-critical.****

14
15 **## 2. Why**

16
17 The user's financial life today is fragmented across many silos:
18
19 - 325 bank accounts (statements arrive monthly as password-protected PDFs)
20 - Multiple credit cards (monthly bills, also password-protected PDFs)
21 - Brokerage and mutual-fund accounts (quarterly statements, portfolio snapshots)
22 - Grocery and shopping receipts (Swiggy, Zomato, Amazon, Flipkart, BigBasket, Blinkit ? mostly plain HTML/PDF in email body)
23 - Utility bills, insurance premiums, EMI schedules
24 - Promotional offers and coupons (often forgotten)

25
26 Each silo has its own portal, login, and format. There is no consolidated view. ****The inbox is the only place where they already converge.****

27
28 Concrete use-cases the user wants once this exists:
29
30 1. ****Grocery & shopping price intelligence**** ? track what was bought, where, and at what price, over time. Detect when a merchant is no longer competitive on staples.
31 2. ****Investment portfolio timeline**** ? a single time-series of holdings across brokers, so allocation drift and rebalancing decisions are obvious.
32 3. ****Bank/card comparison**** ? fees paid, interest earned, reward redemption efficiency, surfaced as objective numbers.
33 4. ****Item catalog**** ? every significant purchase linked to its receipt/invoice, so warranty windows, return windows, and replacement cycles are queryable.
34 5. ****Forgotten-deals reminder**** ? surface the unused coupons and offers already sitting in the inbox (this is the original goal of the predecessor project; see § 11).

35
36 **## 3. Origin**

37
38 muneem is the successor to `~/Projects/Email-Mining`, a small prototype the user built earlier that extracted promo codes from manually-saved email PDFs using PyMuPDF + Gemini for image OCR. Same insight ("the inbox contains structured value I'm forgetting"), generalized scope.

39
40 See [docs/prior-art-email-mining.md](./docs/prior-art-email-mining.md) for what it did, what to inherit, and what not to.

41
42 **## 4. The pipeline (five stages)**

43
44 Each stage is independent enough to develop and test in isolation. They communicate through well-defined data boundaries (raw files on disk ? unlocked files on disk ? structured records in DB ? derived views).

45
46 **### Stage 1 ? Email ingestion**

47
48 ****Goal:**** pull messages and attachments from Gmail and Outlook into a local staging area.

49
50 - ****Gmail:**** [Gmail API](https://developers.google.com/gmail/api) over OAuth2. Read-only scope (`gmail.readonly`). Supports server-side search queries (`has:attachment filename:pdf from:hdfc newer\_than:90d`) which is far cheaper than pulling everything and filtering locally. Has good rate limits for personal use.

51 - ****Outlook:**** [Microsoft Graph API](https://learn.microsoft.com/en-us/graph/api/resources/mail-api-overview) over OAuth2. Read-only scope (`Mail.Read`). Similar search capability.

52 - ****Why not IMAP:**** worse search, no labels, harder OAuth, and we lose Gmail's powerful query syntax. Use the native APIs.

53

54 Mechanically straightforward. The main design questions are: ****how is "financial document" detected**** (sender allowlist? subject patterns? attachment heuristics? hybrid?), and ****how is incremental sync state tracked**** (Gmail ``historyId`` / Graph ``deltaLink``).

55

56 **### Stage 2 ? Attachment unlocking**

57

58 ****Goal:**** decrypt password-protected PDFs.

59

60 The decryption itself is trivial ? every mainstream PDF library handles the standard security handler. In Go, ``github.com/pdfcpu/pdfcpu``; in Python, ``pikepdf`` or ``pypdf``.

61

62 The real design question is ****where the passwords come from.**** Indian bank and CC statement passwords are almost always derived from user attributes:

63

64 - "First 4 letters of name (uppercase) + DDMM of DOB" ? common at HDFC, ICICI

65 - "PAN number" ? common at some brokers

66 - "Last 4 digits of card number" ? common for CC statements

67 - "Customer ID" ? some banks

68 - "Mobile number" ? utilities

69

70 Three plausible approaches, each with trade-offs:

71

72 | Approach | Pro | Con |

73 |---|---|---|

74 | ****Per-sender rule template**** the user configures once (e.g., ``from:alerts@hdfcbank.net`` ? ``first4(name)+ddmm(dob)``) | Deterministic, no guessing, auditable | Requires upfront setup; rules must be maintained as banks change formats |

75 | ****Candidate dictionary**** (try a small list of derived strings per PDF) | Zero config; "just works" | Brute-forceable feel; fails noisily on unknown senders |

76 | ****Interactive prompt**** (ask the user the first time, cache for that sender) | Safe, simple | Friction during initial backfill of years of statements |

77

78 A hybrid is likely best: candidate dictionary first, fall back to rule template, fall back to interactive prompt. ****Decision parked.**** See § 8.

79

80 **### Stage 3 ? Document parsing**

81

82 ****This is the hard 80% of the project.**** Every issuer's PDF layout is different and changes over time.

83

84 The space of approaches, in increasing robustness and cost:

85

86 1. ****Per-issuer regex/template parsers.**** Precise; brittle. One parser per format, breaks when the issuer redesigns. Realistic for the top ~5 senders that dominate volume.

87 2. ****Text-layer extraction + structural heuristics.**** Pull text with ``pdfminer.six`` / ``pdfplumber`` (both MIT); rely on coordinates and font runs to recover tables. Works well for "born digital" PDFs (most bank statements); fails on scanned/image PDFs. (We avoid PyMuPDF/``fitz`` ? it is AGPL; see § 5 rule 8.)

88 3. ****OCR (Tesseract) for image-based PDFs and embedded raster content.**** Slower; need layout reconstruction.

89 4. ****LLM-assisted structured extraction.**** Hand page text (+ optionally page rasters) to a model with a target JSON schema. Generalizes across layouts; expensive per page; ****must be local for sensitive document categories****. Options: Ollama-hosted models (Llama 3, Qwen, Mistral), or a small purpose-built model.

90

91 In practice this will be ****a hybrid****: deterministic template parsers for the top ~5 high-volume senders (where they pay for themselves), LLM-assisted extraction as the long-tail fallback. Promo emails and other non-sensitive categories can use cloud LLMs; bank/card/investment statements ****must not**** without explicit per-category approval.

92

93 Embedded image deduplication (Email-Mining used MD5 hash + SSIM) is a real win here: bank statements repeat the same logo/letterhead on every page, and we don't want to OCR that 30

times per document.

94

95 ### Stage 4 ? Normalization & storage

96

97 **Goal:** map heterogeneous parser outputs into one schema, persist locally.

98

99 Storage: **SQLite**. Single file, no server, easy to back up, easy to encrypt at rest (SQLCipher), good enough performance for years of personal data. The schema needs to span:

100

101 - `documents` ? every ingested file with sender, date, type, hash, path

102 - `accounts` ? bank accounts, cards, broker accounts, identified by issuer + last-4 or similar

103 - `transactions` ? line items from statements, normalized (date, amount, currency, counterparty, category, source_document)

104 - `holdings` ? point-in-time snapshots of investment positions per account

105 - `merchants` ? canonicalized merchant identities (so "SWIGGY*BANGALORE", "Swiggy Instamart", and "SWIGGY BNGLR" collapse to one)

106 - `items` ? line-items from itemized receipts (grocery, shopping)

107 - `offers` ? promotional offers/coupons with expiry, source, redemption status

108

109 The merchant canonicalization step is non-trivial ? it deserves its own thought (fuzzy match + manual override table is the usual answer).

110

111 ### Stage 5 ? Analytics & insights

112

113 **Goal:** turn the normalized DB into the use-cases in § 2.

114

115 Once the data is in the schema, the analytics are mostly SQL + light application logic. Likely surface: a local CLI initially, possibly a small local web UI later. Notebook-style exploration is fine for prototyping queries before they're promoted to first-class views.

116

117 ---

118

119 ## 5. Security posture (first class)

120

121 The threat model deserves to be stated plainly: **muneem aggregates the user's entire financial life ? every account, every transaction, every holding ? into one local store. That store is a honeypot.** A leak would be catastrophic.

122

123 Standing rules:

124

125 1. **All raw statement data stays on the local machine.** No cloud sync of the staging dir or the DB.

126 2. **No cloud LLM APIs for sensitive document categories** (bank, CC, broker, MF, insurance) without explicit per-category user approval discussed in the current session. The default extraction path for these is local (Tesseract + local LLM via Ollama, or deterministic parsers).

127 3. **Non-sensitive categories** (promo emails, generic newsletters, public receipts) may use cloud LLMs after a privacy review of the specific content type.

128 4. **DB encrypted at rest** (SQLCipher or equivalent), key never in the repo.

129 5. **OAuth tokens** stored in the OS keychain (Windows Credential Manager / macOS Keychain), not on disk.

130 6. **Logs must not contain extracted statement text.** Sanitize aggressively; redact account numbers, balances, line-items.

131 7. **Nothing sensitive ever in git.** `.gitignore` blocks real PDFs, the DB file, `.env`, and password files. Do not weaken it.

132 8. **Commercial-license safety.** Only dependencies under permissive, commercial-safe licenses (MIT, BSD, Apache-2.0, ISC, HPND). No copyleft (GPL/LGPL/**AGPL**) or source-available licenses. **PyMuPDF/`fitz` is AGPL-3.0 and is banned** ? use pdfminer.six (MIT) / pypdfium2 (Apache/BSD) instead. Enforced by `tests/test\_dependency\_licenses.py`; tracked in [docs/DEPENDENCIES.md](./docs/DEPENDENCIES.md).

133 9. **Minimize paid dependencies.** Prefer local / self-hosted / open-source (and, where needed, own-trained) models and tools. Paid APIs ? cloud LLMs, paid OCR/vision ? are a **fallback only**, gated behind explicit per-use approval. This extends rule 2 from privacy to cost.

```

134
135     ## 6. Non-goals (for v1)
136
137     - **Multi-user / SaaS / hosted product.** Single user, single machine.
138     - **Cloud sync of raw data.** Period.
139     - **Cracking unknown passwords.** muneem decrypts PDFs the user is authorized to read,
using passwords the user knows or can derive.
140     - **Browser scraping bank portals.** Email APIs only. Portal scraping is a fragile, ToS-
fraught rabbit hole; we deliberately avoid it.
141     - **Real-time alerting / push.** Batch-pull on a schedule is sufficient.
142     - **Tax preparation, investment advice, or anything regulated.** muneem surfaces data;
the user decides.
143
144     ## 7. Recommended first vertical slice
145
146     Before generalizing, prove the end-to-end pipeline on the smallest meaningful scope:
147
148     > **Gmail ? one specific bank's monthly statements ? unlocked ? parsed into
`transactions` ? stored in SQLite ? a CLI command that lists last month's transactions.**
149
150     This exercises every stage exactly once, on a real input the user cares about, with no
detours. It will surface the actual hard parts (which we can only guess at now) and force the
open decisions in § 8 to get made on real evidence.
151
152     Concretely: pick **one** Indian bank statement format the user has many examples of
(likely HDFC or ICICI given the user's geography), build the parser for that one format only,
and put the rest of the pipeline around it. Then evaluate before generalizing.
153
154     ## 8. Open architectural decisions
155
156     These were parked for explicit user discussion (**don't pick silently**). Items **8.1,
8.2, and 8.4 are now resolved** ? see the inline resolution notes below and
[docs/ROADMAP.md](./docs/ROADMAP.md) § 1. Items **8.3 and 8.5 remain open**.
157
158     ### 8.1 Language
159
160     | Option | Pro | Con |
161     |---|---|---|
162     | **Go only** | Single binary, easy distribution, great concurrency for the ingestion
side, the user already uses Go for the badminton-highlight-indexer project | Weak PDF parsing /
OCR / ML ecosystem; rolling our own is painful |
163     | **Python only** | Best-in-class PDF, OCR, and ML libraries (pdfplumber, PyMuPDF,
Tesseract bindings, Ollama clients, every parser under the sun); fastest to prototype | Heavier
deployment story; less ergonomic for the long-running ingestion daemon |
164     | **Go ingestion + Python parsing** | Plays to each language's strengths | Two-language
project complexity; inter-process boundary to maintain |
165
166     Recommended default: **Python-only for v1**, accept the deployment-story cost in
exchange for moving fast on the hard parts. Revisit if the ingestion side outgrows Python.
167
168     > **Resolved (2026-05-30): Python-only for v1.** Adopted the recommended default. See
[docs/ROADMAP.md](./docs/ROADMAP.md) § 1.
169
170     ### 8.2 Password sourcing strategy
171
172     See § 4 Stage 2. The hybrid (dictionary ? template ? prompt) is the natural answer, but
the design of the rule-template config format and the candidate dictionary needs a user call.
173
174     > **Resolved (2026-05-30): Hybrid ? candidate dictionary ? per-sender rule template ?
interactive prompt**, caching the winning method per sender; password material never in the repo
or logs. The rule-template config format and dictionary design are deferred to implementation
(ROADMAP Milestone C.1). See [docs/ROADMAP.md](./docs/ROADMAP.md) § 1.
175
176     ### 8.3 LLM locality, per document category
177

```

178 The default is "local for sensitive, cloud allowed for non-sensitive after review." But
 which models? Ollama with Llama 3 / Qwen / Mistral on the user's hardware? A small fine-tuned
 model? Cloud (Anthropic / OpenAI / Gemini) with strict redaction for the non-sensitive
 categories?

179

180 This decision is closely coupled to the user's hardware capacity (GPU? RAM?) and
 willingness to wait on inference latency.

181

182 ### 8.4 First-slice scope confirmation

183

184 § 7 proposes "Gmail + one bank ? CLI". Confirm with user before building.

185

186 > **Resolved (2026-05-30): Build both tracks in parallel** ? the promo-corpus
 parse?store?query spine *and* the Gmail ingestion plumbing, as two decoupled tracks that
 converge at the first bank slice; first bank = **HDFC**. This refines § 7 (which proposed a
 single end-to-end bank slice first). See [`docs/ROADMAP.md`](./docs/ROADMAP.md) §§ 1 and 4.

187

188 ### 8.5 Merchant canonicalization approach

189

190 Fuzzy string matching has well-known failure modes on merchant names. Do we use a
 curated rules file (user maintains)? A library? A learned model? Defer until real data exposes
 the actual mess.

191

192 ## 9. Proposed (not built) file layout

193

194 This is a *suggestion* for once the language decision is made. Don't materialize it
 prematurely.

195

196 ```

197 muneem/
 198 ??? README.md
 199 ??? CLAUDE.md
 200 ??? DESIGN.md
 201 ??? docs/
 202 ? ??? prior-art-email-mining.md
 203 ? ??? (future: parser-notes-per-issuer.md, schema.md, threat-model.md)
 204 ??? testdata/
 205 ? ??? promo-emails/ # non-sensitive corpus (in repo)
 206 ? ??? statements/ # GITIGNORED ? real statements, never committed
 207 ??? (source tree TBD based on § 8.1)
 208 ??? (DB file, GITIGNORED)
 209 ```

210

211 ## 10. Glossary

212

213 - **Muneem** (?????) ? traditional Indian household/business bookkeeper.
 214 - **Issuer** ? the bank, card network, broker, or merchant that produced a document.
 215 - **Statement** ? a periodic financial summary from an issuer (bank statement, CC bill,
 broker statement, MF report).
 216 - **Holding** ? a position in an investment account at a point in time.
 217 - **Line item** ? a single transaction or itemized purchase row within a statement or
 receipt.

218 - **Born-digital PDF** ? a PDF generated from a software source with a real text layer
 (most bank statements). Contrast with scanned/image PDFs.

219

220 ## 11. Vision continuity from Email-Mining

221

222 The predecessor project (`~/Projects/Email-Mining`) extracted promo codes and expiry
 dates from manually-saved promo email PDFs, using PyMuPDF for text+image extraction and Gemini
 for image OCR and summarization. Its scope was just "find the coupon I forgot about."

223

224 muneem **subsumes** that scope. The deals/coupons feature survives as **one of several
 extraction targets** ? one of the analytics surfaces, fed by the same five-stage pipeline that
 handles bank statements and everything else. The promo-email PDFs from that project are
 preserved in [`testdata/promo-emails/`](./testdata/promo-emails/) as the first regression

corpus.

225

226 What was learned there and worth keeping: the PDF text+image extraction *shape* is right (but PyMuPDF/fitz itself is AGPL-3.0 ? muneem uses pdfminer.six (MIT) instead; see § 5 rule 8); image dedup by hash + SSIM is a real win for repeated chrome; the shape "page text + per-image OCR ? unified text ? structured extraction" generalizes well. What we change: programmatic Gmail/Outlook fetching instead of manual save-as-PDF; local extraction by default for sensitive categories; persistent normalized DB instead of one-shot stdout.

227

228 Full details in [docs/prior-art-email-mining.md](./docs/prior-art-email-mining.md).

229

228. assistant (2026-05-31T09:15:20.538Z)

DESIGN.md is already strongly "full financial life" in its framing ? good. Let me see the remaining ICICI doc contexts before editing:

229. user (2026-05-31T09:15:22.065Z)

```
C:\Users\avidu\Projects\muneem\docs\parser-architecture.md-7-> **Steps 1?2 of the plan (§10)
have landed** ? the verifier + concept schema, and the generic
docs\parser-architecture.md:8:> concept-mapping engine + profiles (ICICI runs through it; HDFC
is profiled). The Axis rewrite
C:\Users\avidu\Projects\muneem\docs\parser-architecture.md-9-> (step 3) is next; the §11 open
questions remain (generic-vs-profile has a provisional answer).
--
C:\Users\avidu\Projects\muneem\docs\parser-architecture.md-142- an instrument family
(equity/MF/ETF/bond) reuses the same control flow with its own concepts +
docs\parser-architecture.md:143: oracle. Issuer knowledge is `profiles.py` (`ICICI_PROFILE`,
`HDFC_PROFILE`) ? **data, not code**.
docs\parser-architecture.md:144: **ICICI now runs fully through the engine**; HDFC consumes
its profile (account pattern) while
C:\Users\avidu\Projects\muneem\docs\parser-architecture.md-145- keeping its bespoke
coordinate-clustering extraction (the documented `extract_tables` exception).
--
C:\Users\avidu\Projects\muneem\docs\DEPENDENCIES.md-38-| requests | Apache-2.0 | HTTP library
for Outlook provider |
docs\DEPENDENCIES.md:39:| pdfplumber | MIT | table extraction for statements (HDFC/ICICI/Axis) |
C:\Users\avidu\Projects\muneem\docs\DEPENDENCIES.md-40-| pytest *(dev)* | MIT | tests |
--
C:\Users\avidu\Projects\muneem\docs\CURRENT_STATE.md-20-[Omitted long context line]
docs\CURRENT_STATE.md:21:* **ICICI Bank** (`icici.py`): Transaction / savings statements
(`icici_transactions`); now **mapped through the generic concept-mapping engine** (`engine.py`)
driven by `ICICI_PROFILE`; **reconciles** + records status.
C:\Users\avidu\Projects\muneem\docs\CURRENT_STATE.md-22-
docs\CURRENT_STATE.md:23:[Omitted long matching line]
C:\Users\avidu\Projects\muneem\docs\CURRENT_STATE.md-24-
--
C:\Users\avidu\Projects\muneem\docs\CURRENT_STATE.md-27-## 3. Database & CLI
docs\CURRENT_STATE.md:28:* Schema v4 with `documents`, `offers`, and per-bank txn tables
(`axis_`, `icici_transactions`, `hdfc_transactions`). Dedup on sha256. No versioned migration
scripts yet (IF NOT EXISTS + user_version pragma).
docs\CURRENT_STATE.md-29-* CLI: `ingest <pdf>` (promo or bank_statement via parser), `offers
list`, `txns list <bank>`.
```

230. assistant (2026-05-31T09:16:21.887Z)

Now I have all ICICI doc contexts. CURRENT_STATE needs the most reframing, so let me re-read it fresh, then rewrite it with the total-finance view + ICICI removed + HDFC/Axis/AU focus:

231. user (2026-05-31T09:16:22.128Z)

```
1 # Current State
2
```

```

3      > **Last Updated:** 2026-06 (post tech-debt audit + ruff cleanup)
4      > This document summarizes the current capabilities, completed features, and near-term
next steps for `muneem`.
5      > See [ROADMAP.md](./ROADMAP.md) for the authoritative milestone tracking.
6
7      ## 1. Core Ingestion Pipeline
8      The ingestion pipeline supports a generic `SearchCriteria` interface and multiple
providers (Track B plumbing):
9
10     * **Gmail** (`muneem.ingestion.gmail`): Full OAuth2 read-only + search + attachment
download implemented.
11     * **IMAP** (`muneem.ingestion.imap`): Basic auth + search implemented; `get_headers` /
attachment methods are NotImplemented (experimental).
12     * **Outlook** (`muneem.ingestion.outlook`): Device-flow auth + basic search implemented;
attachment/header/download NotImplemented (experimental).
13
14     Companion one-time auth tools live in `tools/`. No high-level `muneem fetch` CLI command
yet (see Roadmap B.4).
15
16     ## 2. Statement Parsers (Track A + C progress)
17     Deterministic parsers live under `muneem.parsers`, registered via decorator and selected
by `get_parser` on text heuristics + sender. Submodules are **auto-loaded on package import**
(`autoload_parsers`), so the registry is populated in the production path (not just under
tests).
18
19     * **HDFC Bank** (`hdfc.py`): Savings / transaction-account statements via **coordinate
clustering** (deliberately *not* `extract_tables()` ? HDFC's table is semi-ruled). Extracts
date, narration, value date, ref, withdrawal, deposit, closing balance. **Self-validating:**
balance reconciliation + a cross-check against the bank's own end-of-statement SUMMARY (Dr/Cr
counts, opening/closing balance); records `documents.status` = `reconciled`/`unreconciled`.
Verified complete on real statements (37 & 31 txns) without inspecting any value.
20     * **Axis Bank** (`axis.py`): Savings (`axis_transactions`) + Credit Card
(`axis_cc_transactions`). ?? The savings `extract_tables()` mapping is **unreliable on real Axis
layouts** ? column counts and date placement vary between statements (e.g. Jun vs Oct 2025), so
it can mislabel rows. Reconciliation catches this, so as a **stop-gap the savings parser refuses
to store rows that don't reconcile (marks the document `unreconciled`, inserts 0) instead of
persisting garbage. **Pending real fix: an HDFC-style coordinate-clustering rewrite **validated
by total reconciliation (`opening` + `?Deposits` ? `?Withdrawals` == `closing`). Credit-card has no
running balance (parse/persist split + count only).
21     * **ICICI Bank** (`icici.py`): Transaction / savings statements (`icici_transactions`);
now **mapped through the generic concept-mapping engine** (`engine.py`) driven by
`ICICI_PROFILE`; **reconciles** + records status.
22
23     All balance-bearing parsers share one **verifier** (`muneem.parsers.validation`) and
split parsing from DB-insert, so the row logic is unit-testable without a PDF/DB and a parse is
validated before it's trusted. **The concept-core rework (parser-architecture.md step 1) has
landed: **the verifier now spans the per-row walk + total reconcile (`opening` + `?Cr` ? `?Dr` ==
`closing`) + a concept-level bookend + a recorded confidence verdict, all targeting a bank-
agnostic **concept schema** (`muneem.parsers.concepts`); **HDFC is refactored onto it as the
reference implementation** with no behavior change. **Step 2 has also landed:** the generic
concept-mapping engine (`engine.py` ? L0 header lexicon / profile column-order / L2 content-
search, family-agnostic via a `FamilySpec` + injected oracle) and issuer **profiles**
(`profiles.py`). **ICICI now runs fully through the engine**; HDFC is profiled (its bespoke
coordinate extraction stays ? the documented `extract_tables` exception). The **Axis coordinate-
clustering rewrite onto the engine is step 3** (the must-fix). Parsers use `pdfplumber` (with
optional password); HDFC parses by word-position clustering, Axis/ICICI by `extract_tables()`
(reconciliation will flag it if that drops rows on a real statement). **Password resolution**
(password_rules + hybrid unlock) is **not yet wired** into the main `muneem ingest --doc-type
bank_statement` path; it is exercised only via the `@pytest.mark.local_statements` test in
`tests/parsers/test_local_axis.py`. Unlock hybrid logic exists in `unlock.py` (pypdf +
candidates/prompt) and is tested.
24
25     See `password_rules.py` (now prefers user config dir) and `cli.py:ingest`.
26
27     ## 3. Database & CLI

```

```

28      * Schema v4 with `documents`, `offers`, and per-bank txn tables (`axis_`,
`icici_transactions`, `hdfc_transactions`). Dedup on sha256. No versioned migration scripts yet
(IF NOT EXISTS + user_version pragma).
29      * CLI: `ingest <pdf>` (promo or bank_statement via parser), `offers list`, `txns list
<bank>`.
30      * Config uses platformdirs for local dirs; secrets via keyring.
31
32      ## 4. Testing & Hygiene
33      * Strong offline regression on `testdata/promo-emails/` (T1/T3 in default CI).
34      * Security guards in `test_repo_hygiene.py` + license allowlist test.
35      * Local-only tests for real statements (gitignored data).
36
37      ## 5. Next Steps (Roadmap Alignment)
38      Focus remains on completing Milestone C (HDFC vertical slice + unlock wiring) then D
(harden, logging, evaluation) before generalize. Open items (DESIGN §8.3 LLM for sensitive, §8.5
merchant canonicalization) stay parked.
39
40      **Tech debt reduced in recent cleanup PR** (see PR for details): deduped helpers, DRY DB
inserts, improved password rule loading, ruff format enforcement, doc accuracy.
41
42      The project stays **local-first**, secrets in OS keychain, promo corpus safe for CI/LLM
experiments.
43

```

232. assistant (2026-05-31T09:16:58.216Z)

Now I'll rewrite CURRENT_STATE with the total-finance framing, ICICI removed, and HDFC/Axis/AU focus:

233. assistant (2026-05-31T09:17:19.884Z)

Current State

```

> **Last Updated:** 2026-06 (concept-core engine + issuer refocus: HDFC / Axis / AU Bank)
> Summarizes current capabilities, completed features, and near-term next steps for `muneem`.
> See [ `ROADMAP.md` ](./ROADMAP.md) for milestone tracking and [ `parser-
architecture.md` ](./parser-architecture.md) for the parser design.

> **Scope reminder ? muneem is a _whole-finance_ bookkeeper, not a bank tool.** Bank & credit-
card
> statements are the first vertical slice; **equities, mutual funds, ETFs, and bonds are core
> targets**, parsed into the same local ledger, with transaction/instrument-event classification
to
> follow. The parser core is deliberately **family-agnostic** (see §2), so adding an instrument
> family is *new data + a reconciliation rule*, not a new pipeline.

```

1. Core Ingestion Pipeline

The ingestion pipeline supports a generic `SearchCriteria` interface and multiple providers (Track B plumbing):

```

* **Gmail** (`muneem.ingestion.gmail`): Full OAuth2 read-only + search + attachment download
implemented.
* **IMAP** (`muneem.ingestion.imap`): Basic auth + search implemented; `get_headers` /
attachment methods are NotImplemented (experimental).
* **Outlook** (`muneem.ingestion.outlook`): Device-flow auth + basic search implemented;
attachment/header/download NotImplemented (experimental).

```

Companion one-time auth tools live in `tools/`. No high-level `muneem fetch` CLI command yet (see Roadmap B.4).

2. Statement Parsers (concept-core architecture)

Deterministic parsers live under `muneem.parsers`, registered via decorator and selected by `get\_parser` on text heuristics + sender. Submodules are **auto-loaded on package import**

(`\_autoload\_parsers`), so the registry is populated in the production path (not just under tests).

The parser core is **concept-first and verification-first** (see [parser-architecture.md](./parser-architecture.md)): an extractor maps an issuer's columns onto a bank-agnostic **concept schema** (`concepts.py`); a generic **engine** (`engine.py`) does the column-to-concept mapping (L0 header lexicon ? profile column-order ? L2 content search); and a shared **verifier** (`validation.py`) decides trust by *arithmetic* ? per-row balance walk + total reconcile + SUMMARY bookend + a recorded confidence verdict. The engine is **family-agnostic**: parameterized by a `FamilySpec` + an injected reconciliation oracle, so a future **instrument family** (equity/MF/ETF/bond holdings + buy/sell/dividend/fee events) reuses the same control flow with its own concepts + oracle. Issuer knowledge lives in `profiles.py` as **data, not code**.

Active issuers ? we have real statements + passwords, so these get validated on real data:

- * **HDFC Bank** (`hdfc.py`): Savings / transaction-account statements via **coordinate clustering** (deliberately *not* `extract_tables()` ? HDFC's table is semi-ruled). **Self-validating**: *running-balance reconciliation* + a cross-check against the bank's own SUMMARY (Dr/Cr counts, opening/closing balance); records `documents.status` = `'reconciled'/'unreconciled'`. Refactored onto the concept core as the **reference implementation**. ? **Verified on the user's 4 real statements** ? all reconcile *and* bookend, without inspecting any value.
- * **Axis Bank** (`axis.py`): Savings + Credit Card. ?? The old savings `extract_tables()` mapping was **unreliable across real layouts** (column counts and date placement vary between vintages ? e.g. Jun vs Oct 2025). **Rewrite in progress (step 3)**: **HDFC-style coordinate clustering** ? generic engine mapping ? total reconcile (`'opening' + ?Cr ? ?Dr == closing'`), oracle-gated so it can never persist unreconciled rows. Credit-card has no running balance (parse/persist split + count only).
- * **AU Bank** (planned, step-3 sibling): engine-driven profile on the same coordinate-clustering + total-reconcile path as Axis. 4 real statements on hand.

> **Dropped**: **ICICI** ? no statements on hand to validate against, so it was removed to focus effort on the issuers we can actually verify (HDFC/Axis/AU). It can return as a profile when data exists._

All balance-bearing parsers share the verifier and split parsing from DB-insert, so row logic is unit-testable without a PDF/DB and a parse is validated before it's trusted. **Concept-core steps 1?2 have landed** (the verifier + concept schema; the generic mapping engine + profiles). Parsers use `pdfplumber` (with optional password). **Password resolution** (`password_rules` + hybrid unlock in `unlock.py`) is **not yet wired** into the main `muneem ingest --doc-type bank_statement` path; it is exercised via the `@pytest.mark.local_statements` tests on real (gitignored) statements.

See `password_rules.py` (prefers user config dir) and `cli.py:ingest`.

3. Database & CLI

- * Schema v4 with `documents`, `offers`, and per-bank txn tables (`axis_*`, `hdfc_transactions`). Dedup on sha256. No versioned migration scripts yet (IF NOT EXISTS + `user_version` pragma). A unified `transactions` table (plus `holdings` for instruments) is planned once the per-bank schemas stabilize (parser-architecture.md §9).
- * CLI: `ingest <pdf>` (promo or bank_statement via parser), `offers list`, `txns list <bank>`.
- * Config uses `platformdirs` for local dirs; secrets via keyring.

4. Testing & Hygiene

- * Strong offline regression on `testdata/promo-emails/` + synthetic parser/engine fixtures (default CI).
- * Security guards in `test_repo_hygiene.py` + license allowlist test.
- * Local-only tests for real statements (gitignored data; passwords via env/keyring, never committed).

5. Next Steps

Near-term: finish the **Axis + AU** engine-driven parsers (step 3) and precision-test them on real statements alongside HDFC; wire password unlock into the main ingest path; then harden

(Milestone D). ****Beyond banks (core scope, not backlog):**** a second ****instrument concept family**** for broker / MF / ETF / bond statements ? `holdings` + buy/sell/dividend/fee classification. Parked: DESIGN §8.3 (local LLM for sensitive categories), §8.5 (merchant canonicalization).

The project stays ****local-first****, secrets in the OS keychain, promo corpus safe for CI/LLM experiments.

234. user (2026-05-31T09:17:19.906Z)

The file C:\Users\avidu\Projects\muneem\docs\CURRENT_STATE.md has been updated successfully. (file state is current in your context ? no need to Read it back)

235. assistant (2026-05-31T09:18:16.678Z)

Now the remaining docs ? README, DESIGN, DEPENDENCIES, parser-architecture, and ROADMAP ? reframed to total-finance + ICICI removed. Batching across files:

236. user (2026-05-31T09:18:20.428Z)

The file C:\Users\avidu\Projects\muneem\README.md has been updated successfully. (file state is current in your context ? no need to Read it back)

237. user (2026-05-31T09:18:21.842Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

238. user (2026-05-31T09:18:24.192Z)

The file C:\Users\avidu\Projects\muneem\DESIGN.md has been updated successfully. (file state is current in your context ? no need to Read it back)

239. user (2026-05-31T09:18:27.013Z)

The file C:\Users\avidu\Projects\muneem\docs\parser-architecture.md has been updated successfully. (file state is current in your context ? no need to Read it back)

240. user (2026-05-31T09:18:29.617Z)

The file C:\Users\avidu\Projects\muneem\docs\ROADMAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

241. assistant (2026-05-31T09:18:35.409Z)

`DEPENDENCIES.md` needs a read first. Let me read it and finish the remaining edits:

242. user (2026-05-31T09:18:35.835Z)

```
1      # muneem ? Dependency policy & license allowlist
2
3      muneem must be **safe to use commercially** and must **minimize paid dependencies**.
4      These are standing principles (DESIGN.md § 5, rules 8?9). This doc is the allowlist;
5      `tests/test_dependency_licenses.py` enforces it (a dependency not listed here fails CI).
6
7      ## Rules
8
9      1. Only permissive, commercial-safe licenses: **MIT, BSD (2/3-clause), Apache-2.0, ISC,
HPND**.
10     2. **Banned:** GPL, LGPL, **AGPL**, and source-available / non-commercial licenses.
11         ? **PyMuPDF / `fitz` is AGPL-3.0 ? do not use it.** (Its own metadata reads
12         **"Dual Licensed - GNU AFFERO GPL 3.0 or Artifex Commercial"*; the non-AGPL option
13         is a *paid* license, which also violates rule 4.)
14     3. **MPL-2.0** (weak, file-level copyleft) only as a fallback when no permissive option
```



```

15     exists, and flagged here.
16 4. Prefer **local / self-hosted / open-source** runtimes over paid APIs. Paid APIs
17 (cloud LLMs, paid OCR/vision) are a **fallback only**, gated behind explicit
approval.
18 We are willing to **train our own** image/OCR models rather than take a paid
dependency.
19 5. Adding a dependency requires updating **both** this table **and** the allowlist in
20 `tests/test_dependency_licenses.py` ? otherwise the build fails.
21
22 ## Current dependencies
23
24 | Package | License | Role |
25 |---|---|---|
26 | typer | MIT | CLI |
27 | platformdirs | MIT* | OS config/data dirs |
28 | keyring | MIT | OS keychain (secrets) |
29 | pdfminer.six | MIT | PDF text extraction (replaces AGPL PyMuPDF) |
30 | pypdfium2 | Apache-2.0 / BSD-3 | PDF page rendering for OCR |
31 | pytesseract | Apache-2.0 | local OCR (Tesseract) wrapper |
32 | Pillow | HPND | image handling for OCR |
33 | pypdf | BSD-3-Clause | decrypt password-protected statements (Stage 2) |
34 | google-api-python-client | Apache-2.0 | Gmail API (read-only) |
35 | google-auth | Apache-2.0 | OAuth credentials |
36 | google-auth-oauthlib | Apache-2.0 | installed-app OAuth consent flow |
37 | msal | MIT | Microsoft Graph API OAuth |
38 | requests | Apache-2.0 | HTTP library for Outlook provider |
39 | pdfplumber | MIT | table extraction for statements (HDFC/ICICI/Axis) |
40 | pytest *(dev)* | MIT | tests |
41 | ruff *(dev)* | MIT | lint / format |
42 | PyYAML *(dev)* | MIT* | read the test answer key |
43
44 \* PyMuPDF (AGPL), PyYAML (MIT) and platformdirs (MIT) were confirmed directly from
45 installed package metadata; the rest are MIT per their upstream projects.
46
47 ## System tools (not Python packages)
48
49 - **Tesseract OCR** (Apache-2.0) ? local OCR engine, invoked via `pytesseract`. Install
50 separately (e.g. `winget install UB-Mannheim.TesseractOCR`). muneem finds it on PATH,
51 at the Windows default location, or via the `MUNEEM_TESSERACT_CMD` env var.
52
53 ## Current / recently added (permissive)
54
55 - **pypdf** (BSD-3) ? PDF decryption (Stage 2, used by unlock.py and parsers). Chosen
over
56 pikepdf (MPL-2.0 flagged fallback) for license simplicity.
57 - **scikit-image / numpy** (BSD) ? planned for image dedup (SSIM) in future; not yet in
58 pyproject (Pillow covers basic image needs for OCR today).
59
60 ## Planned (future, permissive, no paid)
61
62 - A **purpose-trained OCR model** for stylized promo codes (Tesseract is approximate on
63 some banners like Swiggy). Aligns with "train our own" (DESIGN §5 rule 9).
64
65 **Optional deps** (outlook extra): msal + requests ? only loaded when OutlookProvider is
66 used; guarded by ImportError in code.
67

```

243. assistant (2026-05-31T09:18:50.694Z)

Now the final doc edits:

244. user (2026-05-31T09:18:52.590Z)

The file C:\Users\avidu\Projects\muneem\docs\DEPENDENCIES.md has been updated successfully.
(file state is current in your context ? no need to Read it back)

245. user (2026-05-31T09:18:54.002Z)

The file C:\Users\avidu\Projects\muneem\DESIGN.md has been updated successfully. (file state is current in your context ? no need to Read it back)

246. user (2026-05-31T09:18:57.769Z)

The file C:\Users\avidu\Projects\muneem\docs\parser-architecture.md has been updated successfully. (file state is current in your context ? no need to Read it back)

247. user (2026-05-31T09:19:00.188Z)

The file C:\Users\avidu\Projects\muneem\docs\ROADMAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

248. assistant (2026-05-31T09:19:07.506Z)

Last doc edit ? step 3 in the plan now covers Axis + AU:

249. user (2026-05-31T09:19:09.725Z)

The file C:\Users\avidu\Projects\muneem\docs\parser-architecture.md has been updated successfully. (file state is current in your context ? no need to Read it back)

250. user (2026-05-31T09:19:22.683Z)

```
--- any icici left anywhere (excluding CLAUDE.md general bank list)? ---
docs/CURRENT_STATE.md:33:> _Dropped: **ICICI** ? no statements on hand to validate against, so
it was removed to focus effort on the issuers we can actually verify (HDFC/Axis/AU). It can
return as a profile when data exists._
docs/ROADMAP.md:19:> coordinate-clustering + total-reconcile path. **ICICI was removed** (no
statements on hand to
--- lint + offline suite ---
All checks passed!
===== XPASSES =====
===== short test summary info =====
XPASS tests/test_ocr.py::test_swiggy_image_code_recovered_fuzzily - recovery of the stylized
boxed code is OCR-engine/version dependent ? succeeds on Tesseract 5.4, returns nothing on the
apt build; the planned fix is a purpose-trained model (docs/DEPENDENCIES.md). strict=False so it
never fails the build.
103 passed, 9 deselected, 1 xpassed in 6.19s
```

251. assistant (2026-05-31T09:19:36.298Z)

Only the two intentional "ICICI was dropped" notes remain ? correct. Lint clean, 103 pass.
Shipping PR1 (refocus):

252. user (2026-05-31T09:19:41.582Z)

```
[chore/refocus-remove-icici 5c07b40] chore(refocus): remove ICICI; reframe docs to the whole-
finance view (HDFC/Axis/AU)
15 files changed, 64 insertions(+), 283 deletions(-)
delete mode 100644 src/muneem/parsers/icici.py
delete mode 100644 tests/parsers/test_icici.py
```

253. user (2026-05-31T09:19:46.248Z)

branch 'chore/refocus-remove-icici' set up to track 'origin/chore/refocus-remove-icici'.
To <https://github.com/avidullu/muneem.git>
* [new branch] chore/refocus-remove-icici -> chore/refocus-remove-icici

254. user (2026-05-31T09:19:58.015Z)

<https://github.com/avidullu/muneem/pull/24>

255. user (2026-05-31T09:20:08.754Z)

```
15 files changed, 64 insertions(+), 283 deletions(-)
delete mode 100644 src/muneem/parsers/icici.py
delete mode 100644 tests/parsers/test_icici.py
Already on 'main'
- [deleted]          (none)      -> origin/chore/refocus-remove-icici
Already up to date.
2a589ee chore(refocus): remove ICICI; reframe docs to the whole-finance view (HDFC/Axis/AU)
(#24)
```

256. assistant (2026-05-31T09:21:06.015Z)

PR1 merged as [muneem#24](<https://github.com/avidullu/muneem/pull/24>) ? ICICI gone, docs now lead with the whole-finance view.

Now ****Step 3 ? Axis rewrite****, and crucially I can now ****validate against your 4 real Axis statements**** (with `SUNE0106`, surfacing counts/booleans only). Let me set up a fresh branch from the refocused main and rebuild the coordinate-clustering module:

257. user (2026-05-31T09:21:06.679Z)

Updated task #7 status

258. user (2026-05-31T09:21:07.369Z)

Updated task #4 status

259. user (2026-05-31T09:21:08.453Z)

```
Deleted branch feat/axis-coordinate-clustering (was 7ea7f7a).
stash@{0}: On feat/axis-coordinate-clustering: axis-step3-wip
Switched to a new branch 'feat/axis-clustering-rewrite'
feat/axis-clustering-rewrite
```

260. user (2026-05-31T09:21:41.240Z)

File created successfully at: C:\Users\avidu\Projects\muneem\src\muneem\parsers\clustering.py
(file state is current in your context ? no need to Read it back)

261. assistant (2026-05-31T09:21:27.408Z)

Now rebuild the coordinate-clustering module and add the Axis profile:

262. assistant (2026-05-31T09:21:41.222Z)

"""Coordinate clustering ? turn positioned words into a clean cell-matrix.

When ``pdfplumber.extract\_tables()`` mis-groups a semi-ruled or whitespace-aligned statement (the Axis case: a date merged into the narration cell, column counts drifting between vintages), we fall back to reading ****word positions**** directly. Words are grouped into visual rows by their ``top``, column boundaries are inferred from the vertical whitespace shared across all rows, and each word is dropped into the column its x-centre falls in. The result is a rectangular cell-matrix the generic ``engine`` can map ? and because columns come from geometry, a leading date sits in its **own** column instead of being glued to the narration.

This is bank-agnostic extraction (no issuer constants, no x-positions hard-coded); the engine's oracle still decides whether the result is trustworthy. Pure geometry in, strings out ? no value is logged.

```
"""
```

```
from __future__ import annotations
```

```
from collections.abc import Sequence
```

```
Word = dict # a pdfplumber word: {"text", "x0", "x1", "top", "bottom", ...}
```

```
def group_lines(words: Sequence[Word], tol: float = 2.5) -> list[list[Word]]:
    """Group words into visual lines by ``top`` (tolerant of sub-pixel jitter), left-to-right."""
    lines: list[list[Word]] = []
    anchor_top: float | None = None
    for w in sorted(words, key=lambda w: (w["top"], w["x0"])):
        if lines and anchor_top is not None and abs(w["top"] - anchor_top) <= tol:
            lines[-1].append(w)
        else:
            lines.append([w])
            anchor_top = w["top"]
    return [sorted(line, key=lambda w: w["x0"]) for line in lines]
```

```
def infer_columns(words: Sequence[Word], gap: float = 10.0) -> list[tuple[float, float]]:
    """Infer column x-extents by merging word x-intervals separated by less than ``gap`` points.
```

```

    The gaps wider than ``gap`` are the inter-column whitespace; everything else coalesces into
    a
    column. Returns ``(x_start, x_end)`` per column, left to right.
    """
```

```
    intervals = sorted((float(w["x0"]), float(w["x1"])) for w in words)
    merged: list[list[float]] = []
    for x0, x1 in intervals:
        if merged and x0 <= merged[-1][1] + gap:
            merged[-1][1] = max(merged[-1][1], x1)
        else:
            merged.append([x0, x1])
    return [(a, b) for a, b in merged]
```

```
def _column_of(center: float, columns: list[tuple[float, float]]) -> int:
    for i, (a, b) in enumerate(columns):
        if a <= center <= b:
            return i
    return min(
        range(len(columns)),
        key=lambda i: min(abs(center - columns[i][0]), abs(center - columns[i][1])),
    )
```

```
def line_to_cells(line: Sequence[Word], columns: list[tuple[float, float]]) -> list[str]:
    """Place each word into its column; join words that share a column (in x order)."""
    cells = [" " for _ in columns]
    for w in line:
        idx = _column_of((float(w["x0"]) + float(w["x1"])) / 2, columns)
        cells[idx] = (cells[idx] + " " + str(w["text"])).strip()
    return cells
```

```
def cluster_to_matrix(
    words_by_page: Sequence[Sequence[Word]], *, line_tol: float = 2.5, col_gap: float = 10.0
```

```

) -> list[list[str]]:
    """Cluster words from one or more pages into a single cell-matrix with consistent columns.

    Column boundaries are inferred **once across all pages** (a statement keeps its layout), so
rows
    from every page land in the same columns; rows are emitted in page-then-top order.
    """
    all_words = [w for page in words_by_page for w in page]
    if not all_words:
        return []
    columns = infer_columns(all_words, col_gap)
    matrix: list[list[str]] = []
    for page in words_by_page:
        for line in group_lines(page, line_tol):
            matrix.append(line_to_cells(line, columns))
    return matrix

__all__ = ["group_lines", "infer_columns", "line_to_cells", "cluster_to_matrix"]

```

263. assistant (2026-05-31T09:21:54.675Z)

Now add `AXIS\_PROFILE` to profiles.py (two edits):

264. user (2026-05-31T09:21:59.521Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\profiles.py has been updated successfully. (file state is current in your context ? no need to Read it back)

265. user (2026-05-31T09:22:08.194Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\profiles.py has been updated successfully. (file state is current in your context ? no need to Read it back)

266. assistant (2026-05-31T09:23:01.422Z)

Now the core rewrite ? `axis.py` savings onto coordinate clustering + engine + total reconcile (CC parser preserved):

267. assistant (2026-05-31T09:23:32.551Z)

```

"""Axis Bank statement parsers: savings/transaction account + credit card.

**Savings now runs on the concept core.** Extraction proposes candidate cell-matrices ?
coordinate
clustering (whole statement, consistent columns; handles the date-merged / column-count drift
across
vintages) and, as a fallback, `extract_tables()` ? and the generic `engine` maps each onto
the
canonical concepts. **Reconciliation disposes:** a parse is trusted only if the per-row balance
walks
or (when there's no per-row balance, e.g. some vintages) the totals reconcile against the
printed
`opening + ?Cr ? ?Dr == closing`. The parser is **oracle-gated**: it persists only an endorsed
parse, so a layout it can't yet read stores nothing (status `unreconciled`) rather than
mislabeled
rows. This replaces the old fixed-column `extract_tables()` mapping that mislabeled rows.

Credit-card statements carry no running balance, so `AxisCreditCardParser` can't reconcile
(parse/persist split + count only). No statement content is logged here.
"""

from __future__ import annotations

```

```

import re
import sqlite3
from dataclasses import dataclass, field
from pathlib import Path

import pdfplumber

from muneem import db as dbmod

from . import register_parser
from .base import StatementParser
from .clustering import cluster_to_matrix
from .concepts import ConceptRow, StatementConcepts
from .engine import map_table
from .profiles import AXIS_PROFILE
from .validation import Confidence, parse_amount, reconcile_running_balance, reconcile_total

_CC_DATE_RE = re.compile(r"\d{2}/\d{2}/\d{4}") # Axis credit card: dd/mm/yyyy
_CARD_RES = [re.compile(r"\d{4}XXX\d{4}"), re.compile(r"(\d{6}\*\d{4})")]
_OPENING_RE = re.compile(r"opening\s+balance[^\d-]*(-?\d{1,2}|\.\d{2})", re.IGNORECASE)
_CLOSING_RE = re.compile(r"closing\s+balance[^\d-]*(-?\d{1,2}|\.\d{2})", re.IGNORECASE)

def _first_match(text: str, patterns: list[re.Pattern]) -> str | None:
    for rx in patterns:
        m = rx.search(text)
        if m:
            return m.group(1).strip()
    return None

```

--- Savings / transaction account (concept core) -----

```

@dataclass
class AxisSavings:
    """Outcome of a savings parse: concept rows + whether reconciliation endorsed them."""

    account_number: str
    rows: list[ConceptRow] = field(default_factory=list)
    verified: bool = False

def parse_axis_summary(text: str) -> StatementConcepts:
    """Read opening/closing balance from the statement text (for the total-reconcile oracle).

    Best-effort regex; ``None`` when not found (the oracle then relies on the per-row walk).
    """
    opening = _first_match(text, [_OPENING_RE])
    closing = _first_match(text, [_CLOSING_RE])
    return StatementConcepts(
        opening_balance=parse_amount(opening),
        closing_balance=parse_amount(closing),
    )

def _savings_oracle(opening: float | None, closing: float | None):
    """Trust a parse if the per-row balance walks, else if the totals reconcile to the
    bookends."""

    def check(rows: list[ConceptRow]) -> bool:
        if reconcile_running_balance([(r.debit, r.credit, r.balance) for r in rows]).ok:
            return True
        if rows and opening is not None and closing is not None:

```

```

        return reconcile_total(rows, opening, closing)
    return False

return check

def parse_axis_savings(pdf_path: Path, password: str | None = None) -> AxisSavings:
    """Open an Axis savings statement and return concept rows + a reconciliation verdict.

    Tries a coordinate-clustered matrix (whole statement) first, then concatenated
    ``extract_tables()`` rows, and keeps the first mapping the reconciliation oracle endorses.
    """
    with pdfplumber.open(pdf_path, password=password) as pdf:
        words_by_page = [
            page.extract_words(use_text_flow=False, keep_blank_chars=False) for page in
pdf.pages
        ]
        extracted = [row for page in pdf.pages for t in (page.extract_tables() or []) for row in
t]
        head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))

        account_number = AXIS_PROFILE.first_account(head) or "UNKNOWN"
        summary = parse_axis_summary(head)
        oracle = _savings_oracle(summary.opening_balance, summary.closing_balance)

        candidates = [cluster_to_matrix(words_by_page), extracted]
        best: AxisSavings | None = None
        for matrix in candidates:
            result = map_table(matrix, AXIS_PROFILE, oracle=oracle)
            if result is None:
                continue
            if result.verified:
                return AxisSavings(account_number, result.rows, True)
            if best is None:
                best = AxisSavings(account_number, result.rows, False)
        return best or AxisSavings(account_number, [], False)

@register_parser
class AxisParser(StatementParser):
    def can_handle(self, text: str, sender: str | None = None, subject: str | None = None) ->
bool:
        if sender and "axisbank.com" in sender.lower():
            return True
        t = text or ""
        return "Axis Bank" in t and ("Statement of Account" in t or "Account Statement" in t)

    def parse_and_insert(
        self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str | None = None
    ) -> int:
        """Persist a savings parse **only if it reconciles**; otherwise store nothing.

        Sets ``documents.status`` to ``reconciled`` / ``unreconciled`` accordingly, so a layout
the
        parser can't yet read surfaces loudly instead of saving mislabeled rows.
        """
        try:
            parsed = parse_axis_savings(pdf_path, password)
        except Exception as exc:
            raise RuntimeError(f"Failed to parse Axis statement {pdf_path}: {exc}") from exc

        if not parsed.verified:
            conn.execute(
                "UPDATE documents SET status = ? WHERE id = ?",

```

```

        (Confidence.UNRECONCILED.value, doc_id),
    )
    return 0

for r in parsed.rows:
    dbmod.insert_axis_transaction(
        conn,
        document_id=doc_id,
        account_number=parsed.account_number,
        txn_date=r.txn_date,
        value_date=r.value_date or "",
        particulars=r.narration,
        debit=r.debit,
        credit=r.credit,
        balance=r.balance,
    )
    conn.execute(
        "UPDATE documents SET status = ? WHERE id = ?",
        (Confidence.RECONCILED.value, doc_id),
    )
return len(parsed.rows)

```

--- Credit card (no running balance; count only) -----

```

@dataclass
class AxisCCTxn:
    txn_date: str
    particulars: str
    merchant_category: str
    amount: float
    type: str # "Cr" or "Dr"

def _rows_from_cc_tables(tables: list) -> list[AxisCCTxn]:
    """Build credit-card transactions from extracted tables. Pure: no PDF, no DB."""
    rows: list[AxisCCTxn] = []
    for table in tables or []:
        for row in table or []:
            if not row or len(row) < 9:
                continue
            date_str = str(row[0]).strip() if row[0] else ""
            if not _CC_DATE_RE.match(date_str):
                continue
            amount_str = str(row[8]).strip() if row[8] else ""
            is_cr = "Cr" in amount_str
            amount = parse_amount(re.sub(r"^\d.", "", amount_str)) or 0.0
            rows.append(
                AxisCCTxn(
                    txn_date=date_str,
                    particulars=str(row[2]).strip() if row[2] else "",
                    merchant_category=str(row[7]).strip() if row[7] else "",
                    amount=amount,
                    type="Cr" if is_cr else "Dr",
                )
            )
    return rows

def parse_axis_cc(pdf_path: Path, password: str | None = None) -> tuple[str, list[AxisCCTxn]]:
    """Open an Axis credit-card statement and return (card_number, transactions)."""
    card_number = "UNKNOWN"
    rows: list[AxisCCTxn] = []

```



```

with pdfplumber.open(pdf_path, password=password) as pdf:
    head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))
    card_number = _first_match(head, _CARD_RES) or "UNKNOWN"
    for page in pdf.pages:
        rows.extend(_rows_from_cc_tables(page.extract_tables()))
    return card_number, rows

@register_parser
class AxisCreditCardParser(StatementParser):
    def can_handle(self, text: str, sender: str | None = None, subject: str | None = None) ->
bool:
        return "Axis Bank" in text and "Credit Card Statement" in text

    def parse_and_insert(
        self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str | None = None
    ) -> int:
        try:
            card_number, rows = parse_axis_cc(pdf_path, password)
        except Exception as exc:
            raise RuntimeError(f"Failed to parse Axis CC statement {pdf_path}: {exc}") from exc
        for t in rows:
            dbmod.insert_axis_cc_transaction(
                conn,
                document_id=doc_id,
                account_number=card_number,
                txn_date=t.txn_date,
                particulars=t.particulars,
                merchant_category=t.merchant_category,
                amount=t.amount,
                type=t.type,
            )
        # Credit-card statements carry no running balance, so reconciliation does not apply;
        # documents.status is left as set by the caller.
        return len(rows)

```

268. user (2026-05-31T09:23:32.586Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py has been updated successfully. (file state is current in your context ? no need to Read it back)

269. assistant (2026-05-31T09:23:49.134Z)

Now the real test. Let me run the new Axis parser against your 4 real statements (`SUNE0106`, structural booleans/counts only ? no values surface):

270. user (2026-05-31T09:23:54.429Z)

AXIS BANK	Statement for August 2024.pdf	account=MISSING rows= 6 verified=True
AXIS BANK	Statement for June 2025.pdf	account=MISSING rows= 3 verified=False
AXIS BANK	Statement for May 2025.pdf	account=MISSING rows= 7 verified=False
AXIS BANK	Statement for October 2025.pdf	account=MISSING rows= 15 verified=True

271. assistant (2026-05-31T09:24:27.082Z)

Real progress ? **2 of 4 verify** (Aug 2024: 6 rows, Oct 2025: 15 rows). June/May don't reconcile yet, and the account regex misses on all. Let me debug **structurally** (column map, layer, balance presence, summary/label detection ? never values):

272. user (2026-05-31T09:24:35.728Z)

```

AXIS BANK Statement for August 2024.pdf
clustered rows=169 ncols=1 | summary open=True close=True

```

```

labels=['account no', 'a/c no', 'customer id', 'account statement']
layer=None mapped_cols=[] nrows=0 nbal=0 verified=None
AXIS BANK Statement for June 2025.pdf
clustered rows=143 ncols=1 | summary open=True close=True
labels=['account no', 'account number', 'a/c no', 'customer id', 'account statement']
layer=None mapped_cols=[] nrows=0 nbal=0 verified=None
AXIS BANK Statement for May 2025.pdf
clustered rows=141 ncols=1 | summary open=True close=True
labels=['account no', 'account number', 'a/c no', 'customer id', 'account statement']
layer=None mapped_cols=[] nrows=0 nbal=0 verified=None
AXIS BANK Statement for October 2025.pdf
clustered rows=167 ncols=1 | summary open=True close=True
labels=['account no', 'account number', 'a/c no', 'customer id', 'account statement']
layer=None mapped_cols=[] nrows=0 nbal=0 verified=None

```

273. assistant (2026-05-31T09:25:34.481Z)

Key finding: the **clustered** matrix collapsed to `ncols=1` (my `infer_columns`` gets saturated by the wide narration column across 169 lines) ? so the 2 wins (Aug/Oct) came from the **extract_tables** fallback, and Jun/May fail there too. Let me diagnose the `extract_tables` path + per-table shapes (counts/shapes/booleans only):

274. user (2026-05-31T09:25:42.744Z)

```

AXIS BANK Statement for August 2024.pdf: tables=9 shapes=[(1, 4), (6, 6), (10, 8), (9, 5), (4,
8), (4, 8), (4, 8), (4, 10), (11, 4)]
layer=L0 cols=['balance', 'credit', 'debit', 'narration', 'txn_date'] nrows=6 nbal=6
perrow=True total=True verified=True
AXIS BANK Statement for June 2025.pdf: tables=7 shapes=[(1, 4), (6, 6), (9, 8), (6, 6), (2,
10), (6, 6), (11, 4)]
layer=L0 cols=['balance', 'credit', 'debit', 'narration', 'txn_date', 'value_date'] nrows=3
nbal=1 perrow=False total=False verified=False
AXIS BANK Statement for May 2025.pdf: tables=6 shapes=[(1, 4), (6, 6), (9, 8), (6, 8), (6, 6),
(11, 4)]
layer=L0 cols=['balance', 'credit', 'debit', 'narration', 'txn_date'] nrows=7 nbal=7
perrow=False total=False verified=False
AXIS BANK Statement for October 2025.pdf: tables=7 shapes=[(1, 4), (6, 6), (19, 9), (4, 7), (2,
10), (6, 6), (11, 4)]
layer=L0 cols=['balance', 'credit', 'debit', 'narration', 'txn_date'] nrows=15 nbal=0
perrow=False total=True verified=True

```

275. assistant (2026-05-31T09:27:15.624Z)

The clustering's `infer_columns`` saturates because it projects *all* 169 lines (headers, address, summary), where transactions are a tiny fraction. The fix: `infer_columns` from **transaction-like lines only** (date + money token), and use a **coverage histogram** so an occasional wide line can't collapse columns. Let me rewrite the clustering with a row-filter and coverage-based column detection:

276. assistant (2026-05-31T09:27:36.640Z)

""Coordinate clustering ? turn positioned words into a clean cell-matrix.

When `pdfplumber.extract_tables()` mis-groups a semi-ruled or whitespace-aligned statement (the Axis case: a date merged into the narration cell, column counts drifting between vintages), we read **word positions** directly. Words are grouped into visual rows by their `top``; column boundaries are inferred from the vertical whitespace **shared across the transaction rows**; and each word is dropped into the column its x-centre falls in. Because columns come from geometry, a leading date sits in its *own* column instead of being glued to the narration.

Two robustness lessons baked in:

```
* **Infer columns from transaction-like rows only.** Headers, address blocks, and summaries span
the
page and would erase the inter-column whitespace. A caller-supplied ``keep_line`` predicate
(e.g.
"has a date *and* a money token") restricts both column inference and the emitted matrix to
the
transaction band.
* **Use a coverage histogram, not interval-merging.** A column boundary is an x-channel that is
empty
in *most* rows; a single long narration can't collapse two columns.
```

Bank-agnostic extraction (no issuer constants, no x-positions hard-coded); the engine's oracle
still
decides whether the result is trustworthy. Pure geometry in, strings out ? no value is logged.
"""

```
from __future__ import annotations
```

```
from collections.abc import Callable, Sequence
```

```
Word = dict # a pdfplumber word: {"text", "x0", "x1", "top", "bottom", ...}
```

```
def group_lines(words: Sequence[Word], tol: float = 2.5) -> list[list[Word]]:
    """Group words into visual lines by ``top`` (tolerant of sub-pixel jitter), left-to-
right."""
    lines: list[list[Word]] = []
    anchor_top: float | None = None
    for w in sorted(words, key=lambda w: (w["top"], w["x0"])):
        if lines and anchor_top is not None and abs(w["top"] - anchor_top) <= tol:
            lines[-1].append(w)
        else:
            lines.append([w])
            anchor_top = w["top"]
    return [sorted(line, key=lambda w: w["x0"]) for line in lines]
```

```
def infer_columns(
    lines: Sequence[Sequence[Word]], bin_w: float = 1.0, min_frac: float = 0.3
) -> list[tuple[float, float]]:
    """Infer column x-extents from a coverage histogram over ``lines``.
```

An x-bin is "filled" if at least ``min\_frac`` of the lines have a word covering it; columns
are

the runs of filled bins, and the empty runs between them are the inter-column whitespace.

Robust

to a few full-width lines because it thresholds on the *fraction* of rows, not on any single
one.

"""

```
words = [w for line in lines for w in line]
if not words:
    return []
x0 = min(float(w["x0"]) for w in words)
x1 = max(float(w["x1"]) for w in words)
nbins = max(1, int((x1 - x0) / bin_w) + 1)
cover = [0] * nbins
for line in lines:
    marked: set[int] = set()
    for w in line:
        a = int((float(w["x0"]) - x0) / bin_w)
        b = int((float(w["x1"]) - x0) / bin_w)
        marked.update(range(max(0, a), min(nbins - 1, b) + 1))
    for k in marked:
```

```

        cover[k] += 1
    thresh = max(1.0, min_frac * len(lines))
    filled = [c >= thresh for c in cover]
    cols: list[tuple[float, float]] = []
    start: int | None = None
    for i, f in enumerate(filled):
        if f and start is None:
            start = i
        elif not f and start is not None:
            cols.append((x0 + start * bin_w, x0 + i * bin_w))
            start = None
    if start is not None:
        cols.append((x0 + start * bin_w, x0 + nbins * bin_w))
    return cols

def _column_of(center: float, columns: list[tuple[float, float]]) -> int:
    for i, (a, b) in enumerate(columns):
        if a <= center <= b:
            return i
    return min(
        range(len(columns)),
        key=lambda i: min(abs(center - columns[i][0]), abs(center - columns[i][1])),
    )

def line_to_cells(line: Sequence[Word], columns: list[tuple[float, float]]) -> list[str]:
    """Place each word into its column; join words that share a column (in x order)."""
    cells = [" " for _ in columns]
    for w in line:
        idx = _column_of((float(w["x0"]) + float(w["x1"])) / 2, columns)
        cells[idx] = (cells[idx] + " " + str(w["text"])).strip()
    return cells

def cluster_to_matrix(
    words_by_page: Sequence[Sequence[Word]],
    *,
    line_tol: float = 2.5,
    min_frac: float = 0.3,
    keep_line: Callable[[list[str]], bool] | None = None,
) -> list[list[str]]:
    """Cluster words across pages into one cell-matrix with consistent columns.

    ``keep_line`` (texts of a line ? bool) restricts both column inference and the output to the
    transaction band ? without it, full-width headers/footers swamp the column structure.
Columns
are inferred once across all kept lines (a statement keeps its layout); rows are emitted in
page-then-top order.
    """
    lines: list[list[Word]] = []
    for page in words_by_page:
        lines.extend(group_lines(page, line_tol))
    if keep_line is not None:
        lines = [ln for ln in lines if keep_line([str(w["text"]) for w in ln])]
    if not lines:
        return []
    columns = infer_columns(lines, min_frac=min_frac)
    if not columns:
        return []
    return [line_to_cells(line, columns) for line in lines]

__all__ = ["group_lines", "infer_columns", "line_to_cells", "cluster_to_matrix"]

```

277. user (2026-05-31T09:27:36.658Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\clustering.py has been updated successfully. (file state is current in your context ? no need to Read it back)

278. assistant (2026-05-31T09:27:50.237Z)

Now wire a transaction-line predicate into the Axis clustering call:

279. user (2026-05-31T09:27:54.921Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py has been updated successfully. (file state is current in your context ? no need to Read it back)

280. user (2026-05-31T09:27:58.779Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py has been updated successfully. (file state is current in your context ? no need to Read it back)

281. assistant (2026-05-31T09:28:04.080Z)

Now re-run the diagnostic on the **clustered** path with the row filter (structural only):

282. user (2026-05-31T09:28:16.100Z)

```
AXIS BANK Statement for August 2024.pdf
  CLUSTER matrix rows=10 ncols=8 | layer=L2 cols=['balance', 'credit', 'debit', 'narration',
'txn_date', 'value_date'] nrows=4 nbal=3 perrow=False total=False
  END-TO-END verified=True rows=6
AXIS BANK Statement for June 2025.pdf
  CLUSTER matrix rows=9 ncols=6 | layer=L2 cols=['credit', 'debit', 'narration', 'txn_date',
'value_date'] nrows=8 nbal=0 perrow=False total=False
  END-TO-END verified=False rows=8
AXIS BANK Statement for May 2025.pdf
  CLUSTER matrix rows=9 ncols=5 | layer=L2 cols=['balance', 'credit', 'debit', 'narration',
'txn_date'] nrows=9 nbal=4 perrow=False total=False
  END-TO-END verified=False rows=9
AXIS BANK Statement for October 2025.pdf
  CLUSTER matrix rows=17 ncols=3 | layer=None cols=None nrows=0 nbal=0 perrow=False total=None
  END-TO-END verified=True rows=15
```

283. assistant (2026-05-31T09:29:19.304Z)

Clustering now yields multi-column matrices, but it's not the winning candidate for any statement ? extract_tables+engine carries Aug/Oct, and June/May fail both. My hypothesis: **concatenating all tables misaligns columns** (each statement has 6?9 tables of different widths). Let me probe per-table mapping to see if the transaction table reconciles on its own:

284. user (2026-05-31T09:29:26.640Z)

```
AXIS BANK Statement for August 2024.pdf (open=True close=True)
  table2 shape=(10, 8) layer=L0 cols=['balance', 'credit', 'debit', 'narration', 'txn_date']
nrows=6 nbal=6 verified=True
  table7 shape=(4, 10) layer=L2 cols=['balance', 'credit', 'debit', 'narration', 'txn_date',
'value_date'] nrows=3 nbal=3 verified=False
AXIS BANK Statement for June 2025.pdf (open=True close=True)
  table3 shape=(6, 6) layer=L0 cols=['balance', 'credit', 'debit', 'narration', 'txn_date',
'value_date'] nrows=2 nbal=0 verified=False
  table4 shape=(2, 10) layer=L2 cols=['balance', 'credit', 'debit', 'narration', 'txn_date',
'value_date'] nrows=1 nbal=1 verified=False
AXIS BANK Statement for May 2025.pdf (open=True close=True)
  table2 shape=(9, 8) layer=L0 cols=['balance', 'credit', 'debit', 'narration', 'txn_date']
nrows=5 nbal=5 verified=True
  table3 shape=(6, 8) layer=L0 cols=['balance', 'credit', 'debit', 'narration', 'txn_date']
```

```
nrows=2 nbal=2 verified=True
AXIS BANK Statement for October 2025.pdf (open=True close=True)
    table2 shape=(19, 9) layer=L0 cols=['balance', 'credit', 'debit', 'narration', 'txn_date']
nrows=15 nbal=0 verified=True
    table4 shape=(2, 10) layer=L2 cols=['balance', 'credit', 'debit', 'narration', 'txn_date',
'value_date'] nrows=1 nbal=1 verified=False
```

285. assistant (2026-05-31T09:30:27.801Z)

****This is the key insight:**** per-table mapping verifies the transaction tables that concatenation was breaking ? Aug (table2, 6 rows ?), May (****split across table2+table3****, 5+2 rows, each ?), Oct (table2, 15 rows ?). Concatenating everything misaligned columns. June's table is genuinely shredded by extract_tables (2+1 rows, no balance).

Let me switch to ****per-table mapping ? combine the oracle-endorsed tables ? re-verify the whole set****, with clustering as fallback:

286. user (2026-05-31T09:30:38.356Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py has been updated successfully. (file state is current in your context ? no need to Read it back)

287. user (2026-05-31T09:30:45.605Z)

```
AXIS BANK Statement for August 2024.pdf      rows= 6 verified=True
AXIS BANK Statement for June 2025.pdf        rows= 8 verified=False
AXIS BANK Statement for May 2025.pdf         rows= 7 verified=False
AXIS BANK Statement for October 2025.pdf     rows= 15 verified=True
--- 2/4 Axis statements reconcile ---
```

288. assistant (2026-05-31T09:31:57.149Z)

May went `verified=False` even though table2 (5) and table3 (2) each verified alone ? because the ****combined set still fails total reconcile**, which means extract_tables is **missing** some of May's transactions (incomplete ledger), and the oracle correctly refuses an incomplete parse. June is similarly shredded. So I'm at a ****safe, honest 2/4**** (Aug + Oct fully reconcile; June/May are vintages where extract_tables mangles the table and my clustering isn't yet good enough ? they store **nothing**, never garbage).

That's a strict improvement over the stop-gap (0/4) and the safety invariant holds. Rather than chase June/May blind, let me ship this and report honestly. First, let me see what test breakage the rewrite caused:

289. user (2026-05-31T09:31:59.958Z)

```
E501 Line too long (101 > 100)
--> src\muneem\parsers\axis.py:6:101
|
4 | clustering (whole statement, consistent columns; handles the date-merged / column-count
drift across
5 | vintages) and, as a fallback, ``extract_tables()`` ? and the generic ``engine`` maps each
onto the
6 | canonical concepts. **Reconciliation disposes:** a parse is trusted only if the per-row
balance walks
|
^
7 | or (when there's no per-row balance, e.g. some vintages) the totals reconcile against the
printed
8 | ``opening + ?Cr ? ?Dr == closing``. The parser is **oracle-gated**: it persists only an
endorsed
|
```

```
E501 Line too long (101 > 100)
--> src\muneem\parsers\clustering.py:16:101
```

```

|
14 | "has a date *and* a money token") restricts both column inference and the emitted matrix
to the
15 | transaction band.
16 | * **Use a coverage histogram, not interval-merging.** A column boundary is an x-channel
that is empty
|
^
17 | in *most* rows; a single long narration can't collapse two columns.
|
--- axis/registry tests ---
pdf_path = WindowsPath('x.pdf')
conn = <sqlite3.Connection object at 0x00000228A70544F0>, doc_id = 1
password = None

def parse_and_insert(
    self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str | None = None
) -> int:
    """Persist a savings parse **only if it reconciles**; otherwise store nothing.

    Sets ``documents.status`` to ``reconciled`` / ``unreconciled`` accordingly, so a layout
the
    parser can't yet read surfaces loudly instead of saving mislabeled rows.
    """
    try:
        parsed = parse_axis_savings(pdf_path, password)
    except Exception as exc:
>         raise RuntimeError(f"Failed to parse Axis statement {pdf_path}: {exc}") from exc
E         RuntimeError: Failed to parse Axis statement x.pdf: module 'muneem.parsers.axis' has
no attribute 'AxisTxn'

```

```

src\muneem\parsers\axis.py:155: RuntimeError
===== short test summary info =====
FAILED tests/parsers/test_axis.py::test_rows_from_savings_tables_skips_non_txn_rows
FAILED tests/parsers/test_axis.py::test_parse_and_insert_via_pdfplumber_mock
FAILED tests/parsers/test_axis.py::test_parse_and_insert_records_reconciled
FAILED tests/parsers/test_axis.py::test_parse_and_insert_refuses_to_store_unreconciled
4 failed, 4 passed in 0.93s

```

290. assistant (2026-05-31T09:33:43.888Z)

Per-table mapping is the right design (and correctly refuses incomplete parses). Let me fix the two lint lines and rewrite the Axis tests for the new architecture (the old savings tests referenced the deleted `AxisTxn`), plus add clustering tests:

291. user (2026-05-31T09:33:45.790Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py has been updated successfully. (file state is current in your context ? no need to Read it back)

292. user (2026-05-31T09:33:47.196Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\clustering.py has been updated successfully. (file state is current in your context ? no need to Read it back)

293. assistant (2026-05-31T09:34:07.898Z)

```

"""Tests for the Axis parsers (savings via the concept core + credit card).

```

```

Savings runs end-to-end on synthetic data through a pdfplumber mock (extract_tables /
extract_text /
extract_words), so the per-table ? engine ? reconcile path is exercised without a real PDF;
`parse_and_insert`'s store-iff-reconciled gate is checked by monkeypatching
`parse_axis_savings`.

```

Real encrypted statements run under the ``local\_statements`` marker (test_local_axis.py).

"""

```
from pathlib import Path
from unittest.mock import MagicMock, patch
```

```
import muneem.db as dbmod
from muneem.parsers import axis
from muneem.parsers.axis import AxisCreditCardParser, AxisParser, AxisSavings,
parse_axis_summary
from muneem.parsers.concepts import ConceptRow
```

```
def test_can_handle():
    assert AxisParser().can_handle("x", sender="alerts@axisbank.com")
    assert AxisParser().can_handle("Axis Bank Statement of Account")
    assert AxisParser().can_handle("Axis Bank Account Statement") # the real wording on
statements
    assert not AxisParser().can_handle("HDFC Bank")
    assert AxisCreditCardParser().can_handle("Axis Bank Credit Card Statement")
    assert not AxisCreditCardParser().can_handle("Axis Bank Statement of Account")
```

```
def test_parse_axis_summary_reads_opening_closing():
    s = parse_axis_summary("Opening Balance 1,000.00\n...rows...\nClosing Balance 2,500.50")
    assert s.opening_balance == 1000.0 and s.closing_balance == 2500.5
```

```
def test_is_txn_line():
    assert axis._is_txn_line(["01-02-2026", "UPI/Zomato", "500.00", "", "9500.00"])
    assert not axis._is_txn_line(["Tran Date", "Transaction", "Withdrawals", "Balance"]) #
header
    assert not axis._is_txn_line(["01-02-2026", "narration but no amount"]) # date, no money
```

```
def _run_savings(page_tables, text="", page_words=None):
    """Drive parse_axis_savings against a single synthetic page via a pdfplumber mock."""
    page = MagicMock()
    page.extract_tables.return_value = page_tables
    page.extract_text.return_value = text
    page.extract_words.return_value = page_words or []
    pdf = MagicMock()
    pdf.pages = [page]
    with patch("pdfplumber.open") as m:
        m.return_value.__enter__.return_value = pdf
        return axis.parse_axis_savings(Path("x.pdf"), password="")
```

```
def test_savings_per_row_reconcile_via_tables():
    table = [
        ["Tran Date", "Transaction", "Withdrawals", "Deposits", "Balance"],
        ["01-02-2026", "UPI Zomato", "500.00", "", "9500.00"],
        ["02-02-2026", "Salary", "", "2000.00", "11500.00"],
        ["03-02-2026", "ATM", "1500.50", "", "9999.50"],
    ]
    r = _run_savings([table])
    assert r.verified and len(r.rows) == 3
    assert r.rows[0].debit == 500.0 and r.rows[0].balance == 9500.0
    assert r.rows[1].credit == 2000.0
```

```
def test_savings_total_reconcile_when_no_per_row_balance():
    # No Balance column ? reconciles only against the printed opening/closing totals (Oct
vintage).
    table = [
```



```

        ["Tran Date", "Transaction", "Withdrawals", "Deposits"],
        ["01-02-2026", "UPI", "500.00", ""],
        ["02-02-2026", "Salary", "", "2000.00"],
    ]
    r = _run_savings([table], text="Opening Balance 1,000.00 ... Closing Balance 2,500.00")
    assert r.verified and len(r.rows) == 2
    assert r.rows[0].debit == 500.0 and r.rows[0].balance is None

def test_savings_refuses_when_unreconciled():
    table = [
        ["Tran Date", "Transaction", "Withdrawals", "Deposits", "Balance"],
        ["01-02-2026", "UPI", "500.00", "", "9500.00"],
        ["02-02-2026", "Salary", "", "2000.00", "1.00"], # balance breaks the walk
    ]
    r = _run_savings([table])
    assert not r.verified

def test_parse_and_insert_stores_when_verified(tmp_path, monkeypatch):
    rows = [ConceptRow(txn_date="01-02-2026", narration="UPI", debit=500.0, balance=9500.0)]
    monkeypatch.setattr(axis, "parse_axis_savings", lambda *a, **k: AxisSavings("ACC1", rows,
True))
    with dbmod.connect(tmp_path / "t.db") as conn:
        dbmod.init_db(conn)
        doc_id = dbmod.insert_document(conn, source="file", sha256="ax1", status="parsed")
        n = AxisParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
        assert n == 1
        stored = dbmod.list_axis_transactions(conn)
        assert len(stored) == 1 and stored[0]["account_number"] == "ACC1" and stored[0]["debit"]
== 500.0
        assert dbmod.get_document_by_sha(conn, "ax1")["status"] == "reconciled"

def test_parse_and_insert_refuses_when_unverified(tmp_path, monkeypatch):
    rows = [ConceptRow(txn_date="01-02-2026", narration="UPI", debit=500.0, balance=1.0)]
    monkeypatch.setattr(axis, "parse_axis_savings", lambda *a, **k: AxisSavings("ACC1", rows,
False))
    with dbmod.connect(tmp_path / "t.db") as conn:
        dbmod.init_db(conn)
        doc_id = dbmod.insert_document(conn, source="file", sha256="ax2", status="parsed")
        n = AxisParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
        assert n == 0
        assert dbmod.list_axis_transactions(conn) == []
        assert dbmod.get_document_by_sha(conn, "ax2")["status"] == "unreconciled"

def test_rows_from_cc_tables():
    tables = [
        [
            ["Date", "c1", "Particulars", "c3", "c4", "c5", "c6", "Category", "Amount"],
            ["05/02/2026", "x", "Swiggy", "", "", "", "", "Food", "250.00 Dr"],
            ["06/02/2026", "x", "Refund", "", "", "", "", "Food", "100.00 Cr"],
            ["bad", "too", "short"], # < 9 cols -> skipped
        ]
    ]
    rows = axis._rows_from_cc_tables(tables)
    assert len(rows) == 2
    assert rows[0].amount == 250.0 and rows[0].type == "Dr" and rows[0].merchant_category ==
"Food"
    assert rows[1].amount == 100.0 and rows[1].type == "Cr"

```

294. user (2026-05-31T09:34:07.915Z)

The file C:\Users\avidu\Projects\muneem\tests\parsers\test_axis.py has been updated

successfully. (file state is current in your context ? no need to Read it back)

295. assistant (2026-05-31T09:34:13.308Z)

Now the clustering unit tests:

296. assistant (2026-05-31T09:34:24.657Z)

```
"""Tests for coordinate clustering (muneem.parsers.clustering).
```

```
Synthetic word geometry only ? proves lines group by ``top``, columns come from the transaction
band (a full-width header must not collapse them), and a single wide cell can't merge two
columns.
"""
```

```
from muneem.parsers.clustering import cluster_to_matrix, group_lines, infer_columns
```

```
def W(text, x0, top, x1=None):
    """A minimal pdfplumber-style word dict."""
    return {
        "text": text,
        "x0": float(x0),
        "x1": float(x1 if x1 is not None else x0 + 5 * len(text)),
        "top": float(top),
        "bottom": float(top) + 8,
    }
```

```
def _has_date(texts):
    return any(t.count("-") == 2 for t in texts)
```

```
def _txn_row(date, narr, amt, bal, top):
    return [W(date, 40, top, 80), W(narr, 110, top, 230), W(amt, 300, top, 340), W(bal, 420,
top, 470)]
```

```
def test_group_lines_clusters_by_top():
    words = [W("a", 10, 100), W("b", 60, 101), W("c", 10, 130)]
    lines = group_lines(words)
    assert len(lines) == 2
    assert [w["text"] for w in lines[0]] == ["a", "b"]
```

```
def test_cluster_keeps_only_txn_lines_and_finds_columns():
    words = [W("ACCOUNT STATEMENT", 40, 50, 520)] # full-width header ? dropped by keep_line
    words += _txn_row("01-02-2026", "UPI", "500.00", "9500.00", 100)
    words += _txn_row("02-02-2026", "Salary", "2000.00", "11500.00", 120)
    m = cluster_to_matrix([words], keep_line=_has_date)
    assert len(m) == 2 # the header line is excluded
    assert all(len(row) == 4 for row in m) # date | narration | amount | balance
    assert m[0][0] == "01-02-2026" and m[0][3] == "9500.00"
```

```
def test_one_wide_cell_does_not_collapse_columns():
    # A single long narration spanning toward the amount column must not merge the two columns;
    # the coverage histogram thresholds on the fraction of rows, so one outlier can't.
    words = _txn_row("01-02-2026", "UPI", "500.00", "9500.00", 100)
    words += _txn_row("02-02-2026", "Salary", "2000.00", "11500.00", 120)
    words += [W("01-02-2026", 40, 140, 80), W("a very long narration line here", 110, 140, 295),
              W("750.00", 300, 140, 340), W("9000.00", 420, 140, 470)]
    m = cluster_to_matrix([words], keep_line=_has_date)
    assert len(m) == 3
    assert all(len(row) == 4 for row in m)
```

```
def test_infer_columns_returns_runs():
    lines = [_txn_row("01-02-2026", "UPI", "500.00", "9500.00", 100)]
    cols = infer_columns(lines, min_frac=0.3)
    assert len(cols) == 4
```

297. user (2026-05-31T09:34:24.670Z)

File created successfully at: C:\Users\avidu\Projects\muneeem\tests\parsers\test_clustering.py
(file state is current in your context ? no need to Read it back)

298. user (2026-05-31T09:34:29.981Z)

```
Exit code 1
2 files reformatted, 9 files left unchanged
E501 Line too long (101 > 100)
--> src\muneeem\parsers\clustering.py:50:101
|
48 |     An x-bin is "filled" if at least ``min_frac`` of the lines have a word covering it;
columns are
49 |     the runs of filled bins, and the empty runs between them are the inter-column
whitespace. Robust
50 |     to a few full-width lines because it thresholds on the *fraction* of rows, not on any
single one.
|
^
51 |     """
52 |     words = [w for line in lines for w in line]
|

E501 Line too long (101 > 100)
--> tests\parsers\test_axis.py:5:101
|
3 | Savings runs end-to-end on synthetic data through a pdfplumber mock (extract_tables /
extract_text /
4 | extract_words), so the per-table ? engine ? reconcile path is exercised without a real PDF;
5 | ``parse_and_insert``'s store-iff-reconciled gate is checked by monkeypatching
``parse_axis_savings``.
|
^
6 | Real encrypted statements run under the ``local_statements`` marker (test_local_axis.py).
7 | """
|
```

Found 2 errors.

299. assistant (2026-05-31T09:34:34.963Z)

Two more docstring lines to trim:

300. user (2026-05-31T09:34:40.174Z)

The file C:\Users\avidu\Projects\muneeem\src\muneeem\parsers\clustering.py has been updated successfully. (file state is current in your context ? no need to Read it back)

301. user (2026-05-31T09:34:40.345Z)

The file C:\Users\avidu\Projects\muneeem\tests\parsers\test_axis.py has been updated successfully. (file state is current in your context ? no need to Read it back)

302. user (2026-05-31T09:34:50.813Z)

All checks passed!

--- full offline suite ---

.....X..... [64%]

```
..... [100%]
===== XPASSES =====
===== short test summary info =====
XPASS tests/test_ocr.py::test_swiggy_image_code_recovered_fuzzily - recovery of the stylized
boxed code is OCR-engine/version dependent ? succeeds on Tesseract 5.4, returns nothing on the
apt build; the planned fix is a purpose-trained model (docs/DEPENDENCIES.md). strict=False so it
never fails the build.
110 passed, 9 deselected, 1 xpassed in 5.74s
```

303. assistant (2026-05-31T09:35:18.097Z)

110 passing, lint clean ? Step 3 is functionally done (synthetic coverage + **2/4 real Axis statements reconcile**, oracle-gated). Let me update the local-statements test wording (it's no longer a "stop-gap") and verify the real result once more before shipping:

304. user (2026-05-31T09:35:22.394Z)

The file C:\Users\avidu\Projects\muneem\tests\parsers\test_local_axis.py has been updated successfully. (file state is current in your context ? no need to Read it back)

305. user (2026-05-31T09:35:27.009Z)

The file C:\Users\avidu\Projects\muneem\tests\parsers\test_local_axis.py has been updated successfully. (file state is current in your context ? no need to Read it back)

306. assistant (2026-05-31T09:35:34.819Z)

Let me confirm the safety invariant holds on all 4 real Axis statements (and recount verified) before shipping:

307. user (2026-05-31T09:35:44.031Z)

```
--- Axis local safety-invariant test (real statements) ---
.... [100%]
4 passed in 2.93s
--- recount verified ---
Axis: 2/4 reconcile + persist; 2/4 store nothing (oracle-gated, safe)
```

308. assistant (2026-05-31T09:35:59.842Z)

Safety invariant holds on all 4 real statements; **2/4 reconcile + persist (Aug, Oct), 2/4 store nothing (safe)** ? a strict improvement over the stop-gap's 0/4. Shipping Step 3:

309. user (2026-05-31T09:36:06.515Z)

```
[feat/axis-clustering-rewrite 0f50272] feat(parsers): Axis savings rewrite onto the concept core
(clustering + total reconcile)
6 files changed, 475 insertions(+), 202 deletions(-)
create mode 100644 src/muneem/parsers/clustering.py
create mode 100644 tests/parsers/test_clustering.py
```

310. user (2026-05-31T09:36:11.219Z)

branch 'feat/axis-clustering-rewrite' set up to track 'origin/feat/axis-clustering-rewrite'.
To <https://github.com/avidullu/muneem.git>
* [new branch] feat/axis-clustering-rewrite -> feat/axis-clustering-rewrite

311. user (2026-05-31T09:36:26.573Z)

<https://github.com/avidullu/muneem/pull/25>

312. user (2026-05-31T09:36:36.376Z)

```
create mode 100644 src/muneem/parsers/clustering.py
create mode 100644 tests/parsers/test_clustering.py
Already on 'main'
- [deleted]      (none)      -> origin/feat/axis-clustering-rewrite
Already up to date.
5156b5a feat(parsers): Axis savings rewrite onto the concept core (clustering + total reconcile) (#25)
```

313. assistant (2026-05-31T09:37:29.282Z)

Step 3 merged as [muneem#25](https://github.com/avidullu/muneem/pull/25). Now ****AU Bank**** (task 8). Before building the full parser, let me diagnose whether AU's layout reconciles through the engine ? with a draft AU profile against the 4 real AU statements (structural booleans only):

314. user (2026-05-31T09:37:29.763Z)

Updated task #4 status

315. user (2026-05-31T09:37:30.235Z)

Updated task #5 status

316. user (2026-05-31T09:37:30.704Z)

Updated task #8 status

317. user (2026-05-31T09:37:43.288Z)

```
Statement_2024MTH07_XXXXX859_146.pdf: tables=11 open=False close=False | per_table rows=17
v=True | cluster rows=24 v=False
Statement_2026MTH01_XXXXX859_721.pdf: tables=12 open=False close=False | per_table rows=31
v=True | cluster rows=40 v=False
Statement_2026MTH03_XXXXX859_108.pdf: tables=11 open=False close=False | per_table rows=26
v=True | cluster rows=36 v=False
Statement_2026MTH04_XXXXX859_849.pdf: tables=12 open=False close=False | per_table rows=31
v=True | cluster rows=40 v=False
```

318. assistant (2026-05-31T09:38:37.127Z)

****AU Bank is the cleanest of all ? 4/4 statements verify**** via per-table + per-row reconcile (17/31/26/31 txns)! The engine handles AU beautifully. Since Axis and AU share the exact same recipe, I'll extract a shared engine-driven savings module (DRY) and add AU on top. Setting up:

319. user (2026-05-31T09:38:38.273Z)

Switched to a new branch 'feat/au-bank-parser'
feat/au-bank-parser

320. user (2026-05-31T09:38:38.742Z)

Chapter marked: "AU Bank parser + shared savings". Continue your current work.

321. user (2026-05-31T09:39:04.368Z)

File created successfully at: C:\Users\avidu\Projects\muneem\src\muneem\parsers\savings.py (file state is current in your context ? no need to Read it back)

322. assistant (2026-05-31T09:38:42.067Z)

Now the shared engine-driven savings module:

323. assistant (2026-05-31T09:39:04.351Z)

```
"""Shared engine-driven savings-statement parsing (Axis, AU Bank, and future issuers).
```

Every issuer whose statement is a transaction table follows the same recipe, so it lives here once
and each bank supplies only a :class:`~muneem.parsers.engine.Profile`:

1. **Extraction proposes** candidate cell-matrices ? one per ``extract\_tables()`` table, plus a coordinate-clustered whole-statement matrix (for layouts ``extract\_tables`` mis-segments).
2. The generic **engine** maps each candidate's columns onto the canonical concepts.
3. **Reconciliation disposes**: a parse is trusted only if its per-row balance walks, or ? when a vintage has no per-row balance ? the totals reconcile to the printed ``opening``/``closing``.

A statement splits its ledger across several extracted tables (page breaks), so we map each table on its own, keep the rows the oracle endorses, and re-check the *combined* set (which catches a cross-table balance break, i.e. a missing transaction). The result is **oracle-gated**: a layout we can't yet read returns ``verified=False`` and the caller persists nothing ? never mislabeled rows.

No statement content is logged; only counts/booleans travel out.
"""

```
from __future__ import annotations
```

```
import re
from dataclasses import dataclass, field
from pathlib import Path
```

```
import pdfplumber
```

```
from .clustering import cluster_to_matrix
from .concepts import ConceptRow, StatementConcepts
from .engine import Profile, map_table
from .validation import parse_amount, reconcile_running_balance, reconcile_total
```

```
_MONEY_TOKEN = re.compile(r"^\d{1,3}(\.\d{2})?(?:cr|dr)?$", re.IGNORECASE)
_OPENING_RE = re.compile(r"opening\s+balance[^\d-]*(-?\d{1,3}(\.\d{2})?)", re.IGNORECASE)
_CLOSING_RE = re.compile(r"closing\s+balance[^\d-]*(-?\d{1,3}(\.\d{2})?)", re.IGNORECASE)
```

```
@dataclass
```

```
class SavingsParse:
```

```
    """Outcome of a savings parse: concept rows + whether reconciliation endorsed them."""
```

```
    account_number: str
    rows: list[ConceptRow] = field(default_factory=list)
    verified: bool = False
```

```
def parse_statement_summary(text: str) -> StatementConcepts:
```

```
    """Read opening/closing balance from statement text (for the total-reconcile oracle).
```

```
    Best-effort regex; ``None`` when not found ? the oracle then relies on the per-row walk.
    """
```

```
    opening = _OPENING_RE.search(text)
    closing = _CLOSING_RE.search(text)
    return StatementConcepts(
        opening_balance=parse_amount(opening.group(1)) if opening else None,
        closing_balance=parse_amount(closing.group(1)) if closing else None,
    )
```

```
def txn_line_predicate(profile: Profile):
```

```

"""A line is transaction-like if it carries one of the profile's dates *and* a money token.

Used to restrict coordinate clustering to the transaction band (drops headers / summaries /
address blocks that would otherwise wreck column inference).
"""

patterns = profile.date_patterns

def keep(texts: list[str]) -> bool:
    has_date = any(p.match(t) for p in patterns) for t in texts)
    return has_date and any(_MONEY_TOKEN.match(t) for t in texts)

return keep

def reconcile_oracle(opening: float | None, closing: float | None):
    """Trust a parse if the per-row balance walks, else if the totals reconcile to the
    bookends."""

    def check(rows: list[ConceptRow]) -> bool:
        if reconcile_running_balance([(r.debit, r.credit, r.balance) for r in rows]).ok:
            return True
        if rows and opening is not None and closing is not None:
            return reconcile_total(rows, opening, closing)
        return False

    return check

def parse_savings(pdf_path: Path, profile: Profile, password: str | None = None) ->
SavingsParse:
    """Parse a savings statement into concept rows + a reconciliation verdict, driven by
    ``profile``.

    Per-table mapping first (each extracted table mapped on its own; oracle-endorsed rows
    combined
    and re-checked), then a coordinate-clustering fallback. Returns ``verified=False`` if
    neither
    reconciles, so the caller can refuse to persist.
    """
    with pdfplumber.open(pdf_path, password=password) as pdf:
        words_by_page = [
            page.extract_words(use_text_flow=False, keep_blank_chars=False) for page in
pdf.pages
        ]
        tables = [t for page in pdf.pages for t in (page.extract_tables() or [])]
        head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))

        account_number = profile.first_account(head) or "UNKNOWN"
        summary = parse_statement_summary(head)
        oracle = reconcile_oracle(summary.opening_balance, summary.closing_balance)

        confirmed: list[ConceptRow] = []
        for table in tables:
            res = map_table(table, profile, oracle=oracle)
            if res and res.verified:
                confirmed.extend(res.rows)
        if confirmed and oracle(confirmed):
            return SavingsParse(account_number, confirmed, True)

        clustered = map_table(
            cluster_to_matrix(words_by_page, keep_line=txn_line_predicate(profile)),
            profile,
            oracle=oracle,
        )

```

```

    if clustered and clustered.verified:
        return SavingsParse(account_number, clustered.rows, True)

    fallback = confirmed or (clustered.rows if clustered else [])
    return SavingsParse(account_number, fallback, False)

__all__ = [
    "SavingsParse",
    "parse_savings",
    "parse_statement_summary",
    "txn_line_predicate",
    "reconcile_oracle",
]

```

324. assistant (2026-05-31T09:39:09.912Z)

Now refactor `axis.py` to use the shared module (with back-compat names so the tests keep working). Let me read its current state:

325. user (2026-05-31T09:39:10.581Z)

```

1      """Axis Bank statement parsers: savings/transaction account + credit card.
2
3      **Savings now runs on the concept core.** Extraction proposes candidate cell-matrices ?
coordinate
4      clustering (whole statement, consistent columns; handles the date-merged / column-count
drift across
5      vintages) and, as a fallback, ``extract_tables()`` ? and the generic ``engine`` maps
each onto the
6      canonical concepts. **Reconciliation disposes:** a parse is trusted only if its balance
walks
7      or (when there's no per-row balance, e.g. some vintages) the totals reconcile against
the printed
8      ``opening + ?Cr ? ?Dr == closing``. The parser is **oracle-gated**: it persists only an
endorsed
9      parse, so a layout it can't yet read stores nothing (status ``unreconciled``) rather
than mislabeled
10     rows. This replaces the old fixed-column ``extract_tables()`` mapping that mislabeled
rows.
11
12     Credit-card statements carry no running balance, so ``AxisCreditCardParser`` can't
reconcile
13     (parse/persist split + count only). No statement content is logged here.
14     """
15
16     from __future__ import annotations
17
18     import re
19     import sqlite3
20     from dataclasses import dataclass, field
21     from pathlib import Path
22
23     import pdfplumber
24
25     from muneem import db as dbmod
26
27     from . import register_parser
28     from .base import StatementParser
29     from .clustering import cluster_to_matrix
30     from .concepts import ConceptRow, StatementConcepts
31     from .engine import map_table
32     from .profiles import AXIS_PROFILE
33     from .validation import Confidence, parse_amount, reconcile_running_balance,
reconcile_total

```



```

34
35     _CC_DATE_RE = re.compile(r"\d{2}/\d{2}/\d{4}") # Axis credit card: dd/mm/yyyy
36     _CARD_RES = [re.compile(r"(\d{4}XXXX\d{4})"), re.compile(r"(\d{6}\*\d{4})")]
37     _OPENING_RE = re.compile(r"opening\s+balance[^\d-]*(-?\d,]+\.\d{2})", re.IGNORECASE)
38     _CLOSING_RE = re.compile(r"closing\s+balance[^\d-]*(-?\d,]+\.\d{2})", re.IGNORECASE)
39     _SAVINGS_DATE = re.compile(r"^\d{2}[-/]\d{2}[-/]\d{4}$") # dd-mm-yyyy or dd/mm/yyyy
40     _MONEY_TOKEN = re.compile(r"^\d{2}[-/]\d{2}[-/]\d{4}$") # dd-mm-yyyy or dd/mm/yyyy
41
42
43     def _is_txn_line(texts: list[str]) -> bool:
44         """A transaction row carries a date *and* a money amount (drops
45         headers/summaries/address)."""
46         return any(_SAVINGS_DATE.match(t) for t in texts) and any(_MONEY_TOKEN.match(t) for
47         t in texts)
48
49     def _first_match(text: str, patterns: list[re.Pattern]) -> str | None:
50         for rx in patterns:
51             m = rx.search(text)
52             if m:
53                 return m.group(1).strip()
54         return None
55
56     # --- Savings / transaction account (concept core)
57     -----
58
59     @dataclass
60     class AxisSavings:
61         """Outcome of a savings parse: concept rows + whether reconciliation endorsed
62         them."""
63         account_number: str
64         rows: list[ConceptRow] = field(default_factory=list)
65         verified: bool = False
66
67     def parse_axis_summary(text: str) -> StatementConcepts:
68         """Read opening/closing balance from the statement text (for the total-reconcile
69         oracle).
70
71         Best-effort regex; ``None`` when not found (the oracle then relies on the per-row
72         walk).
73
74         """
75         opening = _first_match(text, [_OPENING_RE])
76         closing = _first_match(text, [_CLOSING_RE])
77         return StatementConcepts(
78             opening_balance=parse_amount(opening),
79             closing_balance=parse_amount(closing),
80         )
81
82     def _savings_oracle(opening: float | None, closing: float | None):
83         """Trust a parse if the per-row balance walks, else if the totals reconcile to the
84         bookends."""
85
86         def check(rows: list[ConceptRow]) -> bool:
87             if reconcile_running_balance([(r.debit, r.credit, r.balance) for r in rows]).ok:
88                 return True
89             if rows and opening is not None and closing is not None:
90                 return reconcile_total(rows, opening, closing)
91             return False
92
93     return check

```

```

92
93
94     def parse_axis_savings(pdf_path: Path, password: str | None = None) -> AxisSavings:
95         """Open an Axis savings statement and return concept rows + a reconciliation
verdict.
96
97         Tries a coordinate-clustered matrix (whole statement) first, then concatenated
98         ``extract_tables()`` rows, and keeps the first mapping the reconciliation oracle
endorses.
99         """
100         with pdfplumber.open(pdf_path, password=password) as pdf:
101             words_by_page = [
102                 page.extract_words(use_text_flow=False, keep_blank_chars=False) for page in
pdf.pages
103             ]
104             tables = [t for page in pdf.pages for t in (page.extract_tables() or [])]
105             head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))
106
107             account_number = AXIS_PROFILE.first_account(head) or "UNKNOWN"
108             summary = parse_axis_summary(head)
109             oracle = _savings_oracle(summary.opening_balance, summary.closing_balance)
110
111             # Strategy 1 ? per-table. A statement splits its ledger across several extracted
tables (page
112             # breaks); concatenating them all misaligns columns. Instead map each table on its
own and keep
113             # the rows from tables the oracle endorses, then re-check the *combined* set
(catches a cross-
114             # table balance break). This is "extraction proposes, reconciliation disposes" at
table scope.
115             confirmed: list[ConceptRow] = []
116             for table in tables:
117                 res = map_table(table, AXIS_PROFILE, oracle=oracle)
118                 if res and res.verified:
119                     confirmed.extend(res.rows)
120             if confirmed and oracle(confirmed):
121                 return AxisSavings(account_number, confirmed, True)
122
123             # Strategy 2 ? coordinate clustering across the whole statement (for layouts
extract_tables
124             # mis-segments, e.g. a date merged into the narration cell).
125             clustered = map_table(

```

326. user (2026-05-31T09:39:15.079Z)

```

125         clustered = map_table(
126             cluster_to_matrix(words_by_page, keep_line=_is_txn_line), AXIS_PROFILE,
oracle=oracle
127         )
128         if clustered and clustered.verified:
129             return AxisSavings(account_number, clustered.rows, True)
130
131         # Neither reconciled ? refuse to persist (store nothing; mark the document
unreconciled).
132         fallback = confirmed or (clustered.rows if clustered else [])
133         return AxisSavings(account_number, fallback, False)
134
135
136     @register_parser
137     class AxisParser(StatementParser):
138         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
139             if sender and "axisbank.com" in sender.lower():
140                 return True

```

```

141         t = text or ""
142         return "Axis Bank" in t and ("Statement of Account" in t or "Account Statement"
in t)
143
144     def parse_and_insert(
145         self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
146     ) -> int:
147         """Persist a savings parse **only if it reconciles**; otherwise store nothing.
148
149         Sets ``documents.status`` to ``reconciled`` / ``unreconciled`` accordingly, so a
layout the
150         parser can't yet read surfaces loudly instead of saving mislabeled rows.
151         """
152         try:
153             parsed = parse_axis_savings(pdf_path, password)
154         except Exception as exc:
155             raise RuntimeError(f"Failed to parse Axis statement {pdf_path}: {exc}") from
exc
156
157         if not parsed.verified:
158             conn.execute(
159                 "UPDATE documents SET status = ? WHERE id = ?",
160                 (Confidence.UNRECONCILED.value, doc_id),
161             )
162             return 0
163
164         for r in parsed.rows:
165             dbmod.insert_axis_transaction(
166                 conn,
167                 document_id=doc_id,
168                 account_number=parsed.account_number,
169                 txn_date=r.txn_date,
170                 value_date=r.value_date or "",
171                 particulars=r.narration,
172                 debit=r.debit,
173                 credit=r.credit,
174                 balance=r.balance,
175             )
176             conn.execute(
177                 "UPDATE documents SET status = ? WHERE id = ?",
178                 (Confidence.RECONCILED.value, doc_id),
179             )
180         return len(parsed.rows)
181
182
183     # --- Credit card (no running balance; count only)
-----
184
185
186     @dataclass
187     class AxisCCTxn:
188         txn_date: str
189         particulars: str
190         merchant_category: str
191         amount: float
192         type: str # "Cr" or "Dr"
193
194
195     def _rows_from_cc_tables(tables: list) -> list[AxisCCTxn]:
196         """Build credit-card transactions from extracted tables. Pure: no PDF, no DB."""
197         rows: list[AxisCCTxn] = []
198         for table in tables or []:
199             for row in table or []:

```

327. assistant (2026-05-31T09:40:18.588Z)

Now refactor `axis.py` to delegate to the shared module while keeping back-compat names for the tests. Four targeted edits:

328. user (2026-05-31T09:40:25.843Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py has been updated successfully. (file state is current in your context ? no need to Read it back)

329. user (2026-05-31T09:40:29.453Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py has been updated successfully. (file state is current in your context ? no need to Read it back)

330. user (2026-05-31T09:40:46.377Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py has been updated successfully. (file state is current in your context ? no need to Read it back)

331. assistant (2026-05-31T09:41:00.366Z)

Update the module docstring to reflect the shared path, then verify the Axis refactor is green:

332. user (2026-05-31T09:41:06.273Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py has been updated successfully. (file state is current in your context ? no need to Read it back)

333. user (2026-05-31T09:41:10.481Z)

```
Exit code 1
E501 Line too long (101 > 100)
--> src\muneem\parsers\axis.py:4:101
|
3 | **Savings runs on the shared engine-driven path** (:mod:`muneem.parsers.savings`): per-table
+
4 | coordinate-clustered candidates ? generic engine mapping ? reconciliation oracle, **oracle-
gated** so
|
^
5 | a vintage it can't read stores nothing (status ``unreconciled``) rather than mislabeled
rows. The
6 | Axis specifics are entirely in ``AXIS_PROFILE``. This replaced the old fixed-column
``extract_tables``
|

E501 Line too long (102 > 100)
--> src\muneem\parsers\axis.py:6:101
|
4 | coordinate-clustered candidates ? generic engine mapping ? reconciliation oracle, **oracle-
gated** so
5 | a vintage it can't read stores nothing (status ``unreconciled``) rather than mislabeled
rows. The
6 | Axis specifics are entirely in ``AXIS_PROFILE``. This replaced the old fixed-column
``extract_tables``
|
^^
7 | mapping that mislabeled rows across vintages.
|

E501 Line too long (101 > 100)
--> src\muneem\parsers\savings.py:89:101
|
```

```

88 | def parse_savings(pdf_path: Path, profile: Profile, password: str | None = None) ->
SavingsParse:
89 |     """Parse a savings statement into concept rows + a reconciliation verdict, driven by
``profile``.
    |
    ^
90 |
91 |     Per-table mapping first (each extracted table mapped on its own; oracle-endorsed rows
combined
    |

```

Found 3 errors.

334. user (2026-05-31T09:41:25.197Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py has been updated successfully. (file state is current in your context ? no need to Read it back)

335. user (2026-05-31T09:41:30.179Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\savings.py has been updated successfully. (file state is current in your context ? no need to Read it back)

336. user (2026-05-31T09:41:39.217Z)

```

All checks passed!
--- axis tests ---
..... [100%]
9 passed in 0.40s
--- real Axis still 2/4 after refactor ---
Axis 2/4 verified (unchanged by refactor)

```

337. assistant (2026-05-31T09:42:15.775Z)

Axis refactor is clean (9 tests pass, still 2/4 on real data). Now build AU. Let me add the profile, DB table, CLI dispatch, and probe AU's detection markers (for `can\_handle`):

338. user (2026-05-31T09:42:20.490Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\profiles.py has been updated successfully. (file state is current in your context ? no need to Read it back)

339. user (2026-05-31T09:42:22.848Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\db.py has been updated successfully. (file state is current in your context ? no need to Read it back)

340. assistant (2026-05-31T09:42:33.810Z)

Now the DB helpers, CLI dispatch, `\_\_all\_\_`, and the marker probe:

341. user (2026-05-31T09:42:38.066Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\db.py has been updated successfully. (file state is current in your context ? no need to Read it back)

342. user (2026-05-31T09:42:40.419Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\cli.py has been updated successfully. (file state is current in your context ? no need to Read it back)

343. user (2026-05-31T09:42:41.829Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\profiles.py has been updated successfully. (file state is current in your context ? no need to Read it back)

344. user (2026-05-31T09:42:45.416Z)

```
Statement_2024MTH07_XXXXX859_146.pdf -> ['aubank']
Statement_2026MTH01_XXXXX859_721.pdf -> ['aubank']
Statement_2026MTH03_XXXXX859_108.pdf -> ['aubank']
Statement_2026MTH04_XXXXX859_849.pdf -> ['aubank']
```

345. assistant (2026-05-31T09:43:05.165Z)

AU statements are detected by the `***aubank***` marker (likely the ``aubank.in`` domain). Let me write the AU parser and its tests:

346. assistant (2026-05-31T09:43:13.524Z)

```
"""AU Small Finance Bank savings / transaction-account statement parser.
```

```
Runs on the **shared engine-driven path** (:mod:`muneem.parsers.savings`): per-table +
coordinate-clustered candidates ? generic engine mapping ? reconciliation oracle, **oracle-
gated** so
a vintage it can't read stores nothing rather than mislabeled rows. The AU specifics are
entirely in
`AUBANK_PROFILE` ? this parser is mostly the DB wiring. No statement content is logged here.
"""
```

```
from __future__ import annotations
```

```
import sqlite3
from pathlib import Path
```

```
from muneem import db as dbmod
```

```
from . import register_parser
from .base import StatementParser
from .profiles import AUBANK_PROFILE
from .savings import SavingsParse, parse_savings
from .validation import Confidence
```

```
def parse_au_savings(pdf_path: Path, password: str | None = None) -> SavingsParse:
    """Parse an AU Bank savings statement via the shared engine-driven path."""
    return parse_savings(pdf_path, AUBANK_PROFILE, password)
```

```
@register_parser
```

```
class AUParser(StatementParser):
```

```
    def can_handle(self, text: str, sender: str | None = None, subject: str | None = None) ->
bool:
```

```
        if sender and "aubank" in sender.lower():
            return True
        low = (text or "").lower()
        return "au small finance bank" in low or "aubank" in low
```

```
    def parse_and_insert(
```

```
        self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str | None = None
) -> int:
```

```
        """Persist a savings parse **only if it reconciles**; otherwise store nothing."""
```

```
        try:
```

```
            parsed = parse_au_savings(pdf_path, password)
```

```
        except Exception as exc:
```

```
            raise RuntimeError(f"Failed to parse AU statement {pdf_path}: {exc}") from exc
```

```

if not parsed.verified:
    conn.execute(
        "UPDATE documents SET status = ? WHERE id = ?",
        (Confidence.UNRECONCILED.value, doc_id),
    )
    return 0

for r in parsed.rows:
    dbmod.insert_au_transaction(
        conn,
        document_id=doc_id,
        account_number=parsed.account_number,
        txn_date=r.txn_date,
        value_date=r.value_date or "",
        particulars=r.narration,
        debit=r.debit,
        credit=r.credit,
        balance=r.balance,
    )
conn.execute(
    "UPDATE documents SET status = ? WHERE id = ?",
    (Confidence.RECONCILED.value, doc_id),
)
return len(parsed.rows)

```

347. user (2026-05-31T09:43:13.534Z)

File created successfully at: C:\Users\avidu\Projects\muneeem\src\muneeem\parsers\au.py (file state is current in your context ? no need to Read it back)

348. assistant (2026-05-31T09:43:30.421Z)

"""Tests for the AU Bank parser.

AU runs on the shared engine-driven savings path (covered in depth by test_axis / test_engine), so here we check AU detection, an end-to-end per-row reconcile via a pdfplumber mock, and the store-iff-reconciled gate. Real statements run under local_statements (test_local_au.py)."""

```

from pathlib import Path
from unittest.mock import MagicMock, patch

import muneem.db as dbmod
from muneem.parsers import au
from muneem.parsers.au import AUParser, parse_au_savings
from muneem.parsers.concepts import ConceptRow
from muneem.parsers.savings import SavingsParse

def test_can_handle():
    assert AUParser().can_handle("...customercare@aubank.in...")
    assert AUParser().can_handle("AU Small Finance Bank Statement")
    assert AUParser().can_handle("anything", sender="statements@aubank.in")
    assert not AUParser().can_handle("HDFC Bank savings account details")
    assert not AUParser().can_handle("Axis Bank Statement of Account")

def test_parse_au_savings_per_row_reconcile():
    table = [
        ["Date", "Transaction", "Debit", "Credit", "Balance"],
        ["01-02-2026", "UPI", "500.00", "", "9500.00"],
        ["02-02-2026", "Salary", "", "2000.00", "11500.00"],
    ]
    page = MagicMock()

```

```

page.extract_tables.return_value = [table]
page.extract_text.return_value = "aubank statement"
page.extract_words.return_value = []
pdf = MagicMock()
pdf.pages = [page]
with patch("pdfplumber.open") as m:
    m.return_value.__enter__.return_value = pdf
    r = parse_au_savings(Path("x.pdf"), password="")
assert r.verified and len(r.rows) == 2
assert r.rows[0].debit == 500.0 and r.rows[1].credit == 2000.0

def test_parse_and_insert_stores_when_verified(tmp_path, monkeypatch):
    rows = [ConceptRow(txn_date="01-02-2026", narration="UPI", debit=500.0, balance=9500.0)]
    monkeypatch.setattr(au, "parse_au_savings", lambda *a, **k: SavingsParse("AU1", rows, True))
    with dbmod.connect(tmp_path / "t.db") as conn:
        dbmod.init_db(conn)
        doc_id = dbmod.insert_document(conn, source="file", sha256="au1", status="parsed")
        n = AUParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
        assert n == 1
        stored = dbmod.list_au_transactions(conn)
        assert len(stored) == 1 and stored[0]["account_number"] == "AU1" and stored[0]["debit"]
== 500.0
        assert dbmod.get_document_by_sha(conn, "au1")["status"] == "reconciled"

def test_parse_and_insert_refuses_when_unverified(tmp_path, monkeypatch):
    rows = [ConceptRow(txn_date="01-02-2026", narration="UPI", debit=500.0, balance=1.0)]
    monkeypatch.setattr(au, "parse_au_savings", lambda *a, **k: SavingsParse("AU1", rows,
False))
    with dbmod.connect(tmp_path / "t.db") as conn:
        dbmod.init_db(conn)
        doc_id = dbmod.insert_document(conn, source="file", sha256="au2", status="parsed")
        n = AUParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
        assert n == 0
        assert dbmod.list_au_transactions(conn) == []
        assert dbmod.get_document_by_sha(conn, "au2")["status"] == "unreconciled"

```

349. user (2026-05-31T09:43:30.433Z)

File created successfully at: C:\Users\avidu\Projects\muneem\tests\parsers\test_au.py (file state is current in your context ? no need to Read it back)

350. assistant (2026-05-31T09:43:50.654Z)

"""Local-only checks for the AU Bank parser against real encrypted statements.

Skipped unless an AU password is available (`password\_rules.yaml` or `MUNEEM\_TEST\_AU\_PW`);
never
runs in CI. The parser is **oracle-gated**, so this asserts the safety invariant: it either
stored
nothing (a vintage it can't yet read) or everything it stored reconciled ? never mislabeled
rows.
Surfaces counts / booleans / parser names only ? no statement content.
"""

```

import os
from pathlib import Path

```

```

import pdfplumber
import pytest

```

```

from muneem import db as dbmod
from muneem.parsers import get_parser
from muneem.password_rules import get_password_for_bank

```



```

AU_PDFS = sorted(Path("testdata/aubank-statements").glob("*.pdf"))

@pytest.mark.local_statements
@pytest.mark.parametrize("pdf_path", AU_PDFS, ids=lambda p: p.name)
def test_local_au_never_stores_unreconciled(pdf_path, tmp_path):
    pw = (
        get_password_for_bank("aubank")
        or get_password_for_bank("au")
        or os.environ.get("MUNEEM_TEST_AU_PW")
    )
    if not pw:
        pytest.skip("needs the AU password (password_rules.yaml or MUNEEM_TEST_AU_PW)")

    with pdfplumber.open(pdf_path, password=pw) as pdf:
        text = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(2,
len(pdf.pages))))
        parser = get_parser(text)
        if parser is None or type(parser).__name__ != "AUParser":
            pytest.skip(f"{pdf_path.name}: not routed to AUParser")

    with dbmod.connect(tmp_path / "t.db") as conn:
        dbmod.init_db(conn)
        doc_id = dbmod.insert_document(conn, source="file", sha256=pdf_path.name,
status="parsed")
        n = parser.parse_and_insert(pdf_path, conn, doc_id, password=pw)
        status = dbmod.get_document_by_sha(conn, pdf_path.name)["status"]
        stored = dbmod.list_au_transactions(conn)

        assert (n == 0 and stored == []) or status == "reconciled", (
            f"{pdf_path.name}: stored {n} row(s) with status={status} ? must not persist
unreconciled"
        )

```

351. user (2026-05-31T09:43:50.671Z)

File created successfully at: C:\Users\avidu\Projects\muneem\tests\parsers\test_local_au.py
(file state is current in your context ? no need to Read it back)

352. user (2026-05-31T09:43:55.844Z)

```

Exit code 1
2 files reformatted, 2 files left unchanged
E501 Line too long (101 > 100)
--> src\muneem\parsers\au.py:4:101
|
3 | Runs on the **shared engine-driven path** (:mod:`muneem.parsers.savings`): per-table +
4 | coordinate-clustered candidates ? generic engine mapping ? reconciliation oracle, **oracle-
gated** so
|
^
5 | a vintage it can't read stores nothing rather than mislabeled rows. The AU specifics are
entirely in
6 | ``AUBANK_PROFILE`` ? this parser is mostly the DB wiring. No statement content is logged
here.
|

Found 1 error.

```

353. user (2026-05-31T09:44:04.796Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\au.py has been updated successfully.
(file state is current in your context ? no need to Read it back)

354. user (2026-05-31T09:44:27.371Z)

```
All checks passed!
--- full offline suite ---
..... [100%]
===== XPASSES =====
===== short test summary info =====
XPASS tests/test_ocr.py::test_swiggy_image_code_recovered_fuzzily - recovery of the stylized
boxed code is OCR-engine/version dependent ? succeeds on Tesseract 5.4, returns nothing on the
apt build; the planned fix is a purpose-trained model (docs/DEPENDENCIES.md). strict=False so it
never fails the build.
114 passed, 13 deselected, 1 xpassed in 6.26s
--- AU real-data precision + routing (counts/booleans only) ---
.... [100%]
4 passed in 3.39s
AU: 4/4 reconcile+persist; routed-to-AUParser 4/4
```

355. assistant (2026-05-31T09:44:41.823Z)

****AU Bank: 4/4 reconcile + persist, 4/4 correctly routed**, all 114 offline tests pass, lint clean. AU is the cleanest issuer. Let me update the docs to reflect AU landed + the shared savings module, then ship. Reading CURRENT_STATE \$2:**

356. user (2026-05-31T09:44:42.533Z)

```
16      * **Gmail** (`muneem.ingestion.gmail`): Full OAuth2 read-only + search + attachment
download implemented.
17      * **IMAP** (`muneem.ingestion.imap`): Basic auth + search implemented; `get_headers` /
attachment methods are NotImplemented (experimental).
18      * **Outlook** (`muneem.ingestion.outlook`): Device-flow auth + basic search implemented;
attachment/header/download NotImplemented (experimental).
19
20      Companion one-time auth tools live in `tools/`. No high-level `muneem fetch` CLI command
yet (see Roadmap B.4).
21
22      ## 2. Statement Parsers (concept-core architecture)
23      Deterministic parsers live under `muneem.parsers`, registered via decorator and selected
by `get_parser` on text heuristics + sender. Submodules are **auto-loaded on package import**
(`autoload_parsers`), so the registry is populated in the production path (not just under
tests).
24
25      The parser core is **concept-first and verification-first** (see [`parser-
architecture.md`](./parser-architecture.md)): an extractor maps an issuer's columns onto a bank-
agnostic **concept schema** (`concepts.py`); a generic **engine** (`engine.py`) does the
column?concept mapping (L0 header lexicon ? profile column-order ? L2 content search); and a
shared **verifier** (`validation.py`) decides trust by **arithmetic** ? per-row balance walk +
total reconcile + SUMMARY bookend + a recorded confidence verdict. The engine is **family-
agnostic**: parameterized by a `FamilySpec` + an injected reconciliation oracle, so a future
**instrument family** (equity/MF/ETF/bond holdings + buy/sell/dividend/fee events) reuses the
same control flow with its own concepts + oracle. Issuer knowledge lives in `profiles.py` as
**data, not code**.
26
27      **Active issuers** ? we have real statements + passwords, so these get validated on real
data:
28
29      * **HDFC Bank** (`hdfc.py`): Savings / transaction-account statements via **coordinate
clustering** (deliberately **not** `extract_tables()`) ? HDFC's table is semi-ruled). **Self-
validating**: running-balance reconciliation + a cross-check against the bank's own SUMMARY
(Dr/Cr counts, opening/closing balance); records `documents.status` =
`reconciled`/`unreconciled`. Refactored onto the concept core as the **reference
implementation**. ? **Verified on the user's 4 real statements** ? all reconcile **and** bookend,
without inspecting any value.
30      * **Axis Bank** (`axis.py`): Savings + Credit Card. ?? The old savings
`extract_tables()` mapping was **unreliable across real layouts** (column counts and date
placement vary between vintages ? e.g. Jun vs Oct 2025). **Rewrite in progress (step 3):** HDFC-
```

style ****coordinate clustering ? generic engine mapping ? total reconcile**** (`opening + ?Cr ? ?Dr == closing`), oracle-gated so it can never persist unreconciled rows. Credit-card has no running balance (parse/persist split + count only).

31 * ****AU Bank**** (planned, step-3 sibling): engine-driven profile on the same coordinate-clustering + total-reconcile path as Axis. 4 real statements on hand.

32

33 > **_Dropped: **ICICI**** ? no statements on hand to validate against, so it was removed to focus effort on the issuers we can actually verify (HDFC/Axis/AU). It can return as a profile when data exists._

357. assistant (2026-05-31T09:45:00.861Z)

AU works end to end. Let me update CURRENT_STATE ? Axis (landed, 2/4) and AU (landed, 4/4) bullets, plus the shared `savings.py` note:

358. user (2026-05-31T09:45:06.453Z)

The file C:\Users\avidu\Projects\muneeem\docs\CURRENT_STATE.md has been updated successfully. (file state is current in your context ? no need to Read it back)

359. user (2026-05-31T09:45:13.484Z)

The file C:\Users\avidu\Projects\muneeem\docs\CURRENT_STATE.md has been updated successfully. (file state is current in your context ? no need to Read it back)

360. assistant (2026-05-31T09:45:17.064Z)

Now the "steps landed" line in CURRENT_STATE and the parser-architecture step 3 status. Let me read both:

361. user (2026-05-31T09:45:18.022Z)

34

35 All balance-bearing parsers share the verifier and split parsing from DB-insert, so row logic is unit-testable without a PDF/DB and a parse is validated before it's trusted. ****Concept-core steps 1?2 have landed**** (the verifier + concept schema; the generic mapping engine + profiles). Parsers use `pdfplumber` (with optional password). ****Password resolution**** (password_rules + hybrid unlock in `unlock.py`) is ****not yet wired**** into the main `muneeem ingest --doc-type bank\_statement` path; it is exercised via the `@pytest.mark.local\_statements` tests on real (gitignored) statements.

36

37 See `password\_rules.py` (prefers user config dir) and `cli.py:ingest`.

38

362. user (2026-05-31T09:45:18.795Z)

146 issuers driven through the engine are ****Axis + AU Bank**** (step 3).

147 3. ****Axis + AU Bank rewrite onto the core**** ? the must-fix ***and*** the first real test of

148 "generic-first": coordinate clustering ? concept mapping ? total reconcile, oracle-gated.

149 Validates on the real Axis (Aug'24, May/Jun/Oct'25) and AU Bank statements.

150 4. ****L3 local-vision fallback**** (after the research): a verified extractor that only fires when the

151 deterministic ladder fails to reconcile, with the number-traps from \$7.

152 5. ****Unify storage**** (\$9) + begin ****Tier-2 transaction semantics****.

153

154 Each step is a green, reversible PR; the verifier guarantees we never regress into storing garbage.

155

156 ## 11. Open questions (for discussion)

157

363. user (2026-05-31T09:45:33.537Z)

The file C:\Users\avidu\Projects\muneem\docs\CURRENT_STATE.md has been updated successfully.
(file state is current in your context ? no need to Read it back)

364. user (2026-05-31T09:45:37.733Z)

The file C:\Users\avidu\Projects\muneem\docs\parser-architecture.md has been updated successfully. (file state is current in your context ? no need to Read it back)

365. user (2026-05-31T09:45:40.030Z)

The file C:\Users\avidu\Projects\muneem\docs\parser-architecture.md has been updated successfully. (file state is current in your context ? no need to Read it back)

366. assistant (2026-05-31T09:45:44.463Z)

Docs updated. Final check, then ship the AU PR:

367. user (2026-05-31T09:45:52.577Z)

All checks passed!

```
===== short test summary info =====
XPASS tests/test_ocr.py::test_swiggy_image_code_recovered_fuzzily - recovery of the stylized
boxed code is OCR-engine/version dependent ? succeeds on Tesseract 5.4, returns nothing on the
apt build; the planned fix is a purpose-trained model (docs/DEPENDENCIES.md). strict=False so it
never fails the build.
114 passed, 13 deselected, 1 xpassed in 6.09s
--- status ---
M docs/CURRENT_STATE.md
M docs/parser-architecture.md
M src/muneem/cli.py
M src/muneem/db.py
M src/muneem/parsers/axis.py
M src/muneem/parsers/profiles.py
?? src/muneem/parsers/au.py
?? src/muneem/parsers/savings.py
?? tests/parsers/test_au.py
?? tests/parsers/test_local_au.py
```

368. user (2026-05-31T09:46:02.743Z)

```
[feat/au-bank-parser 467b6fe] feat(parsers): AU Bank parser + extract shared engine-driven
savings path
10 files changed, 396 insertions(+), 106 deletions(-)
create mode 100644 src/muneem/parsers/au.py
create mode 100644 src/muneem/parsers/savings.py
create mode 100644 tests/parsers/test_au.py
create mode 100644 tests/parsers/test_local_au.py
```

369. user (2026-05-31T09:46:07.796Z)

```
branch 'feat/au-bank-parser' set up to track 'origin/feat/au-bank-parser'.
To https://github.com/avidullu/muneem.git
* [new branch]      feat/au-bank-parser -> feat/au-bank-parser
```

370. user (2026-05-31T09:46:19.558Z)

<https://github.com/avidullu/muneem/pull/26>

371. user (2026-05-31T09:46:29.701Z)

```
create mode 100644 tests/parsers/test_au.py
create mode 100644 tests/parsers/test_local_au.py
Already on 'main'
```

- [deleted] (none) -> origin/feat/au-bank-parser
Already up to date.
5f41467 feat(parsers): AU Bank parser + extract shared engine-driven savings path (#26)

372. assistant (2026-05-31T09:46:59.712Z)

AU merged as [muneem#26](https://github.com/avidullu/muneem/pull/26). Now the finale you asked for ? **the consolidated precision comparison, HDFC vs Axis vs AU on real statements** (counts/booleans only):

373. user (2026-05-31T09:47:00.665Z)

Updated task #8 status

374. user (2026-05-31T09:47:01.140Z)

Updated task #6 status

375. user (2026-05-31T09:47:15.982Z)

ISSUER STATEMENT		ROWS	RECONCILED	BOOKENDED
HDFC	Avi_Dullu_28022026_183854522.pdf	37	True	True
HDFC	Avi_Dullu_30042026_001858675.pdf	31	True	True
HDFC	Avi_Dullu_30112025_110905081.pdf	40	True	True
HDFC	Avi_Dullu_31082025_205409615.pdf	63	True	True
AXIS	AXIS BANK Statement for August 2024.pdf	6	True	n/a (no SUMMARY parse)
AXIS	AXIS BANK Statement for June 2025.pdf	8	False	n/a (no SUMMARY parse)
AXIS	AXIS BANK Statement for May 2025.pdf	7	False	n/a (no SUMMARY parse)
AXIS	AXIS BANK Statement for October 2025.pdf	15	True	n/a (no SUMMARY parse)
AU	Statement_2024MTH07_XXXXX859_146.pdf	17	True	n/a
AU	Statement_2026MTH01_XXXXX859_721.pdf	31	True	n/a
AU	Statement_2026MTH03_XXXXX859_108.pdf	26	True	n/a
AU	Statement_2026MTH04_XXXXX859_849.pdf	31	True	n/a
TOTALS: HDFC 4/4 (reconcile + bookend vs SUMMARY) AXIS 2/4 AU 4/4				
All unverified statements store NOTHING (oracle-gated) ? never mislabeled rows.				

376. assistant (2026-05-31T09:47:37.984Z)

All tasks complete. Let me mark the final task done, clean up the stale stash, and confirm the final state:

377. user (2026-05-31T09:47:38.464Z)

Updated task #6 status

378. user (2026-05-31T09:47:39.859Z)

stash@{0}: On feat/axis-coordinate-clustering: axis-step3-wip
Dropped refs/stash@{0} (0373b1099b0841d8f1d7d5a604156e53261d2880)
--- recent history ---
5f41467 feat(parsers): AU Bank parser + extract shared engine-driven savings path (#26)
5156b5a feat(parsers): Axis savings rewrite onto the concept core (clustering + total reconcile) (#25)
2a589ee chore(refocus): remove ICICI; reframe docs to the whole-finance view (HDFC/Axis/AU) (#24)
7ea7f7a feat(parsers): generic concept-mapping engine (L0/L1/L2) + profiles; ICICI runs through it (#23)
7555fb2 feat(parsers): concept-core verifier + schema; HDFC refactored onto it (no behavior change) (#22)
6f6141f docs: parser architecture proposal (concept-core, verification-first) [DRAFT] (#21)
128fc36 docs(roadmap): flag Axis savings parser as a must-fix (needs thorough rewrite) (#20)
--- tree clean? ---

(clean)

379. assistant (2026-05-31T09:48:01.306Z)

Everything's shipped and `main` is clean + green. Here's the wrap-up.

? Precision: real statements, all three issuers

```
| Issuer | Result | Transactions parsed |
|---|---|---|
| **HDFC** | ? **4/4** ? reconcile ** + bookend** vs the bank's own SUMMARY | 37, 31, 40, 63 |
| **AU Bank** | ? **4/4** ? reconcile + persist | 17, 31, 26, 31 |
| **Axis** | ?? **2/4** ? Aug'24 (6) + Oct'25 (15) reconcile | May/Jun'25 store **nothing** |
```

****10 of 12 real statements fully reconcile.**** The 2 Axis misses aren't wrong data ?
`extract\_tables` mangles those vintages into an **incomplete** ledger, the total-reconcile oracle catches it, and they ****store nothing**** rather than mislabeled rows. The safety invariant is verified on all 12. HDFC's are the strongest (they also bookend against the printed Dr/Cr counts + opening/closing).

What shipped this session (5 squash-merged PRs)

1. [#22](https://github.com/avidullu/muneem/pull/22) ? ****concept core****: verifier + concept schema; HDFC refactored on (no behavior change).
2. [#23](https://github.com/avidullu/muneem/pull/23) ? ****generic mapping engine**** (L0/L1/L2) + profiles; family-agnostic seam.
3. [#24](https://github.com/avidullu/muneem/pull/24) ? ****refocus****: removed ICICI; reframed README/DESIGN/CURRENT_STATE/architecture/ROADMAP to the ****whole-finance**** view.
4. [#25](https://github.com/avidullu/muneem/pull/25) ? ****Axis rewrite**** onto coordinate clustering + total reconcile.
5. [#26](https://github.com/avidullu/muneem/pull/26) ? ****AU Bank**** + extracted the shared `savings.py` path (adding a bank is now mostly a profile).

Concept-core ****steps 1?3 are done****. 114 offline tests pass; your passwords were used only as transient env vars (never written, committed, or echoed) and every check surfaced counts/booleans only.

What remains

- ****The 2 Axis vintages (May/June 2025).**** `extract\_tables` shreds them; recovering them means improving the coordinate clusterer (the column inference is still fragile on those layouts). It's isolated follow-up work ? and since I can iterate against your real files (booleans only), it's tractable.
- ****Step 4/5**** (per the roadmap): the local-VLM L3 fallback (still pending the research run), then storage unification + the ****instrument family**** (equity/MF/ETF/bonds) ? the next big piece of the total-finance vision.

Want me to (a) dig into the 2 Axis vintages next, (b) start the instrument family, or (c) wire the password-unlock into the main `muneem ingest` path so this all runs from the CLI, not just tests?

380. user (2026-05-31T09:49:41.822Z)

Can you make sure the current state and architecture, next steps etc. are all updated as per the current state? We will take a pause and resume later if the slate is clean and updated to remote

381. assistant (2026-05-31T09:50:07.564Z)

On it ? let me audit the three state docs against what actually shipped, fix anything stale (the ROADMAP "Axis in progress" note is the likely culprit), then commit + push so remote is clean. Reading them in full first:

382. user (2026-05-31T09:50:08.547Z)

1 # Current State

```

2
3   > **Last Updated:** 2026-06 (concept-core engine + issuer refocus: HDFC / Axis / AU
Bank)
4   > Summarizes current capabilities, completed features, and near-term next steps for
`muneem`.
5   > See [ROADMAP.md](./ROADMAP.md) for milestone tracking and [parser-
architecture.md](./parser-architecture.md) for the parser design.
6
7   > **Scope reminder ? muneem is a _whole-finance_ bookkeeper, not a bank tool.** Bank &
credit-card
8   > statements are the first vertical slice; **equities, mutual funds, ETFs, and bonds are
core
9   > targets**, parsed into the same local ledger, with transaction/instrument-event
classification to
10  > follow. The parser core is deliberately **family-agnostic** (see §2), so adding an
instrument
11  > family is *new data + a reconciliation rule*, not a new pipeline.
12
13  ## 1. Core Ingestion Pipeline
14  The ingestion pipeline supports a generic SearchCriteria interface and multiple
providers (Track B plumbing):
15
16  * **Gmail** (muneem.ingestion.gmail`): Full OAuth2 read-only + search + attachment
download implemented.
17  * **IMAP** (muneem.ingestion.imap`): Basic auth + search implemented; get_headers` /
attachment methods are NotImplemented (experimental).
18  * **Outlook** (muneem.ingestion.outlook`): Device-flow auth + basic search implemented;
attachment/header/download NotImplemented (experimental).
19
20  Companion one-time auth tools live in tools/`. No high-level muneem fetch` CLI command
yet (see Roadmap B.4).
21
22  ## 2. Statement Parsers (concept-core architecture)
23  Deterministic parsers live under muneem.parsers`, registered via decorator and selected
by get_parser` on text heuristics + sender. Submodules are **auto-loaded on package import**
(_autoload_parsers`), so the registry is populated in the production path (not just under
tests).
24
25  The parser core is **concept-first and verification-first** (see [parser-
architecture.md](./parser-architecture.md)): an extractor maps an issuer's columns onto a bank-
agnostic **concept schema** (concepts.py`); a generic **engine** (engine.py`) does the
column?concept mapping (L0 header lexicon ? profile column-order ? L2 content search); and a
shared **verifier** (validation.py`) decides trust by *arithmetic* ? per-row balance walk +
total reconcile + SUMMARY bookend + a recorded confidence verdict. The engine is **family-
agnostic**: parameterized by a FamilySpec` + an injected reconciliation oracle, so a future
**instrument family** (equity/MF/ETF/bond holdings + buy/sell/dividend/fee events) reuses the
same control flow with its own concepts + oracle. Issuer knowledge lives in profiles.py` as
**data, not code**. Bank **savings** parsers share one engine-driven path ? savings.py` (per-
table + coordinate-clustered candidates ? engine ? oracle), so adding a savings issuer is mostly
a profile; HDFC keeps bespoke coordinate clustering (the documented extract_tables` exception).
26
27  **Active issuers** ? we have real statements + passwords, so these get validated on real
data:
28
29  * **HDFC Bank** (hdfc.py`): Savings / transaction-account statements via **coordinate
clustering** (deliberately *not* extract_tables() ? HDFC's table is semi-ruled). **Self-
validating:** running-balance reconciliation + a cross-check against the bank's own SUMMARY
(Dr/Cr counts, opening/closing balance); records documents.status` =
reconciled`/`unreconciled`. Refactored onto the concept core as the **reference
implementation**. ? **Verified on the user's 4 real statements** ? all reconcile *and* bookend,
without inspecting any value.
30  * **Axis Bank** (axis.py`): Savings + Credit Card. **Rewrite landed (step 3):** savings
runs on the shared engine-driven path (savings.py`) ? per-table + coordinate-clustered
candidates ? engine mapping ? reconciliation oracle (per-row walk, else total reconcile),
oracle-gated so it never persists unreconciled rows. ? **2/4 real statements reconcile +

```

persist** (Aug 2024, Oct 2025); the other 2 (May/June 2025) are vintages `extract\_tables` mangles into an incomplete ledger ? they store **nothing** (safety invariant verified on all 4), pending better extraction. Credit-card has no running balance (count only).

31 * **AU Bank** (`au.py`): engine-driven savings via the shared `savings.py` path ? same recipe, just `AUBANK\_PROFILE`. ? **4/4 real statements reconcile + persist** (the cleanest issuer so far).

32

33 > _Dropped: **ICICI** ? no statements on hand to validate against, so it was removed to focus effort on the issuers we can actually verify (HDFC/Axis/AU). It can return as a profile when data exists._

34

35 All balance-bearing parsers share the verifier and split parsing from DB-insert, so row logic is unit-testable without a PDF/DB and a parse is validated before it's trusted. **Concept-core steps 1?3 have landed** (the verifier + concept schema; the generic mapping engine + profiles; the shared savings path with Axis + AU Bank on it). Parsers use `pdfplumber` (with optional password). **Password resolution** (password_rules + hybrid unlock in `unlock.py`) is **not yet wired** into the main `muneem ingest --doc-type bank\_statement` path; it is exercised via the `@pytest.mark.local\_statements` tests on real (gitignored) statements.

36

37 See `password\_rules.py` (prefers user config dir) and `cli.py:ingest`.

38

39 ## 3. Database & CLI

40 * Schema v4 with `documents`, `offers`, and per-bank txn tables (`axis\_`, `hdfc\_transactions`). Dedup on sha256. No versioned migration scripts yet (IF NOT EXISTS + user_version pragma). A unified `transactions` table (plus `holdings` for instruments) is planned once the per-bank schemas stabilize (parser-architecture.md §9).

41 * CLI: `ingest <pdf>` (promo or bank_statement via parser), `offers list`, `txns list <bank>`.

42 * Config uses platformdirs for local dirs; secrets via keyring.

43

44 ## 4. Testing & Hygiene

45 * Strong offline regression on `testdata/promo-emails/` + synthetic parser/engine fixtures (default CI).

46 * Security guards in `test\_repo\_hygiene.py` + license allowlist test.

47 * Local-only tests for real statements (gitignored data; passwords via env/keyring, never committed).

48

49 ## 5. Next Steps

50 Near-term: finish the **Axis + AU** engine-driven parsers (step 3) and precision-test them on real statements alongside HDFC; wire password unlock into the main ingest path; then harden (Milestone D). **Beyond banks (core scope, not backlog):** a second **instrument concept family** for broker / MF / ETF / bond statements ? `holdings` + buy/sell/dividend/fee classification. Parked: DESIGN §8.3 (local LLM for sensitive categories), §8.5 (merchant canonicalization).

51

52 The project stays **local-first**, secrets in the OS keychain, promo corpus safe for CI/LLM experiments.

53

383. user (2026-05-31T09:50:08.851Z)

1 # muneem ? Project Plan & Roadmap

2

3 > Status: **active plan, pre-implementation.** This is the "how and in what order" companion to [DESIGN.md](../DESIGN.md) (the "what and why"). DESIGN.md owns the architecture and the threat model; this doc owns sequencing, milestones, and concrete tasks. Where they disagree, DESIGN.md wins and this doc should be corrected.

4 >

5 > Created 2026-05-30.

6 >

7 > **Scope:** muneem is a _whole-finance_ bookkeeper. Bank/CC statements are the first slice;

8 > **equity, mutual-fund, ETF, and bond statements are core scope** (not merely the § 6 "generalize"

9 > backlog), enabled by the family-agnostic concept core. Sequencing still leads with the


```

bank slice,
10     > but the data model and parser core are built to host an instrument family +
instrument-event
11     > classification (buy / sell / dividend / fees).
12
13     ---
14
15     ## ?? Known issues ? must-fix
16
17     > **Update (2026-06):** the concept-core rework has landed (parser-architecture.md steps
172); the
18     > Axis rewrite onto it is **in progress (step 3)**, with **AU Bank** as a sibling issuer
on the same
19     > coordinate-clustering + total-reconcile path. **ICICI was removed** (no statements on
hand to
20     > validate). HDFC verifies on the user's 4 real statements.
21
22     - **Axis savings parser needs a thorough rewrite.** The current `extract_tables()`-based
23     mapping is **wrong and unreliable on real Axis layouts**: the real columns are
24     `Txn Date | Transaction | Withdrawals | Deposits | Balance | Other Information` (not
the
25     assumed `Debit/Credit`), the column count and date placement **vary between
statements**
26     (Jun 2025 = 8 cols, per-row balance, date merged into the narration cell; Oct 2025 = 9
27     cols, clean date in col 0, no per-row balance), `can_handle` matches "Statement of
Account"
28     but the text says "Account Statement", and the account-number regex misses. **Stop-
gapped
29     in PR #19** ? `AxisParser` now refuses to persist a parse that doesn't reconcile
(stores 0
30     rows, marks the document `unreconciled`), so no mislabeled data is saved, but it
parses
31     nothing useful yet. **Real fix:** an HDFC-style **coordinate-clustering** rewrite,
validated
32     by **total reconciliation** (`opening + ?Deposits ? ?Withdrawals == closing`,
confirmed TRUE
33     on the Oct 2025 statement) plus the opening/closing bookend. Validate against the June
+
34     October 2025 statements and any future Axis files. Credit-card
(`AxisCreditCardParser`) is
35     separate and unverified on real data. **The rewrite should land on the new concept-
core
36     architecture ? see [ `parser-architecture.md` ](./parser-architecture.md) (it's step 3
of that
37     plan and the first real test of "generic-first").**
38
39     ---
40
41     ## 1. Decisions locked (2026-05-30)
42
43     These resolve open decisions from DESIGN.md § 8. They were made by the user explicitly;
this section is the start of the decision log.
44
45     | DESIGN.md ref | Decision | Choice |
46     |---|---|---|
47     | § 8.1 Language | Implementation language for v1 | **Python-only.** Accept the heavier
deployment story in exchange for the best PDF/OCR/LLM ecosystem and fastest path on the hard
parsing work. Revisit only if the ingestion daemon outgrows Python. |
48     | § 8.4 First slice | What to build first | **Both tracks in parallel** ? the promo-
corpus parse?store?query spine *and* the Gmail ingestion plumbing, developed as two decoupled
tracks that converge at the first bank slice. |
49     | § 8.2 Passwords | Password sourcing strategy | **Hybrid: candidate dictionary ? per-
sender rule template ? interactive prompt.** Cache the winning method per sender. Secrets never
touch the repo or logs. |
50     | (practical) | First real bank parser target | **HDFC** ? born-digital statements,

```

name+DDMM-style PDF password (per CLAUDE.md), and the issuer with the longest consistent statement history to test against. |

51

52 ****Still parked**** (do not pick silently ? see § 9 below): § 8.3 LLM model for ***sensitive*** categories, § 8.5 merchant canonicalization, and the timing of encryption-at-rest (SQLCipher).

53

54 ---

55

56 **## 2. Guiding principles** (inherited from DESIGN.md § 5 + CLAUDE.md)

57

58 1. ****Local by default.**** All raw statement data and the DB stay on the machine. OCR/LLM work runs locally for any sensitive category. No cloud sync of staging or DB, ever.

59 2. ****No cloud LLM on sensitive contents**** (bank, CC, broker, MF, insurance) without explicit per-category approval in-session. The ****promo corpus** is the one non-sensitive playground** where cloud LLMs are policy-permitted ? so it's where we validate any LLM-extraction pattern ***before*** it could ever see a statement.

60 3. ****Test on safe, committed data wherever possible.**** `testdata/promo-emails/` is the regression corpus and runs in CI. Statement parsing gets CI coverage via ****synthetic/redacted fixtures**** in the issuer's layout ? never real statements.

61 4. ****Secrets discipline.**** OAuth tokens ? OS keychain (Windows Credential Manager). Nothing sensitive in git; don't weaken `.gitignore`. Logs are redacted (no account numbers, balances, line items, or statement text).

62 5. ****Decisions parked for humans stay parked.**** Don't write code that silently locks in § 9's open items.

63

64 ---

65

66 **## 3. Proposed Python stack**

67

68 User-locked is Python-only. The libraries below are ***recommendations*** (well-known OSS, per CLAUDE.md's "batteries-included stdlib + well-known OSS over novel dependencies") ? open to override, not parked decisions. Anything that is a cloud ****service**** is flagged and needs a nod before adoption.

69

70 | Concern | Proposed | Notes |

71 |---|---|---|

72 | Packaging / deps / venv | ****uv**** + `pyproject.toml` | Fast, modern. Poetry is an equally fine fallback. |

73 | Lint + format | ****ruff**** | One tool, fast. |

74 | Tests | ****pytest**** | Regression suite runs offline in CI on the promo corpus + synthetic fixtures. |

75 | PDF text + image extraction | ****PyMuPDF (`fitz`)**** | Carried from Email-Mining; confirmed the right library. |

76 | Table recovery (born-digital statements) | ****pdfplumber**** | Coordinate/word-level extraction for HDFC's transaction tables. |

77 | PDF decryption | ****pikepdf**** (libqpdf) | Robust standard-security-handler support. `pypdf` is the fallback. |

78 | Image dedup | ****Pillow + scikit-image SSIM**** | Carried from Email-Mining (MD5 hash + SSIM ? 0.95). |

79 | OCR (image PDFs, later) | ****Tesseract**** via `pytesseract` | Local. Windows needs the Tesseract binary installed (setup task). |

80 | Gmail | ****google-api-python-client + google-auth-oauthlib**** | Read-only scope `gmail.readonly`. |

81 | Secret storage | ****keyring**** | Uses Windows Credential Manager on win32. |

82 | DB | ****sqlite3 (stdlib)**** + lightweight versioned migrations | Encryption-at-rest (SQLCipher via `sqlcipher3`) deferred to Milestone D per DESIGN.md. |

83 | CLI | ****Typer**** | Clean ergonomics; `argparse` (stdlib) is the zero-dependency fallback. |

84 | LLM (promo only, non-sensitive) | deterministic-first; cloud LLM ****only if needed and approved**** | See § 5 Track A. Sensitive-doc LLM is parked (§ 9 / DESIGN § 8.3). |

85

86 ---

87

88 **## 4. Build order at a glance**

89

```

90     ...
91     Milestone 0 ? Decisions & scaffolding (no sensitive data)
92         ?
93         ?????????????????????????????????????????????????????????????
94         ?                                     ?
95         TRACK A: Promo spine                TRACK B: Gmail ingestion        (run in parallel,
96         parse ? store ? query                fetch ? stage ? catalog        fully decoupled)
97         (committed corpus, CI)              (real email, local only)
98         ?                                     ?
99         ?????????????????????????????????
100             ?
101         Milestone C ? HDFC vertical slice (CONVERGENCE)
102         unlock (hybrid passwords) ? HDFC parser ? transactions ? CLI
103         ? completes DESIGN.md § 7
104             ?
105             ?
106         Milestone D ? Harden & evaluate (the pre-generalize checkpoint)
107             ?
108             ?
109         Generalize (more issuers, local-LLM fallback, holdings, items, analytics, UI)
110     ...
111
112     Tracks A and B are genuinely independent: A needs no OAuth and no secrets; B needs no
113     parsing. As a single developer you can interleave them in any order ? neither blocks the other.
114     **Both must land before Milestone C**, where a fetched-and-unlocked HDFC PDF flows through the
115     parser for the first time.
116
117     Effort tags below are rough (S / M / L), for relative sequencing only ? not calendar
118     estimates.
119
120     ---
121
122     ## 5. Milestones & tasks
123
124     ### Milestone 0 ? Decisions & scaffolding · no sensitive data
125
126     - [ ] **0.1 (S)** Project skeleton: `pyproject.toml` (uv), package dir (name TBD, e.g.
127     `muneem/`), `ruff` + `pytest` configured. *Do not over-materialize the source tree (DESIGN § 9)
128     ? create only what 0.x needs.*
129     - [ ] **0.2 (S)** Offline CI: GitHub Actions (or local pre-commit) that installs deps
130     and runs `pytest` against committed data only. A trivial `test_smoke` proves the loop is green.
131     - [ ] **0.3 (S)** Config + secrets foundation: a config loader (env + a gitignored
132     `config.toml`), and a `keyring`-backed secret accessor stub. Confirm `.gitignore` covers
133     `config.toml` and any new secret paths.
134     - [ ] **0.4 (S)** `gitignore` audit: verify the proposed local-only dirs
135     (`testdata/statements/`, staging dir, `*.db`) are blocked. Add a
136     `testdata/statements/.gitkeep`-style note or README so the expected-but-absent dir is
137     documented.
138     - **Exit:** `uv run pytest` is green in CI on a clean checkout; repo layout agreed.
139
140     ### Track A ? Promo spine: parse ? store ? query · committed corpus, fully CI-testable
141
142     > Resurrects the Email-Mining deals feature as muneem's first working output. Zero
143     secrets, zero OAuth.
144
145     - [ ] **A.1 (M)** Port Email-Mining's extraction into a real module: PyMuPDF per-page
146     `get_text` + `get_images`, size filter (?300x300), MD5 + SSIM dedup. Replace the edit-line-55
147     hardcoded path with proper arguments/config. Replace one-shot stdout with a return value.
148     - [ ] **A.2 (S)** Minimal schema slice + migration v1: `documents` (id, source,
149     external_id, sender, subject, received_date, doc_type, sha256, path, status, created_at) and
150     `offers` (id, document_id, merchant, promo_code, description, expiry_date, redeemed,
151     source_text). Full schema is future work (`docs/schema.md`).
152     - [ ] **A.3 (M)** Promo extraction, **deterministic-first**: regex/heuristics for promo
153     codes + expiry dates over the unified text. Where deterministic extraction misses, an **LLM
154     fallback** is permitted *because promo is non-sensitive* ? but that pulls in a cloud service, so

```

it's gated on a quick okay (reuse Email-Mining's Gemini path, go local via Ollama, or use Anthropic ? your call when we get there).

135 - [] ****A.4 (M)**** `muneem ingest <pdf>` writes a `documents` + `offers` row; `muneem offers list` reads them back. ****Ground-truth + regression test:**** establish the known promo codes for each of the 10 corpus PDFs, then a `pytest` that runs the pipeline over `testdata/promo-emails/` and asserts recovery. (Building the ground-truth answer key is itself part of this task.)

136 - ****Exit:**** `ingest ? SQLite ? offers list` works end-to-end; green regression test on the committed corpus. Stages 3?4?5 proven with zero risk.

137

138 **### Track B ? Gmail ingestion: fetch ? stage ? catalog . real email, local only**

139

140 > First contact with real (sensitive) email. Everything it writes is gitignored.

141

142 - [] ****B.1 (M)**** Gmail OAuth2: register a Google Cloud project, `gmail.readonly` scope, store the token in `keyring`. Document the one-time consent flow.

143 - [] ****B.2 (M)**** Financial-document detection v1: a ****sender allowlist**** + server-side query (`has:attachment filename:pdf from:? newer\_than:~`). Start narrow and precise; broaden later. Pull matching messages + PDF attachments into a gitignored staging dir.

144 - [] ****B.3 (S)**** Incremental sync: persist Gmail `historyId`; on re-run, fetch only what's new. Idempotent (re-running never duplicates `documents`, keyed by message id + attachment hash).

145 - [] ****B.4 (S)**** Catalog fetched attachments as `documents` rows (status `locked`/`unparsed`), with sha256 + staging path. `muneem fetch --since 90d` is the entry point.

146 - ****Exit:**** `muneem fetch` populates the staging dir + `documents` rows from a narrow real-Gmail query. (Manual/local verification ? not CI; it touches a real account.)

147

148 **### Milestone C ? HDFC vertical slice . CONVERGENCE . completes DESIGN.md § 7**

149

150 > A fetched HDFC statement gets unlocked and parsed into real transactions. This is the moment every stage runs once on real data the user cares about.

151

152 - [] ****C.1 (M)**** Unlocking (Stage 2) with the ****hybrid**** strategy: candidate dictionary derived from user attributes (e.g. `first4(name).upper() + DDMM(dob)`) ? per-sender rule template ? interactive prompt. Cache the winning method per sender in the keychain. ****No password material in repo or logs.**** Decrypt with pikepdf; advance `documents.status` to `unlocked`.

153 - [] ****C.2 (L)**** HDFC parser (Stage 3, the hard 80%): born-digital text-layer + pdfplumber table recovery ? normalized transactions (txn_date, value_date, amount, direction, narration, running balance). ****Synthetic HDFC-layout fixtures**** committed for CI; real statements stay gitignored locally.

154 - [] ****C.3 (M)**** Schema migration v2: `accounts` (issuer, account_type, last4, holder) + `transactions` (account_id, document_id, txn fields, category, merchant_id-nullable). Populate from the parser. `muneem txns list --last-month` prints real parsed transactions.

155 - ****Exit:**** ****DESIGN.md § 7 vertical slice complete**** ? Gmail ? HDFC statement ? unlocked ? parsed ? SQLite ? CLI lists last month's transactions, on real data, locally.

156

157 **### Milestone D ? Harden & evaluate . the pre-generalize checkpoint**

158

159 - [] ****D.1 (M)**** Redacted structured logging (assert no statement text / account numbers leak); idempotent, resumable re-runs across all stages.

160 - [] ****D.2 (S)**** Encryption-at-rest decision (DESIGN § 5.4): adopt SQLCipher now or schedule it. (Parked item ? bring evidence, then decide.)

161 - [] ****D.3 (M)**** ****Evaluation writeup:**** how well did the deterministic HDFC parser hold across statement vintages? Where does it break? Which categories will need the local-LLM fallback? This is the evidence DESIGN § 7 asks for ***before*** generalizing. Capture in `docs/parser-notes-hdfc.md`.

162 - ****Exit:**** the slice is robust and re-runnable; a written go/no-go on generalization with real evidence.

163

164 ---

165

166 **## 6. Generalize (post-checkpoint backlog ? scope after Milestone D)**

167

168 Ordered roughly by leverage, not committed:

```

169
170     - Second/third bank parsers + the first **credit-card bill** parser (reuses unlock +
parser scaffolding).
171     - **Local-LLM long-tail fallback** (Ollama) for issuers that don't justify a bespoke
parser ? **local, because these are sensitive** (resolves DESIGN § 8.3).
172     - Broker / MF statements ? `holdings` (point-in-time positions; the investment-timeline
use-case).
173     - Itemized receipts (Swiggy/Amazon/BigBasket/etc.) ? `items` (the price-intelligence +
catalog use-cases).
174     - **Merchant canonicalization** ? `merchants` (DESIGN § 8.5 ? defer until real data
exposes the actual mess, then decide rules-file vs. library vs. learned).
175     - Analytics surfaces for the DESIGN § 2 use-cases (CLI views first; optional small
**local** web UI later).
176
177     ---
178
179     ## 7. Cross-cutting workstreams (run throughout, not a phase)
180
181     - **Security & secrets hygiene** ? keychain for tokens + cached passwords; redacted
logs; gitignore audits each milestone; encryption-at-rest tracked to D.2.
182     - **Schema migrations from day one** ? versioned migration scripts; the schema *will*
churn as new issuers land. Keep `docs/schema.md` current.
183     - **Test-corpus curation** ? ground-truth answer key for the promo corpus (A.4);
synthetic/redacted statement fixtures for CI (C.2). Never commit real statements.
184     - **Decision log** ? append resolutions to § 1 here and mark the corresponding DESIGN.md
§ 8 item resolved.
185
186     ---
187
188     ## 8. Risks & mitigations
189
190     | Risk | Mitigation |
191     |---|---|
192     | Sensitive data leaks into git, logs, or a cloud LLM | Local-by-default; promo-only LLM
playground; redacted logging (D.1); gitignore audits; the § 2 principles are non-negotiable. |
193     | HDFC parser is brittle across statement vintages | Synthetic fixtures + the D.3
evaluation gate *before* generalizing; deterministic-first with a planned local-LLM fallback for
the long tail. |
194     | Password sourcing fails noisily on backfill | Hybrid strategy degrades to interactive
prompt; cache per sender so it's a one-time cost. |
195     | Gmail OAuth / quota friction | Read-only scope, server-side queries (cheap),
incremental `historyId` sync; narrow allowlist first. |
196     | Scope creep into the full § 2 vision before the slice works | Hard checkpoint at
Milestone D; generalization is explicitly post-checkpoint. |
197     | Two parallel tracks diverge / integration surprises at C | Both write through the
*same* `documents` table and schema-migration discipline from A.2/B.4, so convergence is a join,
not a rewrite. |
198
199     ---
200
201     ## 9. Still-open decisions (do NOT pick silently)
202
203     Carried from DESIGN.md § 8, not yet resolved ? surface and ask when a milestone forces
them:
204
205     - **§ 8.3 ? LLM model for sensitive categories.** Coupled to your hardware (GPU? RAM?)
and latency tolerance. Forced at the start of the generalize phase (local-LLM fallback). Ollama
+ Llama/Qwen/Mistral is the likely shape, but unconfirmed.
206     - **§ 8.5 ? Merchant canonicalization.** Deferred until real receipt data exposes the
mess. Forced when `items`/`merchants` land.
207     - **Encryption-at-rest timing** (D.2) ? SQLCipher now vs. later.
208     - **Promo-track LLM** (A.3) ? only reached if deterministic extraction underperforms;
cloud is policy-permitted for promo but still needs a quick okay on which vendor.
209
210     ---

```

211
212 ## 10. Definition of done for v1
213
214 The DESIGN.md § 7 slice, hardened: ****`muneem fetch`** pulls HDFC statements from Gmail ?
unlocks them with the hybrid password strategy ? the HDFC parser writes normalized
`transactions` to SQLite ? ****`muneem txns list`** shows last month's activity ? all locally, with
redacted logs, idempotent re-runs, green CI on synthetic fixtures, and a written evaluation of
where to go next.** The promo "forgotten deals" feature ships alongside as the Track A
deliverable.
215

384. assistant (2026-05-31T09:52:42.263Z)

Now I can see the staleness clearly. ****CURRENT_STATE §5**** still says "finish Axis + AU" (they're done), and ****ROADMAP**** is the most drifted: its "must-fix" still describes Axis as broken/stop-gapped, its status says "pre-implementation," its stack table still lists the ****banned PyMuPDF****, and there's no signal that Milestone 0 / Track A / the parsers have landed. ****parser-architecture.md`** is current (updated in #26). Let me fix ****CURRENT_STATE §5** and bring ****ROADMAP** in line:

385. user (2026-05-31T09:52:49.714Z)

The file C:\Users\avidu\Projects\muneem\docs\CURRENT_STATE.md has been updated successfully.
(file state is current in your context ? no need to Read it back)

386. assistant (2026-05-31T09:52:58.949Z)

I duplicated the closing line ? let me check and fix:

387. user (2026-05-31T09:52:59.610Z)

49 ## 5. Next Steps
50 Concept-core ****steps 1?3 are done**** (verifier + schema; mapping engine + profiles; the
shared savings path). Real-statement precision: ****HDFC 4/4, AU 4/4, Axis 2/4**** (the 2 Axis
misses store nothing ? oracle-gated). In rough order from here:
51
52 * ****Recover the 2 Axis vintages**** (May/June 2025): ****`extract\_tables`** mangles them into an
incomplete ledger; needs a stronger coordinate clusterer. Isolated and safe (they currently
store nothing).
53 * ****Wire password unlock into `muneem ingest`**** (Roadmap C.1): today the hybrid unlock
(****`unlock.py`**) is exercised only by the ****`local\_statements`** tests, not the main path.
54 * ****Step 4 ? L3 local-vision fallback**** (pending the local-VLM deep-research run) for
layouts the deterministic ladder can't reconcile.
55 * ****Step 5 ? unify storage**** (****`transactions`** + ****`holdings`**) and begin the ****instrument
family**** (equity/MF/ETF/bond ? holdings + buy/sell/dividend/fee classification) ? the next big
piece of the whole-finance vision.
56
57 Parked: DESIGN §8.3 (local LLM for sensitive categories), §8.5 (merchant
canonicalization), encryption-at-rest timing (D.2).
58
59 The project stays ****local-first****, secrets in the OS keychain, promo corpus safe for
CI/LLM experiments.
60
61 The project stays ****local-first****, secrets in the OS keychain, promo corpus safe for
CI/LLM experiments.
62

388. user (2026-05-31T09:53:07.134Z)

The file C:\Users\avidu\Projects\muneem\docs\CURRENT_STATE.md has been updated successfully.
(file state is current in your context ? no need to Read it back)

389. assistant (2026-05-31T09:53:16.198Z)

Now the ROADMAP ? status line, a progress snapshot, the must-fix reframe, the banned-PyMuPDF stack fix, and milestone-header status annotations:

390. user (2026-05-31T09:53:19.974Z)

The file C:\Users\avidu\Projects\muneem\docs\ROADMAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

391. user (2026-05-31T09:53:35.064Z)

The file C:\Users\avidu\Projects\muneem\docs\ROADMAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

392. user (2026-05-31T09:53:39.345Z)

The file C:\Users\avidu\Projects\muneem\docs\ROADMAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

393. assistant (2026-05-31T09:53:44.708Z)

Now annotate the milestone headers with their real status:

394. user (2026-05-31T09:53:46.145Z)

The file C:\Users\avidu\Projects\muneem\docs\ROADMAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

395. user (2026-05-31T09:53:48.046Z)

The file C:\Users\avidu\Projects\muneem\docs\ROADMAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

396. user (2026-05-31T09:53:49.910Z)

The file C:\Users\avidu\Projects\muneem\docs\ROADMAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

397. user (2026-05-31T09:53:51.386Z)

The file C:\Users\avidu\Projects\muneem\docs\ROADMAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

398. assistant (2026-05-31T09:54:05.038Z)

Let me verify nothing stale remains across the docs and check `docs/TESTING.md`:

399. user (2026-05-31T09:54:08.326Z)

```
--- 'stop-gap' / Axis 'in progress' / 'PyMuPDF' outside allowed spots ---
docs/parser-architecture.md:105:  flagged; today's stop-gap already refuses to persist
unreconciled rows)
--- PyMuPDF mentions (should be 'banned' context only) ---
DESIGN.md:38:muneem is the successor to `~/Projects/Email-Mining`, a small prototype the user
built earlier that extracted promo codes from manually-saved email PDFs using PyMuPDF + Gemini
for image OCR. Same insight ("the inbox contains structured value I'm forgetting"), generalized
scope.
DESIGN.md:163:| **Python only** | Best-in-class PDF, OCR, and ML libraries (pdfplumber, PyMuPDF,
Tesseract bindings, Ollama clients, every parser under the sun); fastest to prototype | Heavier
deployment story; less ergonomic for the long-running ingestion daemon |
DESIGN.md:222:The predecessor project (`~/Projects/Email-Mining`) extracted promo codes and
expiry dates from manually-saved promo email PDFs, using PyMuPDF for text+image extraction and
Gemini for image OCR and summarization. Its scope was just "find the coupon I forgot about."
docs/ROADMAP.md:130:- [ ] **A.1 (M)** Port Email-Mining's extraction into a real module: PyMuPDF
```

per-page `get\_text` + `get\_images`, size filter (?300×300), MD5 + SSIM dedup. Replace the edit-line-55 hardcoded path with proper arguments/config. Replace one-shot stdout with a return value.

docs/TESTING.md:22:Direct inspection of all 10 PDFs ? PyMuPDF text-layer extraction (`get\_text`) + large-image count (`get\_images`, ?300×300 px, matching the Email-Mining dedup filter) + visual rendering of a sample. This is the empirical basis for everything below.

docs/TESTING.md:41:- ****Ligatures in the text layer.**** PyMuPDF returns `0?er` / `0?` (U+FB00 `?`), not `Offer`/`Off`. Naïve substring/regex matching for "Off"/"Offer" silently fails. ? ****NFKC normalization**** before matching, with a dedicated unit test (`airasia-2`, `airasia-3`).

docs/prior-art-email-mining.md:37:PyMuPDF (fitz).open()

--- TESTING.md parser/icici refs ---

(none in TESTING.md)

400. assistant (2026-05-31T09:54:43.481Z)

Mostly clean ? the PyMuPDF mentions in DESIGN/prior-art/TESTING are accurate historical context (the predecessor used it; it's listed as banned where relevant). Two real fixes: `parser-architecture.md`'s "today's stop-gap" (no longer a stop-gap) and the stale PyMuPDF in ROADMAP's A.1 task. Let me see the architecture context:

401. user (2026-05-31T09:54:44.546Z)

```
96     profile, and only drop to custom code if the generic ladder genuinely can't cope.
97
98     ## 7. Failsafes ? the "traps" (loud drift, never silent)
99
100    The Axis lesson: a broken parse must *announce itself*. Built-in tripwires, each ? flag
+ provenance,
101    never store-and-pretend:
102
103    - header label matches no lexicon entry ? "unknown field, mapped by structure ? review"
104    - reconciliation breaks, or parsed counts ? printed summary ? **quarantine** (store
nothing or store
105    flagged; today's stop-gap already refuses to persist unreconciled rows)
106    - a profile's expected anchor is missing ? "format may have changed for <bank>"
107    - _(L3)_ VLM output won't reconcile, or disagrees with a deterministic parse ? reject
the VLM
```

402. user (2026-05-31T09:54:49.056Z)

The file C:\Users\avidu\Projects\muneem\docs\parser-architecture.md has been updated successfully. (file state is current in your context ? no need to Read it back)

403. user (2026-05-31T09:54:52.429Z)

The file C:\Users\avidu\Projects\muneem\docs\ROADMAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

404. assistant (2026-05-31T09:55:37.673Z)

For internal consistency, let me tick the checkboxes under the two "? DONE" sections (Milestone 0 and Track A):

405. user (2026-05-31T09:55:39.062Z)

The file C:\Users\avidu\Projects\muneem\docs\ROADMAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

406. user (2026-05-31T09:55:40.510Z)

The file C:\Users\avidu\Projects\muneem\docs\ROADMAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

407. user (2026-05-31T09:55:41.923Z)

The file C:\Users\avidu\Projects\muneeem\docs\ROADMAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

408. user (2026-05-31T09:55:43.337Z)

The file C:\Users\avidu\Projects\muneeem\docs\ROADMAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

409. user (2026-05-31T09:55:44.756Z)

The file C:\Users\avidu\Projects\muneeem\docs\ROADMAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

410. user (2026-05-31T09:55:46.163Z)

The file C:\Users\avidu\Projects\muneeem\docs\ROADMAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

411. user (2026-05-31T09:55:47.345Z)

The file C:\Users\avidu\Projects\muneeem\docs\ROADMAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

412. assistant (2026-05-31T09:55:54.485Z)

All docs updated. Let me verify the ROADMAP top reads coherently and confirm the suite is still green before committing:

413. user (2026-05-31T09:55:55.440Z)

```
1      # muneeem ? Project Plan & Roadmap
2
3      > Status: **active plan; mid-implementation.** Milestone 0 + Track A are done and the
parser concept-core has landed (see the Progress snapshot below;
[ `CURRENT_STATE.md` ](./CURRENT_STATE.md) is the live status). This is the "how and in what
order" companion to [ `DESIGN.md` ](../DESIGN.md) (the "what and why"); DESIGN.md owns the
architecture and threat model, this doc owns sequencing. Where they disagree, DESIGN.md wins.
4      >
5      > Created 2026-05-30.
6      >
7      > **Scope:** muneeem is a _whole-finance_ bookkeeper. Bank/CC statements are the first
slice;
8      > **equity, mutual-fund, ETF, and bond statements are core scope** (not merely the § 6
"generalize"
9      > backlog), enabled by the family-agnostic concept core. Sequencing still leads with the
bank slice,
10     > but the data model and parser core are built to host an instrument family +
instrument-event
11     > classification (buy / sell / dividend / fees).
12     >
13     > **Progress snapshot (2026-06):** Milestone 0 (scaffolding / offline CI / config +
keyring) ? and
14     > **Track A** (promo spine: `ingest` ? SQLite ? `offers list`, green regression) ? are
done.
15     > **Track B:** Gmail OAuth2 + search + attachment download ?; the `muneeem fetch` CLI
(B.4) is not
16     > built yet. **Parsers:** the concept-core rework (parser-architecture.md **steps 1?3**)
landed ?
17     > HDFC 4/4 real, AU Bank 4/4, Axis 2/4, ICICI removed. **Not yet wired:** password
unlock into the
18     > main `ingest` path (C.1) and the unified `transactions`/`holdings` schema (C.3 / step
5). The
```

```

19      > checkbox lists below predate the concept-core pivot ? treat
[ `CURRENT_STATE.md` ](./CURRENT_STATE.md)
20      > as the live status.
21
22      ---
23
24      ## ?? Known issues
25
26      > **Axis savings ? rewrite landed (2026-06), partially complete.** The old fixed-column
27      > `extract_tables()` mapping (wrong columns, layouts that drift between vintages) was
replaced by the
28      > concept-core path: coordinate clustering + per-table mapping ? engine ? reconciliation
oracle
29      > (per-row walk, else total reconcile), oracle-gated. See [ `parser-
architecture.md` ](./parser-architecture.md)
30      > step 3. **Real-data precision: 2/4** ? Aug 2024 and Oct 2025 reconcile + persist;
**May & June 2025
31      > still don't** (`extract_tables` mangles those vintages into an incomplete ledger,
which the oracle
32      > correctly rejects). The 2 misses **store nothing** ? never mislabeled rows.
33      >
34      > **Remaining:** a stronger coordinate clusterer to recover May/June 2025 (and future
Axis vintages).
35      > The credit-card parser (`AxisCreditCardParser`) is still unverified on real data.

```

414. user (2026-05-31T09:56:03.550Z)

```

===== short test summary info =====
XPASS tests/test_ocr.py::test_swiggy_image_code_recovered_fuzzily - recovery of the stylized
boxed code is OCR-engine/version dependent ? succeeds on Tesseract 5.4, returns nothing on the
apt build; the planned fix is a purpose-trained model (docs/DEPENDENCIES.md). strict=False so it
never fails the build.
114 passed, 13 deselected, 1 xpassed in 6.23s
--- changed docs ---
  M docs/CURRENT_STATE.md
  M docs/ROADMAP.md
  M docs/parser-architecture.md

```

415. assistant (2026-05-31T09:56:14.680Z)

ROADMAP top reads coherently, 114 tests green, docs-only changes. Shipping the docs-sync as a final PR:

416. user (2026-05-31T09:56:19.639Z)

```

Switched to a new branch 'docs/sync-state'
[docs/sync-state a0f8fd6] docs: sync ROADMAP / CURRENT_STATE / parser-architecture to the
shipped state
 3 files changed, 46 insertions(+), 41 deletions(-)

```

417. user (2026-05-31T09:56:30.523Z)

```

To https://github.com/avidullu/muneem.git
* [new branch]      docs/sync-state -> docs/sync-state
https://github.com/avidullu/muneem/pull/27

```

418. user (2026-05-31T09:56:43.569Z)

```

docs/parser-architecture.md | 2 +-
3 files changed, 46 insertions(+), 41 deletions(-)
Already on 'main'
- [deleted]          (none)      -> origin/docs/sync-state
Already up to date.
=== CLEAN-SLATE CHECK ===

```

```
branch: main
sync:    ## main...origin/main
tree:    0 uncommitted change(s)
open PRs: 0
8b65ad4 docs: sync ROADMAP / CURRENT_STATE / parser-architecture to the shipped state (#27)
```

419. assistant (2026-05-31T09:57:01.465Z)

```
? **Clean slate, fully synced to remote.**

- **Branch:** `main`, even with `origin/main` (nothing ahead/behind)
- **Working tree:** 0 uncommitted changes
- **Open PRs:** 0
- **Tip:** `8b65ad4` (docs sync, #27)
```

Docs are now accurate to the shipped state

```
- **[CURRENT_STATE.md](docs/CURRENT_STATE.md)** ? concept-core steps 1?3 done; HDFC/Axis/AU
parsers with real precision; §5 Next Steps rewritten to the actual remaining work.
- **[ROADMAP.md](docs/ROADMAP.md)** ? status flipped to mid-implementation; a Progress
snapshot at the top; Axis "must-fix" reframed as landed-but-partial (2/4); the banned-PyMuPDF
stack entry corrected; Milestone 0 + Track A ticked.
- **[parser-architecture.md](docs/parser-architecture.md)** ? steps 1?3 marked landed; "stop-
gap" language removed.
```

Where it stands (for when you resume)

The whole arc this session: ***6 squash-merged PRs*** ([#22](https://github.com/avidullu/muneem/pull/22)?[#27](https://github.com/avidullu/muneem/pull/27)) ? concept core ? mapping engine ? ICICI removal + total-finance docs ? Axis rewrite ? AU Bank ? docs sync. Real-statement precision: ***HDFC 4/4, AU 4/4, Axis 2/4*** (misses store nothing, oracle-gated). 114 offline tests green.

Top of the resume queue (in CURRENT_STATE §5 / ROADMAP):

1. Recover the 2 Axis vintages (May/June 2025) ? stronger coordinate clusterer.
2. Wire password unlock into `muneem ingest` (C.1).
3. Step 4 (L3 local-vision fallback, pending the VLM research) ? Step 5 (unified storage + the ***instrument family*** for equity/MF/ETF/bonds).

Everything's safe to pause on. Good place to pick up later ? enjoy the break.

420. user (2026-05-31T09:57:03.680Z)

Can you please also complete the "wire the password-unlock into the main muneem ingest path so this all runs from the CLI"? and update a document with step instructions to use and /or enable them

Also ensure that cI is green after you are all done and the repo and local are in sync. With that we take a break.

421. assistant (2026-05-31T09:57:32.951Z)

On it ? this is Roadmap C.1 (wire the hybrid unlock into `muneem ingest`), plus a setup doc, then green CI and in-sync repo. Let me first understand the existing unlock/extract/config plumbing before wiring it together.

422. user (2026-05-31T09:57:33.911Z)

```
1      """Password rule loader for statement decryption (hybrid strategy support).
2
3      Prefers the user config dir (via platformdirs, e.g. ~/.config/muneem/password_rules.yaml
4      on Linux) for real rules. Falls back to legacy CWD `password_rules.yaml` (common
during
5      dev) with a DeprecationWarning. Never commit real rules or the file itself.
6
7      See DESIGN.md §8.2 and CLAUDE.md security rules. Create the file in your config dir
```

```

8     with bank: password-or-rule entries (never commit it).
9     """
10
11     from __future__ import annotations
12
13     import warnings
14     from pathlib import Path
15
16     import yaml
17
18     from muneem.config import CONFIG_DIR
19
20
21     def get_password_for_bank(bank_name: str) -> str | None:
22         """Return the password (or derived rule string) for a bank, or None.
23
24         Resolution order:
25         1. <config_dir>/password_rules.yaml (recommended location)
26         2. ./password_rules.yaml (legacy, emits DeprecationWarning)
27         """
28         candidates = [
29             (CONFIG_DIR / "password_rules.yaml", False),
30             (Path("password_rules.yaml"), True),
31         ]
32         for path, is_legacy in candidates:
33             if path.exists():
34                 if is_legacy:
35                     warnings.warn(
36                         "Loading password_rules from CWD (./password_rules.yaml) is
37 deprecated. "
38                         f"Move it to {CONFIG_DIR / 'password_rules.yaml'} for future-
39 proofing. "
40                         "See docs and CLAUDE.md (never commit real password material).",
41                         DeprecationWarning,
42                         stacklevel=2,
43                     )
44                 with path.open(encoding="utf-8") as f:
45                     rules = yaml.safe_load(f) or {}
46                     val = rules.get(bank_name)
47                     return str(val) if val else None
48         return None

```

423. user (2026-05-31T09:57:33.972Z)

```

1     """PDF text extraction (the deterministic, born-digital path).
2
3     Uses **pdfminer.six (MIT)** ? a permissive, commercially-safe, pure-Python text
4     extractor. We deliberately do *not* use PyMuPDF/``fitz``: it is **AGPL-3.0**,
5     incompatible with muneem's commercial-safety principle (see docs/DEPENDENCIES.md).
6
7     NFKC normalization is applied because the real corpus returns ligatures
8     (e.g. U+FB00 '?') that would otherwise break naive matching. Image extraction +
9     OCR (for codes rasterized into banners, e.g. Swiggy's CTSWIGGYUX) is the image
10    tier and lands with the OCR work (pypdfium2 + Tesseract, both permissive).
11    """
12
13    from __future__ import annotations
14
15    import hashlib
16    import unicodedata
17    from dataclasses import dataclass
18    from pathlib import Path
19
20    from pdfminer.high_level import extract_text

```

```

21
22
23 @dataclass
24 class Extracted:
25     """The result of reading a PDF's text layer."""
26
27     path: Path
28     sha256: str
29     text: str
30
31
32 def _normalize(text: str) -> str:
33     """NFKC-normalize so ligatures fold to ASCII (U+FB00 '?' -> 'ff')."""
34     return unicodedata.normalize("NFKC", text)
35
36
37 def file_sha256(path: Path | str) -> str:
38     """Stream the file and return its hex SHA-256 (used for document dedup)."""
39     digest = hashlib.sha256()
40     with open(path, "rb") as fh:
41         for chunk in iter(lambda: fh.read(65536), b""):
42             digest.update(chunk)
43     return digest.hexdigest()
44
45
46 def extract(pdf_path: Path | str) -> Extracted:
47     """Extract and NFKC-normalize the full text layer of a PDF."""
48     path = Path(pdf_path)
49     text = extract_text(str(path))
50     return Extracted(path=path, sha256=file_sha256(path), text=_normalize(text))
51

```

424. assistant (2026-05-31T09:58:13.020Z)

Now the config, any keyring/secrets module, the CI workflow, and the existing unlock/CLI tests so I match the patterns:

425. user (2026-05-31T09:58:13.562Z)

```

1     """Local paths and configuration.
2
3     muneem is local-first: config lives in the OS config dir, data (DB + staging)
4     in the OS data dir. Nothing here touches the repository or the network, and no
5     secrets are stored here ? those go through :mod:`muneem.secrets` (the keychain).
6     """
7
8     from __future__ import annotations
9
10    import tomllib
11    from dataclasses import dataclass, field
12    from pathlib import Path
13    from typing import Any
14
15    from platformdirs import PlatformDirs
16
17    _dirs = PlatformDirs(appname="muneem", appauthor=False)
18
19    CONFIG_DIR = Path(_dirs.user_config_dir)
20    DATA_DIR = Path(_dirs.user_data_dir)
21    CONFIG_FILE = CONFIG_DIR / "config.toml"
22    DB_PATH = DATA_DIR / "muneem.db"
23    STAGING_DIR = DATA_DIR / "staging"
24
25
26    @dataclass(frozen=True)

```

```

27     class Config:
28         """Resolved local paths plus any values from ``config.toml``."""
29
30         config_dir: Path = CONFIG_DIR
31         data_dir: Path = DATA_DIR
32         config_file: Path = CONFIG_FILE
33         db_path: Path = DB_PATH
34         staging_dir: Path = STAGING_DIR
35         values: dict[str, Any] = field(default_factory=dict)
36
37
38     def load_config() -> Config:
39         """Load configuration, reading ``config.toml`` if it exists. Read-only."""
40         values: dict[str, Any] = {}
41         if CONFIG_FILE.is_file():
42             with CONFIG_FILE.open("rb") as fh:
43                 values = tomlib.load(fh)
44         return Config(values=values)
45
46
47     def ensure_dirs() -> None:
48         """Create the local config/data/staging dirs if missing. Never touches the repo."""
49         for directory in (CONFIG_DIR, DATA_DIR, STAGING_DIR):
50             directory.mkdir(parents=True, exist_ok=True)
51

```

426. user (2026-05-31T09:58:14.092Z)

```

src\muneem\secrets.py
.github\workflows\ci.yml
tools\gmail_auth.py
tools\imap_auth.py
tools\outlook_auth.py

```

427. user (2026-05-31T09:58:14.432Z)

```

1     import os
2     from pathlib import Path
3
4     import pypdf
5     import pytest
6
7     from muneem import unlock
8
9
10    def _write_pdf(path: Path, password: str | None = None) -> Path:
11        writer = pypdf.PdfWriter()
12        writer.add_blank_page(width=200, height=200)
13        if password:
14            writer.encrypt(password)
15        with open(path, "wb") as fh:
16            writer.write(fh)
17        return path
18
19
20    @pytest.fixture
21    def encrypted_pdf(tmp_path):
22        return _write_pdf(tmp_path / "enc.pdf", password="secret123")
23
24
25    @pytest.fixture
26    def plain_pdf(tmp_path):
27        return _write_pdf(tmp_path / "plain.pdf")
28
29

```

```

30     def test_is_encrypted(encrypted_pdf, plain_pdf):
31         assert unlock.is_encrypted(encrypted_pdf)
32         assert not unlock.is_encrypted(plain_pdf)
33
34
35     def test_unlock_via_candidate(encrypted_pdf, tmp_path):
36         res = unlock.unlock(encrypted_pdf, candidates=["wrong", "secret123"], dest=tmp_path
/ "out.pdf")
37         assert res.ok and res.method == "candidate" and res.password == "secret123"
38         assert res.output_path.exists()
39         assert not unlock.is_encrypted(res.output_path) # the decrypted copy opens without
a password
40
41
42     def test_unlock_via_prompt(encrypted_pdf):
43         res = unlock.unlock(encrypted_pdf, candidates=["nope"], prompt_fn=lambda:
"secret123")
44         assert res.ok and res.method == "prompt" and res.password == "secret123"
45
46
47     def test_unlock_fails_and_leaks_no_password(encrypted_pdf):
48         res = unlock.unlock(encrypted_pdf, candidates=["a", "b"])
49         assert not res.ok and res.password is None and res.output_path is None
50
51
52     def test_plaintext_is_copied(plain_pdf, tmp_path):
53         res = unlock.unlock(plain_pdf, dest=tmp_path / "copy.pdf")
54         assert res.ok and res.method == "plaintext" and res.output_path.exists()
55
56
57     @pytest.mark.local_statements
58     def test_real_hdfc_statements_decrypt():
59         """Local-only: set MUNEEM_TEST_HDFC_PW and place statements in testdata/hdfc-
statements/. """
60         pw = os.environ.get("MUNEEM_TEST_HDFC_PW")
61         pdfs = sorted(Path("testdata/hdfc-statements").glob("*.pdf"))
62         if not pw or not pdfs:
63             pytest.skip("needs MUNEEM_TEST_HDFC_PW + real statements (never in CI)")
64         for pdf in pdfs:
65             assert unlock.is_encrypted(pdf)
66             assert unlock.unlock(pdf, candidates=[pw]).ok
67

```

428. assistant (2026-05-31T09:58:19.084Z)

There's a `secrets.py` (keyring) and a CI workflow. Let me read those plus the CLI tests:

429. user (2026-05-31T09:58:20.048Z)

```

1     """Secret storage via the OS keychain.
2
3     OAuth tokens and cached statement passwords are stored here (Windows Credential
4     Manager / macOS Keychain / Secret Service), never in the repo, on disk in
5     plaintext, or in logs. See DESIGN.md section 5.
6
7     This is the thin foundation used later by ingestion (token) and unlocking
8     (per-sender cached password) ? see docs/ROADMAP.md milestones B.1 and C.1.
9     """
10
11     from __future__ import annotations
12
13     import keyring
14     from keyring.errors import PasswordDeleteError
15
16     _SERVICE = "muneem"

```

```

17
18
19 def get_secret(name: str) -> str | None:
20     """Return the stored secret, or None if absent."""
21     return keyring.get_password(_SERVICE, name)
22
23
24 def set_secret(name: str, value: str) -> None:
25     """Store (or replace) a secret in the OS keychain."""
26     keyring.set_password(_SERVICE, name, value)
27
28
29 def delete_secret(name: str) -> None:
30     """Delete a secret if present; a no-op if it is not."""
31     try:
32         keyring.delete_password(_SERVICE, name)
33     except PasswordDeleteError:
34         pass
35

```

430. user (2026-05-31T09:58:20.559Z)

```

1     name: ci
2
3     on:
4         push:
5             branches: [main]
6         pull_request:
7         workflow_dispatch:
8
9     jobs:
10        test:
11            runs-on: ubuntu-latest
12            strategy:
13                fail-fast: false
14            matrix:
15                python-version: ["3.11", "3.12", "3.13"]
16            steps:
17                - uses: actions/checkout@v6
18                - uses: actions/setup-python@v6
19                  with:
20                      python-version: ${ matrix.python-version }
21                - cache: pip
22                  cache-dependency-path: pyproject.toml
23                - name: Install
24                  run: |
25                      python -m pip install --upgrade pip
26                      python -m pip install -e '[dev]'
27                - name: Lint
28                  run: ruff check .
29                - name: Format check
30                  run: ruff format --check .
31                - name: Test
32                  run: pytest
33
34        ocr:
35            # OCR tier needs the Tesseract binary; runs on demand (Linux) to avoid per-PR
36            if: github.event_name == 'workflow_dispatch'
37            runs-on: ubuntu-latest
38            steps:
39                - uses: actions/checkout@v6
40                - uses: actions/setup-python@v6
41                  with:
42                      python-version: "3.13"

```



```
43         cache: pip
44         cache-dependency-path: pyproject.toml
45     - name: Install Tesseract
46       run: sudo apt-get update && sudo apt-get install -y tesseract-ocr
47     - name: Install
48       run: |
49         python -m pip install --upgrade pip
50         python -m pip install -e '[dev]'
51     - name: OCR tests
52       run: pytest -m ocr
53
```

431. user (2026-05-31T09:58:20.693Z)

```
1     from typer.testing import CliRunner
2
3     from muneem.cli import app
4
5     runner = CliRunner()
6
7
8     def test_version_flag():
9         result = runner.invoke(app, ["--version"])
10        assert result.exit_code == 0
11        assert "muneem" in result.stdout
12
13
14    def test_info_lists_paths():
15        result = runner.invoke(app, ["info"])
16        assert result.exit_code == 0
17        assert "config dir" in result.stdout
18        assert "staging" in result.stdout
19
```

432. user (2026-05-31T10:00:21.708Z)

Chapter marked: "C.1 ? wire unlock into ingest". Continue your current work.

433. assistant (2026-05-31T10:00:18.544Z)

Now I have the full picture: `secrets.py` (keyring), CI runs `ruff check . && ruff format --check . && pytest`, and the unlock primitives exist ? they're just not wired into `ingest`. Let me set up the branch and confirm `.gitignore` protects the rules file:

434. user (2026-05-31T10:00:21.156Z)

```
Switched to a new branch 'feat/wire-statement-unlock'
--- .gitignore: password/config coverage ---
8:*.db
11:muneem.db*
23:password_rules.yaml
24:password_rules.yml
25:password_rules.json
26:passwords.txt
90:Thumbs.db
```

435. assistant (2026-05-31T10:00:45.441Z)

`gitignore` already blocks `password\_rules.\*` and `passwords.txt` ? good. Now the building blocks: `extract()` gains a password, `password\_rules` gains a bank-agnostic candidate list, and a new `statement\_unlock` ties the hybrid strategy together:

436. user (2026-05-31T10:00:49.226Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\extract.py has been updated successfully.
(file state is current in your context ? no need to Read it back)

437. assistant (2026-05-31T10:00:59.634Z)

```
"""Password rule loader for statement decryption (hybrid strategy support).
```

```
Prefers the user config dir (via platformdirs, e.g. ~/.config/muneem/password_rules.yaml
on Linux) for real rules. Falls back to legacy CWD "password_rules.yaml" (common during
dev) with a DeprecationWarning. **Never commit real rules or the file itself** ? `.gitignore`
blocks `password_rules.*`.
```

```
See DESIGN.md §8.2, CLAUDE.md security rules, and docs/ingesting-statements.md for the file
format. Entries are either ``<bank>: <password>`` or a top-level ``candidates: [...]`` list.
"""
```

```
from __future__ import annotations
```

```
import warnings
from pathlib import Path
from typing import Any
```

```
import yaml
```

```
from muneem.config import CONFIG_DIR
```

```
def _load_rules() -> dict[str, Any]:
    """Load the rules dict from the config dir (preferred) or legacy CWD file; ``{}`` if absent.
```

```
    Resolution order:
```

1. ``<config\_dir>/password\_rules.yaml`` (recommended location)
2. ``./password\_rules.yaml`` (legacy, emits DeprecationWarning)

```
    """
```

```
    for path, is_legacy in (
        (CONFIG_DIR / "password_rules.yaml", False),
        (Path("password_rules.yaml"), True),
    ):
```

```
        if path.exists():
```

```
            if is_legacy:
```

```
                warnings.warn(
```

```
                    "Loading password_rules from CWD (./password_rules.yaml) is deprecated. "
```

```
                    f"Move it to {CONFIG_DIR / 'password_rules.yaml'} for future-proofing. "
```

```
                    "See docs and CLAUDE.md (never commit real password material).",
```

```
                    DeprecationWarning,
```

```
                    stacklevel=3,
```

```
                )
```

```
                with path.open(encoding="utf-8") as f:
```

```
                    return yaml.safe_load(f) or {}
```

```
    return {}
```

```
def get_password_for_bank(bank_name: str) -> str | None:
```

```
    """Return the password (or derived rule string) for a bank, or ``None``."""
```

```
    val = _load_rules().get(bank_name)
```

```
    return str(val) if val else None
```

```
def candidate_passwords() -> list[str]:
```

```
    """Every configured password to try as an unlock candidate, **bank-agnostic**.
```

The ingest path resolves the password *before* it knows the issuer (it needs the decrypted text to pick a parser), so it tries all configured passwords. Collects each per-bank value plus any entries under a top-level ``candidates`` list; order preserved, de-duplicated. Quote numeric passwords in the YAML so leading zeros survive.

```

"""
rules = _load_rules()
collected: list[str] = []
for key, val in rules.items():
    if key == "candidates" and isinstance(val, list):
        collected.extend(str(v) for v in val if v not in (None, ""))
    elif isinstance(val, str | int | float):
        collected.append(str(val))
seen: set[str] = set()
unique: list[str] = []
for pw in collected:
    if pw and pw not in seen:
        seen.add(pw)
        unique.append(pw)
return unique

```

438. user (2026-05-31T10:00:59.654Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\password_rules.py has been updated successfully. (file state is current in your context ? no need to Read it back)

439. assistant (2026-05-31T10:01:02.858Z)

Now the orchestrator that ties candidates ? keyring cache ? interactive prompt:

440. assistant (2026-05-31T10:01:12.814Z)

""""Resolve a statement PDF's password for the ingest path ? the hybrid strategy, wired.

DESIGN.md §8.2 / ROADMAP C.1. Resolution order when a statement is encrypted:

1. **Candidate dictionary** ? passwords cached in the OS keychain (from earlier successful unlocks) plus those configured in ``password\_rules.yaml`` (see :mod:`muneem.password\_rules`).
2. **Interactive prompt** ? a hidden ``getpass`` prompt (skipped with ``interactive=False``, e.g. automation / CI).

A password that works **via the prompt** is cached in the keychain so the next ingest is friction-free. Nothing here logs a password, writes one to the repo, or echoes it; the cache is the OS keychain (:mod:`muneem.secrets`), never a file.

```

from __future__ import annotations

import getpass
import json
from pathlib import Path

from muneem import secrets
from muneem.password_rules import candidate_passwords
from muneem.unlock import is_encrypted, unlock

```

Keychain entry holding a JSON list of known-good statement passwords (the learned

```
_CACHE_KEY = "statement_unlock_candidates"
```

```

def _cached_candidates() -> list[str]:
    raw = secrets.get_secret(_CACHE_KEY)
    if not raw:
        return []
    try:
        return [p for p in json.loads(raw) if isinstance(p, str) and p]
    except (ValueError, TypeError):
        return []

```

```

def _remember(password: str) -> None:
    """Add a working password to the keychain candidate cache (deduped). Never logged."""
    if not password:
        return
    current = _cached_candidates()
    if password not in current:
        secrets.set_secret(_CACHE_KEY, json.dumps([*current, password]))

def resolve_password(pdf_path: Path | str, *, interactive: bool = True) -> str | None:
    """Return a working password for ``pdf_path``.

    ``""`` means the PDF is not encrypted (no password needed). ``None`` means it is encrypted
    and
    no candidate (or prompt) unlocked it. A prompt-resolved password is cached for next time.
    """
    path = Path(pdf_path)
    if not is_encrypted(path):
        return ""

    candidates = [*_cached_candidates(), *candidate_passwords()]
    prompt_fn = None
    if interactive:

        def prompt_fn() -> str | None: # noqa: E306 ? local prompt; no echo, never logged
            return getpass.getpass(f"Password for {path.name}: ") or None

    result = unlock(path, candidates=candidates, prompt_fn=prompt_fn)
    if not result.ok:
        return None
    if result.method == "prompt" and result.password:
        _remember(result.password)
    return result.password or ""

```

441. user (2026-05-31T10:01:12.827Z)

File created successfully at: C:\Users\avidu\Projects\muneem\src\muneem\statement_unlock.py
(file state is current in your context ? no need to Read it back)

442. assistant (2026-05-31T10:02:19.194Z)

Now wire it into `cli.py` ? restructure `ingest` so bank statements go through a helper that unlocks ? extracts with the password ? selects parser ? stores, and add a `--no-prompt` flag:

443. user (2026-05-31T10:02:26.741Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\cli.py has been updated successfully. (file state is current in your context ? no need to Read it back)

444. user (2026-05-31T10:02:34.462Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\cli.py has been updated successfully. (file state is current in your context ? no need to Read it back)

445. assistant (2026-05-31T10:03:35.168Z)

Now the tests (offline, CI-safe) for the resolver and the CLI wiring, with keyring isolated to an in-memory store:

446. assistant (2026-05-31T10:03:45.883Z)

```

"""Tests for the ingest-path password resolver (muneem.statement_unlock).

```

Synthetic encrypted PDFs (pypdf), no real statements. The OS keychain is replaced with an in-

```

memory
dict so tests never touch the real keyring (and run cleanly in headless CI). No real password.
"""

import json
from pathlib import Path

import pypdf
import pytest

from muneem import statement_unlock

def _pdf(path: Path, password: str | None = None) -> Path:
    writer = pypdf.PdfWriter()
    writer.add_blank_page(width=200, height=200)
    if password:
        writer.encrypt(password)
    with open(path, "wb") as fh:
        writer.write(fh)
    return path

@pytest.fixture(autouse=True)
def fake_keychain(monkeypatch):
    """Replace muneem.secrets with an in-memory store (no real keyring, CI-safe)."""
    store: dict[str, str] = {}
    monkeypatch.setattr(statement_unlock.secrets, "get_secret", store.get)
    monkeypatch.setattr(statement_unlock.secrets, "set_secret", lambda k, v:
store.__setitem__(k, v))
    # default: no configured candidates unless a test overrides
    monkeypatch.setattr(statement_unlock, "candidate_passwords", list)
    return store

def test_plaintext_pdf_needs_no_password(tmp_path):
    assert statement_unlock.resolve_password(_pdf(tmp_path / "plain.pdf")) == ""

def test_resolves_via_configured_candidate(tmp_path, monkeypatch):
    monkeypatch.setattr(statement_unlock, "candidate_passwords", lambda: ["wrong", "secret123"])
    pdf = _pdf(tmp_path / "enc.pdf", password="secret123")
    assert statement_unlock.resolve_password(pdf, interactive=False) == "secret123"

def test_no_prompt_failure_returns_none(tmp_path, monkeypatch):
    monkeypatch.setattr(statement_unlock, "candidate_passwords", lambda: ["wrong"])
    pdf = _pdf(tmp_path / "enc.pdf", password="secret123")
    assert statement_unlock.resolve_password(pdf, interactive=False) is None

def test_prompt_resolves_then_caches_for_next_time(tmp_path, monkeypatch, fake_keychain):
    pdf = _pdf(tmp_path / "enc.pdf", password="secret123")
    monkeypatch.setattr(statement_unlock.getpass, "getpass", lambda prompt="": "secret123")
    assert statement_unlock.resolve_password(pdf, interactive=True) == "secret123"
    # the winning password is cached in the (fake) keychain
    assert "secret123" in json.loads(fake_keychain[statement_unlock._CACHE_KEY])

    # a *new* file with the same password now unlocks from the cache ? no prompt
    def _no_prompt(prompt=""):
        raise AssertionError("should not prompt once the password is cached")

    monkeypatch.setattr(statement_unlock.getpass, "getpass", _no_prompt)
    pdf2 = _pdf(tmp_path / "enc2.pdf", password="secret123")
    assert statement_unlock.resolve_password(pdf2, interactive=True) == "secret123"

```

447. user (2026-05-31T10:03:45.898Z)

File created successfully at: C:\Users\avidu\Projects\muneeem\tests\test_statement_unlock.py
(file state is current in your context ? no need to Read it back)

448. assistant (2026-05-31T10:03:49.623Z)

Now the CLI wiring tests:

449. user (2026-05-31T10:03:59.832Z)

The file C:\Users\avidu\Projects\muneeem\tests\test_cli.py has been updated successfully. (file state is current in your context ? no need to Read it back)

450. assistant (2026-05-31T10:04:05.438Z)

Let me run lint + the new tests before writing the doc:

451. user (2026-05-31T10:04:09.228Z)

3 files reformatted, 3 files left unchanged
All checks passed!
--- new tests + full offline suite ---
..... [100%]
8 passed in 0.72s
[This command modified 3 files you've previously read: tests\test_statement_unlock.py,
tests\test_cli.py, src\muneeem\cli.py. Call Read before editing.]

452. assistant (2026-05-31T10:04:13.906Z)

Lint clean, 8 new tests pass. Now the full offline suite to confirm nothing regressed:

453. user (2026-05-31T10:04:21.824Z)

..... [100%]
===== XPASSES =====
===== short test summary info =====
XPASS tests/test_ocr.py::test_swiggy_image_code_recovered_fuzzily - recovery of the stylized
boxed code is OCR-engine/version dependent ? succeeds on Tesseract 5.4, returns nothing on the
apt build; the planned fix is a purpose-trained model (docs/DEPENDENCIES.md). strict=False so it
never fails the build.
120 passed, 13 deselected, 1 xpassed in 6.20s

454. assistant (2026-05-31T10:04:59.069Z)

120 passed. Now let me verify the **full CLI path end-to-end on real statements** ? password in-
process only (no disk, no keyring), output is counts/status only:

455. user (2026-05-31T10:05:07.974Z)

The PDF <_io.BufferedReader name='testdata\\hdfc-statements\\Avi_Dullu_28022026_183854522.pdf'>
contains a metadata field indicating that it should not allow text extraction. Ignoring this
field and proceeding. Use the check_extractable if you want to raise an error in this case
HDFC:
 ingested Avi_Dullu_28022026_183854522.pdf (doc 1); parsed 37 transaction(s) via HDFCParser
[reconciled]
AXIS (Aug 2024):
 ingested AXIS BANK Statement for August 2024.pdf (doc 1); parsed 6 transaction(s) via
AxisParser [reconciled]
AU:
 ingested Statement_2024MTH07_XXXXX859_146.pdf (doc 1); parsed 17 transaction(s) via AUParser
[reconciled]

456. assistant (2026-05-31T10:05:25.183Z)

****End-to-end on real statements works from the CLI**** ? HDFC 37 txns, Axis(Aug) 6, AU 17, all ``[reconciled]``, password resolved in-process. Now the setup doc:

457. assistant (2026-05-31T10:05:49.365Z)

Ingesting bank statements (with password unlock)

``muneem ingest`` reads a bank-statement PDF, ****unlocks it if it's password-protected****, parses it into normalized transactions, and stores only a parse that ****reconciles**** (the running balance walks, or the totals match the printed opening/closing). Supported issuers today: ****HDFC, Axis, AU Bank****.

```
```bash
muneem ingest /path/to/statement.pdf --doc-type bank_statement
```
```

If the PDF is encrypted, muneem resolves the password with a ****hybrid strategy**** (no password is ever written to the repo or logged):

1. ****Cached candidates**** ? passwords from earlier successful unlocks, stored in your ****OS keychain****.
2. ****Configured candidates**** ? entries in ``password_rules.yaml`` (see below).
3. ****Interactive prompt**** ? a hidden prompt (``getpass``). A password entered here that works is ****cached in the keychain****, so the next ingest of any statement with the same password is friction-free.

On success you'll see, e.g.:

```
```
ingested statement.pdf (doc 1); parsed 37 transaction(s) via HDFCParser [reconciled]
```
```

``[reconciled]`` means the parse is trusted; ``[unreconciled]`` means it couldn't be verified, so ****nothing was stored**** (muneem never persists mislabeled rows). Read parsed rows back with ``muneem txns list hdfc`` (or ``axis`` / ``au``).

Enable unattended unlocking (recommended)

For batch/backfill or ``--no-prompt`` runs, put your statement passwords in a ****gitignored**** rules file so you're never prompted.

1. Find your config directory:

```
```bash
muneem info # ? "config dir : /home/you/.config/muneem" (varies by OS)
```
```

2. Create ``<config dir>/password_rules.yaml``. ****Never commit it**** ? ``gitignore`` already blocks ``password_rules.*``, and it lives in your OS config dir (outside the repo) anyway.

```
```yaml
<config-dir>/password_rules.yaml ? NEVER commit this file.
Quote every password (keeps leading zeros; avoids YAML parsing "0106" as a number).

Per-issuer passwords:
hdfc: "<your-hdfc-pdf-password>"
axis: "<your-axis-pdf-password>"
aubank: "<your-au-pdf-password>"

?or a bank-agnostic list of passwords to try (handy when the issuer label varies):
```

```

candidates:
- "<password-a>"
- "<password-b>"
...

```

muneem tries **all** of these (it has to pick the password before it can read the PDF to know the issuer), so either form works. A legacy `./password_rules.yaml` in the current directory is still read but warns ? prefer the config dir.

### 3. Ingest without prompts:

```

```bash
muneem ingest statement.pdf --doc-type bank_statement --no-prompt
```

```

`--no-prompt` uses only cached + configured candidates and **never** opens an interactive prompt (so it's safe in cron/CI). Without it, an unresolved password falls back to the hidden prompt.

## Security

- **Nothing sensitive is committed or logged.** `password_rules.*` is gitignored; resolved passwords live only in memory and the **OS keychain** (Windows Credential Manager / macOS Keychain / Linux Secret Service), via `muneem.secrets`.
- **No decrypted copy is written to disk** ? the password is handed straight to the text extractor and the parser in memory.
- Transaction parses are **reconciliation-gated**: an unverifiable parse stores nothing rather than risk mislabeled rows.

## Troubleshooting

```

Message	Meaning / fix
`Could not unlock <file>: no working password`	None of the cached/configured candidates worked. Run without `--no-prompt` to type it, or add it to `password_rules.yaml`.
`No parser recognizes <file> as a supported bank statement`	The issuer isn't supported yet (today: HDFC, Axis, AU Bank), or the text didn't match.
`? [unreconciled]` (0 transactions)	The parser couldn't verify the parse (e.g. an Axis vintage `extract_tables` mangles); nothing was stored. See [parser-architecture.md](./parser-architecture.md).

```

## Headless Linux note

Keychain storage needs a backend. Use a desktop keyring (GNOME Keyring / KWallet), or `pip install keyrings.alt` for a file-based backend. Until then, `password_rules.yaml` + `--no-prompt` works without any keychain (you just won't get prompt-caching).

### 458. user (2026-05-31T10:05:49.384Z)

File created successfully at: C:\Users\avidu\Projects\muneem\docs\ingesting-statements.md (file state is current in your context ? no need to Read it back)

### 459. assistant (2026-05-31T10:07:02.516Z)

Now reflect **C.1 done** in CURRENT\_STATE and ROADMAP:



#### 460. user (2026-05-31T10:07:05.811Z)

The file C:\Users\avidu\Projects\muneem\docs\CURRENT\_STATE.md has been updated successfully.  
(file state is current in your context ? no need to Read it back)

#### 461. user (2026-05-31T10:07:08.300Z)

The file C:\Users\avidu\Projects\muneem\docs\CURRENT\_STATE.md has been updated successfully.  
(file state is current in your context ? no need to Read it back)

#### 462. user (2026-05-31T10:07:13.895Z)

The file C:\Users\avidu\Projects\muneem\docs\CURRENT\_STATE.md has been updated successfully.  
(file state is current in your context ? no need to Read it back)

#### 463. user (2026-05-31T10:07:20.456Z)

<tool\_use\_error>File has been modified since read, either by the user or by a linter. Read it again before attempting to write it.</tool\_use\_error>

#### 464. user (2026-05-31T10:07:22.797Z)

<tool\_use\_error>File has been modified since read, either by the user or by a linter. Read it again before attempting to write it.</tool\_use\_error>

#### 465. user (2026-05-31T10:07:24.093Z)

<tool\_use\_error>File has been modified since read, either by the user or by a linter. Read it again before attempting to write it.</tool\_use\_error>

#### 466. assistant (2026-05-31T10:07:36.502Z)

ROADMAP's file-state went stale (the branch checkout). Let me re-read the regions and re-apply:

#### 467. user (2026-05-31T10:07:37.456Z)

```
13 > **Progress snapshot (2026-06):** Milestone 0 (scaffolding / offline CI / config +
keyring) ? and
14 > **Track A** (promo spine: `ingest` ? SQLite ? `offers list`, green regression) ? are
done.
15 > **Track B:** Gmail OAuth2 + search + attachment download ?; the `muneem fetch` CLI
(B.4) is not
16 > built yet. **Parsers:** the concept-core rework (parser-architecture.md **steps 1?3**)
landed ?
17 > HDFC 4/4 real, AU Bank 4/4, Axis 2/4, ICICI removed. **Not yet wired:** password
unlock into the
18 > main `ingest` path (C.1) and the unified `transactions`/`holdings` schema (C.3 / step
5). The
19 > checkbox lists below predate the concept-core pivot ? treat
[`CURRENT_STATE.md`] (./CURRENT_STATE.md)
20 > as the live status.
21
```

#### 468. user (2026-05-31T10:07:38.002Z)

```
155 ### Milestone D ? Harden & evaluate · the pre-generalize checkpoint
156
157 - [] **D.1 (M)** Redacted structured logging (assert no statement text / account
numbers leak); idempotent, resumable re-runs across all stages.
158 - [] **D.2 (S)** Encryption-at-rest decision (DESIGN § 5.4): adopt SQLCipher now or
schedule it. (Parked item ? bring evidence, then decide.)
159 - [] **D.3 (M)** **Evaluation writeup:** how well did the deterministic HDFC parser
hold across statement vintages? Where does it break? Which categories will need the local-LLM
```

fallback? This is the evidence DESIGN § 7 asks for \*before\* generalizing. Capture in  
`docs/parser-notes-hdfc.md`.  
160 - \*\*Exit:\*\* the slice is robust and re-runnable; a written go/no-go on generalization  
with real evidence.  
161  
162 ---

#### 469. user (2026-05-31T10:07:45.402Z)

146 ### Milestone C ? HDFC vertical slice · IN PROGRESS (HDFC/Axis/AU parsers ?; unlock-  
wiring C.1 + unified schema C.3 pending)  
147  
148 > A fetched HDFC statement gets unlocked and parsed into real transactions. This is the  
moment every stage runs once on real data the user cares about.  
149  
150 - [ ] \*\*C.1 (M)\*\* Unlocking (Stage 2) with the \*\*hybrid\*\* strategy: candidate dictionary  
derived from user attributes (e.g. `first4(name).upper() + DDMM(dob)` ) ? per-sender rule  
template ? interactive prompt. Cache the winning method per sender in the keychain. \*\*No  
password material in repo or logs.\*\* Decrypt with pikepdf; advance `documents.status` to  
`unlocked`.  
151 - [ ] \*\*C.2 (L)\*\* HDFC parser (Stage 3, the hard 80%): born-digital text-layer +  
pdfplumber table recovery ? normalized transactions (txn\_date, value\_date, amount, direction,  
narration, running balance). \*\*Synthetic HDFC-layout fixtures\*\* committed for CI; real  
statements stay gitignored locally.  
152 - [ ] \*\*C.3 (M)\*\* Schema migration v2: `accounts` (issuer, account\_type, last4, holder)  
+ `transactions` (account\_id, document\_id, txn fields, category, merchant\_id-nullable). Populate  
from the parser. `muneem txns list --last-month` prints real parsed transactions.  
153 - \*\*Exit:\*\* \*\*DESIGN.md § 7 vertical slice complete\*\* ? Gmail ? HDFC statement ?  
unlocked ? parsed ? SQLite ? CLI lists last month's transactions, on real data, locally.

#### 470. assistant (2026-05-31T10:07:51.326Z)

Now re-apply the ROADMAP edits (and tick C.2 too ? the HDFC parser is done):

#### 471. user (2026-05-31T10:07:54.501Z)

The file C:\Users\avidu\Projects\muneem\docs\ROADMAP.md has been updated successfully. (file  
state is current in your context ? no need to Read it back)

#### 472. user (2026-05-31T10:07:56.361Z)

The file C:\Users\avidu\Projects\muneem\docs\ROADMAP.md has been updated successfully. (file  
state is current in your context ? no need to Read it back)

#### 473. user (2026-05-31T10:08:00.589Z)

The file C:\Users\avidu\Projects\muneem\docs\ROADMAP.md has been updated successfully. (file  
state is current in your context ? no need to Read it back)

#### 474. user (2026-05-31T10:08:02.089Z)

The file C:\Users\avidu\Projects\muneem\docs\ROADMAP.md has been updated successfully. (file  
state is current in your context ? no need to Read it back)

#### 475. assistant (2026-05-31T10:08:08.894Z)

A quick README pointer to the new guide for discoverability ? re-reading first since the branch  
checkout staled it:

#### 476. user (2026-05-31T10:08:09.519Z)

17 - Forgotten deals and coupons sitting unread in your inbox  
18

19        ## Status  
20

**477. user (2026-05-31T10:08:16.053Z)**

19        ## Status  
20  
21        \*\*Early implementation.\*\* The design and plan are settled ? see  
[ DESIGN.md ](./DESIGN.md) (architecture + threat model), [ `docs/ROADMAP.md` ](./docs/ROADMAP.md)  
(milestones), and [ `docs/TESTING.md` ](./docs/TESTING.md) (test strategy). The Python project  
skeleton, CLI, and offline CI are in place; build proceeds by the roadmap's milestones. Prior  
art: [ `docs/prior-art-email-mining.md` ](./docs/prior-art-email-mining.md).

**478. user (2026-05-31T10:08:23.240Z)**

The file C:\Users\avidu\Projects\muneem\README.md has been updated successfully. (file state is  
current in your context ? no need to Read it back)

**479. assistant (2026-05-31T10:08:29.733Z)**

All docs updated. Now let me run the \*\*exact CI trio locally\*\* (`ruff check .`, `ruff format  
--check .`, `pytest`) to guarantee green CI:

**480. user (2026-05-31T10:08:34.510Z)**

```
=== ruff check . ===
All checks passed!
=== ruff format --check . ===
55 files already formatted
=== pytest (CI default) ===
..... [60%]
..... [100%]
119 passed, 15 deselected in 1.83s
```

**481. assistant (2026-05-31T10:08:53.447Z)**

**\*\*CI trio green locally:\*\*** `ruff check` clean, all 55 files formatted, `pytest` 119 passed.  
Shipping C.1:

**482. user (2026-05-31T10:08:59.731Z)**

```
Exit code 1
M README.md
M docs/CURRENT_STATE.md
M docs/ROADMAP.md
M src/muneem/cli.py
M src/muneem/extract.py
M src/muneem/password_rules.py
M tests/test_cli.py
?? docs/ingesting-statements.md
?? src/muneem/statement_unlock.py
?? tests/test_statement_unlock.py
muneem pre-commit: refusing to commit sensitive file(s):
 x src/muneem/password_rules.py
 password rules
```

These must never reach git (CLAUDE.md security rules; see .gitignore).  
If this is a deliberate, reviewed exception: git commit --no-verify

**483. assistant (2026-05-31T10:09:27.951Z)**

The pre-commit security guard is flagging **\*\*`src/muneem/password\_rules.py`\*\*** ? but that's the  
legitimate *\*loader module\** (code), not a secrets file. The hook is matching the filename too  
broadly. Let me inspect the hook and its test before deciding how to handle this correctly (not

just bypass it):

#### 484. user (2026-05-31T10:09:28.903Z)

```
1 """Tests for the pre-commit sensitive-file guard (.github/precommit_check.py).
2
3 Tests the pure rule function + the exit-code path (via PRECOMMIT_TEST_FILES), so
4 no shell, git mutation, or real staging is needed ? portable across OSes.
5 """
6
7 from __future__ import annotations
8
9 import importlib.util
10 from pathlib import Path
11
12 import pytest
13
14 REPO_ROOT = Path(__file__).resolve().parents[1]
15 CHECK = REPO_ROOT / ".github" / "precommit_check.py"
16
17
18 def _load_checker():
19 spec = importlib.util.spec_from_file_location("precommit_check", CHECK)
20 assert spec and spec.loader
21 mod = importlib.util.module_from_spec(spec)
22 spec.loader.exec_module(mod)
23 return mod
24
25
26 pc = _load_checker()
27
28
29 @pytest.mark.parametrize(
30 "path",
31 [
32 "testdata/hdfc-statements/Avi_Dullu_28022026.pdf",
33 "testdata/axis-statements/AXIS BANK Statement for October 2025.pdf", # spaces
34 "testdata/aubank-statements/logs_71559053525.zip", # non-PDF under testdata/
35 "testdata/credentials/client_secret.json",
36 "some/dir/Statement.PDF", # PDF anywhere, any case
37 "muneem.db",
38 "data/ledger.sqlite3",
39 "credentials.json",
40 "src/token.json",
41 "oauth_token_cache",
42 "password_rules.yaml",
43 ".env",
44 ".env.local",
45 "secrets/server.pem",
46 "id_rsa.key",
47],
48)
49 def test_sensitive_paths_are_blocked(path):
50 assert pc.is_sensitive(path) is not None, f"should block: {path}"
51
52
53 @pytest.mark.parametrize(
54 "path",
55 [
56 "testdata/promo-emails/swiggy.pdf", # the allowed corpus
57 "testdata/synthetic/fixture.pdf",
58 "src/muneem/hdfc.py",
59 "tests/test_hdfc.py",
60 "README.md",
61 ".env.example",
```

```

62 "docs/DESIGN.md",
63 "pyproject.toml",
64 "Makefile",
65],
66)
67 def test_allowed_paths_pass(path):
68 assert pc.is_sensitive(path) is None, f"should allow: {path}"
69
70
71 def test_main_blocks_when_sensitive_staged(monkeypatch):
72 monkeypatch.setenv("PRECOMMIT_TEST_FILES", "src/muneem/hdfc.py\ntestdata/hdfc-
statements/x.pdf")
73 assert pc.main() == 1
74
75
76 def test_main_passes_when_clean(monkeypatch):
77 monkeypatch.setenv("PRECOMMIT_TEST_FILES", "src/muneem/hdfc.py\nREADME.md")
78 assert pc.main() == 0
79

```

#### 485. user (2026-05-31T10:09:29.403Z)

```

.git\hooks\applypatch-msg.sample
.git\hooks\commit-msg.sample
.git\hooks\fsmonitor-watchman.sample
.git\hooks\post-update.sample
.git\hooks\pre-applypatch.sample
.git\hooks\pre-commit.sample
.git\hooks\pre-merge-commit.sample
.git\hooks\pre-push.sample
.git\hooks\pre-rebase.sample
.git\hooks\pre-receive.sample
.git\hooks\prepare-commit-msg.sample
.git\hooks\push-to-checkout.sample
.git\hooks\sendemail-validate.sample
.git\hooks\update.sample
tools\gmail_auth.py
tools\imap_auth.py
tools\outlook_auth.py
.githooks\pre-commit
.githooks\precommit_check.py
.githooks__pycache__\precommit_check.cpython-313.pyc

```

#### 486. assistant (2026-05-31T10:09:43.790Z)

The hook is ``.githubhooks/precommit_check.py`` with an ``.is_sensitive()`` rule.  
``.src/muneem/password_rules.py`` (a tracked code module since PR #11, before the hook) is a  
**\*\*false positive\*\*** ? the pattern matches the ``.password_rules`` substring including the ``.py``  
loader. The right fix is to narrow the rule (still block ``.password_rules.{yaml,yml,json}``, allow  
the ``.py`` module) + add a regression test ? not bypass the guard. Let me read it:

#### 487. user (2026-05-31T10:09:44.063Z)

```

1 #!/usr/bin/env python3
2 """muneem sensitive-file guard (invoked by the pre-commit hook).
3
4 Refuses to *stage* files that policy says must never be committed: real
5 statements (any PDF, or anything under ``testdata/`` outside the allowed
6 corpora), OAuth secrets, password rules, local databases, key material, and
7 ``.env`` files. This is the PRE-commit backstop to ``.gitignore`` ? it stops the
8 ``git add -f`` / ``git add -A`` mistake before it becomes a commit (the very
9 thing that caused an earlier statement leak). The post-commit complement is
10 ``tests/test_repo_hygiene.py``.
11

```

```

12 Exit 0 when nothing sensitive is staged, 1 otherwise. A deliberate, reviewed
13 exception can be committed with ``git commit --no-verify``.
14 """
15
16 from __future__ import annotations
17
18 import fnmatch
19 import os
20 import subprocess
21 import sys
22
23 # Only these subtrees of testdata/ are non-sensitive and may be committed.
24 ALLOWED_PREFIXES = ("testdata/promo-emails/", "testdata/synthetic/")
25 SECRET_NAMES = {"credentials.json", "token.json", "token.pickle", "passwords.txt"}
26
27
28 def is_sensitive(path: str) -> str | None:
29 """Return a human-readable reason if ``path`` must not be committed, else None."""
30 p = path.replace("\\", "/").strip()
31 if not p or p.startswith(ALLOWED_PREFIXES):
32 return None
33 name = p.rsplit("/", 1)[-1]
34 low = p.lower()
35 nlow = name.lower()
36 if p.startswith("testdata/"):
37 return "under testdata/ ? sensitive by default (only promo-emails/ and
synthetic/ allowed)"
38 if low.endswith(".pdf"):
39 return "PDF outside the allowed corpora (real statements must stay local)"
40 if low.endswith((".db", ".sqlite", ".sqlite3")):
41 return "local database (may contain parsed statement data)"
42 if low.endswith((".pem", ".key")):
43 return "key material"
44 if nlow in SECRET_NAMES:
45 return "OAuth / credential secret"
46 if nlow.startswith("oauth_token"):
47 return "OAuth token"
48 if fnmatch.fnmatch(nlow, "password_rules.*"):
49 return "password rules"
50 if (nlow == ".env" or nlow.startswith(".env.")) and nlow != ".env.example":
51 return ".env file (may contain secrets)"
52 return None
53
54
55 def _staged_paths() -> list[str]:
56 # Tests inject a newline-separated list via PRECOMMIT_TEST_FILES (no git needed).
57 override = os.environ.get("PRECOMMIT_TEST_FILES")
58 if override is not None:
59 return [ln for ln in override.replace("\r\n", "\n").split("\n") if ln.strip()]
60 # -z keeps paths with spaces/unicode intact (the real statements have spaces).
61 out = subprocess.run(
62 ["git", "diff", "--cached", "--name-only", "-z", "--diff-filter=ACM"],
63 capture_output=True,
64 text=True,
65 check=True,
66).stdout
67 return [p for p in out.split("\0") if p.strip()]
68
69
70 def main() -> int:
71 offenders = [(p, reason) for p in _staged_paths() if (reason := is_sensitive(p))]
72 if not offenders:
73 return 0
74 print("muneem pre-commit: refusing to commit sensitive file(s):", file=sys.stderr)
75 for p, reason in offenders:

```

```

76 print(f" x {p}\n {reason}", file=sys.stderr)
77 print(
78 "\nThese must never reach git (CLAUDE.md security rules; see .gitignore).\n"
79 "If this is a deliberate, reviewed exception: git commit --no-verify",
80 file=sys.stderr,
81)
82 return 1
83
84
85 if __name__ == "__main__":
86 raise SystemExit(main())
87

```

#### 488. assistant (2026-05-31T10:10:10.589Z)

Found it: line 48 `fnmatch(nlow, "password\_rules.\*")` matches the `.py` loader too. The intent is to block the rules `**data** file (.yaml/.yml/.json)`, not the code module. Let me narrow it and add a regression test:

#### 489. user (2026-05-31T10:10:13.091Z)

The file C:\Users\avidu\Projects\muneem\githubhooks\precommit\_check.py has been updated successfully. (file state is current in your context ? no need to Read it back)

#### 490. assistant (2026-05-31T10:11:27.637Z)

API Error: The socket connection was closed unexpectedly. For more information, pass `verbose: true` in the second argument to fetch()

#### 491. user (2026-05-31T10:11:31.651Z)

The file C:\Users\avidu\Projects\muneem\tests\test\_precommit\_hook.py has been updated successfully. (file state is current in your context ? no need to Read it back)

#### 492. user (2026-05-31T10:11:36.031Z)

The file C:\Users\avidu\Projects\muneem\tests\test\_precommit\_hook.py has been updated successfully. (file state is current in your context ? no need to Read it back)

#### 493. assistant (2026-05-31T10:11:43.804Z)

Now verify the hook fix + full CI trio, then re-commit:

#### 494. user (2026-05-31T10:11:48.993Z)

```

2 files left unchanged
=== ruff check . ===
All checks passed!
=== ruff format --check . ===
55 files already formatted
=== pytest ===
..... [59%]
..... [100%]
122 passed, 15 deselected in 1.91s

```

#### 495. assistant (2026-05-31T10:11:54.118Z)

122 passed (the hook fix's regression cases included), CI trio green. Now re-commit ? the guard will pass on the `.py` loader while still blocking the secret file:

#### 496. user (2026-05-31T10:12:03.015Z)

[feat/wire-statement-unlock e63ed44] feat(cli): wire hybrid password-unlock into `muneem ingest`

```
(C.1) + setup doc
12 files changed, 423 insertions(+), 64 deletions(-)
create mode 100644 docs/ingesting-statements.md
create mode 100644 src/muneem/statement_unlock.py
create mode 100644 tests/test_statement_unlock.py
```

#### 497. assistant (2026-05-31T10:12:16.149Z)

Committed (guard passed on the `.py`, still blocks the secret). Now push, open the PR, and **\*\*watch CI\*\*** before merging:

#### 498. user (2026-05-31T10:12:29.343Z)

```
To https://github.com/avidullu/muneem.git
* [new branch] feat/wire-statement-unlock -> feat/wire-statement-unlock
https://github.com/avidullu/muneem/pull/28
```

#### 499. assistant (2026-05-31T10:12:37.630Z)

PR [#28](https://github.com/avidullu/muneem/pull/28) up. Let me watch CI (3 Python versions) to confirm green before merging:

#### 500. user (2026-05-31T10:13:20.759Z)

```
ocr skipping 0
https://github.com/avidullu/muneem/actions/runs/26709798188/job/78717766505
Refreshing checks status every 20 seconds. Press Ctrl+C to quit.
```

```
test (3.11) pending 0
https://github.com/avidullu/muneem/actions/runs/26709798188/job/78717766267
test (3.12) pending 0
https://github.com/avidullu/muneem/actions/runs/26709798188/job/78717766264
test (3.13) pending 0
https://github.com/avidullu/muneem/actions/runs/26709798188/job/78717766265
ocr skipping 0
https://github.com/avidullu/muneem/actions/runs/26709798188/job/78717766505
test (3.11) pass 27s
https://github.com/avidullu/muneem/actions/runs/26709798188/job/78717766267
test (3.12) pass 24s
https://github.com/avidullu/muneem/actions/runs/26709798188/job/78717766264
test (3.13) pass 26s
https://github.com/avidullu/muneem/actions/runs/26709798188/job/78717766265
ocr skipping 0
https://github.com/avidullu/muneem/actions/runs/26709798188/job/78717766505
test (3.11) pass 27s
https://github.com/avidullu/muneem/actions/runs/26709798188/job/78717766267
test (3.12) pass 24s
https://github.com/avidullu/muneem/actions/runs/26709798188/job/78717766264
test (3.13) pass 26s
https://github.com/avidullu/muneem/actions/runs/26709798188/job/78717766265
ocr skipping 0
https://github.com/avidullu/muneem/actions/runs/26709798188/job/78717766505
--- exit: 0 ---
```

#### 501. assistant (2026-05-31T10:13:26.206Z)

CI green on all three Python versions. Squash-merging and syncing:

#### 502. user (2026-05-31T10:13:34.474Z)

```
create mode 100644 src/muneem/statement_unlock.py
create mode 100644 tests/test_statement_unlock.py
Already on 'main'
- [deleted] (none) -> origin/feat/wire-statement-unlock
```



Already up to date.

d4f071e feat(cli): wire hybrid password-unlock into `muneem ingest` (C.1) + setup doc (#28)

### 503. assistant (2026-05-31T10:13:43.173Z)

Merged as [#28](https://github.com/avidullu/muneem/pull/28). The push to `main` triggers CI again ? let me confirm that run is green too, since "CI green after done" was the bar:

### 504. user (2026-05-31T10:14:20.317Z)

```
watching main run 26709819277
? test (3.13) in 21s (ID 78717824936)
 ? Set up job
 ? Run actions/checkout@v6
 ? Run actions/setup-python@v6
 ? Install
 ? Lint
 ? Format check
 ? Test
 ? Post Run actions/setup-python@v6
 ? Post Run actions/checkout@v6
 ? Complete job
- ocr in 0s (ID 78717825008)
--- exit: 0 ---
```

### 505. assistant (2026-05-31T10:14:25.848Z)

CI green on `main` too (Lint ?, Format check ?, Test ?; run exit 0). Final clean-slate confirmation:

### 506. user (2026-05-31T10:14:29.730Z)

```
branch: main
sync: ## main...origin/main
tree: 0 uncommitted change(s)
open PRs: 0
failed to parse jq expression (line 1, column 19)
 [0].conclusion + \" @\" + ([0].headSha[0:7])
 ^ unexpected token "\"\"
latest main CI:
d4f071e feat(cli): wire hybrid password-unlock into `muneem ingest` (C.1) + setup doc (#28)
```

### 507. assistant (2026-05-31T10:14:36.513Z)

Clean slate confirmed (main, even with origin, 0 uncommitted, 0 open PRs). My jq quoting broke ? let me re-confirm the CI conclusion cleanly:

### 508. user (2026-05-31T10:14:38.356Z)

```
[{"conclusion":"success","displayTitle":"feat(cli): wire hybrid password-unlock into `muneem ingest` (C.1) + s?","headSha":"d4f071eb830bb2e0b3f7fd468ffde50036baf3a","status":"completed"}]
```

### 509. assistant (2026-05-31T10:14:57.125Z)

? **Done**, CI green, clean slate. **`muneem ingest`** now unlocks and parses encrypted statements from the CLI.

## What shipped ? [muneem#28](https://github.com/avidullu/muneem/pull/28) (C.1)

- **Wired** the hybrid unlock into ``muneem ingest --doc-type bank_statement``: keychain-cached + ``password_rules.yaml`` candidates ? hidden prompt, with a prompt-resolved password **cached** in the OS keychain for next time. The password is resolved before extraction, passed in-memory to the parser, and **never** logged or written to the repo; no decrypted copy touches disk.
- **no-prompt** flag for unattended/cron runs; output shows the

```
`[reconciled]`/[unreconciled]` verdict.
- Setup doc: docs/ingesting-statements.md ? how to use it,
how to enable unattended unlocking (the `password_rules.yaml` format + location), security
notes, and troubleshooting. Linked from README + CURRENT_STATE.
- Bonus fix: the pre-commit guard was false-positiving on `src/muneem/password_rules.py`
(the loader `module`) ? narrowed it to block only the secret `password_rules.{yaml,yml,json}`,
regression-tested.
```

Verified **end-to-end** on your real statements (password in-process; counts/status only): HDFC  
37 / Axis Aug'24 6 / AU 17 transactions, all `[reconciled]`.

## State at the pause

```
- CI: `success` on `main` (and the PR) across Python 3.11/3.12/3.13 ? `ruff check` + `ruff
format --check` + `pytest` (**122 tests**).
- Repo + local in sync: on `main`, even with `origin/main`, **0 uncommitted changes, 0 open
PRs**. Tip: `d4f071e`.
- Docs reflect reality: ROADMAP C.1 + C.2 ticked, Milestone C status updated.
```

**Resume queue** (unchanged, in CURRENT\_STATE §5): recover the 2 Axis vintages (May/June) ? Step  
4 (L3 local-vision, pending VLM research) ? Step 5 (unified `transactions`/`holdings` + the  
instrument family). Enjoy the break.

### 510. user (2026-05-31T10:14:59.269Z)

Can we add some github action tests which avoid regression when working from other branches of  
this repo using the hdfc, aubank statements that we have? The repo will always be private so  
privacy can be a bit relaxed.

### 511. assistant (2026-05-31T10:16:47.967Z)

Happy to set this up ? running the real HDFC/AU statements in CI is the strongest regression  
guard. But this is the one decision in this project I don't want to make for you, because it's  
**hard to reverse**: the real statements are exactly what the local-first design + the repo's  
own guards (`.gitignore`, the pre-commit hook, `test\_repo\_hygiene`) exist to keep off any cloud.

The thing "private repo" doesn't soften: **git history is permanent and lives on GitHub's  
servers**. Once a real (even encrypted) statement is committed, it's on GitHub's cloud forever ?  
recoverable if the password ever leaks, if the repo is later made public/forked, or if a  
collaborator/token is compromised. That's different from "only my machine has it."

The good news: there's a way to get **real-statement CI** on every branch without any of that ?  
so let me get your call on the approach.

### 512. user (2026-05-31T10:18:15.104Z)

Your questions have been answered: "How should the real HDFC/AU statements drive branch CI?  
(They're your most sensitive asset, so the trade-offs differ a lot.)"="A smart mixture of 1. and  
2. seems the best way to go ahead.". You can now continue with these answers in mind.

### 513. assistant (2026-05-31T10:22:35.963Z)

A smart mix it is: **synthetic full-pipeline fixtures in normal cloud CI** (every branch, no  
secrets) + a **self-hosted runner** that exercises your **real** HDFC/AU statements (which never  
leave your machine). Let me track this and build it.

### 514. user (2026-05-31T10:22:38.794Z)

Task #9 created successfully: Minimal stdlib PDF generator for synthetic fixtures

### 515. user (2026-05-31T10:22:41.146Z)

Task #10 created successfully: Synthetic full-pipeline PDF tests (HDFC + savings) for cloud CI

**516. user (2026-05-31T10:22:43.968Z)**

Task #11 created successfully: Self-hosted runner workflow for real statements + unify password sourcing

**517. user (2026-05-31T10:22:46.032Z)**

Task #12 created successfully: Regression-CI setup doc + green CI + merge

**518. user (2026-05-31T10:22:51.210Z)**

Updated task #9 status

**519. user (2026-05-31T10:22:50.972Z)**

Switched to a new branch 'feat/regression-ci'

**520. user (2026-05-31T10:23:07.440Z)**

[Request interrupted by user]

**521. user (2026-05-31T10:23:53.053Z)**

Can you write a note about this in next steps doc to track somewhere and not go ahead with code for now. Make sure to include enough details to ground our decisions

**522. assistant (2026-05-31T10:25:37.329Z)**

Got it ? I'll stop the build and instead capture this as a grounded, tracked plan so we can resume cleanly. Let me clean up the in-session code tasks, switch to a docs branch, and read the exact spots to anchor the note.

**523. user (2026-05-31T10:25:37.755Z)**

Updated task #9 deleted

**524. user (2026-05-31T10:25:38.721Z)**

Updated task #10 deleted

**525. user (2026-05-31T10:25:39.166Z)**

Updated task #11 deleted

**526. user (2026-05-31T10:25:39.689Z)**

Updated task #12 deleted

**527. user (2026-05-31T10:25:40.816Z)**

Switched to branch 'main'  
Deleted branch feat/regression-ci (was d4f071e).  
Switched to a new branch 'docs/track-regression-ci'

**528. user (2026-05-31T10:25:48.893Z)**

178:## 7. Cross-cutting workstreams (run throughout, not a phase)  
187:## 8. Risks & mitigations

**529. assistant (2026-05-31T10:25:48.226Z)**

Now read the two anchor spots fresh (CURRENT\_STATE \$5 and ROADMAP's cross-cutting section):

### 530. user (2026-05-31T10:25:48.504Z)

```
48
49 ## 5. Next Steps
50 Concept-core **steps 1?3 are done**, and **password unlock is now wired into `muneem
ingest`** (C.1 ? hybrid candidates ? prompt, keychain-cached; see [ingesting-
statements.md](./ingesting-statements.md)). Real-statement precision: **HDFC 4/4, AU 4/4, Axis
2/4** (the 2 Axis misses store nothing ? oracle-gated). In rough order from here:
51
52 * **Recover the 2 Axis vintages** (May/June 2025): `extract_tables` mangles them into an
incomplete ledger; needs a stronger coordinate clusterer. Isolated and safe (they currently
store nothing).
53 * **Step 4 ? L3 local-vision fallback** (pending the local-VLM deep-research run) for
layouts the deterministic ladder can't reconcile.
54 * **Step 5 ? unify storage** (`transactions` + `holdings`) and begin the **instrument
family** (equity/MF/ETF/bond ? holdings + buy/sell/dividend/fee classification) ? the next big
piece of the whole-finance vision.
55
56 Parked: DESIGN §8.3 (local LLM for sensitive categories), §8.5 (merchant
canonicalization), encryption-at-rest timing (D.2).
57
```

### 531. user (2026-05-31T10:25:53.053Z)

```
178 ## 7. Cross-cutting workstreams (run throughout, not a phase)
179
180 - **Security & secrets hygiene** ? keychain for tokens + cached passwords; redacted
logs; gitignore audits each milestone; encryption-at-rest tracked to D.2.
181 - **Schema migrations from day one** ? versioned migration scripts; the schema *will*
churn as new issuers land. Keep `docs/schema.md` current.
182 - **Test-corpus curation** ? ground-truth answer key for the promo corpus (A.4);
synthetic/redacted statement fixtures for CI (C.2). Never commit real statements.
183 - **Decision log** ? append resolutions to § 1 here and mark the corresponding DESIGN.md
§ 8 item resolved.
184
185 ---
186
187 ## 8. Risks & mitigations
```

### 532. assistant (2026-05-31T10:26:43.301Z)

I have the anchors. Writing the grounded plan into ROADMAP (a clear "Planned, deferred" section between §7 and §8) plus a pointer in CURRENT\_STATE §5:

### 533. user (2026-05-31T10:26:59.238Z)

The file C:\Users\avidu\Projects\muneem\docs\ROADMAP.md has been updated successfully. (file state is current in your context ? no need to Read it back)

### 534. user (2026-05-31T10:27:08.210Z)

The file C:\Users\avidu\Projects\muneem\docs\CURRENT\_STATE.md has been updated successfully. (file state is current in your context ? no need to Read it back)

### 535. assistant (2026-05-31T10:27:19.784Z)

Docs-only note captured. Committing, opening the PR, and merging (then sync) so it's tracked on remote:

### 536. user (2026-05-31T10:27:23.437Z)

```
M docs/CURRENT_STATE.md
M docs/ROADMAP.md
[docs/track-regression-ci 99bdc9b] docs: track the regression-CI plan (synthetic cloud + self-
```

hosted real) ? decided, deferred  
2 files changed, 72 insertions(+)

#### 537. user (2026-05-31T10:27:36.595Z)

To <https://github.com/avidullu/muneem.git>  
\* [new branch] docs/track-regression-ci -> docs/track-regression-ci  
<https://github.com/avidullu/muneem/pull/29>

#### 538. user (2026-05-31T10:28:22.396Z)

```
test (3.13) pass 22s
https://github.com/avidullu/muneem/actions/runs/26710098419/job/78718590937
ocr skipping 0
https://github.com/avidullu/muneem/actions/runs/26710098419/job/78718590990
test (3.11) pass 20s
https://github.com/avidullu/muneem/actions/runs/26710098419/job/78718590939
test (3.12) pass 19s
https://github.com/avidullu/muneem/actions/runs/26710098419/job/78718590936
test (3.13) pass 22s
https://github.com/avidullu/muneem/actions/runs/26710098419/job/78718590937
ocr skipping 0
https://github.com/avidullu/muneem/actions/runs/26710098419/job/78718590990
--- checks exit: 0 ---
```

#### 539. assistant (2026-05-31T10:28:26.065Z)

CI green. Merging and syncing:

#### 540. user (2026-05-31T10:28:35.093Z)

```
docs/ROADMAP.md | 71 ++++++
2 files changed, 72 insertions(+)
Already on 'main'
- [deleted] (none) -> origin/docs/track-regression-ci
Already up to date.
=== clean slate ===
branch: main | sync: ## main...origin/main | uncommitted: 0 | open PRs: 0
f784e71 docs: track the regression-CI plan (synthetic cloud + self-hosted real) ? decided,
deferred (#29)
```

#### 541. assistant (2026-05-31T10:28:48.532Z)

Done ? captured as a tracked plan, **no code written**, slate clean.

### What I tracked (and where)

- **[ROADMAP.md](docs/ROADMAP.md)** ? new "Planned ? Regression CI for statement parsing" section ? the grounded decision: the two-layer "smart mix" (synthetic full-pipeline fixtures in cloud CI + a self-hosted runner for the real statements), **why we're not committing real statements** (permanent GitHub-cloud git history is the irreversible risk), both layers' concrete design (the stdlib PDF generator, HDFC x-band targeting, the savings-engine fixture, `MUNEEM_LOCAL_TESTDATA` staging, `password_rules.yaml` sourcing with nothing on GitHub), the one-time runner setup you'd do, and the caveats.`
- **[CURRENT\_STATE.md \$5](docs/CURRENT\_STATE.md)** ? a pointer bullet marked **decided, deferred** ? not yet built\* so it's visible from the live-status doc.

Merged via [muneem#29](<https://github.com/avidullu/muneem/pull/29>) (docs-only); CI green on all three Python versions.

### State

``main``, even with ``origin/main``, **0 uncommitted, 0 open PRs**, tip ``f784e71``. I cleaned up the in-session build tasks since the work is deferred ? the durable record is the ROADMAP section above.

When you want to build it, that section has enough to start cold: implement Layer 1 (synthetic fixtures + broaden `ci.yml` to all branches) and Layer 2 (the `real-statements.yml` self-hosted workflow + `password\_rules.yml` unification), no re-deciding needed.